LuaTEX 参考手册

中文版

Hans Hagen, Taco Hoekwater, Luigi Scarso, Hartmut Henkel

版本 1.24

2026 年 11 月 10 日

目录

第 1 章 简介 **1**

第 2 章 前言 **4**

第 3 章 TEX 的基本增强 **6**

3.1 引言 . . . 6
3.1.1 原语行为 . . . 6
3.1.2 版本信息 . . . 6
3.2 Unicode 文本支持 . . . 7
3.2.1 扩展范围 . . . 7
3.2.2 `Uchar` . . . 8
3.2.3 扩展的表 . . . 8
3.3 属性 . . . 8
3.3.1 节点 . . . 8
3.3.2 属性寄存器 . . . 8
3.3.3 盒子属性 . . . 9
3.4 与 Lua 相关的原语 . . . 10
3.4.1 `directlua` . . . 10
3.4.2 `luabytecode` . . . 11
3.5 类别码表 . . . 12
3.5.1 类别码 . . . 12
3.5.2 `catcodetable` . . . 12
3.5.3 `initcatcodetable` . . . 12
3.5.4 `savecatcodetable` . . . 12
3.6 抑制错误 . . . 13
3.6.1 `suppressfontnotfounderror` . . . 13
3.6.2 `suppresslongerror` . . . 13
3.6.3 `suppressifcsnameerror` . . . 13
3.6.4 `suppressoutererror` . . . 13
3.6.5 `suppressmathparerror` . . . 13
3.6.6 `suppressprimitiveerror` . . . 13
3.7 字体 . . . 14

3.7.1 字体语法 . . . 14
3.7.2 fontid . . . 14
3.7.3 noligs . . . 14
3.7.4 nospaces . . . 14
3.8 记号、命令与字符串 . . . 15
3.8.1 scantextokens . . . 15
3.8.2 toksapp . . . 15
3.8.3 csstring . . . 15
3.8.4 clearmarks . . . 16
3.8.5 alignmark . . . 16
3.8.6 letcharcode . . . 16
3.8.7 glet . . . 16
3.8.8 expanded . . . 16

第 4 章 修改 **17**
4.1 合并后的引擎 . . . 17
4.1.1 变革的必要性 . . . 17
4.1.2 与 TeX 3.1415926 相比的变化 . . . 17
4.1.3 与 e-TeX 2.2 相比的变化 . . . 18
4.1.4 与 pdfTeX 1.40 相比的变化 . . . 18
4.1.5 与 Aleph RC4 相比的变化 . . . 21
4.1.6 来自各处的变化 . . . 22
4.1.7 与标准 WEBC 相比的变化 . . . 22
4.2 后端原语 . . . 22
4.2.1 更少的原语 . . . 22
4.2.2 pdfextension . . . 22
4.2.3 默认值 . . . 27
4.2.4 向后兼容 . . . 27
4.3 方向 . . . 28
4.3.1 四种方向 . . . 28
4.3.2 工作原理 . . . 29
4.3.3 用 breakafterdirmode . . . 30
4.3.4 用 shapemode . . . 30
4.3.5 符号或数字 . . . 31
4.4 实现说明 . . . 31
4.4.1 内存分配 . . . 31
4.4.2 稀疏数组 . . . 31
4.4.3 简单的单字符 csname . . . 32
4.4.4 压缩的格式文件 . . . 32
4.4.5 二进制文件读取 . . . 32

4.4.6 制表符与空格 . . . 32
4.4.7 超链接 . . . 32

第 5 章 使用 LuaTEX **34**
5.1 初始化 . . . 34
5.1.1 作为 Lua 解释器的 LuaTEX . . . 34
5.1.2 作为 Lua 字节编译器的 LuaTEX . . . 34
5.1.3 其他命令行处理 . . . 34
5.2 Lua 行为 . . . 38
5.2.1 Lua 版本 . . . 38
5.2.2 融入 TDS 生态系统 . . . 38
5.2.3 加载库 . . . 38
5.2.4 执行外部程序 . . . 38
5.2.5 多字节 `string` . . . 39
5.2.6 `os` . . . 40
5.2.7 用 `fio` . . . 41
5.2.8 用 `sio` . . . 42
5.2.9 符合 `sha2` . . . 42
5.2.10 区域设置 . . . 42
5.3 Lua 模块 . . . 42
5.4 测试 . . . 43

第 6 章 语言、字符、字体与字形 **44**
6.1 引言 . . . 44
6.2 字符、字形与断字 . . . 44
6.3 主控制循环 . . . 46
6.4 加载断字模式与例外词 . . . 48
6.5 应用断字 . . . 49
6.5.1 `hyphenate` . . . 50
6.5.2 `[set|get]hjcode` . . . 50

第 7 章 字体结构 **52**
7.1 字体表 . . . 52
7.2 真实字体 . . . 58
7.3 虚拟字体 . . . 59
7.3.1 结构 . . . 59
7.3.2 人造字体 . . . 62
7.3.3 虚拟字体示例 . . . 62
7.4 `vf` . . . 63
7.5 `font` . . . 63
7.5.1 加载 TFM 文件 . . . 63

7.5.2 加载 VF 文件 . . . 63
7.5.3 fonts 数组 . . . 63
7.5.4 检查字体状态 . . . 64
7.5.5 直接定义字体 . . . 64
7.5.6 Unicode 映射 . . . 65
7.5.7 扩展字体 . . . 65
7.5.8 预计的下一个字体 id . . . 66
7.5.9 字体 id . . . 66
7.5.10 遍历所有字体 . . . 66
7.5.11 `glyphdimensionsmode` . . . 66
7.5.12 `discretionaryligaturemode` . . . 67
7.5.13 `tracinglostchars` . . . 67

第 8 章 数学排版 68

8.1 传统方式与 OpenType 并用 . . . 68
8.2 Unicode 数学字符 . . . 68
8.3 数学样式 . . . 70
8.3.1 `mathstyle` . . . 70
8.3.2 `Ustack` . . . 71
8.3.3 压缩数学样式 . . . 71
8.4 数学参数设置 . . . 72
8.4.1 大量新的 `Umath*` 原语 . . . 72
8.4.2 基于字体的数学参数 . . . 74
8.5 数学间距 . . . 78
8.5.1 行间环绕间距 . . . 78
8.5.2 固定上下标 . . . 78
8.5.3 Penalty：`mathpenaltiesmode` . . . 79
8.5.4 公式间距：`matheqnogapstep` . . . 79
8.6 数学构造 . . . 80
8.6.1 重音符号处理 . . . 80
8.6.2 根式扩展 . . . 80
8.6.3 上标与下标 . . . 80
8.6.4 可伸缩符号上的上下标 . . . 81
8.6.5 末行 . . . 81
8.7 数学模式 . . . 82
8.7.1 单字符数学命令的详细版本 . . . 82
8.7.2 脚本命令 `Unosuperscript` 和 `Unosubscript` . . . 82
8.7.3 非数学模式中允许的数学命令 . . . 82
8.8 附加特性 . . . 83
8.8.1 已废弃 . . . 83

8.8.2 扁平化：mathflattenmode 83
8.8.3 减少追踪 83

第 9 章 节点 **84**
9.1 Lua 节点表示 84
9.2 主要文本节点 84
9.2.1 hlist 节点 84
9.2.2 vlist 节点 85
9.2.3 rule 节点 85
9.2.4 ins 节点 86
9.2.5 mark 节点 86
9.2.6 adjust 节点 87
9.2.7 disc 节点 87
9.2.8 math 节点 88
9.2.9 glue 节点 88
9.2.10 kern 节点 89
9.2.11 penalty 节点 89
9.2.12 glyph 节点 89
9.2.13 boundary 节点 91
9.2.14 local_par 节点 91
9.2.15 dir 节点 91
9.2.16 marginkern 节点 92
9.3 数学 noad 92
9.3.1 数学核心子节点 92
9.3.2 math_char 与 math_text_char 子节点 92
9.3.3 sub_box 与 sub_mlist 子节点 92
9.3.4 delim 子节点 93
9.3.5 数学核心节点 93
9.3.6 简单 noad 节点 94
9.3.7 accent 节点 94
9.3.8 style 节点 94
9.3.9 choice 节点 94
9.3.10 radical 节点 95
9.3.11 fraction 节点 95
9.3.12 fence 节点 95
9.4 前端 whatsit 96
9.4.1 open 96
9.4.2 write 96
9.4.3 close 96
9.4.4 user_defined 97

9.4.5 save_pos . . . 97
9.4.6 late_lua . . . 97
9.5 DVI 后端 whatsit . . . 98
9.5.1 special . . . 98
9.6 PDF 后端 whatsit . . . 98
9.6.1 pdf_literal 与 pdf_late_literal . . . 98
9.6.2 pdf_refobj . . . 99
9.6.3 pdf_annot . . . 99
9.6.4 pdf_start_link . . . 99
9.6.5 pdf_end_link . . . 99
9.6.6 pdf_dest . . . 99
9.6.7 pdf_action . . . 100
9.6.8 pdf_thread . . . 101
9.6.9 pdf_start_thread . . . 101
9.6.10 pdf_end_thread . . . 101
9.6.11 pdf_colorstack . . . 101
9.6.12 pdf_setmatrix . . . 102
9.6.13 pdf_save . . . 102
9.6.14 pdf_restore . . . 102
9.7 node . . . 102
9.7.1 引言 . . . 102
9.7.2 is_node . . . 103
9.7.3 types 与 whatsits . . . 103
9.7.4 id . . . 103
9.7.5 type 与 subtype . . . 103
9.7.6 fields . . . 104
9.7.7 has_field . . . 104
9.7.8 new . . . 104
9.7.9 free、flush_node 与 flush_list . . . 104
9.7.10 copy 与 copy_list . . . 105
9.7.11 prev 与 next . . . 105
9.7.12 current_attr . . . 105
9.7.13 hpack . . . 106
9.7.14 vpack . . . 106
9.7.15 prepend_prevdepth . . . 106
9.7.16 dimensions 与 rangedimensions . . . 106
9.7.17 mlist_to_hlist . . . 107
9.7.18 slide . . . 107
9.7.19 tail . . . 108
9.7.20 length 与 count . . . 108

9.7.21 is_char 与 is_glyph . . . 108
9.7.22 traverse . . . 108
9.7.23 traverse_id . . . 109
9.7.24 traverse_char 与 traverse_glyph . . . 109
9.7.25 traverse_list . . . 110
9.7.26 has_glyph . . . 110
9.7.27 end_of_math . . . 110
9.7.28 remove . . . 110
9.7.29 insert_before . . . 110
9.7.30 insert_after . . . 111
9.7.31 first_glyph . . . 111
9.7.32 ligaturing . . . 111
9.7.33 kerning . . . 111
9.7.34 unprotect_glyph[s] . . . 111
9.7.35 protect_glyph[s] . . . 112
9.7.36 last_node . . . 112
9.7.37 write . . . 112
9.7.38 protrusion_skippable . . . 112
9.8 弹性间距处理 . . . 112
9.8.1 setglue . . . 112
9.8.2 getglue . . . 113
9.8.3 is_zero_glue . . . 113
9.9 属性处理 . . . 113
9.9.1 属性 . . . 113
9.9.2 attribute_list 节点 . . . 113
9.9.3 attr 节点 . . . 113
9.9.4 has_attribute . . . 114
9.9.5 get_attribute . . . 114
9.9.6 find_attribute . . . 114
9.9.7 set_attribute . . . 114
9.9.8 unset_attribute . . . 114
9.9.9 slide . . . 114
9.9.10 check_discretionary、check_discretionaries . . . 115
9.9.11 flatten_discretionaries . . . 115
9.9.12 family_font . . . 115
9.10 两种访问模型 . . . 115
9.11 属性表（properties） . . . 122

第 10 章 Lua 回调 **125**

10.1 注册回调 . . . 125
10.2 文件发现回调 . . . 125
10.2.1 find_read_file 与 find_write_file . . . 126
10.2.2 find_font_file . . . 126
10.2.3 find_output_file . . . 126
10.2.4 find_format_file . . . 126
10.2.5 find_vf_file . . . 127
10.2.6 find_map_file . . . 127
10.2.7 find_enc_file . . . 127
10.2.8 find_pk_file . . . 127
10.2.9 find_data_file . . . 127
10.2.10 find_opentype_file . . . 127
10.2.11 find_truetype_file 与 find_type1_file . . . 127
10.2.12 find_image_file . . . 127
10.3 文件读取回调 . . . 128
10.3.1 open_read_file . . . 128
10.3.2 通用文件读取器 . . . 128
10.4 数据处理回调 . . . 129
10.4.1 process_input_buffer . . . 129
10.4.2 process_output_buffer . . . 129
10.4.3 process_jobname . . . 130
10.5 节点列表处理回调 . . . 130
10.5.1 contribute_filter . . . 130
10.5.2 buildpage_filter . . . 130
10.5.3 build_page_insert . . . 131
10.5.4 pre_linebreak_filter . . . 131
10.5.5 linebreak_filter . . . 132
10.5.6 append_to_vlist_filter . . . 132
10.5.7 post_linebreak_filter . . . 133
10.5.8 hpack_filter . . . 133
10.5.9 vpack_filter . . . 133
10.5.10 hpack_quality . . . 134
10.5.11 vpack_quality . . . 134
10.5.12 process_rule . . . 134
10.5.13 pre_output_filter . . . 134
10.5.14 hyphenate . . . 134
10.5.15 ligaturing . . . 135
10.5.16 kerning . . . 135
10.5.17 insert_local_par . . . 135

10.5.18 mlist_to_hlist . . . 135
10.6 信息报告回调 . . . 136
10.6.1 pre_dump . . . 136
10.6.2 start_run . . . 136
10.6.3 stop_run . . . 136
10.6.4 start_page_number . . . 136
10.6.5 stop_page_number . . . 136
10.6.6 show_error_hook . . . 136
10.6.7 show_error_message . . . 137
10.6.8 show_ignored_error_message . . . 137
10.6.9 show_lua_error_hook . . . 137
10.6.10 start_file . . . 137
10.6.11 stop_file . . . 137
10.6.12 call_edit . . . 137
10.6.13 finish_synctex . . . 138
10.6.14 wrapup_run . . . 138
10.7 与 PDF 相关的回调 . . . 138
10.7.1 finish_pdffile . . . 138
10.7.2 finish_pdfpage . . . 138
10.7.3 page_order_index . . . 138
10.7.4 process_pdf_image_content . . . 139
10.8 与字体相关的回调 . . . 139
10.8.1 define_font . . . 139
10.8.2 glyph_not_found 与 glyph_info . . . 139
10.8.3 provide_charproc_data . . . 140

第 11 章 与 TEX 相关的库 141

11.1 lua . . . 141
11.1.1 版本信息 . . . 141
11.2 status . . . 141
11.3 tex . . . 143
11.3.1 简介 . . . 144
11.3.2 内部参数值：set . . . 144
11.3.3 转换命令 . . . 148
11.3.4 最后一项命令 . . . 148
11.3.5 访问寄存器：set* . . . 149
11.3.6 字符码寄存器：[get|set*code[s] . . . 151
11.3.7 Box 寄存器：[get|setbox . . . 152
11.3.8 重用 box：[use|saveboxresource 与 getboxresourcedimensions . . . 153
11.3.9 triggerbuildpage . . . 153

11.3.10 splitbox . . . 153
11.3.11 访问数学参数：[get|setmath . . . 153
11.3.12 特殊列表头：[get|setlist . . . 154
11.3.13 语义嵌套层级：getnest . . . 155
11.3.14 打印函数 . . . 156
11.3.15 辅助函数 . . . 158
11.3.16 处理原语的函数 . . . 160
11.3.17 核心功能接口 . . . 161
11.3.18 随机数生成器 . . . 164
11.3.19 与 synctex 相关的函数 . . . 164
11.4 texconfig . . . 164
11.5 texio . . . 167
11.5.1 write . . . 167
11.5.2 write_nl . . . 167
11.5.3 setescape . . . 167
11.5.4 closeinput . . . 167
11.6 token . . . 168
11.6.1 扫描器 . . . 168
11.7 kpse . . . 169
11.7.1 set_program_name . . . 170
11.7.2 record_input_file . . . 170
11.7.3 find_file . . . 170
11.7.4 lookup . . . 171
11.7.5 init_prog . . . 172
11.7.6 readable_file . . . 172
11.7.7 expand_path . . . 172
11.7.8 expand_var . . . 172
11.7.9 expand_braces . . . 172
11.7.10 in_name_ok . . . 172
11.7.11 in_name_ok_silent_extended . . . 172
11.7.12 out_name_ok . . . 173
11.7.13 out_name_ok_silent_extended . . . 173
11.7.14 show_path . . . 173
11.7.15 var_value . . . 173
11.7.16 version . . . 173
11.7.17 check_permission . . . 173

第 12 章 图形库 **174**

12.1 img . . . 174
12.1.1 new . . . 174

12.1.2 fields . . . 175
12.1.3 scan . . . 176
12.1.4 copy . . . 177
12.1.5 write、immediatewrite、immediatewriteobject . . . 177
12.1.6 node . . . 178
12.1.7 types . . . 178
12.1.8 boxes . . . 178
12.2 mplib . . . 179
12.2.1 new . . . 179
12.2.2 statistics . . . 180
12.2.3 execute . . . 180
12.2.4 finish . . . 181
12.2.5 结果表 . . . 181
12.2.6 辅助表格式 . . . 184
12.2.7 笔和 pen_info . . . 185
12.2.8 字符尺寸信息 . . . 185

第 13 章 字体加载器 187
13.1 快速获取字体信息 . . . 187
13.2 加载 OpenType 或 TrueType 文件 . . . 188
13.3 应用 “特性文件 . . . 189
13.4 应用 “AFM 文件 . . . 189
13.5 fontloader 字体表 . . . 189
13.6 表类型 . . . 190
13.6.1 主表 . . . 190
13.6.2 glyphs . . . 192
13.6.3 map . . . 196
13.6.4 private . . . 197
13.6.5 cidinfo . . . 197
13.6.6 pfminfo . . . 197
13.6.7 names . . . 199
13.6.8 anchor_classes . . . 199
13.6.9 gpos . . . 199
13.6.10 gsub . . . 201
13.6.11 ttf_tables . . . 201
13.6.12 mm . . . 201
13.6.13 mark_classes . . . 202
13.6.14 math . . . 202
13.6.15 validation_state . . . 202
13.6.16 horiz_base . . . 202

13.6.17 altuni . . . 203
13.6.18 vert_variants . . . 203
13.6.19 mathkern . . . 204
13.6.20 kerns . . . 204
13.6.21 vkerns . . . 204
13.6.22 texdata . . . 204
13.6.23 lookups . . . 204

第 14 章 HarfBuzz 库 **207**

14.1 luaharfbuzz . . . 207
14.1.1 示例 . . . 208

第 15 章 后端库 **213**

15.1 pdf . . . 213
15.1.1 mapfile . . . 213
15.1.2 [set|get . . . 213
15.1.3 [set|get . . . 213
15.1.4 [set|get . . . 213
15.1.5 [set|getversion . . . 213
15.1.6 getcreationdate . . . 214
15.1.7 [set|getinclusionerrorlevel 和 [set|get]ignoreunknownimages . . . 214
15.1.8 [set|get]suppressoptionalinfo, [set|get]trailerid, [set|get]omitcidset, [set|get]omitinfo, [set|get]omitmediabox, [set|get]omitprocset, [set|get]ptexprefix . . . 214
15.1.9 [set|getcompresslevel 和 [set|get]recompress . . . 214
15.1.10 [set|getgentounicode . . . 214
15.1.11 [set|getdecimaldigits . . . 214
15.1.12 [set|getpkresolution . . . 215
15.1.13 getlast[obj|link|annot 和 getretval . . . 215
15.1.14 getmaxobjnum . . . 215
15.1.15 [set|getorigin . . . 215
15.1.16 [set|getimageresolution . . . 215
15.1.17 [set|getmargin . . . 215
15.1.18 get[pos|hpos|vpos . . . 215
15.1.19 [has|getmatrix . . . 216
15.1.20 print . . . 216
15.1.21 immediateobj . . . 216
15.1.22 obj . . . 217
15.1.23 refobj . . . 217
15.1.24 reserveobj . . . 218

15.1.25 getpageref . . . 218
15.1.26 registerannot . . . 218
15.1.27 newcolorstack . . . 218
15.1.28 setfontattributes . . . 218
15.2 pdfe . . . 218
15.2.1 简介 . . . 218
15.2.2 open . . . 219
15.2.3 getsize . . . 220
15.2.4 get[catalog|trailer|info . . . 220
15.2.5 getpage . . . 220
15.2.6 get[string|integer|number|boolean|name, type . . . 220
15.2.7 get[from . . . 221
15.2.8 [open|close|readfrom|whole|stream . . . 221
15.2.9 getfrom[dictionary|array . . . 221
15.2.10 [dictionary|arraytotable . . . 222
15.2.11 getfromreference . . . 222
15.3 内存流 . . . 222
15.4 pdfscanner . . . 223

索引 **226**

索引 **226**

第 1 章 简介

这是 LuaTEX 的参考手册。我们不敢说它完备齐全，并且假定读者已经了解 TEX，正如 “The TEX Book、“e-TEX 手册、“pdfTEX 手册等文献中所描述的那样。更多参考资料发表于用户组期刊以及与 ConTEXt 相关的文档中。

从项目启动到稳定的 1.0 版本，大约经历了十年时间，但这是有充分理由的。后续版本带来了新功能、更强的控制力以及对内部结构的清理。实验性功能逐步演变为稳定功能，或被移除。早在相当早的阶段，LuaTEX 就已经可以用于生产环境，作者们也在日常工作中使用它。后续版本有时需要对 Lua 接口进行调整，但核心概念保持不变。当前版本在功能上可以认为是稳定的，不会再有根本性的变化。当然，我们之后可能会决定推出具有不同特性的 2.00 版本。

不要指望 LuaTEX 的行为与 pdfTEX 完全一致！虽然那个 8 位引擎的核心功能是我们的起点，但它已经与 Omega（Aleph）的方向性支持相结合。而且，LuaTEX 由于采用宽字符（32 位）、众多寄存器以及大内存支持，行为可能有所不同。生成的 PDF 代码与 pdfTEX 不同，但用户通常不会察觉。它原生支持 UTF 输入、支持大型（超过 8 位）字体，并且数学排版机制针对 OpenType 数学做了调优。同时也支持方向性排版。日志输出可能与其他引擎不同，而且随着我们的推进，差异可能会更大。当你运行纯 TEX 时，LuaTEX 无疑比 pdfTEX 慢，但当你运行例如 ConTEXt MKIV 时，在很多情况下它反而更快，尤其是当你有稍微复杂一些的文档或输入时。无论如何，全链路 32 位加上更多功能是有代价的，但在现代机器上这已经不是真正的问题。

测试是在 ConTEXt 下进行的，但 LuaTEX 也应该能与其他宏包良好配合。为此我们提供了通用字体处理器，这些处理器与 ConTEXt 中使用的大体相同。讨论具体的实现细节超出了本手册的范围。即使我们努力让 LuaTEX 保持精干高效，这里已经有足够多的内容值得讨论。

LuaTEX 由若干相互关联但（目前仍）可区分的部分组成。源代码的组织方式经过了调整，以便将所有这些组件粘合在一起。我们持续清理 TEX 引擎中积累的代码所带来的副作用（尤其是不再需要的代码）。

- 我们最初采用了 pdfTEX 1.40.9 版本的大部分代码。代码库被转换为 C 并拆分为多个模块。实验性功能被移除，实用宏也未被继承，因为它们的功能可以用 Lua 编程实现。后端接口命令的数量已减少到少数几个。所谓的扩展功能与核心分离（我们尽量让核心保持与原始 TEX 核心接近）。一些机制如展开和凸排（protrusion）由于清理和优化，行为可能与原始版本不同。一些基于 whatsit 的功能（图像支持和可复用内容）现在已成为核心功能。我们不再与 pdfTEX 的开发同步。
- 包含了 Aleph RC4（源自 Omega）的方向模型。相关的原始命令是 LuaTEX 核心的一部分，但在节点级别，方向性支持不再基于所谓的 whatsit，而是基于具有相关属性的真实节点。方向的数量被限制在有用的集合内，后端也已具备方向感知能力。
- Aleph 的 I/O 转换过程、tcx 文件以及 enctex 均不可用。这些与编码相关的功能已被基于

Lua 的方案（reader 回调）所取代。以类似的方式，所有文件 I/O 都可以被拦截。

- 我们目前使用 Lua 5.3.*。有少数几个 Lua 库我们认为是核心 Lua 机制的一部分，例如 `lpeg`。还有一些额外的 Lua 库用于与 TEX 的内部接口对接。我们也保留了 Lua 5.2 的 `bit32` 库。
- 有各种各样的 TEX 扩展，但仅限于那些无法通过 Lua 接口实现的扩展。数学机制通常有两条代码路径：一条是传统路径，另一条更适合宽 OpenType 字体。在这方面我们尽可能遵循 Microsoft 规范。一些数学功能已经开放了一些接口，以便用户获得更多控制权。
- fontloader 使用了 FontForge 2008.11.17 的部分代码，结合了专为 TEX 引擎使用而编写的额外代码。我们尽量将特定字体支持最小化到 TEX 所需的程度：字符引用和尺寸，并将其他一切委托给 Lua。这样我们可以在不触及核心的情况下让 TEX 保持可扩展。为了最小化依赖，我们可能会在某个时候决定将其设为可选库。
- MetaPost 库是 LuaTEX 的组成部分。这为 TEX 提供了一定的图形能力，使用的是相对高速的图形子系统。同样，Lua 被用作前端和后端之间的桥梁。MetaPost 的进一步开发与 LuaTEX 密切相关。
- TEX 自带的虚拟字体技术已被集成到字体机制中,允许在运行时创建虚拟字体。由于 LuaTEX 同时也可以充当 Lua 解释器，这意味着无需额外程序即可构建完整的 TEX 工作流。
- 从 1.09 版本开始，不再使用 poppler 库来包含（PDF），而是使用一个轻量级的专用库。这移除了一个依赖，但也使得 LuaTEX 的包含代码与 pdfTEX 不同。事实上，由于 Lua 图像接口的存在，它本来就已经大不相同了。

我们努力保持后续版本的兼容性，但中间版本可能包含实验性功能。一般规则是：进入 TeX Live 以及/或者在 ConTEXt 会议前后发布的版本是稳定版本。年度 TeX Live 发布之间的任何版本都应视为 beta 版，在代码仓库中以 trunk 版本的形式存在。我们有一个用于开发的实验分支，但不对其任何实验性功能提供支持。中间版本（来自 trunk）通常通过 ConTEXt 发行渠道（garden 以及所谓的 minimals）提供。

1.10 版本或多或少是开发的终点：这就是你所得到的。因为不仅 ConTEXt （我们可以相当容易地适配）使用 LuaTEX，所以我们不能在没有不可预见后果的情况下改变基础。到目前为止，已经证明 Lua 可以用来扩展核心功能，因此没有必要再添加更多功能，更不用说为（不那么）常见的问题提供硬编码解决方案了。当然，会有 bug 修复，也许还有一些优化，甚至可能有一些新增内容或非侵入式改进，但只能在稳定版本之外经过测试后才能加入。毕竟，二进制文件已经够大了，能获得的收益也不多。

你可能会发现一些尚未文档化的 Lua 辅助函数。这些被认为是实验性的，不过当你在一个已经存在一段时间的 ConTEXt 版本中遇到它们时，可以认为它们会保留下来。当然，也可能只是我们还没来得及文档化它们。

这样的手册并不真正适合作为教程，关于教程我们推荐参阅 ConTEXt 附带的文档、文章等。它对所有读者来说也永远不够完备。我们努力跟上进度，但读者需要意识到，这都是业余时间做的志愿者工作。而且可以肯定的是，抱怨手册不好或文档糟糕并不会真正激励我们花更多时间在上面。话虽如此，我们希望本文档是有用的。

Hans Hagen Harmut Henkel Taco Hoekwater Luigi Scarso

版本	2026 年 11 月 10 日
LuaTEX	1.10
ConTEXt	MkIV

第 2 章 前言

这是一本参考手册，不是教程。这意味着我们讨论的是相对于传统 TEX 的变化，同时也介绍新功能。因此，我们会引用一些假定读者已知的概念，或者将在后面解释的概念。

一般用户不需要了解本手册中的太多内容。例如，字体和语言通常由你所使用的宏包来处理。在用户层面，通常也不需要真正去摆弄节点列表。如果你确实要摆弄，那最好清楚自己在做什么。阅读 Donald Knuth 的 “The TEX Book” 是很好的时间投资，因为了解一切的起源也是有益的。Victor Eijkhout 的 “TEX by Topic” 提供了更具概括性的综述。你也可以翻阅 “e-TEX 手册” 以及关于 pdfTEX 的文档。

但是，如果你是因为 Lua 来到这里的，那么你只需要知道：可以在排版运行中调用它。你使用的宏包可能会提供一些封装机制，但完成这项工作的基本 `directlua` 命令是：

```
\directlua{tex.print("Hi there")}
```

你可以将代码放在花括号之间，但如果代码量较大，也可以将其放入文件中，并使用常规的 Lua 命令加载该文件。

如果你仍然决定继续阅读，那么了解什么是节点会很有帮助，所以我们在这里做一个简要介绍。如果你输入以下这段文字：

```
Hi There
```

最终我们会得到一个节点链表，用 ASCII 艺术表示如下：

```
H <=> i <=> [glue] <=> T <=> h <=> e <=> r <=> e
```

当我们有一个段落时，实际上会得到这样的结构：

```
[localpar] <=> H <=> i <=> [glue] <=> T <=> h <=> e <=> r <=> e <=> [glue]
```

每个字符都会成为一个所谓的 glyph 节点，这是一个包含当前字体、字符编码和当前语言等属性的记录。空格会成为 glue 节点。节点类型有很多，我们将在后面讨论。每个节点指向前一个节点或后一个节点（如果它们存在的话）。

事先了解以下几点也是有益的：TEX 基本上围绕着创建段落和页面来运作。段落构建器接收一个列表并将其断行为多行。我们将水平材质转换为垂直材质。行就是所谓的 box，可以用 glue、penalty 等分隔。页面构建器累积行，在合适时触发输出例程，该例程会接收目前为止的列表。构建实际页面并不是 TEX 的职责，而是通过允许操作 box 的原始命令来完成的。结果交还给 TEX，并输出到（通常是 PDF）文件。

LuaTEX 引擎在处理过程中几乎每个合理的节点都为 Lua 代码提供了钩子：收集内容、断字、应用字体特性、断行等。这意味着你可以重载 TEX 的固有行为，而这种固有行为仍然是基准。当我们提到“回调（callbacks）”时，指的就是这些钩子。

纯 TEX 基本上是一个用于编写特定样式的基础框架，而像 ConTEXt 和 LATEX 这样的宏包则为用户提供了大量额外工具，使文档排版美观。它们隐藏了字体管理、语言需求、将结构转化为排版结果、分页、包含图像等繁琐细节。你应该注意，当你接入自己的代码来操作列表时，这可能会与你所使用的宏包产生冲突。

在接下来的章节中阅读关于节点的内容时，最好记住与它们相关的命令。以下是几个例子：

命令	节点	说明
hbox	hlist	水平 box
vbox	vlist	垂直 box，基线位于底部
vtop	vlist	垂直 box，基线位于顶部
hskip	glue	水平间距，可伸缩可收缩
vskip	glue	垂直间距，可伸缩可收缩
kern	kern	水平或垂直固定间距
discretionary	disc	断字点（前置、后置、替代）
char	glyph	一个字符
hrule	rule	一条水平线
vrule	rule	一条垂直线
textdir(ection)	dir	文本方向的改变

目前这些内容应该足以让你理解接下来的章节了。

第 3 章 TEX 的基本增强

3.1 引言

3.1.1 原语行为

自诞生之日起，LuaTEX 就提供了超越 pdfTEX（包含 e-TEX）与 Aleph 超集的额外特性。这不仅限于通过 `directlua` 执行 Lua 代码的能力，LuaTEX 还通过新增的 TEX 侧原语或对现有原语的扩展来增加功能。

当 LuaTEX 以 “iniluatex 模式（`luatex -ini`）启动时，它仅定义 TEX82 已知的原语命令以及一个额外命令 `directlua`。正如其名所示，必须调用一个 Lua 函数才能将额外的原语添加到用户环境中。获取所有新原语命令的最简单方法是在格式生成文件中加入这一行：

```
\directlua { tex.enableprimitives('',tex.extraprimitives()) }
```

但请注意，在这么早的阶段，花大括号可能尚未被赋予正确的 `catcode`（会引发 “Missing number 错误），因此可能需要在上述行之前先进行如下赋值：

可以进行更细粒度的原语控制，详情请参阅 11.3.16。为简便起见，本手册假设你已经按上述方式执行了 `directlua` 命令。

上面记录的启动行为被认为是稳定的，即不会再有向后不兼容的更改。我们已将一些相当通用的 pdfTEX 原语提升为 LuaTEX 核心原语，而从 Aleph（Omega）继承的少数原语也同样被提升。这实际上意味着我们现在只剩下 `tex`、`etex` 和 `luatex` 三组原语。

在 4 中，我们讨论了若干源自 pdfTEX 和 Aleph（Omega）的原语。此处我们只讨论真正新增的原语。在关于字体和数学的章节中，我们还会讨论更多新增原语。

3.1.2 版本信息

`luatexbanner`

有三个新原语可用于检测 LuaTEX 的版本：

原语	值	说明
`luatexbanner`	[Lua 输出]	命令行上显示的横幅
`luatexversion`	1.10	主版本号与次版本号的组合

luatexrevision	1	修订号，当前值为

LuaTEX 的官方版本号定义如下：

- 主版本号是 luatexversion 除以 100 的整数结果。该原语是一个 “内部变量，因此根据上下文，你可能需要在其使用前加上 the。
- 次版本号是 luatexversion 对 100 取模后得到的两位数结果。
- 修订号由 luatexrevision 报告。该原语展开为一个正整数。
- 完整版本号由主版本号、次版本号和修订号组成，以点号分隔。

formatname

formatname 的语法与 jobname 完全相同。在 initex 中，其展开结果为空。否则，其展开结果就是当前加载的格式在 initex 转储运行期间 jobname 所具有的值。你可以使用这个记号表来提供自己的版本信息。

3.2　Unicode 文本支持

3.2.1　扩展范围

文本输入和输出现在被视为 Unicode 文本，因此输入字符可以使用完整的 Unicode 范围 ($2^{20}+2^{16}-1=$ 0x10FFFF)。后续章节会讨论字符与字形。虽然两者不能互换，但密切相关。在排版时，字符总是被转换为特定字体中该字符的合适图形表示。然而，在处理待排版节点列表时，其内容仍可被视为字符。在 LuaTEX 内部，这两个概念之间没有明确的界限。由于字形节点的子类型可以在 Lua 中更改，这取决于用户自己。子类型大于 255 表示已经进行过字体处理。

少数几个原语受到此影响，且影响方式类似：它们都必须接受更大范围的数值。例如，char 现在接受 0 到 1,114,111 之间的值。对于规范的输入文件，这应该不成问题，但对于那些在旧的基于 TEX 的引擎处理时本应产生错误的输入，这可能造成不兼容。受影响且初始值（等号左侧）或次要值（等号右侧）发生改变的命令有：char、lccode、uccode、hjcode、catcode、sfcode、efcode、lpcode、rpcode、chardef。

就核心引擎而言,所有与文本文件的输入输出均采用 UTF-8 编码。输入文件可以通过 reader 回调进行预处理，这将在 10.3 中说明。Unicode 输入的规范化有意不内建，可由宏包在回调处理期间完成。我们已做出一些务实的选择，用户需要接受这些选择。

要按字节大小输出, 可以使用刚好超出有效 Unicode 范围的字符, 从值 1,114,112 (0x110000) 开始。当需要打印字符 $c >=$ 1,114,112 时，LuaTEX 实际打印的是 c 减去 1,114,112 后对应的单个字节。

输出到终端时, 较低控制范围 ($c < 32$) 的字符使用 \textasciicircum \textasciicircum {} 记法, \textasciicircum \textasciicircum {}I、\textasciicircum \textasciicircum {}J 和 \textasciicircum \textasciicircum {}M 除外。这些被视为 “安全字符，因此按原样打印。你可以通过 texio.setescape(false) 禁用转义，这样控制台上就会显示正常字符。

3.2.2　Uchar

可展开命令 Uchar 读取一个 0 到 1,114,111 之间的数字，并展开为对应的 Unicode 字符。

3.2.3　扩展的表

所有传统 TEX 和 e-TEX 寄存器都可以使用 16 位数字。受影响的命令有：

count dimen skip muskip marks toks countdef dimendef skipdef muskipdef toksdef
insert box unhbox unvbox copy unhcopy unvcopy wd ht dp setbox vsplit

由于字体内存管理已被重写，源自同一度量文件的不同字体实例之间不再共享字体中的字符属性。当然，我们在后端会尽可能共享字体，以使生成的 PDF 文件尽可能高效，但例如扩展和突出也不再像 pdfTEX 那样使用副本。

3.3　属性

3.3.1　节点

当 TEX 读取输入时，它会根据字符的属性来解释输入流。有些字符表示宏名并触发展开，另一些则开启和关闭分组、触发数学模式等。剩下的内容就成为排版文本。在内部，我们得到的是节点的链表。字符变为 glyph 节点，这类节点具有例如 font 和 char 属性，而 \kern 10pt 则变为带有 width 属性的 kern 节点。空格对 TEX 来说是异类，因为它们被转换为 glue 节点。因此，一个简单的段落主要是 glyph 节点序列（单词）和 glue 节点（空格）的混合。

字符序列在某处会被与断字相关的 disc 节点扩展。之后可以应用字体逻辑，得到一个列表，其中某些字符可以被替换——例如多个字符可以变成一个连字，字体字距可以被注入。这一切由字体属性驱动。

盒子（如 hbox 和 vbox）变为 hlist 或 vlist 节点，带有 width、height、depth 和 shift 属性以及指向其实际内容的 list 指针。盒子可以显式构造，也可以是子进程的结果。例如，当行被断成段落时，各行就是 hlist 节点的链表。

总而言之：你输入的所有内容最终都会变成一个节点，通常作为（嵌套）列表结构的一部分。它们占用相对较少的内存，只携带所需的最少量信息。在传统 TEX 中，字符节点只保存字体和槽位编号；在 LuaTEX 中，我们还存储一些语言相关信息、扩展因子等。既然我们可以从 Lua 访问这些节点，让节点携带更多信息就有意义了，这正是属性发挥作用的地方。

3.3.2　属性寄存器

属性是 LuaTEX 中一个全新的概念。从语法上看，它们很像计数器：属性遵循 TEX 的嵌套栈，可以像普通 count 寄存器一样在 the 等之后使用。

```
\attribute <16-bit number> <optional equals> <32-bit number>!crlf
\attributedef <csname> <optional equals> <16-bit number>
```

从概念上讲，属性要么是“已设置，要么是“未设置。未设置的属性具有一个特殊的负值来表示它们未设置，该值是最低合法值：十六进制的 -"7FFFFFFF，即十进制的 −2147483647。因

此，值 -"7FFFFFFF 不能用作合法的属性值，但你可以将 -"7FFFFFFF 赋值给某个属性以“取消设置它。在 initex 中，所有属性都初始处于这种“未设置状态。

属性可以用作额外的计数值，但其用处主要在于：所有“已设置属性的编号和值都会附加到其作用域内创建的所有节点上。然后，任何处理节点的 Lua 代码都可以查询这些属性。关于如何在 Lua 中使用属性进行节点列表处理，详见 9。

属性存储在一个排序的（稀疏）链表中，并在可能时共享。这允许高效的测试和更新。你可以定义成千上万个属性，但通常这么大的数量没有意义，效率也不高，因为每个节点都携带一个（可能共享的）指向当前已设置属性列表的链接。不过它们是一个方便的扩展，也是我们在 LuaTEX 中最早实现的扩展之一。

3.3.3　盒子属性

节点通常会获得创建时生效的属性列表。这个时刻可能相当异步。例如：在段落构建中，各个行盒子是在 par 命令处理之后才创建的，因此它们获得的是当时生效的属性列表，而不是段落中例如第一行或第三行当时生效的属性。

类似情况在 LuaTEX 中经常发生。几个比较明显的问题情况已被处理：在断字、字距调整和连字期间创建的节点，其属性借用周围字形的属性；而且可以直接影响盒子属性。

当你在寄存器中组装一个盒子时，将这样的盒子放置、解包或复制时，其中包含的节点的属性保持不变。在这方面，属性的行为与已转换为字体中字形引用的字符相同。例如，当你使用属性来实现颜色支持时，每个节点都携带关于其最终颜色的信息。在这种情况下，除非你实现相应的处理机制，否则对已装箱的内容应用颜色将无效。请记住，这种不兼容主要是因为使用独立的 specials 和 literal 来表示颜色比使用属性更不自然。

可以通过使用关键字 attr 来微调应用于 hbox、vbox 或 vtop 的属性列表。attr 关键字应出现在 to 或 spread 之前（如果也指定了这两者的话）。示例如下：

```
\attribute997=123
\attribute998=456
\setbox0=\hbox {Hello}
\setbox2=\hbox attr 999 = 789 attr 998 = -"7FFFFFFF{Hello}

  for b=0,2,2 do
    for a=997, 999 do
      tex.sprint("box ", b, " : attr ",a," : ",tostring(tex.box[b]     [a]))
      tex.sprint("\\quad\\quad")
      tex.sprint("list ",b, " : attr ",a," : ",tostring(tex.box[b].list[a]))
      tex.sprint("\\par")
    end
  end
```

盒子 0 现在设置了属性 997 和 998，而盒子 2 设置了属性 997 和 999，但该盒子内部的节点都将设置属性 997 和 998。赋最大负值会导致某个属性被忽略。

为了让你了解这在 Lua 端意味着什么，请看以下代码：

后面我们会看到如何访问节点的属性。这里的盒子是所谓的 hlist 节点，它们有一个 list 字段指向内容。因为属性本身也是一个列表，你可以通过对节点进行索引来访问它们（这里我们用 [a] 来做）。运行这段代码片段会输出类似：

```
box 0 : attr 997 : 123    list 0 : attr 997 : 123
box 0 : attr 998 : 456    list 0 : attr 998 : 456
box 0 : attr 999 : nil    list 0 : attr 999 : nil
box 2 : attr 997 : 123    list 2 : attr 997 : 123
box 2 : attr 998 : nil    list 2 : attr 998 : nil
box 2 : attr 999 : 789    list 2 : attr 999 : nil
```

因为有些值未设置，我们需要在这里使用 `tostring` 函数，以便得到 `nil` 这个词。

3.4　与 Lua 相关的原语

3.4.1　directlua

为了将 Lua 代码与 TEX 输入融合，需要几个新的原语。原语 `directlua` 用于立即执行 Lua 代码。语法为

```
\directlua <general text>!crlf
\directlua <16-bit number> <general text>
```

`<general text>` 会被完全展开，然后送入 Lua 解释器。在对 `<general text>` 进行读取和展开之后，得到的记号列表会被转换为字符串，就像使用 `\the \toks` 显示的那样。在 Lua 端，每个 `directlua` 块都被视为一个独立的 chunk。在这样的 chunk 中，你可以使用 `local` 指令来避免你的变量与宏包使用的变量冲突。

与记号列表之间的转换意味着你通常不能在参数内使用 Lua 行注释（以 `--` 开头）。因为通常只有一个 “行，第一行注释会一直延续到输入末尾。你需要使用 TEX 风格的行注释（以% 开头），或者在局部改变 TEX 的类别码。另一种可能的写法是：

```
\begingroup

\directlua ...
\endgroup
```

这样就可以使用 Lua 行注释了，因为 TEX 不会用空格替换行尾。当然，这种方法取决于你使用的宏包。

`<16-bit number>` 指定一个 Lua chunk 的名称，取自 `lua.name` 数组（参见本手册后面关于 `lua` 表的文档）。当 chunk 名称以 `@` 开头时，它将显示为文件名。这是 Lua 实现错误处理方式的一个副作用。

`directlua` 命令是可展开的。由于它将 Lua 代码传递给 Lua 解释器，从 TEX 的角度来看，其展开通常为空。但是，有些 Lua 函数会产生供 TEX 读取的内容，即所谓的 print 函数。最简单的用法是 `tex.print(<string> s)`。字符串 `s` 的字符会被放置到 TEX 的输入缓冲区中，也就是 “在 TEX 眼前，供 TEX 立即读取。例如：

```
\count10=20
a\directlua{tex.print(tex.count[10]+5)}b
```

展开为

```
 = function() tex.print("!") end
    t[2] = function() tex.print("?") end
}

\luafunction1
\luafunction2
```

当然，函数也可以在单独的文件中定义。除了正常的 Lua 限制外，函数数量没有限制。当然，有限制是不接受参数，但那将涉及解析，从而不会带来任何收益。该函数在实际被调用时会获得一个参数，即索引，因此在下面的例子中会排印数字 8。

```
\directlua {
    local t = lua.get_functions_table()
    t[8] = function(slot) tex.print(slot) end
}
```

luafunctioncall 原语做同样的事情，但不可展开，例如在 edef 中使用。此外，LuaTEX 还提供了一个定义器：

```
                \luadef\MyFunctionA 1
          \global\luadef\MyFunctionB 2
\global\luadef\MyFunctionC 3
```

你确实应该谨慎使用这些命令。某些引用被存储在记号中，并假设当该记号展开时函数可用。另一方面，由于我们已在相对复杂的情况下测试过此功能，正常使用应该不会有问题。

3.4.2 luabytecode

与上一节讨论的函数调用器类似，我们有字节码调用器。同样，call 变体是不可展开的。

```
\directlua {
    lua.bytecode[9998] = function(s)
        tex.sprint(s*token.scan_int())
    end
    lua.bytecode[5555] = function(s)
        tex.sprint(s*token.scan_dimen())
    end
}
```

这与以下命令配合使用：

```
\luabytecode    9998 5  \luabytecode    5555 5sp
\luabytecodecall9998 5  \luabytecodecall5555 5sp
```

代码中的变量 s 是字节码寄存器的编号，可用于诊断目的。字节码寄存器相比函数调用的优势在于它们被存储在格式中（但不带 upvalue）。

3.5　类别码表

3.5.1　类别码

类别码表是一项新特性，允许你用一条语句切换到预定义的类别码规则。你可以拥有实际上无限数量的不同表。此子系统向后兼容：如果你从不使用以下命令，你的文档在行为上与传统 TEX 相比不会有任何差别。每个类别码表的内容独立于任何其他类别码表，其内容会被存储并从格式文件中取回。

3.5.2　catcodetable

```
\catcodetable <15-bit number>
```

原语 catcodetable 切换到不同的类别码表。这样的表必须事先使用以下两个原语之一创建，或者必须为零。表零由 initex 初始化。

3.5.3　initcatcodetable

```
\initcatcodetable <15-bit number>
```

原语 initcatcodetable 创建一个新表，其类别码与 initex 定义的相同。新类别码表是全局分配的：在当前组结束后不会消失。如果提供的数字与当前活动表相同，则会引发错误。初始值为：

类别码	字符	等价字符	类别
0	\		escape
5	^^M	return	car_ret
9	^^@	null	ignore
10	<space>	space	spacer
11	a–z		letter
11	A–Z		letter
12	其他所有字符		other
14	%		comment
15	^^?	delete	invalid_char

3.5.4　savecatcodetable

```
\savecatcodetable <15-bit number>
```

savecatcodetable 将当前的类别码集合复制到一个具有指定编号的新表中。这个新表中的定义都被视为在最外层层级做出的。

新表是全局分配的：在当前组结束后不会消失。如果提供的编号就是当前活动表，则会引发错误。

3.6　抑制错误

3.6.1　suppressfontnotfounderror

如果此整数参数非零，则 LuaTEX 不会抱怨找不到字体度量文件。相反，它会静默跳过字体赋值，使请求的字体 csname 通过 `ifx` 比较等于 `nullfont`，从而可以在不打扰用户的情况下对其进行测试。

```
\suppressfontnotfounderror = 1
```

3.6.2　suppresslongerror

如果此整数参数非零，则 LuaTEX 不会抱怨在通常禁止的上下文中遇到 `par` 命令（最突出的是在未定义为 `long` 的宏的参数中）。

```
\suppresslongerror = 1
```

3.6.3　suppressifcsnameerror

如果此整数参数非零，则 LuaTEX 不会抱怨在 `ifcsname` 展开中间出现不可展开的命令。相反，它会继续从输入中获取展开的记号，直到遇到 `endcsname` 命令。如果输入展开在 `csname …endcsname` 配对方面不平衡，LuaTEX 进程可能会无限期挂起。

```
\suppressifcsnameerror = 1
```

3.6.4　suppressoutererror

如果此新整数参数非零，则 LuaTEX 不会抱怨在通常禁止的上下文中遇到 `outer` 命令。

```
\suppressoutererror = 1
```

3.6.5　suppressmathparerror

以下设置将允许在数学公式中使用 `par` 记号：

```
\suppressmathparerror = 1
```

这样，下面的代码就是合法的了：

```
$ x + 1 =

a $
```

3.6.6　suppressprimitiveerror

当设置为非零值时，以下命令不会发出错误：

```
\suppressprimitiveerror = 1

\primitive\notaprimitive
```

3.7　字体

3.7.1　字体语法

LuaTEX 接受用花括号括起来的参数作为字体名：

```
\font\myfont = {cmr10}
```

这允许嵌入空格，而无需使用双引号。参数内部会进行宏展开。

3.7.2　fontid

```
\fontid\font
```

此原语展开为一个数字。它不是寄存器，因此不需要在前面加 number（而且使用 the 会报错）。当前使用的字体 id 是 < □□□□ id>。下面是更多示例：

样式	命令	字体 id
常规	\rmfamily	0
粗体	\bf	1
斜体	\it	2
粗斜体	\bfseries \itshape	3

这些数字取决于所使用的宏包，因为每个宏包都有自己处理字体的方式。它们也可能因运行而异，因为取决于字体加载的顺序。例如，在 ConTEXt 中使用虚拟数学 Unicode 字体时，我们很容易就有上百个 id 在使用中。并非所有 id 都必须绑定到真实字体，毕竟它只是一个数字。

原语 setfontid 可用于启用具有给定 id 的字体，当然该 id 必须是有效的。

3.7.3　noligs

这些原语禁止在 LuaTEX 主控制循环构建初始节点列表时插入连字和字距。当你想对 “字符进行节点列表处理，而 TEX 的正常处理会造成干扰时，可以启用这些原语。

```
\noligs <integer>!crlf
\nokerns <integer>
```

这些原语也可以通过重载连字构建和字距函数来实现，即为其关联的回调分配哑函数。请记住，当你（使用 Lua）定义字体时，也可以省略字距和连字表，效果与上述相同。

3.7.4　\nospaces

这个新原语可用于在文本流中遇到空格字符时，覆盖通常的 spaceskip 相关启发式规则。值 1 表示不注入任何内容，而 2 则注入一个零 skip。在 3.1 中，我们看到了四个以空格分隔的字符在不同设置下的结果。

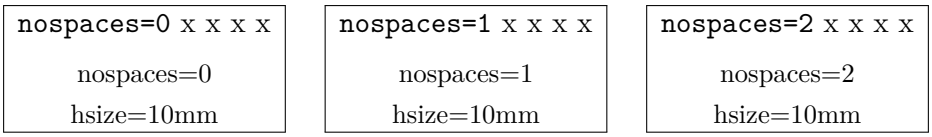


图 3.1: nospaces 选项的效果。

3.8　记号、命令与字符串

3.8.1　scantextokens

scantextokens 的语法与 scantokens 完全相同。该原语是 e-TEX 的 scantokens 的一个略微修改版本。区别在于：

- 最后一行（通常也是唯一一行）不会追加 endlinechar。
- scantextokens 从不引发 EOF 错误，也不执行 everyeof 记号。
- 不执行 “\unknown while end of file \unknown 错误检查。这允许展开在不同的分组层级结束，或者在条件尚不完整时结束。

3.8.2　toksapp

与其使用：

```
\toks0\expandafter{\the\toks0 foo}
```

你可以使用：

```
\etoksapp0{foo}
```

pre 变体是前置而不是追加，e 变体会展开传入的 general text。g 和 x 变体是全局的。

3.8.3　csstring

这些有些特殊。csstring 原语类似于 string，但它省略了前导的转义字符。这比之后再剥离它效率稍高一些。

begincsname 原语类似于 csname，但在不存在该名称时不会创建一个 relaxed 等价物。它等价于

```
\ifcsname foo\endcsname
  \csname foo\endcsname
\fi
```

其优势在于省去了一次查找（不要期望有太多速度提升），但更重要的是它避免了使用 if 测试。lastnamedcs 则需要谨慎使用。上面的例子可以写成：

```
\ifcsname foo\endcsname
  \lastnamedcs
\fi
```

这比构造两次字符串稍微高效一些（在 LuaTEX 深处这还涉及一些 UTF8 处理），但可能更重要的是它节省了几个记号，并使代码更易读。

3.8.4　clearmarks

该原语补充了 e-TEX 的 mark 原语，完全清除一个 mark 类，将三个关联的 mark 文本全部重置为空。它是一个立即命令。

```
\clearmarks <16-bit number>
```

3.8.5　alignmark

原语 alignmark 复制了对齐前言中 ## 的功能，而 aligntab 复制了 & 的功能。

3.8.6　letcharcode

此原语可用于为活动字符赋值，例如：

```
\def\foo{bar} \letcharcode123=\foo
```

这比使用大写技巧（利用 uppercase 对活动字符特殊处理的属性）要优雅一些。

3.8.7　glet

此原语类似于：

```
\def\glet{\global\let}
```

但更快（只有在数百万次调用时才可测量），而且可能更方便（毕竟我们还有 \gdef ）。

3.8.8　expanded

expanded 原语接受一个记号列表并展开其内容，这很方便：它避免了 expandafter 和 noexpand 的复杂组合。你可以将其与 edef 体内部发生的情况进行比较。但这种展开仍然不会展开某些原语操作。

```
\newcount\NumberOfCalls

\def\TestMe{\advance\NumberOfCalls1 }

\edef\Tested{\TestMe foo:\the\NumberOfCalls}
\edef\Tested{\TestMe foo:\the\NumberOfCalls}
\edef\Tested{\TestMe foo:\the\NumberOfCalls}

\meaning\Tested
```

结果是一个宏，其体内包含未展开的代码：

第 4 章 修改

4.1 合并后的引擎

4.1.1 变革的必要性

LuaTEX 的第一个版本仅增加了少量原语, 在很大程度上与 pdfTEX 相同。随后我们将 Aleph 的大量代码合并进来，得到了更多原语。当代码趋于稳定后，我们决定清理该程序相当混杂的本质。这意味着一些原语被提升为核心原语（通常使用不同的名称），另一些则被移除。这使得我们能够开始清理代码库。在 3 中我们讨论了一些新原语，这里将介绍大多数经过调整的原语。

除了由新功能引起的预期变化外，还有一些不太预期的变化。这些有时是新（冲突）特性的副作用，或者更常见的是清理内部接口所必需的变更。这些也会一并提及。

4.1.2 与 TEX 3.1415926 相比的变化

当然，一切都始于传统的 TEX。即使我们从 pdfTEX 起步，大部分仍源自原始版本。但我们有所偏离。

- 当前代码库使用 C 而非 Pascal 编写。我们尽可能使用 CWEB。因此，不再是一个大文件加上变更文件，而是按类别组织的多个文件，如 `tex`、`pdf`、`lang`、`font`、`lua` 等。转换为 C 过程中留下了一些痕迹，但我们会在适当的时候清理源代码并确保文档完善。许多文件采用 CWEB 格式，但其他文件（如与 Lua 接口的文件）则是 C 文件。当然，我们希望尽可能贴近原始版本，以便 Don Knuth 所著的 TEX 基础原理文档仍然适用。
- 有关段落构建、语言处理和断词方面的许多小变化，请参见 6。最重要的变化是：在单词中间插入花括号组（如 `of{}fice`）不再阻止连字的创建。
- 不再有池文件，所有字符串都在编译期间嵌入。
- 说明符 `plus 1 fillll` 不会产生错误。多余的 “l 只会被简单排版出来。
- `endlinechar` 和 `newlinechar` 的上限为 127。
- 缩放（`mag`）仅在 DVI 输出模式下受支持。你可以设置此参数，它甚至能与 `true` 单位一起工作，直到切换到 PDF 输出模式。使用 PDF 输出时，最好不要改动 `mag` 变量。这种模糊行为与最终需要 DVI 输出时却使用 PDF 后端相关功能的情况并无太大区别。
  输出模式被冻结后（通常在第一页 shipped out 时发生）或启用 PDF 输出后，`true` 说明将被忽略。当预加载适配 LuaTEX 的 plain 格式时，`mag` 参数可能已经被设置。
- 当 `\globaldefs` 为正值而请求局部赋值时，如果 `\tracingcommands` 大于 1，日志中会显示 `{\global enforced}`。当 `\globaldefs` 为负值而通过 `\global` 、`\gdef` 等请求全局赋值时，日志会记录 `{\global canceled}`。

4.1.3　与 e-TEX 2.2 相比的变化

作为事实上的标准扩展，我们当然提供 e-TEX 功能，但做了一些小的调整。

- e-TEX 功能始终存在并启用，因此不再需要为 initex 添加前置星号或 `-etex` 开关。
- 不包含 Xetex 扩展，因此原语 `\TeXXeTstate` 、`\beginR` 、`\beginL` 、`\endR` 和 `\endL` 均不存在。相反，我们以 Omega/Aleph 的方向性处理方法为起点。
- e-TEX 的 `tracingassigns` 和 `tracingrestores` 输出的部分跟踪信息不存在。
- LuaTEX 中的寄存器管理采用 Omega/Aleph 模型，因此最大值为 65535，实现使用扁平数组，而非 e-TEX 的扁平与稀疏混合模型。
- 当使用 kpathsea 查找文件时，LuaTEX 使用 `ofm` 文件格式搜索字体度量。这意味着 LuaTEX 查找 `OFMFONTS` 配置变量（如 Omega 和 Aleph），而非 `TFMFONTS`（如 TEX 和 pdfTEX）。虚拟字体同理（LuaTEX 使用变量 `OVFFONTS` 而非 `VFFONTS`）。
- 报告伸展或收缩次序的原语报告的值在 0 到 4 的便利范围内。由于某些宏包可能因此出错，我们还提供 `\eTeXgluestretchorder` 和 `\eTeXglueshrinkorder` ，它们报告与 e-TEX 兼容的值。（新的）`fi` 值报告为 `-1`（因此在 `ifcase` 测试中使用时，该值会使程序进入 `else` 分支）。

4.1.4　与 pdfTEX 1.40 相比的变化

因为我们要生成 PDF，最自然的起点就是广受欢迎的 pdfTEX 程序。我们继承了稳定特性，丢弃了大部分实验性代码，并将一些功能提升为 LuaTEX 核心功能，这进而触发了原语的重命名。

出于兼容性原因，我们仍提及 `\pdf ...` 命令，但 LuaTEX 有不同的后端接口。取代这些原语的是三个接口原语：`pdfextension`、`pdfvariable` 和 `pdffeedback`，它们接受关键字和可选的后续参数（下文我们仍使用 `pdf` 前缀名称作为参考）。这样我们可以在需要时扩展功能，而无需改动核心引擎。前端与后端尽可能解耦。

- （实验性的）snap 节点支持已被移除，因为在节点处理和属性之上构建此功能更为自然。随之消失的相关原语有：`pdfsnaprefpoint`、`pdfsnapy` 和 `pdfsnapycomp`。
- （实验性的）节点周围特殊间距支持也已被移除。随之消失的相关原语有：`pdfadjustinterwordglue`、`pdfprependkern` 和 `pdfappendkern`，以及五个辅助原语 `knbscode`、`stbscode`、`shbscode`、`knbccode` 和 `knaccode`。
- 许多 “pdfTEX 原语已被移除，因为它们可以使用 Lua 实现：`pdfelapsedtime`、`pdfescapehex`、`pdfescapename`、`pdfescapestring`、`pdffiledump`、`pdffilemoddate`、`pdffilesize`、`pdfforcepagebox`、`pdflastmatch`、`pdfmatch`、`pdfmdfivesum`、`pdfmovechars`、`pdfoptionalwaysusepdfpagebox`、`pdfoptionpdfinclusionerrorlevel`、`pdfresettimer`、`pdfshellescape`、`pdfstrcmp` 和 `pdfunescapehex`。
- 版本相关原语 `pdftexbanner`、`pdftexversion` 和 `pdftexrevision` 不再存在，因为与 pdfTEX 开发不再有任何关系。
- 实验性的 snapper 机制已被移除，因此原语 `pdfignoreddimen`、`pdffirstlineheight`、`pdfeachlineheight`、

pdfeachlinedepth 和 pdflastlinedepth 也一并消失。

- 实验性原语 primitive、ifprimitive、ifabsnum 和 ifabsdim 被提升为核心原语。带 \pdf* 前缀的原始版本不可用。
- 由于 LuaTEX 有不同的图像管理子系统，在此期间与祖先版本产生了更多偏离。我们不追随 pdfTEX 的变化。
- 提供了两个额外的令牌列表 pdfxformresources 和 pdfxformattr，作为 pdfxform 关键字的替代方案。
- 图像说明还支持 visiblefilename、userpassword 和 ownerpassword。密码选项仅与加密的 PDF 文件相关。
- 当前版本的 LuaTEX 不再将嵌入的 PDF 文件中的字体替换和/或合并到外层 PDF 文档的字体中。这种功能回退可能是暂时的，取决于重写后的字体后端的形态。
- 原语 pdfpagewidth 和 pdfpageheight 已被移除，因为 pagewidth 和 pageheight 承担了该用途。
- 原语 pdfnormaldeviate、pdfuniformdeviate、pdfsetrandomseed 和 pdfrandomseed 已被提升为不带 pdf 前缀的核心原语，因此原始命令不再被识别。
- 原语 ifincsname、 expanded 和 quitvmode 现在是核心原语。
- 由于 hz 与突出机制已成为核心的一部分,相关原语 lpcode、rpcode、efcode、leftmarginkern、rightmarginkern 被提升为核心原语。两个命令 protrudechars 和 adjustspacing 取代了带 \pdf 前缀的原始版本。
- hz 优化代码已部分重写，因此我们不再需要创建额外的字体实例。前端与后端已解耦，并生成更高效的（PDF）代码。
- 当 adjustspacing 的值为 2 时，hz 优化将应用于字形和字距。当值为 3 时，仅处理字形。小于 2 的值禁用此功能。值为 1 时，字体扩展在 TEX 常规段落断行例程将段落拆分为行之后应用。在这种情况下，断行与标准 TEX 行为一致（与 pdfTEX 相同）。
- tagcode 原语被提升为核心原语。
- letterspacefont 功能现在是核心的一部分，但不会被修改（改进）。我们仅为遗留用途提供它。
- pdfnoligatures 原语现在是 ignoreligaturesinfont。
- pdfcopyfont 原语现在是 copyfont。
- pdffontexpand 原语现在是 expandglyphsinfont。
- 由于位置跟踪在 DVI 模式下也可用，savepos、lastxpos 和 lastypos 命令现在取代了带 pdf 前缀的原始版本。
- 内省原语 \pdflastximagecolordepth 和 \pdfximagebbox 已被移除。可以使用外部应用程序确定这些属性，或使用内置的 img 库。
- 初始化器 pdfoutput 已被 outputmode 取代，pdfdraftmode 现在是 draftmode。
- 像素乘数尺寸 pdfpxdimen 去掉了前缀，现在称为 pxdimen。
- 增加了一个额外的 pdfimageaddfilename 选项，可用于阻止将文件名写入 PDF 文件。

- 原语 pdftracingfonts 现在是 tracingfonts，因为它与后端无关。
- 实验性原语 pdfinsertht 保留为 insertht。
- 对于哪些元数据写入 PDF 文件，有了更多控制。
- 原语提升为核心原语以及字体与后端的分离意味着初始化命名空间 pdftex 已不复存在。

一项变更涉及所谓的 xforms 和 ximages。在 pdfTEX 中它们以所谓的 whatsits 实现。但与其他 whatsit 不同，它们具有尺寸，在例如计算最优断行时需要考虑。在 LuaTEX 中，它们现在被提升为一种特殊类型的规则节点，这简化了需要这些尺寸的代码。

提升的另一个原因是这些概念很有用。后端可以提供在多处使用已渲染内容的能力，图像也很常见。如前文相关章节中已提及，我们现在有：

LuaTEX	pdfTEX
saveboxresource	pdfxform
saveimageresource	pdfximage
useboxresource	pdfrefxform
useimageresource	pdfrefximage
lastsavedboxresourceindex	pdflastxform
lastsavedimageresourceindex	pdflastximage
lastsavedimageresourcepages	pdflastximagepages

有一些 pdffeedback 功能与此相关，但这些是典型的后端特定功能。返回的索引应被视为“只是一个数字，虽然它仍具有与之前相同的含义（与对象相关），但你不应依赖这一点。

突出检测机制略有增强，以支持稍复杂的情况。当识别出突出字符时，会跳过一些节点：

- 零 glue
- 惩罚值
- 空 discretionary
- 常规零 kern
- 零尺寸规则
- 周围为零的数学节点
- dir 节点
- 空水平列表
- 局部 par 节点
- insert、mark 与 adjust
- 边界
- whatsit

由于这可能还不够，你还可以使用突出边界节点来使下一个节点被忽略。当值为 1 或 3 时，在定位左边界条件的测试中会忽略下一个节点。当值为 2 或 3 时，在定位右边界条件时会忽略前一个节点（搜索从右向左进行）。这允许突出与例如移入页边距的内容结合使用：

```
\protrusionboundary1!\quad«Who needs protrusion?»
```

4.1.5　与 Aleph RC4 相比的变化

因为我们需要正确的方向性排版，Aleph 的机制看起来最具吸引力。它们与 Omega 提供的机制相当接近，因此接下来所述适用于这两个程序。

- 扩展的 16 位数学原语（omathcode 等）已被移除。
- OCP 处理已被完全移除,因此以下原语也被移除:ocp、externalocp、ocplist、pushocplist、popocplist、clearocplists、addbeforeocplist、addafterocplist、removebeforeocplist、removeafterocplist 和 ocptracelevel。
- LuaTEX 仅理解 Aleph 的 16 种方向说明符中的 4 种：TLT（拉丁文）、TRT（阿拉伯文）、RTT（中文/日文/韩文）、LTL（蒙古文）。所有其他方向说明符都会产生错误。除了关键字驱动模型外，我们还提供整数驱动模型。
- Aleph 的输入翻译未实现，相关原语不可用：DefaultInputMode、noDefaultInputMode、noInputMode、InputMode、DefaultOutputMode、noDefaultOutputMode、noOutputMode、OutputMode、DefaultInputTranslation、noDefaultInputTranslation、noInputTranslation、InputTranslation、DefaultOutputTranslation、noDefaultOutputTranslation、noOutputTranslation 和 OutputTranslation。
- 修复了若干错误，理清了令人困惑的实现细节。
- 方向说明的扫描器现在允许在方向完全解析后有一个可选空格。
- \textasciicircum \textasciicircum {} 记法已扩展:\textasciicircum \textasciicircum {}\textasciicircum \textasciicircum {} 后期望四个十六进制字符,\textasciicircum \textasciicircum {}\textasciicircum \textasciicircum {}\textasciicircum \textasciicircum {} 后必须给出六个十六进制字符。原始 TEX 解释对 \textasciicircum \textasciicircum {} 情形仍然有效，但四个和六个变体不进行回溯，即当后面没有跟随正确数量的十六进制数字时会发出错误消息。因为 \textasciicircum \textasciicircum {}^ 是正常的 TEX 情形，我们也不支持奇数个的 \textasciicircum \textasciicircum {}\textasciicircum \textasciicircum {}^。
- 方向更改命令紧接其后的 glue 不是合法断点。
- 若干需要从右到左感知的机制已得到改进。例如公式编号的放置。
- 页面尺寸相关原语 pagewidth 和 pageheight 已被提升为核心原语。hoffset 和 voffset 原语已修复。
- 原语 \charwd 、\charht 、\chardp 和 \charit 已被移除，因为我们有 e-TEX 变体 \fontchar *。
- 两个尺寸寄存器 pagerightoffset 和 pagebottomoffset 现在是核心原语。
- 方向相关原语 pagedir、bodydir、pardir、textdir、mathdir 和 boxdir 现在是核心原语。
- 原语提升为核心原语以及移除所有其他原语意味着早期 LuaTEX 版本提供的初始化命名空间 aleph 已不复存在。

以上可以总结为：我们取其 32 位方面和大部分方向机制，将其合并到 pdfTEX 代码库中作为进一步开发的起点。然后我们简化了方向性、修复了它并将其开放。

4.1.6　来自各处的变化

`\partokenname` 和 `\partokencontext` 原语取自发布在实现者邮件列表上的 pdfTEX 变更文件。它们在 pdfTEX 手册中有说明，并归类为 e-TEX 扩展。

4.1.7　与标准 WEBC 相比的变化

编译框架是 WEBC，我们继续使用它，但去掉了 Pascal 到 C 的步骤。该框架还提供一些通用功能，处理从文件读取字节和在 TDS 中定位文件。以下是我们的不同之处：

- 不支持 mltex。
- 不支持 enctex。
- 以下编码相关的命令行开关被静默忽略，即使在非 Lua 模式下也是如此：`-8bit`、`-translate-file`、`-mltex`、`-enc` 和 `-etex`。
- `openout` whatsit 不写入日志文件。
- 部分所谓的 WEBC 扩展在非 kpathsea 模式下难以设置，因为不读取 `texmf.cnf`：`shell-escape` 处于关闭状态（但由于 Lua 的 `os.execute`，这不是问题），并且对 `openin` 和 `openout` 的偏执检查不会发生。然而，Lua 脚本可以通过重载 `io.open` 等来自行完成这些检查。
- “E 选项不做任何有用的事。

4.2　后端原语

4.2.1　更少的原语

在前一节中我们提到一些 pdfTEX 原语被移除，另一些被提升为 LuaTEX 核心原语。这只是故事的一部分。为了在代码中分离后端特定原语，这些命令现在仅由少数几个取代。在传统 TEX 中我们只有 DVI 后端，但现在我们有两个：DVI 和 PDF。额外功能以 TEX 术语中的 “扩展形式实现。通过更严格地分离，我们能够保持核心（前端）干净稳定，并隔离这些扩展。如果因某种原因需要额外的后端选项，可以在不触及核心的情况下实现。三个 PDF 后端相关原语是：

```
\pdfextension command [specification]
\pdfvariable  name
\pdffeedback  name
```

扩展会根据所给命令触发进一步解析。变量是一种寄存器，可以读写，而反馈则是报告某些信息（由于来自后端，它通常是一个令牌序列）。

4.2.2　`pdfextension`

为了让 LuaTEX 超越单纯的 TEX，你需要启用原语。从一开始就是如此。如果你想要传统的 pdfTEX 原语（只要其功能仍然存在），现在可以这样做：

```
\def\pdfliteral             {\pdfextension literal}
\def\pdflateliteral         {\pdfextension lateliteral}
\def\pdfcolorstack          {\pdfextension colorstack}
\def\pdfsetmatrix           {\pdfextension setmatrix}
\def\pdfsave                {\pdfextension save\relax}
\def\pdfrestore             {\pdfextension restore\relax}
\def\pdfobj                 {\pdfextension obj }
\def\pdfrefobj              {\pdfextension refobj }
\def\pdfannot               {\pdfextension annot }
\def\pdfstartlink           {\pdfextension startlink }
\def\pdfendlink             {\pdfextension endlink\relax}
\def\pdfoutline             {\pdfextension outline }
\def\pdfdest                {\pdfextension dest }
\def\pdfthread              {\pdfextension thread }
\def\pdfstartthread         {\pdfextension startthread }
\def\pdfendthread           {\pdfextension endthread\relax}
\def\pdfinfo                {\pdfextension info }
\def\pdfcatalog             {\pdfextension catalog }
\def\pdfnames               {\pdfextension names }
\def\pdfincludechars        {\pdfextension includechars }
\def\pdffontattr            {\pdfextension fontattr }
\def\pdfmapfile             {\pdfextension mapfile }
\def\pdfmapline             {\pdfextension mapline }
\def\pdftrailer             {\pdfextension trailer }
\def\pdfglyphtounicode      {\pdfextension glyphtounicode }
\def\pdfrunninglinkoff      {\pdfextension linkstate 1 }
\def\pdfrunninglinkon       {\pdfextension linkstate 0 }
```

内省原语可以定义为:

```
\def\pdftexversion    {\numexpr\pdffeedback version\relax}
\defpdftex           {\pdffeedback revision}
\def\pdflastlink      {\numexpr\pdffeedback lastlink\relax}
\def\pdfretval        {\numexpr\pdffeedback retval\relax}
\def\pdflastobj       {\numexpr\pdffeedback lastobj\relax}
\def\pdflastannot     {\numexpr\pdffeedback lastannot\relax}
\def\pdfxformname     {\numexpr\pdffeedback xformname\relax}
\def\pdfcreationdate          {\pdffeedback creationdate}
\def\pdffontname      {\numexpr\pdffeedback fontname\relax}
\def\pdffontobjnum    {\numexpr\pdffeedback fontobjnum\relax}
\def\pdffontsize      {\dimexpr\pdffeedback fontsize\relax}
\def\pdfpageref       {\numexpr\pdffeedback pageref\relax}
\def\pdfcolorstackinit        {\pdffeedback colorstackinit}
```

配置相关的寄存器变为:

```
\edef\pdfcompresslevel            {\pdfvariable compresslevel}
\edef\pdfobjcompresslevel         {\pdfvariable objcompresslevel}
\edef\pdfrecompress               {\pdfvariable recompress}
\edef\pdfdecimaldigits            {\pdfvariable decimaldigits}
\edef\pdfgamma                    {\pdfvariable gamma}
\edef\pdfimageresolution          {\pdfvariable imageresolution}
\edef\pdfimageapplygamma          {\pdfvariable imageapplygamma}
```

```
\edef\pdfimagegamma               {\pdfvariable imagegamma}
\edef\pdfimagehicolor             {\pdfvariable imagehicolor}
\edef\pdfimageaddfilename         {\pdfvariable imageaddfilename}
\edef\pdfpkresolution             {\pdfvariable pkresolution}
\edef\pdfpkfixeddpi               {\pdfvariable pkfixeddpi}
\edef\pdfinclusioncopyfonts       {\pdfvariable inclusioncopyfonts}
\edef\pdfinclusionerrorlevel      {\pdfvariable inclusionerrorlevel}
\edef\pdfignoreunknownimages      {\pdfvariable ignoreunknownimages}
\edef\pdfgentounicode             {\pdfvariable gentounicode}
\edef\pdfomitcidset               {\pdfvariable omitcidset}
\edef\pdfomitcharset              {\pdfvariable omitcharset}
\edef\pdfomitinfodict             {\pdfvariable omitinfodict}
\edef\pdfomitinfodict             {\pdfvariable omitinfodict}
\edef\pdfomitmediabox             {\pdfvariable omitmediabox}
\edef\pdfomitprocset              {\pdfvariable omitprocset}
\edef\pdfptexprefix               {\pdfvariable ptexprefix}
\edef\pdfpagebox                  {\pdfvariable pagebox}
\edef\pdfminorversion             {\pdfvariable minorversion}
\edef\pdfuniqueresname            {\pdfvariable uniqueresname}

\edef\pdfhorigin                  {\pdfvariable horigin}
\edef\pdfvorigin                  {\pdfvariable vorigin}
\edef\pdflinkmargin               {\pdfvariable linkmargin}
\edef\pdfdestmargin               {\pdfvariable destmargin}
\edef\pdfthreadmargin             {\pdfvariable threadmargin}
\edef\pdfxformmargin              {\pdfvariable xformmargin}

\edef\pdfpagesattr                {\pdfvariable pagesattr}
\edef\pdfpageattr                 {\pdfvariable pageattr}
\edef\pdfpageresources            {\pdfvariable pageresources}
\edef\pdfxformattr                {\pdfvariable xformattr}
\edef\pdfxformresources           {\pdfvariable xformresources}
\edef\pdfpkmode                   {\pdfvariable pkmode}

\edef\pdfsuppressoptionalinfo     {\pdfvariable suppressoptionalinfo }
\edef\pdftrailerid                {\pdfvariable trailerid }
```

这些变量是内部变量，因此是匿名的。当你询问先前定义的几个变量的含义时：

```
\meaning\pdfhorigin
\meaning\pdfcompresslevel
\meaning\pdfpageattr
```

你会得到：

```
macro:->[internal backend dimension]
macro:->[internal backend integer]
macro:->[internal backend tokenlist]
```

`edef` 也可以是 `def`，但预先展开查找相关寄存器效率更高。

后端派生自 pdfTEX，因此适用相同的语法。不过，`outline` 命令接受一个 `objnum` 后跟一个数字。不进行检查，因此使用时最好是一个有效的（已刷新的）对象。

为了（或多或少）与 pdfTEX 兼容，我们也支持抑制某些信息的选项，但通过位集实现：

```
\pdfvariable suppressoptionalinfo \numexpr
        0
    +   1   % PTEX.FullBanner
    +   2   % PTEX.FileName
    +   4   % PTEX.PageNumber
    +   8   % PTEX.InfoDict
    +  16   % Creator
    +  32   % CreationDate
    +  64   % ModDate
    + 128   % Producer
    + 256   % Trapped
    + 512   % ID
\relax
```

此外，你可以重载 trailer id，但我们不做任何有效性检查，因此你必须传入一个有效的数组。以下类似引擎通常生成的形式。这里你甚至需要包含括号!

```
\pdfvariable trailerid {[
    <FA052949448907805BA83C1E78896398>
    <FA052949448907805BA83C1E78896398>
]}
```

虽然我们从合并 pdfTEX 和 Aleph 起步，但到现在为止代码库和功能已偏离了这些来源。这里我们展示可以传递给扩展的选项。`shipout` 选项是一个兼容性特性。相反，可以使用 `deferred` 前缀。

```
\pdfextension literal
    [shipout] [ direct | page | raw ] { tokens }

\pdfextension dest
    num integer | name { tokens }!crlf
    [ fitbh | fitbv | fitb | fith| fitv | fit |
      fitr <rule spec> | xyz [ zoom <integer> ]

\pdfextension annot
    reserveobjnum | useobjnum <integer>
    { tokens }

\pdfextension save

\pdfextension restore

\pdfextension setmatrix
    { tokens }

[ \immediate ] \pdfextension obj
    reserveobjnum

[ \immediate ] \pdfextension obj
    [ useobjnum <integer> ]
    [ uncompressed ]
    [ stream  [ attr { tokens } ] ]
    [ file ]
    { tokens }
```

```
\pdfextension refobj
    <integer>

\pdfextension colorstack
    <integer>
    set { tokens } | push { tokens } | pop | current

\pdfextension startlink
    [ attr { tokens } ]
    user { tokens } | goto | thread
    [ file { tokens } ]
    [ page <integer> { tokens } | name { tokens } | num  integer ]
    [ newwindow  | nonewwindow ]

\pdfextension endlink

\pdfextension startthread
    num <integer> | name { tokens }

\pdfextension endthread

\pdfextension thread
    num <integer> | name { tokens }

\pdfextension outline
    [ attr { tokens } ]
    [ useobjnum <integer> ]
    [ count <integer> ]
    { tokens }

\pdfextension glyphtounicode
    { tokens }
    { tokens }

\pdfextension catalog
    { tokens }
    [ openaction
      user { tokens } | goto | thread
      [ file { tokens } ]
      [ page <integer> { tokens } | name { tokens } | num <integer> ]
      [ newwindow  | nonewwindow ] ]

\pdfextension fontattr
    <integer>
    {tokens}

\pdfextension mapfile
    {tokens}

\pdfextension mapline
    {tokens}

\pdfextension includechars
    {tokens}
```

```
\pdfextension info
    {tokens}

\pdfextension names
    {tokens}

\pdfextension trailer
    {tokens}
```

4.2.3　默认值

引擎设置以下默认值。

```
\pdfcompresslevel          9
\pdfobjcompresslevel       1 % used: (0,9)
\pdfrecompress             0 % mostly for debugging
\pdfdecimaldigits          4 % used: (3,6)
\pdfgamma               1000
\pdfimageresolution       71
\pdfimageapplygamma        0
\pdfimagegamma          2200
\pdfimagehicolor           1
\pdfimageaddfilename       1
\pdfpkresolution          72
\pdfpkfixeddpi             0
\pdfinclusioncopyfonts     0
\pdfinclusionerrorlevel    0
\pdfignoreunknownimages    0
\pdfgentounicode           0
\pdfomitcidset             0
\pdfomitcharset            0
\pdfomitinfodict           0
\pdfomitmediabox           0
\pdfomitprocset            0
\pdfptexprefix             0
\pdfpagebox                0
\pdfminorversion           4
\pdfuniqueresname          0

\pdfhorigin              1in
\pdfvorigin              1in
\pdflinkmargin           0pt
\pdfdestmargin           0pt
\pdfthreadmargin         0pt
\pdfxformmargin          0pt
```

4.2.4　向后兼容

如果你还想要一些向后兼容，可以添加：

```
\let\pdfpagewidth      \pagewidth
\let\pdfpageheight     \pageheight
```

```
\let\pdfadjustspacing  \adjustspacing
\let\pdfprotrudechars  \protrudechars
\let\pdfnoligatures    \ignoreligaturesinfont
\let\pdffontexpand     \expandglyphsinfont
\let\pdfcopyfont       \copyfont

\let\pdfxform          \saveboxresource
\let\pdflastxform      \lastsavedboxresourceindex
\let\pdfrefxform       \useboxresource

\let\pdfximage         \saveimageresource
\let\pdflastximage     \lastsavedimageresourceindex
\let\pdflastximagepages\lastsavedimageresourcepages
\let\pdfrefximage      \useimageresource

\let\pdfsavepos        \savepos
\let\pdflastxpos       \lastxpos
\let\pdflastypos       \lastypos

\let\pdfoutput         \outputmode
\let\pdfdraftmode      \draftmode

\let\pdfpxdimen        \pxdimen

\let\pdfinsertht       \insertht

\let\pdfnormaldeviate  \normaldeviate
\let\pdfuniformdeviate \uniformdeviate
\let\pdfsetrandomseed  \setrandomseed
\let\pdfrandomseed     \randomseed

\let\pdfprimitive      \primitive
\let\ifpdfprimitive    \ifprimitive

\let\ifpdfabsnum       \ifabsnum
\let\ifpdfabsdim       \ifabsdim
```

甚至:

```
\newdimen\pdfeachlineheight
\newdimen\pdfeachlinedepth
\newdimen\pdflastlinedepth
\newdimen\pdffirstlineheight
\newdimen\pdfignoreddimen
```

4.3　方向

4.3.1　四种方向

LuaTEX 中的方向模型继承自 Omega/Aleph，但我们尝试做了一些改进。曾有一段时间我们尝试恢复模式，但后来发现它与嵌套方向冲突而禁用了它。这本身的副作用是节点列表在方向节点方面不再平衡，当一系列 dir 更改在没有分组的情况下发生时，这可能产生副作用。

在扩展 PDF 后端以支持方向时发现了一些不一致之处，因此我们决定仅支持四种有意义的模型：TLT（拉丁文）、TRT（阿拉伯文）、RTT（中文/日文/韩文）和 LTL（蒙古文）。

4.3.2　工作原理

我们的方法是再次使列表平衡，但尽量避免一些副作用。如果忽略空格（转换为 glue），发生的情况相当直观，但即使在那里，仔细看也能理解。然而，当没有使用适当的嵌套分组时，这种逻辑使组内切换几乎毫无用处：从右到左多次嵌套切换会导致间距因嵌套镜像而一个接一个地排列。当然，一个合理的宏包会为用户管理这些，但我们这里讨论的是底层 dir 注入。

情况如下：

```
textdir TRT nur {textdir TLT run textdir TRT NUR} nur
```

逐步变为：

```
injected: [+TRT]nur {[+TLT]run [+TRT]NUR} nur
balanced: [+TRT]nur {[+TLT]run [-TLT][+TRT]NUR[-TRT]} nur[-TRT]
result  : run {RUNrun } run
```

而这个：

```
textdir TRT nur {nur textdir TLT run textdir TRT NUR} nur
```

变为：

```
injected: [+TRT]nur {nur [+TLT]run [+TRT]NUR} nur
balanced: [+TRT]nur {nur [+TLT]run [-TLT][+TRT]NUR[-TRT]} nur[-TRT]
result  : run {run RUNrun } run
```

现在，在以下示例中注意我们花括号的位置：

```
textdir TRT nur {{textdir TLT run} {textdir TRT NUR}} nur
```

这变为：

```
[def]
\def\ltr{\linedir TLT\relax}
\def\rtl{\linedir TRT\relax}
```

而这一次：

```
run {\rtl nur {\ltr run \rtl NUR \ltr run \rtl NUR} nur}
run {\ltr run {\rtl nur \ltr RUN \rtl nur \ltr RUN} run}
```

输出为间距正确的：

任何比这更复杂的情况，如 skip 和 penalty 的组合或 kern，都应在输入或宏包中处理，因为我们无法预测预期行为。事实上，`linedir` 只是一个便利的附加功能，也可以使用节点列表解析来实现。

方向的复杂性在于它们通常需要跨组工作，因此使用了单独的分组相关堆栈。副作用是可能存在仅包含一个局部 par 节点、后跟方向同步节点的段落。这样的段落被视为空段落，因此被忽

略。因为 `\noindent` 不注入任何内容，而 `\indent` 会注入一个 box，所以仅含缩进和方向的段落被视为有内容的段落处理。

默认情况下，包含 dir 节点的显示公式前的段落从不被忽略。更改此行为可能会破坏现有文档，但当你将 `mathemptydisplaymode` 设为 `1` 时，显示公式前的空段落将被忽略。

4.3.3　用 `breakafterdirmode`

dir 节点后的 glue 在断行决策中被忽略，但你可以通过将 `breakafterdirmode` 设为 `1` 来绕过。下表对比了不同设置下文本方向切换处的断行行为：

breakafterdirmode=0	**breakafterdirmode=1**
pre {textdir TLT xxx} post	pre {textdir TLT xxx} post
pre {textdir TLT xxx }post	pre {textdir TLT xxx }post
pre{ textdir TLT xxx} post	pre{ textdir TLT xxx} post
pre{ textdir TLT xxx }post	pre{ textdir TLT xxx }post
pre { textdir TLT xxx } post	pre { textdir TLT xxx } post

4.3.4　用 `shapemode`

对 Aleph 方向模型的另一项调整是对由 `hangindent` 和 `parshape` 驱动的形状进行控制。这由一个新参数 `shapemode` 控制：

值	`hangindent`	`parshape`
0	正常	正常
1	镜像	正常
2	正常	镜像
3	镜像	镜像

该值在段落结束后重置为零（如 `hangindent` 和 `parshape`）。你可以使用负值来防止这种情况。4.1 中给出了几个示例。

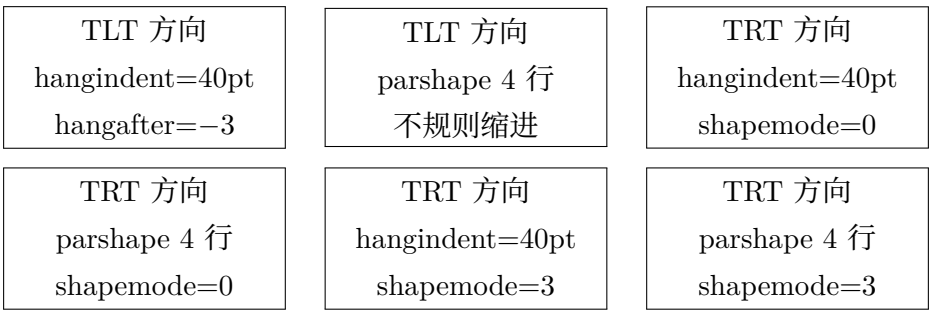


图 4.1: `shapemode` 的效果。

4.3.5　符号或数字

内部实现与 Aleph 不同。首先，我们不使用 whatsit 而是专用节点，而且我们只有 4 个方向映射到 4 个数字。文本方向节点可以标记节点序列的开始或结束，因此有两种状态。在 TEX 端我们看不到这些状态，因为 TEX 本身会在需要时添加适当的结束状态节点。

符号名 TLT、TRT 等源自 Omega。在 LuaTEX 中我们还有基于数字的模型，有时更有意义。

值	等价
0	TLT
1	TRT
2	LTL
3	RTT

我们支持 Omega 原语 textdir、pardir、pagedir、pardir 和 mathdir。它们接受三字符关键字。通过数字设置方向的原语有：textdirection、pardirection、pagedirection、bodydirection 和 mathdirection。为 box 指定方向时，可以使用 bdir 代替 dir。

4.4　实现说明

4.4.1　内存分配

传统 TEX 用于令牌和节点的单一内部内存堆被拆分为两个独立数组。每个数组在需要时动态增长。

与主内存相关的 texmf.cnf 设置不再使用（这些是：main_memory、mem_bot、extra_mem_top 和 extra_mem_bot）。“主内存不足错误仍可能发生，但限制因素现在是系统中的 RAM 容量，而非预定义限制。

此外，节点的内存（解）分配例程已完全重写。相关代码现在位于 C 文件 texnode.c 中，基本上使用十几个 “avail 列表而非双链接模型。增加了一个额外的函数层，使代码可以按类型请求节点，而不是直接申请一定数量的内存字。

由于拆分为两个数组以及由此产生的数据结构差异，部分宏已被复制。例如，现在有 vlink 和 vinfo，以及 token_link 和 token_info。对变量内存数组的所有访问现在都隐藏在一个名为 vmem 的宏之后。我们提到这一点是因为使用 TEXbook 作为参考仍然相当有效，但不适用于内存相关细节。另一个重要细节是我们有双链接节点列表，且大多数节点携带更多数据。

输入行缓冲区和池大小现在也在需要时重新分配，texmf.cnf 设置 buf_size 和 pool_size 被静默忽略。

4.4.2　稀疏数组

mathcode、delcode、catcode、sfcode、lccode 和 uccode（以及新的 hjcode）表现在是用 C 实现的稀疏数组。它们不再是 TEX “等价表的一部分，由于每个表有 110 万条目、每条目占几个内存字，这在内存使用上产生了重大差异。性能并未因此受到实质影响。

catcode、sfcode、lccode、uccode 和 hjcode 赋值在使用 e-TEX 跟踪例程 tracingassigns 和 tracingrestores 时不显示，但我们不认为这是真正的限制。

当前实现的一个副作用是 global 现在在处理上比非全局赋值更昂贵，但不会有多少用户注意到。

字体内的 glyph id 也通过稀疏数组管理，因为 glyph id 可高达索引 $2^{21} - 1$，但从不直接访问，因此用户同样不会注意到。

4.4.3　简单的单字符 csname

单字符命令在内部不再被特殊对待，它们像多字符 csname 一样存储在哈希中。

显示控制序列的代码在决定是否添加尾随空格时，会显式检查长度是否为 1。

活动字符在内部实现为一种特殊类型的多字符控制序列，使用一个在其他情况下不可能获得的前缀。

4.4.4　压缩的格式文件

格式通过 zlib 处理，使其可缩小到未压缩时约一半的大小。这会消耗稍多的 CPU 周期，但磁盘 I/O 少得多，因此总体应该更快。我们使用 3 级压缩，我们发现这是文件大小与解压速度之间的最佳权衡。

4.4.5　二进制文件读取

所有内部代码都做了修改：如果 read_xxx_file 回调之一未设置，则文件由 C 函数读取，基本使用与回调相同的约定：一次性读入足够容纳整个文件内容的缓冲区。虽然这比以前的代码（主要使用 getc 调用）使用更多内存，但可能快不少（取决于你的 I/O 子系统）。

4.4.6　制表符与空格

我们遵循其他 TEX 引擎处理尾随制表符和空格的方式。几十年来，尾随制表符和空格（换行符之前）会从输入中移除，但此行为在 2017 年 9 月被修改为仅处理空格。我们知道这可能在现有工作流中引入兼容性问题，但因为我们不想与上游 TeX Live 有太多差异，所以直接跟进了该补丁（这是功能性变更而非真正的修复）。处理可能的兼容性问题取决于宏包维护者，在 LuaTEX 中他们可以通过处理文件读取的回调来做到。

之前的行为是已知的副作用，而且（由于这类输入通常来自生成的源），如果空格和/或制表符是有意保留的，通常通过在该行添加注释令牌来处理。我们知道这与我们的其他一些选择相矛盾，但与其他引擎保持一致以及在 kpathsea 模式下使用公共文件 I/O 层这一事实，可能会产生破坏兼容性的副作用。我们仍然坚持在日志级别上我们可以（也可能）更不兼容的观点。我们已经暴露了更多细节。

4.4.7　超链接

有一个实验性功能使多行超链接表现稍好，修复了在从右到左排版中出现的一些副作用，但这些副作用也可能出现在从左到右排版中。由于此问题直到 2023 年才被注意到，且它在一定程度上取决于宏包如何处理超链接，因此该修复目前由参数控制：

```
\pdfvariable linking = 1
```

这样（我们希望）遗留文档能按预期输出，无论预期是什么。处理的方面之一涉及（不寻常的）左右 skip。

第 5 章 使用 LuaTEX

5.1 初始化

5.1.1 作为 Lua 解释器的 LuaTEX

在某些情形下，LuaTEX 的行为就像一个独立的 Lua 解释器：

- 命令行上给出了 `--luaonly` 选项，或者
- 可执行文件被命名为 `texlua` 或 `luatexlua`，或者
- 命令行上唯一一个非 |-| 选项参数（文件）的扩展名为 `lua` 或 `luc`。

在这种模式下，它会像 Lua 解释器一样，把 Lua 的 `arg[0]` 设为找到的脚本名，并把前面的选项依次压入负值、把命令行其余部分放入正值。

LuaTEX 在执行完指定的 Lua 脚本后立即退出；实际上，它就是一个体积稍大、附带一批额外预加载库的独立 Lua 解释器。

5.1.2 作为 Lua 字节编译器的 LuaTEX

有两种情形会使 LuaTEX 的行为类似于 Lua 字节编译器：

- 命令行上给出了 `--luaconly` 选项，或者
- 可执行文件被命名为 `texluac`

在这种模式下，LuaTEX 的行为与独立 Lua 发行版中的 `luac` 完全相同，区别仅在于它没有 `-l` 开关，并且会接受（但忽略）`--luaconly` 开关。当前版本的 Lua 可以用 `string.dump` 转储字节码，因此我们将来可能会放弃这个版本的 LuaTEX。

5.1.3 其他命令行处理

LuaTEX 可执行文件启动时，会先查找 `--lua` 命令行选项。如果没有该选项，则命令行的解释方式与其他 TEX 引擎大致相同。有些选项会被接受但不起作用。可识别的命令 |-| 行选项如下：

命令行参数	说明
`--credits`	显示版权信息后退出
`--debug-format`	启用格式文件调试
`--draftmode`	开启草稿模式，即在 PDF 模式下不生成输出文件

--[no-]check-dvi-total-pages	当 DVI 页数超过 65535 时退出（默认：检查）
--[no-]file-line-error	禁用/启用 file:line:error 风格的消息
--[no-]file-line-error-style	--[no-]file-line-error 的别名
--fmt=FORMAT	加载格式文件 FORMAT
--halt-on-error	遇到第一个错误即停止处理
--help	显示帮助信息后退出
--ini	作为 iniluatex 运行, 用于转储格式文件
--interaction=STRING	设置交互模式：batchmode、nonstopmode、scrollmode 或 errorstopmode
--jobname=STRING	将作业名设置为 STRING
--kpathsea-debug=NUMBER	按 NUMBER 的各位设置路径搜索调试标志
--lua=FILE	加载并执行 Lua 初始化脚本
--luadebug	启用 debug 库
--[no-]mktex=FMT	当 FMT 为 tex、tfm 或 fmt 时, 禁用/启用 mktexFMT 的生成
--nosocket	禁用 Lua socket 库
--no-socket	禁用 Lua socket 库
--socket	启用 Lua socket 库
--output-comment=STRING	用 STRING 作为 DVI 文件注释以代替日期（对 PDF 无效）
--output-directory=DIR	将 DIR 用作输出文件目录
--output-format=FORMAT	用 FORMAT 作为作业输出格式; FORMAT 可取 dvi 或 pdf
--progname=STRING	将程序名设置为 STRING
--recorder	启用文件名记录器
--safer	禁用易被滥用的 Lua 命令
--[no-]shell-escape	禁用/启用系统调用
--shell-restricted	将系统调用限制为 texmf.cnf 中给出的命令列表
--synctex=NUMBER	启用 synctex
--utc	适当时使用 UTC 时间
--version	显示版本号后退出

我们不支持 write 18，因为 os.execute 可以完成同样的工作。这样既简化了代码，又允

许使用更多的写入目标。

jobname 的取值按如下方式确定：

- 如果命令行上给出了 --jobname，则其参数将直接作为 jobname 的值，不作任何改动。该参数不会被用于实际读入，因此它所指的文件无需存在。--jobname 开关只控制 jobname 的设置。
- 否则，jobname 将是第一个从文件系统读入的文件名，并去掉其中的路径分量和最后一个扩展名（即最后一个．之后的部分）。
- 上一条有一个例外：如果命令行进入交互模式（以一条命令开头），并且也没有通过 everyjob 读入任何文件，那么最后会把 jobname 设为 texput 作为兜底。

自动生成的输出文件名，是在求得的 jobname 后接上相应的扩展名（log、pdf 等）得到的。这些文件创建在 --output-directory 所指向的目录中；若没有该开关，则创建在当前目录下。如果 --output-directory 非空，其值会被复制到环境变量 TEXMF_OUTPUT_DIRECTORY 中；若该开关为空，则输出目录取 TEXMF_OUTPUT_DIRECTORY 环境变量的值。

在没有 --lua 选项时，命令行处理方式与其他任何基于 WEBC 的排版引擎相同，只是 LuaTEX 多了几个开关、又少了另外几个。而如果存在 --lua 选项，LuaTEX 会进入一种与标准 WEBC 程序不同的命令行处理模式。在这种模式下，会按以下顺序执行一小系列动作：

- 首先，它会像往常一样解析命令行，但只立即解释其中一小部分选项：--safer、--nosocket、--no-socket、--socket、--[no-]shell-escape、--enable-write18、--disable-write18、--shell-restricted、--help、--version 和 --credits。
- 接下来，LuaTEX 查找所请求的 Lua 初始化脚本。如果用命令行上给出的实际文件名找不到，就会进行第二次尝试：在文件名前拼上环境变量 LUATEXDIR 的值（前提是该环境变量已在环境中定义）。
- 然后检查各种安全开关。你可以用它们禁用某些容易被恶意文档滥用的 Lua 命令。目前 --safer 会把以下函数置为 nil：

库	函数
os	execute exec kpsepopen spawn setenv rename remove tmpdir
io	popen output tmpfile
lfs	rmdir mkdir mkdirp chdir lock touch

此外，它还会禁用已编译 Lua 库的加载，并使 io.open() 在以只读以外的方式打开文件时失败。

最后，它会无条件禁用 socket 库（但不禁用 mime 库，后者始终可用）。

从 1.18.0 版起，若使用了 kpathsea，则除 debug.traceback 外，debug 库默认不启用；可以用 --luadebug 开关启用它。在 shell-escape 模式下，debug 库总是启用的。

同样从 1.18.0 版起，若使用了 kpathsea，则当 kpse.in_name_ok_silent_extended 和 kpse.out_name_ok_silent_extended 都校验通过某个路径名时，函数 os.rename、os.remove、lfs.attributes、lfs.chdir、lfs.lock_dir、lfs.dir、lfs.link、lfs.mkdir、lfs.mkdirp、

lfs.rmdir、lfs.symlinkattributes、lfs.touch 返回 true；其中 lfs.attributes、lfs.dir 和 lfs.symlinkattributes 只通过 kpse.in_name_ok_silent_extended 校验。

- LuaTEX 启动时会把 locale 设为一个中性值。如果你因某种原因使用了 os.setlocale，之后务必再把它 nil 掉，否则它可能干扰诸如生成日期之类的代码。你可以这样忽略 locale：

```
os.setlocale(nil,nil)
```

  --nosocket 或 --no-socket 选项会使 socket 库不可用，从而 Lua 无法使用网络；--socket 选项则使 socket 库可用。

  开关 --[no-]shell-escape、--[enable|disable]-write18 和 --shell-restricted 的作用与 pdfTEX 中相同，此外还会使 io.popen()、os.execute、os.exec、os.kpsepopen 和 os.spawn 遵守所请求的选项。

  默认情况下 socket 库不启用：可以用 --socket 或 --shell-escape （但不带 --shell-restricted）启用它，用 --nosocket （或 --no-socket）禁用它，或者用 --safer 无条件禁用它。

  当选项相互冲突时，限制最严格者生效。

  mime 库始终可用。

- 接下来加载并执行初始化脚本。在脚本内部，整个命令行都可通过 Lua 表 arg 访问，从 arg[0] 开始，其中包含可执行文件名。由此带来的结果是，关于无法识别选项的警告会被抑制。

命令行处理发生得非常早。事实上早到 TEX 的任何初始化都还没有进行。因此，与排版相关的表（如 tex、token、node 和 pdf）在启动文件执行期间是禁止使用的（它们被置为 nil）。系统会特别保证 texio.write 和 texio.write_nl 正常工作，这样至少可以在日志文件最终被打开时（并且如果被打开的话）向它报告你的动作（注意此时 TEX 甚至还不知道自己的 jobname）。

你在 Lua 初始化脚本中做的一切，在后续运行期间都保持可见，但 TEX 专用的库（如 tex、token、node 和 pdf 这几个表）除外。脚本执行完毕后，这些表会被初始化到文档中所描述的状态。你不应该用这四个全局名字在变量或表中存放任何东西，因为它们会被完全覆盖。

我们建议你只把启动文件用于与 TEX 无关的自身初始化（如果你确实需要的话），用于解析命令行、在 texconfig 表中设置值，以及注册你所需的回调。

LuaTEX 允许在脚本执行结束时，从 texconfig 表读取值来覆盖部分命令行选项（究竟哪些选项可以被覆盖，见本文稍后对 texconfig 表的说明）。

除非 texconfig 表告诉 LuaTEX 完全不要初始化 kpathsea（为此把 texconfig.kpse_init 设为 false），否则在初始化脚本结束后，LuaTEX 还会继续处理另外一些命令行选项：为了正确初始化内建的 kpathsea 库，LuaTEX 需要知道应当使用哪个正确的程序名；为此它会检查 --progname，若缺少 --progname，则检查 --ini 和 --fmt。

从 1.17.1 版起，在 DVI 模式下，默认启用的新命令行开关 --check-dvi-total-pages 会检查总页数是否超过 65535，若超过则中止本次运行。这破坏了与 pdfTEX 的兼容性：在 pdfTEX 中，正如在 TEX 中一样，当总页数大于 65535 时文件会照常写出。可以用 --[no-]check-dvi-total-pages 恢复以前的行为。

5.2　Lua 行为

5.2.1　Lua 版本

我们目前使用 Lua 5.3，并会跟进该语言的发展，但通常会滞后一段时间。因此用户需要留意该语言相继版本之间（细微）的差异。此外，LuaJITTeX 也存在滞后，因为 LuaJIT 与常规 Lua 的开发并不同步。下面是一个这方面的例子。

Lua 的 `tostring` 函数（以及 `string.format`）可能以科学计数法返回值，当它被用作对 `dimen` 或 `count` 赋值的右操作数时，会令 TEX 这一端感到困惑。这些序列化函数的输出还依赖于 Lua 版本，因此在 Lua 5.3 中得到的输出可能与 5.2 不同。

5.2.2　融入 TDS 生态系统

主流 TEX 发行版都遵循 TEX 目录结构（TDS）。LuaTEX 能够借助 kpathsea 库查找被 `require()` 的模块。为此，`package.searchers[2]` 被替换为另一个加载器函数，它在运行时决定是使用 kpathsea 还是内建的 Lua 核心函数。如果届时 kpathsea 已经初始化，它就使用 kpathsea；否则回退为使用常规的 `package.path` 加载器。

kpathsea 的初始化可以隐式发生（当 LuaTEX 启动且启动脚本未把 `texconfig.kpse_init` 设为 false 时），也可以显式调用 Lua 函数 `kpse.set_program_name()` 来发生。

5.2.3　加载库

LuaTEX 能够使用动态可加载的 Lua 库，除非在命令行上给出了 `--safer` 选项。为此，`package.searchers[3]` 被替换为另一个加载器函数，它在运行时决定是使用 kpathsea 还是内建的 Lua 核心函数。如果届时 kpathsea 已经初始化，它就使用 kpathsea；否则回退为使用常规的 `package.cpath` 加载器。

这项功能需要对 kpathsea 进行扩展。新增了一种 kpathsea 文件格式：`kpse_clua_format`，它搜索扩展名为 `.dll` 和 `.so` 的文件。该变量在 `texmf.cnf` 中的设置名为 `CLUAINPUTS`，默认值为：

```
CLUAINPUTS=.:$SELFAUTOLOC/lib/{$progname,$engine,}/lua//
```

这条路径并不完美（它要求在二进制目录之下有一个 TDS 子树），但体系结构信息必须出现在路径中的某处，而目前最简单的做法就是只在二进制目录之下搜索。当然，编写一个替代加载器并在宏包中使用它也并非难事。往上一级（与 `bin` 平行的 `lib` 目录）本会更理想，但由于 TeX Live 采用 `bin/<arch>` 结构，那样做行不通。

如果同时加载了两个 Lua 库，动态 Lua 库的加载会失败（在 `win32` 上通常会发生这种情况：LuaTEX 内部自带一个 Lua 5.3，而模块自身的 dll 文件又可能链接了另一个）。

5.2.4　执行外部程序

与 TeX Live 中其他 TEX 类程序保持一致，Lua 函数 `os.execute`、`os.kpsepopen` 和 `io.popen`，以及下面将要说明的两个新函数 `os.exec` 和 `os.spawn`，都会考虑 `shell_escape` 和/或 `shell_escape_commands` 的取值。每当 LuaTEX 是以排版文档为预期意图运行时（我们

指的是它以 luatex 名义被调用，而不是 texlua，并且命令行上没有给出 --luaonly 选项），只有当对应的 texmf.cnf 变量或其 texconfig 对应项（见 11.4）允许执行所请求的系统命令时，它才会运行上述四个函数之一。在 LuaTEX 的“脚本解释器”运行方式下，这些设置不起作用，四个函数都保持其原始含义。

有些库还额外提供了若干函数，或是用 C 编写，或是用 Lua 编写。例如，在 LuaTEX 起步时，我们向 luafilesystem 命名空间 lfs 添加了一些辅助函数。两个布尔函数 lfs.isdir 和 lfs.isfile 是用 lfs.attributes 所能做到之事的更快速、更好的变体。额外函数 lfs.shortname 接受一个文件名，在 win32 平台上返回其短文件名；lfs.mkdirp 类似于 lfs.mkdir，但会按需创建父目录。最后，我们仅针对非 |-|win32 平台提供了 lfs.readlink，它接受一个已存在的符号链接作为参数并返回其名字。不过，lfs 库后来自行发展，我们已经放弃了这些函数，转而使用纯 Lua 变体。shortname 辅助函数已过时，现在只是原样返回名字。

5.2.5　多字节 string

string 库还额外提供了几个函数,例如。该函数最多接受两个参数:string.explode(s[,m])，返回一个数组，其中包含按字符串参数 m 的值切分字符串参数 s 后得到的子串。第二个参数是一个字符串，它可以为空串（此时把字符串按字符切分）、单个字符（此时在该字符每次出现处切分，可能产生空字符串），或者单个字符后跟加号 +（这种特殊形式不产生空子串）。m 的默认值是“ +（多个空格）。注意：m 不像用 TEX 宏编写的函数那样被包裹在花括号里隐藏起来。

string 库还额外提供了六个迭代器，它们逐个返回字符串片段：、、、、和。

- string.utfvalues(s)：返回 Unicode 范围内的一个整数值
- string.utfcharacters(s)：返回一个字符串，其中含单个 UTF-8 码元
- string.characters(s)：返回一个含单个字节的字符串
- string.characterpairs(s)：返回两个字符串，各含一个字节；若原字符串长度为奇数，则第二个字符串为空
- string.bytes(s)：返回单个字节值
- string.bytepairs(s)：返回两个字节值；若原字符串长度为奇数，则第二个返回值为 nil 而不是数字

string.characterpairs() 和 string.bytepairs() 迭代器在把 UTF16 编码数据转换为 UTF8 时尤其有用。

string.dump() 还有一种带两个参数的形式。第二个参数是布尔值，若为真，则从转储数据中剥离符号。这与 luajit 中所做的扩展一致。这类函数通常会随 Lua 自身的演进而调整。

string 库的 len、lower、sub 等函数并不具备 Unicode 感知能力。对于 UTF8 编码的字符串（即包含码点大于 127 字符的字符串），可以使用 slnunicode 库中对应的函数，例如 unicode.utf8.len、unicode.utf8.lower 等。例外是 unicode.utf8.find，它总是返回字符串中的字节位置；以及 unicode.utf8.match 和 unicode.utf8.gmatch。后两个函数一般而言是具备 Unicode 感知能力的，但当使用空捕获 () 时会退回到不感知 Unicode 的行为；而其他捕获仍按预期工作。关于 unicode.utf8 函数中字符类的解释，请参阅库源码，地址为 http://luaforge.net/projects/sln。

Lua 5.3 提供了一些原生的 UTF8 支持，但我们也额外添加了几个类似的辅助函数：、和。

- `string.utfvalue(s)`：返回给定字符串中各字符的码点
- `string.utfcharacter(c,...)`：返回一个字符串，其中包含给定码点所对应的字符
- `string.utflength(s)`：返回给定字符串的长度

这三个函数相对快速，且不做太多检查。它们可以作为其他辅助函数的构建块。

5.2.6 os

os 库还额外提供了几个函数和变量：、、、、、、、、、、、、、和，我们在此逐一讨论。

- os.selfdir 是一个变量,保存实际可执行文件所在的目录路径。例如:`\directlua {tex.sprint(os.selfdi`
- `os.exec(commandline)` 是 os.execute 的一个变体。这里的 commandline 既可以是单个字符串，也可以是单个表。
  - 如果参数是表，LuaTEX 首先检查整数索引 0 处是否有值。若有，该值就是要执行的命令；否则使用整数索引 1 处的值。若两者都不存在，则什么也不做。
  - 表中从整数索引 1 开始的连续一组值，是传给命令的参数（索引 1 处的值成为 arg[0]）。命令会在执行路径中查找，因此通常无需传入完整的限定路径名。
  - 如果参数是字符串，则会按空白字符切分，自动转换成一个表。在这种情况下，命令与第一个参数不可能彼此不同。
  - 在字符串参数形式中，可以用单引号或双引号把（部分）参数括起来以保护其中的空白字符。LuaTEX 会解释一层引号，并且把引号文本内出现的 \"、\' 或 \\ 全部反转义。在表形式下，不进行任何字符串处理。

  该函数通常不会把控制权返回给 Lua 脚本：命令会替换当前进程。但如果尝试执行命令时出现问题，它会返回两个值 nil 和 error。
  在 Microsoft Windows 上，当前进程实际上会一直保留在内存中，直到命令执行完毕。这避免了在集成 TEX 环境中运行 texlua 脚本时发生崩溃。
  引入这条命令的初衷,是让当前进程在启动新进程之前先清理干净,因此它特别适合在 texlua 中使用。
- `os.kpsepopen(commandline,[opt])` 类似于 io.popen，但会先对命令行进行一次检查；若检查通过，则返回值与 io.popen 相同；否则返回两个值 nil 和 error。
- os.socketgettime 和 os.socketsleep 分别与 socket.gettime 和 socket.sleep 相同，但它们始终可用。
- `os.spawn(commandline)` 是 os.exec 的带返回值版本，其余调用约定完全相同。
  如果命令正常运行，则返回值为命令的退出状态。否则返回两个值 nil 和 error。
  os.spawn() 还接受一个可选的第二个参数：一个键值对表，用于指定被派生进程的环境。
  注意：如果你确实传入了这个表，它将被用作被派生进程的 N.B. 完整环境，因此你很可能希望从 os.env 出发，按需增删键。
- `os.setenv(key,value)` 在环境中设置一个变量。传入 nil 代替值字符串会删除该变量。

- os.env 是一个哈希表，其中保存着本次运行开始时进程环境中各变量及其值的快照。它是可写的，但实际环境 N.B. 不会自动随之更新。
- os.gettimeofday() 返回当前 “Unix 时间，但以浮点数形式返回。该函数在 SunOS 平台上不可用，因此不要为可移植文档使用此函数。
- os.times() 按照 Unix C 库函数 “times 返回当前进程的用时。该函数在 Microsoft Windows 和 SunOS 平台上不可用，因此不要为可移植文档使用此函数。
- os.sleep(interval[, unit]) 让当前运行暂停给定的秒数。若给出了可选参数 unit，则函数等待 interval / unit 秒。例如 os.sleep(1, 1000) 让程序暂停一毫秒。
- os.tmpdir([template]) 在 “当前目录下创建一个名为 luatex.XXXXXX 的目录，其中的 X 会被替换为一个唯一字符串。该函数同时返回这个字符串，因此你可以 lfs.chdir() 进入它；若创建目录失败则返回 nil。用户有责任在运行结束时自行清理，系统不会自动清理。你也可以为临时文件夹名字使用自己的 template。但传入的字符串必须以六个大写 X 结尾。例如模板 tmp.XXXXXX 可能得到文件夹名 tmp.vX3gPo。
- os.type 是一个字符串，给出操作系统大类的总体指示。目前可能的取值为 windows、unix 和 msdos （你不太可能“在野外”遇到这个值）。
- os.name 是一个字符串，给出更精确的操作系统指示。这些可能取值尚未完全固定；对于 os.type 的 windows 和 msdos 取值，os.name 的值就分别是 windows 和 msdos。
  对于类型 unix，列表更精确：linux、freebsd、kfreebsd、cygwin、openbsd、solaris、sunos（solaris 之前的版本）、hpux、irix、macosx、gnu（hurd）、bsd（未知，但类似 BSD）、sysv（未知，但类似 sysv）、generic（未知）。
- os.uname 返回一个表，其中包含运行时获取的特定操作系统信息。返回表中的键都是字符串值，名字为：sysname、machine、release、version 和 nodename。

5.2.7　用 fio

有一整套辅助函数用于从文件读取数字和字符串：、、、、、、、、、、、、、、、、、。它们作用于普通的 Lua 文件句柄。

除了常规 io 库函数之外，本库还提供了一组从文件读取数字的函数。

readcardinal1(f)	1 字节无符号整数
readcardinal2(f)	2 字节无符号整数
readcardinal3(f)	3 字节无符号整数
readcardinal4(f)	4 字节无符号整数
readcardinaltable(f,n,b)	n 个 b 字节的无符号数
readinteger1(f)	1 字节有符号整数
readinteger2(f)	2 字节有符号整数
readinteger3(f)	3 字节有符号整数
readinteger4(f)	4 字节有符号整数
readintegertable(f,n,b)	n 个 b 字节的整数
readfixed2(f)	2 字节浮点数（用于字体文件）

readfixed4(f)	4 字节浮点数（用于字体文件）
read2dot14(f)	2 字节浮点数（用于字体文件）
setposition(f,p)	跳到位置 p
getposition(f)	获取当前位置
skipposition(f,n)	跳过 n 个位置
readbytes(f,n)	n 个字节
readbytetable(f,n)	n 个字节

针对 cardinal[1-4] 和 integer[1-4] 读取器，还有另外八个小端变体：cardinal[1-4]le 和 integer[1-4]le。

5.2.8　用 sio

sio 库中提供了与 fio 库类似的一组函数：、、、、、、、、、、、、、、、和。这里第一个参数是字符串而不是文件句柄。更多细节见上一节。

5.2.9　符合 sha2

本库是 pdfe 库的副产品，后者需要这类辅助函数。、和 函数接受一个字符串，返回含散列值的字符串。

5.2.10　区域设置

在标准版 Lua 中，许多行为都依赖于当前区域设置。在 LuaTEX 中我们不能这样做，因为这会使文档不可移植。LuaTEX 运行期间会强制设置以下区域：

```
LC_CTYPE=C
LC_COLLATE=C
LC_NUMERIC=C
```

5.3　Lua 模块

有些通常属于 Lua 外部的模块被静态链接进了 LuaTEX，因为它们提供了有用的功能：

- lpeg，作者 Roberto Ierusalimschy，网址 http://www.inf.puc-rio.br/~roberto/lpeg/lpeg.html。该库不具备 Unicode 感知能力，而是按逐字节方式解释字符串。这主要意味着 lpeg.S 不能与以超过两字节编码的 UTF8 字符一起使用，因此 lpeg.S 在匹配时只会去查找这两个字节之一，而不是这两个字节的组合。lpeg.R 也是如此，不过后者在与多字节字符一起使用时会显示错误信息。因此 lpeg.R('aä') 会产生消息 bad argument ##1 to 'R' (range must have two characters)，因为对 lpeg 而言，ä 是两个“字符”（字节），所以 aä 一共是三个。在实践中这并不是真正的问题，稍加留意就能很好地处理 Unicode。
- slnunicode，来自 selene 库，网址 http://luaforge.net/projects/sln。该库经过了轻微扩展，使 unicode.utf8.* 函数也接受第 18 平面的前 256 个值。正如前面所解释的，这

正是 LuaTEX 用于原始二进制输出的范围。我们不打算再提供更多这类扩展，因为基本上你想做的一切都可以在 Lua 中完成。

- luazip，来自 kepler 项目，网址 http://www.keplerproject.org/luazip/。
- luafilesystem,同样来自 kepler 项目,网址 http://www.keplerproject.org/luafilesystem/。
- lzlib，作者 Tiago Dionizio，网址 http://luaforge.net/projects/lzlib/。
- md5,作者 Roberto Ierusalimschy,网址 http://www.inf.puc-rio.br/~roberto/md5/md5-5/md5.html。
- luasocket，作者 Diego Nehab，网址 http://w3.impa.br/~diego/software/luasocket/。luasocket 的.lua 支持模块也被预加载到可执行文件内部，不存在外部文件依赖。

5.4 测试

出于开发需要，你可以影响所使用的启动日期和时间。有两种方式可以做到这一点。

- 设置环境变量 SOURCE_DATE_EPOCH。这会影响 TEX 参数 time 和 date、随机种子、PDF 时间戳，以及同样由时间导出的 PDF id。该变量在 kpathsea 库启用时被查阅；解析工作委托给该库完成。
- 在 texconfig 表中设置 start_time 变量；与其他变量一样，我们在那里使用内部名字。出于兼容性考虑，我们也认下 SOURCE_DATE_EPOCH 这一项。应当注意的是，其他引擎中没有这样的变量，而且这种方法只在整个设置是在 Lua 中完成时才有意义。

当需要世界标准时间时，可以向引擎传入标志 utc。这个属性在日期和时间由 LuaTEX 自身设定时同样有效。它在 texconfig 表中有一个对应的项 use_utc_time。

还可以做一些控制，例如阻止把文件名写入 PDF 文件。这在别处讨论。在 ConTEXt 中，我们提供命令行参数 --nodates，它会更彻底地禁用日期。

第 6 章 语言、字符、字体与字形

6.1 引言

LuaTEX 内部对待最终被排版出来的字符与字形的方式，与 TEX82 处理同一批对象的方式大不相同。解释这种差异最简单的办法，是先聚焦于不受限的水平模式（即段落）和断字。之后再处理水平模式和数学模式中出现的差异就容易了。

在 TEX82 中，你键入的字符在被主控制循环遇到时会被转换为 `char` 节点记录。TEX 在创建这些记录的同时附加并处理字体信息，因此得到的 “水平列表中已经包含连字和隐含字距的最终形态。这种打包是必要的，因为我们可能需要获取比如一个水平盒子的有效宽度。

当需要对段落中的单词进行断字时，TEX 会（一次处理一个词）把 `char` 节点记录转换为一个字符串：用连字的组成部分替换连字，并忽略字距。然后在这个字符串上运行断字算法，再把断字结果转换回一个 “水平列表，依次拼回段落流中。请记住，段落中可能含有未装箱的水平内容，其中已经包含连字和字距，而其中的单词同样参与断字过程。

那些 `char` 节点记录的名字多少有些误导，因为它们其实是特定字体中的字形位置，并非语言学意义上的 “字符。`char` 节点记录内部根本没有语言信息。相反，语言信息是通过水平列表中的 `language whatsit` 节点传递的。

在 LuaTEX 中情况则大不相同。你键入的字符总是被转换为 `glyph` 节点记录，并带有一个特殊子类型，以标明它们是作为语言学字符使用的。LuaTEX 把所需的语言信息存放在这些记录中，但在创建节点时并不做任何与字体相关的处理。它只保存当前字体的索引，以及该字体中某个字符的引用。

当需要排版一个段落时，LuaTEX 先把所有断字点直接插入整个节点列表。接着处理整个列表中的全部字体信息（创建连字并调整字距），最后调整所有子类型标识，使这些记录从此成为 “字形节点。

6.2 字符、字形与断字

TEX82（包括 pdfTEX）区分 `char` 节点和 `lig` 节点。前者是简单条目，只含一个 “字符字段和一个 “字体字段，与记号存在同一块内存中。后者还包含一个组成部分列表，以及一个子类型，用以标明该连字是否由词界产生；它与盒子、字距、粘胶等其他节点存放在同一处。

在 LuaTEX 中，这两种类型被合并为一个稍大的结构，称为 `glyph` 节点。除了原有的字符、字体和组成部分字段外，还有另外几个字段，例如我们将在 9.2.12 中看到的 “attr；这些节点还含有一个子类型，它编码了四种主要类型和两种附加的 “幽灵” 类型。对于连字，可以同时置多位（当一个词只有单个字形时）。

- `character`，用于需要断字的字符：最低位（bit 0）置 1。

- glyph，用于特定字体字形：最低位（bit 0）不置位。
- ligature，用于构造出的连字，bit 1 置位。
- ghost，用于所谓的 “幽灵对象，bit 2 置位。
- left，用于由左词界产生的连字，以及由 leftghost 产生的幽灵，bit 3 置位。
- right，用于由右词界产生的连字，以及由 rightghost 产生的幽灵，bit 4 置位。

glyph 节点还含有语言数据，分为四项，都是节点创建时的当前值：setlanguage（15 位）、lefthyphenmin（8 位）、righthyphenmin（8 位）和 uchyph（1 位）。

顺带一提，LuaTEX 允许 16383 种不同语言，单词最长可达 256 个字符。语言信息随每个字符一同保存。你可以把 firstvalidlanguage 设为例如 1，从而使语言 0 成为一个被忽略的断字语言。

新原语 hyphenationmin 可用于指定一个词的最小长度。该值随（当前）语言一同保存。

由于 uchyph 的值保存在实际节点中，它的处理与 TEX82 略有不同：对 uchyph 的改动立即生效，而不是在当前段落末尾才生效。

已排版的盒子现在总是把语言信息嵌入节点自身，因此不再可能依赖周围的语言设置。在 TEX82 中，段落中部出现的 \unhbox 0 之类语句会使用当前段落语言处理盒子，除非盒子内部发出过 setlanguage。在 LuaTEX 中，所有语言变量都已冻结。

在传统 TEX 中，断字过程由 lccode 驱动。在 LuaTEX 中我们弱化了这种依赖。可以有几种策略。什么都不做时，加载模式、设置例外或对列表断字都会使用当前生效的 lccode。

当你把 savinghyphcodes 设为大于零的值时，当前这组 lccode 会随语言一同保存。这种情况下，之后改动某个 lccode 不再起作用。不过你可以用如下方式调整这组代码：

```
\hjcode`a=`a
```

这个改动是全局性的——如果你记住断字发生的时刻通常是构造段落或水平盒子之时，这就说得通了。语言初始化时若 savinghyphcodes 为零，则你一开始什么都没有；否则你已经有了一组代码。

当某个 hjcode 大于 0 但小于 32 时，它表示要使用的长度。在下面的例子中，我们把一个字符（x）映射到模式中的另一个字符，并告诉引擎 œ 算作一个字符。由于传统上 0 本身被保留用于禁止断字，值 32 被当作 0 处理。

下面是一些例子（我们假设使用法语模式）：

	foobar	foo-bar
\hjcode `x=`o	fxxbar	fxx-bar
\lefthyphenmin 3	œdipus	œdi-pus
\lefthyphenmin 4	œdipus	œdipus
\hjcode `œ=2	œdipus	œdi-pus
\hjcode `i=32 \hjcode `d=32	œdipus	œdipus

让每个字形都携带这全部信息会带来过大的开销，也会使设置这些代码的过程更加复杂。我们曾考虑过用 hjcode 集合的方案，但最终放弃了，因为实践中当前的做法已经足够，而且无论如何都无法做到兼容。

请注意：这些值总是会被保存到格式文件中，与转储格式时 savinghyphcodes 的设置无关。

边界节点通常会标记一个词的结束，这会干扰例如断字插入。为此你可以把 wordboundary 用作触发器。下面是几个用法示例：

```
discrete---discrete
```

在传统 TEX 中，连字构造与断字和换行机制交织在一起。在 LuaTEX 中这些阶段是隔离的。因此我们对（一连串）显式连字符的处理方式有所不同。我们已经添加了一些对断字各方面的控制，而又一项新的控制涉及自动连字符（例如输入中的 - 字符）。

当 automatichyphenmode 取值为 0 时，一个连字符会被转换为自动断字符。它前后的片段不会被断字。一个副作用是，行首连字符可能导致断行，但人们很少遇到这种情况。设置前、后字符会使这一现象更明显。取值为 1 会避免这个副作用；取值为 2 则不把连字符转换为断字符。其他选项的实验（例如允许对两侧单词断字）已被放弃。

```
% before-after \par
% before--after \par
% before---after \par

% -before \par
% after- \par
% --before \par
% after-- \par
% ---before \par
% after--- \par

% before-after \par
% before--after \par
% before---after \par

% ConTeXt \framed □□□□□□□
```

在下面的讨论中，我们展示了三个样本的情况：

输入 A、B、C 的演示输出已省略（原 ConTeXt 图）。

与其他单字符命令的原语同伴一样,- 命令有一个更冗长的原语版本 explicitdiscretionary；而断字器中通常被截获的字符 - （或任何被配置的字符）则可作为 automaticdiscretionary 使用。

6.3　主控制循环

在 LuaTEX 的主循环中，几乎所有待排版的输入字符都会被转换为子类型为 “character 的 glyph 节点记录，但有少数例外。

- accent 原语创建的节点子类型是 “glyph 而不是 “character：一个用于实际的重音符号，一个用于被加重音的字符。这样做的首要原因是，TEX82 中的 accent 明确依赖于当前字体编码，因此给该原语的名字赋予新含义没什么意义，那会使许多旧文档和宏包失效。次要原因是，在 TEX82 中 accent 会禁止对当前单词断字。由于在 LuaTEX 中只对 “character 节点

进行断字，所以可以达到同样的效果。当然，现代具备 Unicode 感知能力的宏包根本不会使用 accent 原语，而是直接映射到组合字符。

char 确实发生了这种含义变化：它现在生成带子类型 character 的 “glyph 节点。在传统 TEX 中，8 位输入编码、断字与从字体中取字形之间存在紧密关系。在 LuaTEX 中我们使用 UTF 输入，大多数情况下它直接映射到字体中的一个字符，字体引擎中的字形替换除外。如果你想直接访问字体中的任意字形，随时可以用 Lua 来做，因为字体是以 Lua 表的形式可用的。

- 数学模式中处理的所有结果，最终都成为子类型为 “glyph 的节点。事实上，数学处理的结果就是一个由字形、字距、粘胶、罚分、盒子等组成的普通列表。
- 由 Aleph 派生的命令 leftghost 和 rightghost 创建第三种子类型的节点：“ghost。这些节点在后续所有处理中被完全忽略，直到添加字形间字距的阶段才会被处理。
- 自动断字符的处理方式不同。TEX82 在感知到一个与当前字体 hyphenchar 匹配的输入字符后，会插入一个空断字符。我们认为这种检测方式是错误的：是否断字不应依赖于当前字体，它是一种语言属性。[1]

  在 LuaTEX 中，工作方式如下：当 LuaTEX 感知到一串输入字符与新整数参数 exhyphenchar 的值匹配时，会在这串节点之后插入一个显式断字符。初始时 TEX 把 \exhyphenchar =`\- 。顺带一提，这是一个全局参数而非语言相关参数，因为根据文档结构而不是文本语言来改变它的值可能更有用。

  在一连串显式连字符之后插入断字符的工作，与其他断字处理同时发生，而不是在主控制循环内部。

  LuaTEX 对 hyphenchar 的唯一用途，是检查一个词是否根本应该被纳入断字。如果一个词中第一个字符节点所附字体的 hyphenchar 为负，则立即放弃对该词的断字。这种行为仅是为了向后兼容而添加的，在新的 LuaTEX 文档中不应再使用 \hyphenchar =-1 作为阻止断字的手段。
- setlanguage 命令不再创建 whatsit。setlanguage 的含义被改为一个与其他参数一样的整数参数。这个整数参数在创建 \glyph _node 时被用来向字形节点添加语言信息。相应地，language 原语也被扩展，使其总是同时更新 setlanguage 的值。
- noboundary 命令（用于在通常会发生词界处理的地方禁止该处理）现在确实会创建节点。这些节点是必需的，因为 noboundary 命令在输入流中的确切位置必须保留到连字和字体处理阶段之后。
- 代码中不再有 main_loop 标号。请记住，TEX82 在向水平列表添加 char_nodes 时做了大量处理。出于速度考虑，它把那段处理代码放在 “主控制循环之外，只有任何 “词的第一个字符才由那个 “主控制循环处理。在 LuaTEX 中不再需要这样做（所有重活都在后面完成），那些（现在非常短小的）字符处理代码已经移回内联。当打开 tracingcommands 时，这一点可以看出来：报告的是整个词，而不仅仅是首字符。

由于我们倾向于把硬编码的行为改为可配置，又添加了几个新原语：

[1] TEX 刚出现时我们还没有 Unicode，而局限于 8 位意味着有时不得不在支持字符输入、字形渲染和断字之间做出妥协。

```
\hyphenpenaltymode
\automatichyphenpenalty
\explicithyphenpenalty
```

第一个参数对自动断字符（即由 exhyphenchar 产生的断字符）有以下影响：

模式	自动断字符 -	显式断字符 -
0	exhyphenpenalty	exhyphenpenalty
1	hyphenpenalty	hyphenpenalty
2	exhyphenpenalty	hyphenpenalty
3	hyphenpenalty	exhyphenpenalty
4	automatichyphenpenalty	explicithyphenpenalty
5	exhyphenpenalty	explicithyphenpenalty
6	hyphenpenalty	explicithyphenpenalty
7	automatichyphenpenalty	exhyphenpenalty
8	automatichyphenpenalty	hyphenpenalty

其他值则执行我们在 LuaTEX 中一直以来的做法：插入 exhyphenpenalty。

6.4　加载断字模式与例外词

尽管我们保留了传统的断字方法（它依然更优越），但 LuaTEX 中断字算法的实现与 TEX82 大不相同。

patterns 的参数经过展开后必须是合法的 UTF8，各模式之间以空格分隔，不允许出现 char 或 chardef 命令。当前实现相当严格，会拒绝所有非-Unicode 字符。同样，hyphenation 展开后的参数也必须是合法的 UTF8，但这里额外提供了一点语法：

- 花括号中的三组参数({}{}{})表示所需的复合断字符,参数与普通文档输入中 discretionary 命令的参数相同。
- 一个 - 表示所需的简单断字符，参见普通文档输入中的 \- 和 \discretionary {-}{}{}。
- 内部命令名会被忽略。这条规则专门为 discretionary 提供，但它也有助于处理可能混入的 relax 命令。
- 一个 = 表示文档输入中的一个（非断字符）连字符。

展开后的参数首先被转换回一个空格分隔的字符串，同时丢弃内部命令名。然后由一个例程把这个字符串转换为字典：通过转换其他列出的项来创建键 |-| 值对。需要重要注意的是，例外字典中的键总是可以由值推导出来。下面是几个例子：

取值	推导键（输入）	效果
ta-ble	table	ta\-ble (= ta\discretionary {-}{}{}ble)
ba{k-}{}{c}ken	backen	ba\discretionary {k-}{}{c}ken

得到的模式和例外字典会保存在 language 当前值所对应的语言代码之下。

在表格的最后一行中，你可以看到值中没有 discretionary 命令：在基于 TEX 的输入语法中，该命令是可选的。其背后的原因是，可以设想把一整本单词字典存为纯文本文件，再用 Lua lang 库中的某个函数加载进 LuaTEX。这种加载方式比走 TEX 语言原语快不少，但如果在加载时还要解释命令序列，那部分（大部分）速度优势就会丧失。

可以用 {-}{}{-} 为复合词中的显式连字符指定额外的断字点（如有需要，把 - 换成实际的显式连字符）。例如，下面这条与单词 “multi|-|word|-|boundaries 匹配，并允许在 “boun 和 “daries 之间额外断行：

```
{}{-}word{-}{}{-}boun-daries}
```

e-TEX 扩展 savinghyphcodes 的动机，是断字过去严重依赖于字体编码。在 LuaTEX 中情况已不再如此，对应的原语基本被忽略了。因为我们现在有了 hjcode，大小写相关代码可以专门用于 uppercase 和 lowercase。

例外词中的三对花括号模式可能有些出人意料，所以我们通过例子来解释。模式 foo{}{}{x}bar 会创建查找键 fooxbar，而模式 foo{}{}{}bar 创建 foobar。然后，当命中发生时，有一个替换文本（x）或没有。由于我们在断字符节点中引入了罚分，例外语法现在还可以带罚分说明。方括号中的值是 exceptionpenalty 的倍数。这里我们把它设为 10000，因此在例子中实际得到 30000。

6.5　应用断字

LuaTEX 在单词中插入断字符所使用的内部结构与 TEX82 非常不同，这意味着在处理上也有一些值得注意的差异。

首要一点是，断字过程中不涉及 “压缩 trie。算法仍然读取由 patgen 生成的模式文件，但 LuaTEX 使用有限状态哈希把模式与待断字单词进行匹配。该算法基于 OpenOffice 所用的 “libhnj 库，而后者又受 TEX 启发。

由于实现方式的原因，LuaTEX 与 TEX82 之间存在一些差异：

- LuaTEX 可以愉快地对整个 Unicode 字符范围断字。
- 模式和例外字典的大小只受可用内存限制，所有分配都是动态进行的。texmf.cnf 中与 trie 相关的设置会被忽略。
- 由于不存在 “trie 准备阶段，语言模式永远不会被冻结。这意味着原语 patterns（及其 Lua 对应项 lang.patterns）可以在任何时候使用，而不仅仅在 iniTEX 中。
- 格式文件中只保存 patterns 和 hyphenation 的字符串表示。在加载格式时，它们只是被重新求值。由此可见，没有真正理由在格式文件中预加载语言。事实上，这样做通常并不好。更聪明的做法是，直到真正需要某种模式时才加载它。
- LuaTEX 在创建隐含断字符时使用语言相关变量 prehyphenchar 和 posthyphenchar，而不是 TEX82 的 hyphenchar；并使用语言相关变量 preexhyphenchar 和 postexhyphenchar 的值来创建显式断字符（而不是 TEX82 的空断字符）。
- 与断字相关的两个计数器 hyphenpenalty 和 exhyphenpenalty 的值，现在保存在断字符节点中。这允许对显式 discretionary 命令进行局部覆盖。应用断字遍历时所使用的是当

时的当前值。不使用回调时，这与传统 TEX 兼容。当你调用 Lua 函数 `lang.hyphenate` 时，使用的也是当前值。

- 断字例外字典以键 |-| 值哈希的形式维护，而且同样是动态的，因此 `hyph_size` 设置也不再被使用。

因为我们在断字符节点中保存了罚分，`discretionary` 命令被扩展为可以接受一个可选的罚分说明，于是你可以这样做：

```
\hsize1mm
1:foo{\hyphenpenalty 10000\discretionary{}{}{}}bar\par
2:foo\discretionary penalty 10000 {}{}{}bar\par
3:foo\discretionary{}{}{}bar\par
```

结果如下（演示输出已省略）。

```
<number> n = lang.prehyphenchar(<language> l)
lang.prehyphenchar(<language> l, <number> n)

<number> n = lang.posthyphenchar(<language> l)
lang.posthyphenchar(<language> l, <number> n)
```

这两个函数用于获取或设置该语言中隐含断字的 “断前和 “断后连字符。初始值为十进制 45（连字符）和十进制 0（表示为空）。

```
<number> n = lang.preexhyphenchar(<language> l)
lang.preexhyphenchar(<language> l, <number> n)

<number> n = lang.postexhyphenchar(<language> l)
lang.postexhyphenchar(<language> l, <number> n)
```

这两个函数用于获取或设置该语言中显式断字的 “断前和 “断后连字符。两者初始值均为十进制 0（表示为空）。

6.5.1　hyphenate

下一个调用在节点列表中插入断字点（断字符节点）。如果以 `tail` 为参数，则处理到该节点即停止。目前，如果 `head`（以及指定的 `tail`）是合法节点，无论是否可能有其他错误，`success` 总是为真。

```
<boolean> success = lang.hyphenate(<node> head)
<boolean> success = lang.hyphenate(<node> head, <node> tail)
```

断字只作用于 “character，即所有字形节点中子类型取值为 1 的那个特殊子类型。子类型不同的字形模式不被处理。更多细节见 6.2。

6.5.2　[set|get]hjcode

以下两条命令可用于设置或查询 hj code：

```
lang.sethjcode(<language> l, <number> char, <number> usedchar)
<number> usedchar = lang.gethjcode(<language> l, <number> char)
```

当你设置一个 hjcode 时，当前这组代码会被初始化，除非该组已经因为 `savinghyphcodes` 大于零而初始化过。

第 7 章 字体结构

7.1 字体表

TEX 的所有字体在 Lua 代码中都表示为表（table），在内部则表示为 C 结构体。下表中的所有键，如果出现在 `define_font` 回调所返回的表中，就会被保存到内部字体结构体中；如果没有定义 `define_font` 回调，则由常规的 TFM|/|VF 读取例程产生的键也会被保存。

列 “VF 表示该键由 `font.read_vf()` 例程创建，列 “TFM 表示该键由 `font.read_tfm()` 例程创建，列 “used 表示 LuaTEX 引擎本身是否会使用该键。表中的顶层键如下：

键	vf	tfm	used	值类型	说明
`name`	yes	yes	yes	string	度量（文件）名
`area`	no	yes	yes	string	（目录）位置，通常为空
`used`	no	yes	yes	boolean	指示是否被使用（初始值: false）
`characters`	yes	yes	yes	table	此字体中已定义的字形
`checksum`	yes	yes	no	number	默认值：0
`designsize`	no	yes	yes	number	期望尺寸（默认值: 655360 == 10pt）
`direction`	no	yes	yes	number	默认值：0
`encodingbytes`	no	no	yes	number	默认值：取决于 `format`
`encodingname`	no	no	yes	string	编码名称
`fonts`	yes	no	yes	table	本地使用的字体
`psname`	no	no	yes	string	这是输入字体源中的 PostScript 字体名，用作 PDF 输出中的字体名标识符。它必须是合法字符串（例如不能包含空格之类的字符），因为后端不会做清理。这样就把完全的控制权交给了加载器。
`fullname`	no	no	yes	string	输出字体名, 当未设置 `psname` 时在 PDF 输出中用作后备
`subfont`	no	no	yes	number	默认值：0，多字体（`ttc`）文件中的子字体索引
`header`	yes	no	no	string	头部注释（如果有）

hyphenchar	no	no	yes	number	默认值：TEX 的 hyphenchar
parameters	no	yes	yes	hash	默认值：7 个参数，全部为零
size	no	yes	yes	number	所需的缩放(默认与 designsize 相同)
skewchar	no	no	yes	number	默认值：TEX 的 skewchar
type	yes	no	yes	string	此字体的基本类型
format	no	no	yes	string	磁盘格式类型
embedding	no	no	yes	string	PDF 嵌入方式
filename	no	no	yes	string	磁盘上的字体文件名
tounicode	no	yes	yes	number	当此项设为 1 时，LuaTEX 假定字体中存在逐字形的 tounicode 条目。
stretch	no	no	yes	number	来自 expandglyphsinfont 的“stretch 值
shrink	no	no	yes	number	来自 expandglyphsinfont 的“shrink 值
step	no	no	yes	number	来自 expandglyphsinfont 的“step 值
expansion_factor	no	no	no	number	已扩展字体的实际扩展系数
attributes	no	no	yes	string	pdffontattr
cache	no	no	yes	string	此键控制 TEX 端对 Lua 表的缓存：yes 表示使用传递给 LuaTEX 的表的引用（这是默认值）；no 表示不保存表引用，也不为该字体缓存任何 Lua 数据；而 renew 表示不保存表引用，但保存对首次访问字体某个字段时所创建表的引用。
nomath	no	no	yes	boolean	此键可为文本字体带来小幅提速。如果存在且为 true，LuaTEX 将不检查字符条目中的数学专用键。
oldmath	no	no	yes	boolean	此键把字体标记为旧式 TEX 数学字体，并禁用 OpenType 代码路径。
slant	no	no	yes	number	此参数使字体倾斜，作用与 Type 1 字体映射文件中的 SlantFont 相同。

extend	no	no	yes	number	此参数在水平方向上缩放字体，作用与 Type 1 字体映射文件中的 ExtendFont 相同。
squeeze	no	no	yes	number	此参数在垂直方向上缩放字体，映射文件中没有对应项。
width	no	no	yes	number	后端会注入设置笔宽的 PDF 运算符。该值（与 TEX 中惯例一样）除以 1000。它与 mode 配合使用。
mode	no	no	yes	number	后端会注入与绘制模式相关的 PDF 运算符：0 为填充，1 为轮廓，2 既绘制又填充，3 完全不绘制。

cache 选项中保存的引用是线程局部的，因此使用协程时要小心：如果表已在一个线程中缓存，而你从另一个线程引用它，就会抛出错误。

键 name 总是必需的。键 stretch、shrink、step 只有一起使用时才有意义：它们可用来替代加载后的 expandglyphsinfont 命令。LuaTEX 不支持 auto_expand 选项。事实上，那些创建扩展或突出副本的原语大概只在与传统字体配合时有用，因为所有这些额外的 OpenType 属性都被排除在外。expansion_factor 是一个可以出现在 font.fonts 中某个字体内部的值。它是由字体扩展算法自动生成的字体的实际扩展系数（取值在 -shrink 与 stretch 之间，步长为 step）。

subfont 参数可用于指定 ttc 字体中的子字体。给出该参数时，它会取代 psname 与 fullname 的组合。第一个子字体的编号为 1。值为零表示使用名字进行查找。

由于我们把扩展的实际状态随每个字形一起存储，并且没有专门的字体实例，因此可以在构造行之前更改一些与字体相关的参数，例如：

```
font.setexpansion(font.current(),100,100,20)
```

这主要是为了实验（或用 Lua 编写的优化例程），因此没有对应的原语。

键 attributes 可用于在 PDF 文件中设置字体属性。键 used 由引擎在字体被实际使用时置位，这会确保字体的定义被写入输出文件（DVI 或 PDF）。TFM 读取器把它设为 false。direction 是一个数字，表示此字体的 “正常方向。共有十六种可能：

#	方向	#	方向	#	方向	#	方向
0	LT	4	RT	8	TT	12	BT
1	LL	5	RL	9	TL	13	BL
2	LB	6	RB	10	TB	14	BB
3	LR	7	RR	11	TR	15	BR

这些是 Omega 风格的方向缩写：第一个字符表示字符字形的 “首边（在书写方向上最先看到的边），第二个字符表示 “顶边。请记住，LuaTEX 的方向模型略有不同，因此这些值实际上并不派什么用场。

parameters 是一个键类型混合的哈希。它有七个可能的字符串键，以及若干个整数索引（从 8 开始向上编号）。这七个字符串实际上用来替代最下面的七个索引，因为这样的用户界面更友好。

这些名字及其内部重映射如下：

名称	重映射
slant	1
space	2
space_stretch	3
space_shrink	4
x_height	5
quad	6
extra_space	7

键 type、format、embedding、fullname 和 filename 用于把 OpenType 字体嵌入结果 PDF 中。

characters 表是一个字符哈希列表，以整数作为索引。该整数就是 TEX 用来识别此字符的 “内部代码。

还可以使用两个非常特殊的字符串索引：left_boundary 是一个虚拟字符，其连字和字距用于处理单词边界。right_boundary 与之类似，但（目前）实际上并没有派什么用场。

每个字符哈希本身也是一个哈希。例如，下面是字体 cmr10 at 10pt 中字符 “f（十进制 102）的例子。表示尺寸的数字单位是缩放点（sp）。

```
[102] = {
    ["width"]  = 200250,
    ["height"] = 455111,
    ["depth"]  = 0,
    ["italic"] = 50973,
    ["kerns"]  = {
        [63] = 50973,
        [93] = 50973,
        [39] = 50973,
        [33] = 50973,
        [41] = 50973
    },
    ["ligatures"] = {
        [102] = { ["char"] = 11, ["type"] = 0 },
        [108] = { ["char"] = 13, ["type"] = 0 },
        [105] = { ["char"] = 12, ["type"] = 0 }
    }
}
```

字符哈希中可以出现以下顶层键：

键	vf	tfm	used	类型	说明
width	yes	yes	yes	number	字符宽度，单位 sp（默认 0）
height	no	yes	yes	number	字符高度，单位 sp（默认 0）
depth	no	yes	yes	number	字符深度，单位 sp（默认 0）
italic	no	yes	yes	number	字符的斜体校正，单位 sp（默认零）
top_accent	no	no	maybe	number	字符的上重音对齐位置，单位 sp（默认零）
bot_accent	no	no	maybe	number	字符的下重音对齐位置，单位 sp（默认零）
left_protruding	no	no	maybe	number	字符的 lpcode
right_protruding	no	no	maybe	number	字符的 rpcode
expansion_factor	no	no	maybe	number	字符的 efcode
tounicode	no	no	maybe	string	字符对应的 Unicode 字符，采用 UTF|-|16BE 十六进制格式
unicode	yes	no	maybe	table	字符对应的 Unicode 字符，以 Unicode 码点（整数）数组形式给出
next	no	yes	yes	number	“下一个更大的字符索引
extensible	no	yes	yes	table	可伸缩配方的组成部分
vert_variants	no	no	yes	table	竖排变体集合的组成部分
horiz_variants	no	no	yes	table	横排变体集合的组成部分
kerns	no	yes	yes	table	字距调整信息
ligatures	no	yes	yes	table	连字信息
commands	yes	no	yes	array	虚拟字体命令
name	no	no	no	string	字符的（PostScript）名字
index	no	no	yes	number	（OpenType 或 TrueType）字体的字形索引
used	no	yes	yes	boolean	已被排版过（默认：false）
mathkern	no	no	yes	table	数学切入（cut-in）规格

top_accent、bot_accent 和 mathkern 的值仅用于数学重音和上标的放置，详见第 8 节。left_protruding 和 right_protruding 的值仅在 protrudechars 非零时使用。是否使用 expansion_factor 取决于字体的全局扩展设置以及 adjustspacing 的值。

tounicode 的用法如下：如果字体在顶层指定了 tounicode=1，那么 LuaTEX 会根据字符级别的 tounicode 字符串（在有提供的地方）为 PDF 字体（或字体子集）构造 /ToUnicode 条目。如果某个字符没有合理的 Unicode 对应字符，就不要提供字符串（不要给空字符串）。

如果没有设置字体级别的 tounicode,LuaTEX 将根据你使用的 TEX 码点来构建 /ToUnicode,

任何字符级别的 tounicodes 都会被忽略。字符串格式正是 Adobe CMap 文件所期望的格式（十六进制编码的 UTF-16BE），只是去掉了外层的尖括号。例如，fi 连字的 tounicode 应为 00660069。如果你传入的是数字，转换会自动完成。

数学字符可以有一个 next 字段，指向下一个更大的字形。不过，如果同时存在 extensible，它会覆盖 next。而 extensible 字段又会被 OpenType 版本的 vert_variants 覆盖。extensible 表非常简单：

键	类型	说明
top	number	顶部字符索引
mid	number	中部字符索引
bot	number	底部字符索引
rep	number	可重复的字符索引

horiz_variants 和 vert_variants 是组件数组。每个组件本身又是一个最多含五个键的哈希：

键	类型	说明
glyph	number	字符索引。注意这是编码数字，不是名字。
extender	number	若此部分可重复则为一（1），否则为零（0）。
start	number	起始侧的最大重叠量（单位：缩放点）。
end	number	结束侧的最大重叠量（单位：缩放点）。
advance	number	此项的总前进宽度。它可以为零或缺失，此时使用字符 component 对应字形的自然尺寸。

kerns 表是一个以字符索引为键的哈希（“字符索引定义为非负整数或字符串值 right_boundary），其值为要应用的字距调整量，单位为缩放点。

ligatures 表是一个以字符索引为键的哈希（“字符索引定义为非负整数或字符串值 right_boundary），其值又是一个小哈希，含两个字段：

键	类型	说明
type	number	此连字命令的类型，默认 0
char	number	结果连字的字符索引

连字中的 char 字段是必需的。连字内部的 type 字段是 TEX 支持的八种连字类型之一的数字或字符串值。当 TEX 插入一个新连字时，它把新字形放在左右两个字形的中间。原来的左

右字形可以选择保留；当至少保留其中之一时，还可以把新的 “插入点向前移动一位或两位。最终位于插入点右侧的字形将成为下一个 “左字形。

文字形式（Knuth）	数字	字符串	结果
`l + r =: n`	0	`=:`	`|n`
`l + r =:| n`	1	`=:|`	`|nr`
`l + r |=: n`	2	`|=:`	`|ln`
`l + r |=:| n`	3	`|=:|`	`|lnr`
`l + r =:|> n`	5	`=:|>`	`n|r`
`l + r |=:> n`	6	`|=:>`	`l|n`
`l + r |=:|> n`	7	`|=:|>`	`l|nr`
`l + r |=:|>> n`	11	`|=:|>>`	`ln|r`

默认值为 0，可以省略。它表示一个 “常规连字，即连字替换原来的两个字形。在此表中，| 表示最终的插入点。

`commands` 数组在下面说明。

7.2　真实字体

一个 TEX 字体是否是应该写入 PDF 文档的 “真实字体，由顶层字体结构中的 `type` 值决定。如果该值为 `real`，则它是一个真正的字体，嵌入机制会尝试把所需的字体对象定义添加到 PDF 中。`type` 的取值如下：

取值	说明
`real`	这是一个基础字体
`virtual`	这是一个虚拟字体

要采取的操作取决于若干不同的变量：

- 所用字体是否适合 8 位编码方案。对于通过 TFM 文件通信的传统 TEX 字体，答案是肯定的。
- 磁盘字体文件的类型，例如位图文件，或者轮廓型 Type 1、TrueType 或 OpenType 字体。
- 请求的嵌入级别，尽管大多数情况下嵌入的是字符子集。完全不嵌入的时代（至少在我们看来）基本上已经过去了。

使用 8 位编码向量之外任何方案的字体,必须以不同方式写入 PDF。当字体表的 `encodingbytes` 设为 2 时，它就是宽字体，其他情况都不是。对于通过 Lua 加载的 OpenType 和 TrueType 字体，值 2 是默认值。对于 Type 1 字体，必须显式把 `encodingbytes` 设为 2。对于 PK 位图字体，完全不支持宽字体编码。

如果不需要特殊处理，LuaTEX 会回退到 pdfTEX 和 dvips 所使用的基于映射文件的方案，从而透明地支持旧版字体。如果使用了 “宽字体, 新的子系统就会启动, 字体结构中必须存在一些

额外字段。在这种情况下，LuaTEX 完全不使用映射文件。这些额外字段是：format、embedding、fullname、cidinfo（如前所述）、filename，以及各个字符中的 index 键。

format 变量可取以下值。type3 字体仅为向后兼容而提供，不支持新的宽编码选项。

取值	说明
type1	这是一个 PostScript Type 1 字体
type3	这是一个位图（PK）字体
truetype	这是一个 TrueType 或基于 TrueType 的 OpenType 字体
opentype	这是一个基于 PostScript 的 OpenType 字体

embedding 变量的合法取值如下：

取值	说明
no	完全不嵌入该字体
subset	包含字体并尝试子集化
full	完整包含此字体

其余字段的用法如下。fullname 将作为 PostScript|/|PDF 字体名。cidinfo 将用作字符集，即 CID 的 /Ordering 和 /Registry 键。filename 指向实际的字体文件。如果你在 filename 中给出完整路径，或者文件就在当前目录下，LuaTEX 的运行效率会稍高一些，因为那种情况下不必重新运行 find_*_file 回调。

请注意：在一个文档中混用新旧字体时，可能会产生 PostScript 名字冲突，从而导致打印错误。出现这种情况时，必须把字体的 fullname 改成更唯一的名字。

排版字符串以宽格式写出，每个字形使用 2 字节，以字符信息中的 index 键作为值。整体效果就像是采用了基于数字的编码，而不是传统的（PostScript）基于名字的重编码。获取 Type 1 字体正确 index 编号的一种方法是通过 fontloader.open 加载字体，并把表索引用作 index 字段。

为了确保最终文档的剪切和粘贴正常工作，最好强制提供 tounicode 向量。并非所有 PDF 阅读器都能正确处理这一点，因此请以 Acrobat 为参照。

7.3　虚拟字体

7.3.1　结构

如果你想让 LuaTEX 把 define_font 返回的表当作虚拟字体处理，必须采取以下步骤：

- 把顶层键 type 设为 virtual。在大多数情况下这是可选的，因为我们反正会查看 commands 条目。

- 确保 fonts 中至少有一个有效条目（见下文），不过在该表缺失时，较新版本的 LuaTEX 会添加一个默认条目。
- 在相关字符上添加 commands 数组。虚拟字符本身也可以指向其他虚拟字符，但要小心嵌套，因为这可能造成循环并使栈溢出（而栈溢出通常也意味着出错了）。

顶层 type 键取特定值 virtual 会触发对表中其余虚拟字体专用字段的处理，但仅仅存在'type' 键就足以阻止 LuaTEX 自行查找虚拟字体。这也可以 "反向运用：如果你完全确定某个字体不是虚拟字体，把 real 赋给 type 会阻止 LuaTEX 查找虚拟字体文件，从而省去一次磁盘搜索。这只在加载 TFM 文件时才有意义。

fonts 是一个（带索引的）Lua 表。其值本身是含一两个键的哈希，每个条目表示虚拟字体中的一个基础字体。如果你的字体要引用自身，可以使用 font.nextid() 函数，它返回下一个将要定义的字体的索引，而那很可能就是当前正在定义的字体。因此，一个表看起来像这样：

```
fonts = {
  { name = "ptmr8a", size = 655360 },
  { name = "psyr", size = 600000 },
  { id = 38 }
}
```

此虚拟字体中第一个被引用的字体（索引 1）是按 10pt 加载的 ptmr8a，第二个是按略大于 9pt 加载的 psyr。第三个是先前定义过的字体，在 LuaTEX 中已知为字体 id 38。数组索引编号由属于各字符的字符命令定义使用。

commands 数组是一个哈希，其中每一项又是一个小数组，第一项表示命令，其余各项是该命令的参数。允许的命令及其参数如下：

命令	参数个数	类型	说明
font	1	number	从本地 fonts 表中选择一个新字体
char	1	number	用当前字体排版此编号字符，并向右移动该字符的宽度
node	1	node	输出此节点（列表），并向右移动该列表的宽度
slot	2	2 numbers	font 与 char 命令组合的快捷方式
push	0		保存当前位置
nop	0		什么也不做
pop	0		弹出位置
rule	2	2 numbers	输出一个 $ht * wd$ 的矩形，并向右移动。
down	1	number	在页面上向下移动
right	1	number	在页面上向右移动
special	1	string	输出一个 special 命令

pdf	2	2 strings	输出一个 PDF 字面量,第一个字符串取 origin、page、text、font、direct 或 raw 之一; 如果只有一个字符串，则假定为 origin
lua	1	string, function	在嵌入字形时执行一段 Lua 脚本; 如果是函数，则会把字体 id 和字符代码传给它
image	1	image	输出一个图像（参数可以是 <image> 变量，也可以是 image_spec 表）
comment	any	any	此命令的参数被忽略

当字体 id 设为 0 时，它会被替换为当前指定的字体 id。这样就避免了对未来 id 做取巧处理。通常可以使用 font.nextid，但如果在此期间又构建了更复杂的字体，可能已经加载了其他实例。

pdf 选项还接受一个 mode 关键字，此时第三个参数设置模式。该选项会以高效方式更改模式（传入空字符串会在 PDF 文件中多出空行)。此选项仅对虚拟字体有意义。font 模式仅在虚拟字体中有意义。各种模式的定义有些模糊，部分继承自 pdfTEX。

模式	说明
origin	进入页面模式并设置位置
page	进入页面模式
text	进入文本模式
font	进入字体模式 (类似文本模式，仅用于虚拟字体)
always	结束当前字符串，并在需要时强制进行变换
raw	结束当前字符串

你始终需要检查生成的 PDF 代码是什么，因为后端的优化可能产生各种干扰，而且字体本身就很复杂。下面是一个相当详尽的字形命令示例，使用了这些键:

```
...
commands = {
    { "push" },                     -- remember where we are
    { "right", 5000 },              -- move right about 0.08pt
    { "font", 3 },                  -- select the fonts[3] entry
    { "char", 97 },                 -- place character 97 (ASCII 'a')
 -- { "slot", 2, 97 },              -- an alternative for the previous two
    { "pop" },                      -- go all the way back
    { "down", -200000 },            -- move upwards by about 3pt
    { "special", "pdf: 1 0 0 rg" }  -- switch to red color
```

```
  -- { "pdf", "origin", "1 0 0 rg" } -- switch to red color (alternative)
     { "rule", 500000, 20000 }       -- draw a bar
     { "special", "pdf: 0 g" }       -- back to black
  -- { "pdf", "origin", "0 g" }      -- back to black (alternative)
}
...
```

在 commands 数组开始处，font 的默认值总是 1。因此，如果虚拟字体本质上只是重编码，通常你不必在数组中显式创建 “font 命令。

commands 数组中的矩形只用两个维度构建：它们没有深度。为了正确的垂直放置，可能需要额外的 down 命令。

无论你在 commands 内部产生了多少移动，输出指针始终恰好移动字符哈希 width 键所给定的宽度。commands 数组内部发生的任何移动在上层都会被忽略。

special 可以带有 pdf:、pdf:origin:、pdf:page:、pdf:direct: 或 pdf:raw: 前缀。当需要拼接字符串时，使用 pdf 命令可能更高效。

7.3.2　人造字体

即使在 “真实字体中，也可以存在虚拟字符。当 LuaTEX 在需要排版某个字符时遇到该字符内部的 commands 字段，它就会像处理真正的虚拟字符一样解释这些命令。在这种情况下，如果你没有创建 “fonts 数组，那么默认（也是唯一）的 “基础字体就是当前字体本身。实际上，这意味着你可以为已有字符创建虚拟副本，这在想要创建复合字符时很有用。

注意：此功能不能反向使用。虚拟字体中不能有 “真实字符！你也不能用这种技术来做字体重编码；那样做需要一个真正的虚拟字体（因为已经存在的字符无法被更改）。

7.3.3　虚拟字体示例

最后，下面是一个展示虚拟字体的 plain TEX 输入文件：

```
\directlua {
  callback.register('define_font',
    function (name,size)
      if name == 'cmr10-red' then
        local f = font.read_tfm('cmr10',size)
        f.name  = 'cmr10-red'
        f.type  = 'virtual'
        f.fonts = {
          { name = 'cmr10', size = size }
        }
        for i,v in pairs(f.characters) do
          if string.char(i):find('[tacohanshartmut]') then
             v.commands = {
               { "special", "pdf: 1 0 0 rg" },
               { "char", i },
               { "special", "pdf: 0 g" },
             }
          end
        end
```

```
        return f
      else
        return font.read_tfm(name,size)
      end
    end
  )
}

\font\myfont  = cmr10-red at 10pt \myfont  This is a line of text \par
\font\myfontx = cmr10     at 10pt \myfontx Here is another line of text \par
```

7.4　vf

当执行字体 commands 中定义的 Lua 代码时，可以使用 vf 库。它提供的函数与命令类似：char、down、fontid、image、node、nop、pop、push、right、rule、special 和 pdf。这个库已经存在一段时间了，但没有怎么宣传和测试，原因之一是它很容易定义出无效字体（或者搞乱 PDF 流）。请记住，每次输出一个字符时都会执行这些 Lua 片段。

7.5　font

font 库提供了访问字体系统内部结构的接口，还包含加载传统 TEX 字体度量格式的辅助函数。其他字体加载功能由 fontloader 库提供，下一节将讨论它。

7.5.1　加载 TFM 文件

本小节所记录的行为被认为是稳定的，今后不会再有向后不兼容的更改。

```
<table> fnt =
    font.read_tfm(<string> name, <number> s)
```

这个数字有点特别：

- 如果为正，它以缩放点指定 “at size（按指定尺寸）。
- 如果为负，其绝对值表示相对于字体 designsize 的 “scaled （缩放）设置。

7.5.2　加载 VF 文件

本小节所记录的行为被认为是稳定的，今后不会再有向后不兼容的更改。

```
<table> vf_fnt =
    font.read_vf(<string> name, <number> s)
```

数字 s 的含义和返回表的格式与 read_tfm 函数中的类似。

7.5.3　fonts 数组

通过一个虚拟数组，Lua 可以访问 TEX 字体的整个表。

```
font.fonts[n] = { ... }
<table> f = font.fonts[n]
```

由于这是一个虚拟数组，你不能对它调用 pairs，但可参见下面的 font.each 迭代器。

实现这个虚拟数组的两个元表函数是：

```
<table> f = font.getfont(<number> n)
font.setfont(<number> n, <table> f)
```

请注意，目前除非你缓存了它，否则每次访问 font.fonts 或调用 font.getfont 都会为整个字体创建一个 Lua 表。如果你想要内部数据的副本，可以使用 font.getcopy：

```
<table> f = font.getcopy(<number> n)
```

这个函数返回一个 TEX 所知参数的表。这些参数可能与缓存表中的不同：

```
<table> p = font.getparameters(<number> n)
```

还要注意：只能对那些已在 TEX 中定义、但自定义以来完全未被访问过的字体进行赋值。这大大限制了写访问 font.fonts 的可用性，今后很可能会实现一套较宽松的规则。

7.5.4　检查字体状态

你可以通过调用此函数来测试字体的状态：

```
<boolean> f =
    font.frozen(<number> n)
```

返回值为 true（不可赋值）、false（可以更改）或 nil（根本不是有效字体）之一。

7.5.5　直接定义字体

你可以通过调用此函数把自己的字体定义到 font.fonts 中：

```
<number> i =
    font.define(<table> f)
```

返回值是所定义字体的内部 id 编号（即 font.fonts 中的索引）。如果创建字体失败，会抛出错误。该表是一个字体结构。另一种调用形式是：

```
<number> i =
    font.define(<number> n, <table> f)
```

其中第一个参数是预留的字体 id（见下文）。

7.5.6　Unicode 映射

glyphtounicode 扩展与 pdfTEX 中的类似，但不提供按字体设置的方法。这与我们存储字体的方式不同、以及可以重载默认字体加载器有关。如果你想调整某个特定字体，可以像下面这样在 plain TEX 代码中使用一个 Lua 辅助函数。

```
% ToUnicode vectors will be added

\pdfvariable gentounicode 1

% Wrap a helper in a macro:

\def\luaglyphtounicode#1#2%
  {\directlua{font.settounicode(\fontid\font,\number#1,"#2")}}

% Use the global mapper:

\pdfextension glyphtounicode {A}{0042}

% Test this:

{A \font\test=cmss10 \test  A}

% Define a non unicode symbol font:

\font\orn = OrnementsADF at 10pt

% Register mappings:

\{ \orn
    \luaglyphtounicode{42}{2B9C}
    \luaglyphtounicode{65}{2B9D}
\}

% Test this:

{\orn \char42 \char65 A} A

% And see if cut-n-paste from the pdf file works.
```

当然，把辅助函数包进宏的做法取决于具体的宏包。此功能是为 2026 版添加的，不应干扰先前的行为。顺带说明：对于 Type 1 字体，向 pdf 文件添加向量只在至少存在一个全局扩展条目时发生，但这种由 Lua 驱动的方式没有这个限制。这意味着当你使用特定于字体的方法时，它也会为其他 8 位字体触发向量的生成。

7.5.7　扩展字体

在合理范围内，你可以在字体定义之后扩展它。由于有些属性最好保持不变，扩展仅限于添加字符。

```
font.addcharacters(<number> n, <table> f)
```

传入的表可以含有 characters 字段，它是一个与 define 中所用类似的（子）表；对于虚拟字体，还可以添加 fonts 表。characters 表中定义的字符会被添加（当尚不存在时）或替换已有条目。请记住，替换可能有副作用，因为某个字符可能已经被使用过。我们不设置限制，而是期望用户自己小心。（setfont 辅助函数是一种更彻底的替换方式。）

7.5.8　预计的下一个字体 id

```
<number> i =
    font.nextid()
```

它返回如果在代码流的这个位置执行 font.define 调用时将会返回的字体 id 编号。这对于需要引用自身的虚拟字体很有用。如果你传入 true 作为参数，该 id 会被预留，之后可作为第一个参数传给 font.define。在创建复杂虚拟字体时这会很方便。

```
<number> i =
    font.nextid(true)
```

7.5.9　字体 id

```
<number> i =
    font.id(<string> csname)
```

它返回与 csname 关联的字体 id；如果 csname 未定义，则返回 −1。

```
<number> i =
    font.max()
```

这是 font.fonts 中已使用的最大索引。

```
<number> i = font.current()
font.current(<number> i)
```

它获取或设置当前使用的字体编号。

7.5.10　遍历所有字体

```
for i,v in font.each() do
  ...
end
```

这是一个遍历每个已定义 TEX 字体的迭代器。返回的第一个值是 font.fonts 中的索引，第二个值是字体本身（以 Lua 表形式）。索引按递增顺序列出，但它们并不总是构成连续数字的数组：在某些情况下序列中可能有空洞。

7.5.11　\glyphdimensionsmode

早在 LuaTEX 的早期，就决定以反映所施垂直偏移的方式来计算字形的有效高度和深度。高度加上该偏移，而深度只在偏移大于零时才减去该偏移。现在我们可以通过这个模式参数更细致

地控制这一行为。偏移会加到高度上，和 |/| 或从深度中减去。有效值永不为负。零模式是默认值。

取值	效果
0	旧行为：把偏移加到高度上，并且仅当偏移为正时才从深度中减去偏移
1	把偏移加到高度上，并从深度中减去偏移
2	把偏移加到高度上并从深度中减去偏移，但保留当前结果与先前结果的较大值
3	使用字形本身的高度和深度，即不应用偏移

7.5.12　\discretionaryligaturemode

此参数控制复杂连字如何与断字点（由连字符处理程序插入的）交互。非零值会阻止构造所谓的 init 和 select 断字点。

7.5.13　\tracinglostchars

这个跟踪器报告缺失的字符。按设计，我们把报告推迟到后端包含它们时、数学排版需要它们时，或创建虚拟字体时（1、2、3）。应 LaTeX 团队的要求，我们另外增加了两个在更早阶段报告的选项，即在字形被打包或在行断行期间处理时报告（4 和 5）。

	效果
0	不报告
1	仅向日志文件发出警告
2	警告并强制输出到终端
3	错误
4	提前警告（并强制输出到终端）
> 4	提前报错

第 8 章 数学排版

8.1 传统方式与 OpenType 并用

LuaTEX 中数学排版的处理方式与 TEX82（以及 pdfTEX）有相当大的不同。首先，LuaTEX 新增了若干原语并扩展了另一些原语，以便轻松使用 Unicode 输入。其次，TEX82 的所有内部特殊值（例如算符间距）都已通过控制序列暴露出来并可修改。第三，新增了若干扩展以便使用 OpenType 数学字体。最后，过去曾被提出或考虑过的一些扩展现已加入引擎。

8.2 Unicode 数学字符

字符处理现已扩展到完整的 Unicode 范围（即 `\U` 前缀），这与 XeTEX 兼容。

TEX 中的数学原语保持原样,但从输入转换为数学命令的那两个除外:`mathcode` 和 `delcode`。这两个命令现在允许在等号左侧使用 21 位字符参数。

一些新的 LuaTEX 原语会读取多个独立值。下表中用加号表示这一点。

此类原语的输入如下所示:

```
\def\overbrace{\Umathaccent 0 1 "23DE }
```

被修改过的 TEX82 原语有:

primitive	min	max		min	max
`mathcode`	0	10FFFF	=	0	8000
`delcode`	0	10FFFF	=	0	FFFFFF

未被修改的有:

primitive	min	max
`mathchardef`	0	8000
`mathchar`	0	7FFF
`mathaccent`	0	7FFF
`delimiter`	0	7FFFFFF
`radical`	0	7FFFFFF

出于实用原因,`mathchardef` 会静默接受大于 `0x8000` 的值,并将其解释为 `Umathcharnumdef`。这是为了兼容旧的宏包。

以下新原语与 XeTEX 兼容：

primitive	min	max		min	max
Umathchardef	0+0+0	7+FF+10FFFF			
Umathcharnumdef[5]	-80000000	7FFFFFFF			
Umathcode	0	10FFFF	=	0+0+0	7+FF+10FFFF
Udelcode	0	10FFFF	=	0+0	FF+10FFFF
Umathchar	0+0+0	7+FF+10FFFF			
Umathaccent	0+0+0	7+FF+10FFFF			
Udelimiter	0+0+0	7+FF+10FFFF			
Uradical	0+0	FF+10FFFF			
Umathcharnum	-80000000	7FFFFFFF			
Umathcodenum	0	10FFFF	=	-80000000	7FFFFFFF
Udelcodenum	0	10FFFF	=	-80000000	7FFFFFFF

其指定格式通常如下：

```
\Umathchardef\xx="1"0"456
\Umathcode   123="1"0"789
```

处理定界符样式对象的新原语不会设置“大字族。用于展示目的的合适字号选择应由字体通过 Umathoperatorsize 参数来处理。

对于其中一些原语，所有信息被打包进单个有符号整数中。对于前两个（Umathcharnum 和 Umathcodenum），最低 21 位是字符编码，其上 3 位表示数学类，字族数据保存在最高位。这意味着数学字族 128–255 的值实际上是负数。对于 Udelcodenum 则没有数学类。数学字族信息保存在紧接字符编码之上的位中。使用这三个命令不如使用双值和三值命令直观，因此除非你确切知道自己在做什么并且确实需要更快输入扫描所带来的加速，否则最好使用冗长形式的命令。

Umathaccent 命令接受可选关键字来控制数学重音符号的各种细节。详见下文 8.6.1。

还有更多新原语，它们将在后续章节中说明：

primitive	value range (in hex)
Uroot	0 + 0–FF + 10FFFF
Uoverdelimiter	0 + 0–FF + 10FFFF
Uunderdelimiter	0 + 0–FF + 10FFFF
Udelimiterover	0 + 0–FF + 10FFFF
Udelimiterunder	0 + 0–FF + 10FFFF

除了伪类变量 (7) 之外，你还可以使用字族号作为信号来指示使用当前字族。这样既可以为字符指定类别，又能让字族自适应。触发性字族通过 variablefam 设置。因此：

```
\variablefam"24
\Umathchardef\foo "3 "24 123
\foo \fam9
```

结果是一个取自字族 9 的花左括号，类别为 “relation，其周围间距也相应处理。

8.3　数学样式

8.3.1　mathstyle

可以用可展开方式探测公式将使用的数学样式（在数学表仍被读取时）。为此，LuaTEX 新增了原语：mathstyle。这是一个类似 romannumeral 的 “转换命令：其值只能读取，不能设置。

返回值在数学模式下为 0 到 7 之间，在其他模式下为 −1。为便于测试，八个数学样式命令已被修改为可用作数值，因此你可以编写如下代码：

```
\ifnum\mathstyle=\textstyle
    \message{normal text style}
\else \ifnum\mathstyle=\crampedtextstyle
    \message{cramped text style}
\fi \fi
```

有时你不会得到预期结果，稍作解释有助于理解发生了什么。当数学被解析和展开时，它会被转换为一个链表。第二趟才构造公式。这与以下事实有关：为了确定自动选择的字号（例如在分式中），后面的内容可以影响前面的字号。其副作用之一是不能修改字体字族的定义（从而重用编号），因为被使用的编号会被存储并在第二趟中使用（所以在公式中途改变 \fam 12 会影响到该字族此前的使用）。

像 textstyle 这样的样式切换原语会被转换为节点,因此其中设置的样式被冻结。mathchoice 原语会构造四个表，第二趟使用其中一个。某些自动样式尚不可知这一事实也意味着 mathstyle 原语展开为当前样式，它当然可能与实际使用的样式不同。它是第一趟状态的快照。因此在下面的示例中，你得到的是被排版的样式编号（第一趟），它实际上可能与所使用的样式（第二趟）不同。当数学选择在未分组的情况下使用时，所选样式在选择之后仍然生效，除非你进行了分组。

```
[a:\mathstyle]\quad
\{
\mathchoice
    {\bf \scriptstyle       (x:d :\mathstyle)}
    {\bf \scriptscriptstyle (x:t :\mathstyle)}
    {\bf \scriptscriptstyle (x:s :\mathstyle)}
    {\bf \scriptscriptstyle (x:ss:\mathstyle)}
\}
\quad[b:\mathstyle]\quad
\mathchoice
    {\bf \scriptstyle       (y:d :\mathstyle)}
    {\bf \scriptscriptstyle (y:t :\mathstyle)}
    {\bf \scriptscriptstyle (y:s :\mathstyle)}
    {\bf \scriptscriptstyle (y:ss:\mathstyle)}
\quad[c:\mathstyle]\quad
\{
\mathchoice
    {\bf \scriptstyle       (z:d :\mathstyle)}
    {\bf \scriptscriptstyle (z:t :\mathstyle)}
    {\bf \scriptscriptstyle (z:s :\mathstyle)}
```

```
        {\bf \scriptscriptstyle (z:ss:\mathstyle)}
    \}
    \quad[d:\mathstyle]

    [a:\mathstyle]\quad
    \begingroup
    \mathchoice
        {\bf \scriptstyle       (x:d :\mathstyle)}
        {\bf \scriptscriptstyle (x:t :\mathstyle)}
        {\bf \scriptscriptstyle (x:s :\mathstyle)}
        {\bf \scriptscriptstyle (x:ss:\mathstyle)}
    \endgroup
    \quad[b:\mathstyle]\quad
    \mathchoice
        {\bf \scriptstyle       (y:d :\mathstyle)}
        {\bf \scriptscriptstyle (y:t :\mathstyle)}
        {\bf \scriptscriptstyle (y:s :\mathstyle)}
        {\bf \scriptscriptstyle (y:ss:\mathstyle)}
    \quad[c:\mathstyle]\quad
    \begingroup
    \mathchoice
        {\bf \scriptstyle       (z:d :\mathstyle)}
        {\bf \scriptscriptstyle (z:t :\mathstyle)}
        {\bf \scriptscriptstyle (z:s :\mathstyle)}
        {\bf \scriptscriptstyle (z:ss:\mathstyle)}
    \endgroup
    \quad[d:\mathstyle]
```

结果如下（演示输出已省略）。

改用 `begingroup` 和 `endgroup` 则得到：

（演示输出已省略。）这看起来可能不对，但它只是 `mathstyle` 展开为当前（第一趟）样式而该编号被注入第二趟转换的表中所产生的副作用。这一切都有道理，也说明了分组的重要性。事实上，数学选择样式在之后仍然生效是有好处的。否则很难做到这一点。

8.3.2 Ustack

TEX 中有少数数学命令，其将要使用的样式并非从一开始就已知。这些命令（`over`、`atop`、`overwithdelims`、`atopwithdelims`）因此通常会为 `mathstyle` 返回错误值。为解决此问题，LuaTEX 引入了一个特殊前缀命令：`Ustack`：

```
$\Ustack {a \over b}$
```

`Ustack` 命令会扫描下一个花括号，并以正确的（分子）数学样式开始一个新的数学组。

8.3.3 压缩数学样式

LuaTEX 有四个新原语可直接设置压缩数学样式：

```
\crampeddisplaystyle
\crampedtextstyle
\crampedscriptstyle
\crampedscriptscriptstyle
```

这些附加命令本身并不那么有用，但作为即将添加的数学参数设置的参数时却很方便。

在 Eijkhout 的《TEX by Topic》中，关于上下标中样式处理的规则描述如下：

- 在任何样式中，上标和下标都取自下一个较小的样式。例外：在展示样式中它们使用脚本样式。
- 下标始终使用该样式的压缩变体；上标仅在原始样式为压缩时才压缩。
- 在..\over ..公式中，任何样式下分子和分母都取自下一个较小的样式。
- 分母始终使用压缩样式；分子仅在原始样式为压缩时才使用压缩样式。
- \sqrt 或 overline 下的公式使用压缩样式。

在 LuaTEX 中可以更细致地设置样式，这意味着有时必须同时设置普通样式和压缩样式才能获得所需效果。（即使）我们使用 scriptstyle 和 crampedscriptstyle 在脚本中强制样式，得到如下结果：

```
\begin{longtable}{l l}
\toprule style         & example \\
\midrule
default       & $b_{x=xx}^{x=xx}$ \\
script        & $b_{\scriptstyle x=xx}^{\scriptstyle x=xx}$ \\
crampedscript & $b_{\crampedscriptstyle x=xx}^{\crampedscriptstyle x=xx}$ \\
\bottomrule
\end{longtable}
```

现在我们设置以下参数：

```
\Umathordrelspacing\scriptstyle=30mu
\Umathordordspacing\scriptstyle=30mu
```

这产生不同的结果（演示输出已省略）。

但是，由于这并非（视觉上）所预期的，我们应该这样说：

```
% \Umathordrelspacing\scriptstyle=30mu
% \Umathordordspacing\scriptstyle=30mu
% \Umathordrelspacing\crampedscriptstyle=30mu
% \Umathordordspacing\crampedscriptstyle=30mu
```

现在我们得到（演示输出已省略）。

8.4　数学参数设置

8.4.1　大量新的 Umath* 原语

在 LuaTEX 中，TEX 在数学排版中使用的字体尺寸参数现在可通过原语命令访问。事实上，数学引擎的重构带来了比以前可访问参数多得多的参数。

primitive name	description
Umathquad	18 个 mu 的宽度

Umathaxis	数学公式垂直中心轴在基线以上的高度
Umathoperatorsize	展示模式中大算符的最小尺寸
Umathoverbarkern	横线上方的垂直间隙
Umathoverbarrule	横线的粗细
Umathoverbarvgap	横线下方的垂直间隙
Umathunderbarkern	横线下方的垂直间隙
Umathunderbarrule	横线的粗细
Umathunderbarvgap	横线上方的垂直间隙
Umathradicalkern	根号上方的垂直间隙
Umathradicalrule	根号线的粗细
Umathradicalvgap	根号线下方的垂直间隙
Umathradicaldegreebefore	放置根号次数之前的前向字距
Umathradicaldegreeafter	放置根号次数之后的后向字距
Umathradicaldegreeraise	次数相对于根号总高和深所提升的百分比；以 percents 表示，因此 60% 表示为整数 60
Umathstackvgap	atop 堆叠中两个元素之间的垂直间隙
Umathstacknumup	atop 堆叠中分子的上移量
Umathstackdenomdown	atop 堆叠中分母的下移量
Umathfractionrule	over 分式中横线的粗细
Umathfractionnumvgap	分子与横线之间的垂直间隙
Umathfractionnumup	over 中分子的上移量
Umathfractiondenomvgap	分母与横线之间的垂直间隙
Umathfractiondenomdown	over 中分母的下移量
Umathfractiondelsize	\...withdelims 的最小定界符尺寸
Umathlimitabovevgap	算符上方极限的垂直间隙
Umathlimitabovebgap	算符上方极限的基线垂直间隙
Umathlimitabovekern	极限顶部预留空间
Umathlimitbelowvgap	算符下方极限的垂直间隙
Umathlimitbelowbgap	算符下方极限的基线垂直间隙
Umathlimitbelowkern	极限底部预留空间
Umathoverdelimitervgap	定界符上方极限的垂直间隙
Umathoverdelimiterbgap	定界符上方极限的基线垂直间隙
Umathunderdelimitervgap	定界符下方极限的垂直间隙
Umathunderdelimiterbgap	定界符下方极限的基线垂直间隙
Umathsubshiftdrop	盒子和子公式的下标下移量
Umathsubshiftdown	字符的下标下移量
Umathsupshiftdrop	盒子和子公式的上标下移量（实际为提升量）
Umathsupshiftup	字符的上标提升量
Umathsubsupshiftdown	存在上标时的下标下移量
Umathsubtopmax	独立下标顶部在基线上方不能超过此高度

Umathsupbottommin	独立上标底部在基线上方不能低于此高度
Umathsupsubbottommax	同时有上下标时，上标底部在基线上方至少要达到此高度
Umathsubsupvgap	上下标之间的垂直间隙
Umathspaceafterscript	上下标之后添加的附加间距
Umathconnectoroverlapmin	可伸缩配方中各部分之间的最小重叠

本节中的每个参数都可以通过如下命令设置：

```
\Umathquad\displaystyle=1em
```

它们遵循分组规则，需要时可以使用 `\the \Umathquad \displaystyle` 。

8.4.2　基于字体的数学参数

虽然有这些数学参数可供调整很好，但手动设置每一个会很繁琐。因此，每当你将字体标识符赋给某个数学字族时，LuaTEX 会初始化一批参数——依据是字体中传统的数学字体尺寸（对于使用基于 TFM 的字体如 cmsy 和 cmex 赋值给数学字族 2 和 3 的情况），或者在通过 Lua 加载字体时依据潜在的 MathConstants 表中的命名值。如果存在 MathConstants 表，它优先于字体尺寸，此时不关心正在赋值给哪个字族：最后赋值字族中的 MathConstants 表会设置所有参数。

下表中的单字母样式缩写和符号化 tfm 字体尺寸名称与 TEXbook 中使用的一致。对 textfont 的赋值设置压缩和非压缩展示及文本样式的值，scriptfont 设置脚本样式，scriptscriptfont 设置脚本脚本样式，因此三种字号共有八个参数。在 TFM 情况下，赋值仅发生在字族 2 和字族 3 中（当然也仅针对有字体尺寸的参数）。

除以下参数外，LuaTEX 还会查看 “space 字体尺寸参数。对于数学字体，此项应设为零。

variable / style	tfm / opentype
\Umathaxis	axis_height AxisHeight
[6] \Umathoperatorsize D, D’	\emdash DisplayOperatorMinHeight
[9] \Umathfractiondelsize D, D’	delim1 FractionDelimiterDisplayStyleSize
[9] \Umathfractiondelsize T, T’, S, S’, SS, SS’	delim2 FractionDelimiterSize
\Umathfractiondenomdown D, D’	denom1 FractionDenominatorDisplayStyleShiftDown
\Umathfractiondenomdown T, T’, S, S’, SS, SS’	denom2 FractionDenominatorShiftDown

\Umathfractiondenomvgap D, D’	3*default_rule_thickness FractionDenominatorDisplayStyleGapMin
\Umathfractiondenomvgap T, T’, S, S’, SS, SS’	default_rule_thickness FractionDenominatorGapMin
\Umathfractionnumup D, D’	num1 FractionNumeratorDisplayStyleShiftUp
\Umathfractionnumup T, T’, S, S’, SS, SS’	num2 FractionNumeratorShiftUp
\Umathfractionnumvgap D, D’	3*default_rule_thickness FractionNumeratorDisplayStyleGapMin
\Umathfractionnumvgap T, T’, S, S’, SS, SS’	default_rule_thickness FractionNumeratorGapMin
\Umathfractionrule	default_rule_thickness FractionRuleThickness
\Umathskewedfractionhgap	math_quad/2 SkewedFractionHorizontalGap
\Umathskewedfractionvgap	math_x_height SkewedFractionVerticalGap
\Umathlimitabovebgap	big_op_spacing3 UpperLimitBaselineRiseMin
[1] \Umathlimitabovekern	big_op_spacing5 0
\Umathlimitabovevgap	big_op_spacing1 UpperLimitGapMin
\Umathlimitbelowbgap	big_op_spacing4 LowerLimitBaselineDropMin
[1] \Umathlimitbelowkern	big_op_spacing5 0
\Umathlimitbelowvgap	big_op_spacing2 LowerLimitGapMin
\Umathoverdelimitervgap	big_op_spacing1 StretchStackGapBelowMin
\Umathoverdelimiterbgap	big_op_spacing3 StretchStackTopShiftUp

\Umathunderdelimitervgap	big_op_spacing2 StretchStackGapAboveMin
\Umathunderdelimiterbgap	big_op_spacing4 StretchStackBottomShiftDown
\Umathoverbarkern	default_rule_thickness OverbarExtraAscender
\Umathoverbarrule	default_rule_thickness OverbarRuleThickness
\Umathoverbarvgap	3*default_rule_thickness OverbarVerticalGap
[1] \Umathquad	math_quad <font_size(f)>
\Umathradicalkern	default_rule_thickness RadicalExtraAscender
[2] \Umathradicalrule	<not set> RadicalRuleThickness
[3] \Umathradicalvgap D, D’	default_rule_thickness+abs(math_x_height)/4 RadicalDisplayStyleVerticalGap
[3] \Umathradicalvgap T, T’, S, S’, SS, SS’	default_rule_thickness+abs(default_rule_thickness)/4 RadicalVerticalGap
[2] \Umathradicaldegreebefore	<not set> RadicalKernBeforeDegree
[2] \Umathradicaldegreeafter	<not set> RadicalKernAfterDegree
[2,7] \Umathradicaldegreeraise	<not set> RadicalDegreeBottomRaisePercent
[4] \Umathspaceafterscript	script_space SpaceAfterScript
\Umathstackdenomdown D, D’	denom1 StackBottomDisplayStyleShiftDown
\Umathstackdenomdown T, T’, S, S’, SS, SS’	denom2 StackBottomShiftDown
\Umathstacknumup D, D’	num1 StackTopDisplayStyleShiftUp

\Umathstacknumup T, T', S, S', SS, SS'	num3 StackTopShiftUp
\Umathstackvgap D, D'	7*default_rule_thickness StackDisplayStyleGapMin
\Umathstackvgap T, T', S, S', SS, SS'	3*default_rule_thickness StackGapMin
\Umathsubshiftdown	sub1 SubscriptShiftDown
\Umathsubshiftdrop	sub_drop SubscriptBaselineDropMin
[8] \Umathsubsupshiftdown	\emdash SubscriptShiftDownWithSuperscript
\Umathsubtopmax	abs(math_x_height*4)/5 SubscriptTopMax
\Umathsubsupvgap	4*default_rule_thickness SubSuperscriptGapMin
\Umathsupbottommin	abs(math_x_height/4) SuperscriptBottomMin
\Umathsupshiftdrop	sup_drop SuperscriptBaselineDropMax
\Umathsupshiftup D	sup1 SuperscriptShiftUp
\Umathsupshiftup T, S, SS,	sup2 SuperscriptShiftUp
\Umathsupshiftup D', T', S', SS'	sup3 SuperscriptShiftUpCramped
\Umathsupsubbottommax	abs(math_x_height*4)/5 SuperscriptBottomMaxWithSubscript
\Umathunderbarkern	default_rule_thickness UnderbarExtraDescender
\Umathunderbarrule	default_rule_thickness UnderbarRuleThickness
\Umathunderbarvgap	3*default_rule_thickness UnderbarVerticalGap

[5]	\Umathconnectoroverlapmin	0
		MinConnectorOverlap

注 1：OpenType 字体将 Umathlimitabovekern 和 Umathlimitbelowkern 设为零，并将 Umathquad 设为所用字体的字号，因为这些在 MATH 表中不受支持。

注 2：传统 TFM 字体不设置 Umathradicalrule，因为 TEX82 改用根号的高度。当 LuaTEX 必须排版根式而此参数确实未设置时，会启用向后兼容模式，假设使用的是旧风格 TEX 字体。此外，它们也不设置 Umathradicaldegreebefore、Umathradicaldegreeafter 和 Umathradicaldegreeraise。这些参数会自动初始化为 5/18 quad、−10/18 quad 和 60。

注 3：如果使用 TFM 字体，Umathradicalvgap 在 LuaTEX 首次必须排版公式之前不会被设置，因为这需要字族 2 和字族 3 的参数。这提供了与 TEX82 的部分向后兼容性，但只是部分兼容：一旦 Umathradicalvgap 被设置，就不再重新计算。

注 4：使用 TFM 字体时，Umathspaceafterscript 也有类似情况：在 LuaTEX 首次必须排版公式之前不会被设置。这提供了一定的与 TEX82 的向后兼容性。但一旦 Umathspaceafterscript 被设置，就不再查看 scriptspace。

注 5：传统 TFM 字体将 Umathconnectoroverlapmin 设为零，因为 TEX82 堆叠可伸缩符号时始终不重叠。

注 6：Umathoperatorsize 仅在 displaystyle 中使用，且仅在 OpenType 字体中设置。在 TFM 字体模式下，它被人为地设置为比初始尝试尺寸大一个缩放点，以便始终尝试“下一个更大的变体，就像在 TEX82 中一样。

注 7：Umathradicaldegreeraise 是特例，因为它是唯一一个以百分比而非缩放点数表示的参数。

注 8：SubscriptShiftDownWithSuperscript 在“标准 OpenType 数学字体 Cambria 中实际并不存在，但它足够有用，所以被加入了。

注 9：FractionDelimiterDisplayStyleSize 和 FractionDelimiterSize 在“标准 OpenType 数学字体 Cambria 中实际并不存在，但足够有用，所以被加入了。

8.5　数学间距

8.5.1　行间环绕间距

行间数学由（可选的）mathsurround 间距环绕，但那是固定尺寸。现在新增了一个参数 mathsurroundskip。当设为非零值（或带有伸缩的零值）时，此参数将取代 mathsurround。通过使用附加参数而非改变 mathsurround 的性质，我们可以保持兼容。同时，通过 mathsurroundmode 增加了更多控制。此指令可取 6 个值，零为默认行为。

```
\mathsurround     10pt
\mathsurroundskip20pt
```

8.5.2　固定上下标

我们有三个参数用于这种固定锚定：

parameter	register
d	`Umathsubshiftdown`
u	`Umathsupshiftup`
s	`Umathsubsupshiftdown`

当我们将 `mathscriptsmode` 设为非零值时，这些参数会用于计算固定位置。这在例如化学排版中是需要的。你可以操作上述变量来获得不同效果。

mode	down	up	example
0	dynamic	dynamic	$CH_2 + CH_2^+ + CH_2^2$
1	d	u	$CH_2 + CH_2^+ + CH_2^2$
2	s	u	$CH_2 + CH_2^+ + CH_2^2$
3	s	$u + s - d$	$CH_2 + CH_2^+ + CH_2^2$
4	$d + (s - d)/2$	$u + (s - d)/2$	$CH_2 + CH_2^+ + CH_2^2$
5	d	$u + s - d$	$CH_2 + CH_2^+ + CH_2^2$

此参数的值遵循分组规则，但作用于整个当前公式。

8.5.3 Penalty：`mathpenaltiesmode`

仅在行间数学中，数学表中才会添加 penalty。你可以通过以下设置强制添加 penalty（即使在展示式数学中）：

```
\mathpenaltiesmode = 1
```

这个原语在 LuaTEX 中并非真正必要，因为你可以使用回调 `mlist_to_hlist`，通过以强制 penalty 调用常规例程来强制添加 penalty。然而，作为开放和控制的一部分，这个原语是有意义的。作为额外补充，我们还提供了两个附加 penalty：

```
\prebinoppenalty = -100 % example value
\prerelpenalty   =  900 % example value
```

它们默认为无穷大，表示不需要插入。设置后，它们会在 binop 或 rel noad 之前注入。这是一个实验性特性。

8.5.4 公式间距：`matheqnogapstep`

默认情况下，TEX 会在公式和编号之间添加一个 quad 间距。这是硬编码的。一个新原语可以控制这一点：

```
\matheqnogapstep = 1000
```

由于使用的是数学文本字体的 math quad 而非一个尺寸，我们使用步长来控制大小。值为零将抑制间隙。该步长除以 1000，这是 TEX 中模拟浮点因子的常用方式。

8.6　数学构造

8.6.1　重音符号处理

LuaTEX 在数学模式下同时支持顶部重音符号和底部重音符号，且数学重音符号会自动伸缩（当然，前提是重音符号所在字体支持此功能）。底部和组合重音符号以及固定宽度数学重音符号由 Umathaccent 后面的可选关键字控制。

Umathaccent 之后的关键字 bottom 表示需要底部重音符号，关键字 both 表示同时需要顶部和底部重音符号（在这种情况下当然需要指定两个重音符号）。

然后读取定义重音符号的三个整数。这组整数可以由 fixed 关键字前缀，以指示请求非伸缩变体（如果是两个重音符号，则此步骤重复）。

一个简单示例：

```
\Umathaccent both fixed 0 0 "20D7 fixed 0 0 "20D7 {example}
```

如果需要放置数学顶部重音符号，且被加着重号者是字符并具有非零 top_accent 值，则使用此值放置重音符号，而非 TEX82 使用的 skewchar 字距。

top_accent 值表示被加着重号者中的一条垂直线。重音符号将水平移动,使其自身的 top_accent 线与被加着重号者的线对齐。如果重音符号的 top_accent 值为零，则改用重音符号宽度的一半加上其斜体修正。

顶部重音符号的垂直位置取决于被加着重号者字体的 x_height（如 TEXbook 所述），但如果值为零且字体有 MathConstants 表，则改用 AccentBaseHeight。

底部重音符号的垂直位置正好在被加着重号者正下方，不进行任何修正。

可能的位置有 top、bottom、both 和 center。未给出位置时默认为 top。可以指定附加参数 fraction，后跟一个数字；例如值 1200 表示判据为核宽度的 1.2 倍。fraction 仅适用于逐步选择的形状，主要用于 overlay 位置。它也适用于其他位置，但那时涉及的是宽度。

8.6.2　根式扩展

新原语 Uroot 允许构造包含次数字段的根式 noad。其语法是 Uradical 的扩展：

```
\Uradical <fam integer> <char integer> <radicand>
\Uroot    <fam integer> <char integer> <degree> <radicand>
```

次数的位置由数学参数 Umathradicaldegreebefore、Umathradicaldegreeafter 和 Umathradicaldegree 控制。次数以 scriptscriptstyle 排版。

8.6.3　上标与下标

通过 Lua 加载的 OpenType 数学字体中的字符字段可以有一个 “mathkern 表。此表的格式与 fontloader 库返回的 “mathkern 表相同，不同之处在于所有高度和字距值必须以实际缩放点指定。

当需要在数学项旁放置上标或下标时，LuaTEX 会检查上标或下标与核是否都是简单字符项。如果是，且两个字符项的字体都是 OpenType 字体（而非传统 TEX 字体），则 LuaTEX 将使用 OpenType 数学算法来决定上标或下标的水平位置。

工作方式如下：

- 计算上下标的垂直位置。
- 默认水平位置紧贴基字符旁边。
- 对于上标，加上基字符的斜体修正。
- 对于上标，计算两个垂直值：上标（上移后）的底部和基字符的顶部。对于下标，两个值是（下移后）下标的顶部和基字符的底部。
- 对于这两个位置中的每一个：
  - 查找基字符在此高度的 math kern 值（对于下标位置，是 `bottom_right` 角；对于上标位置，是 `top_right` 角）
  - 查找上下标在此高度的 math kern 值（对于下标位置，是 `top_left` 角；对于上标位置，是 `bottom_left` 角）
  - 将找到的值相加得到初步结果。
- 要应用的水平字距是上一步两个结果中的较小者。

特定高度处的 math kern 值是由下一个更高高度和字距对指定的字距值，或者是字符中最高的那个（如果字符中没有足够高的值），或者就是零（如果字符完全没有 math kern 对）。

8.6.4　可伸缩符号上的上下标

原语 `Uunderdelimiter` 和 `Uoverdelimiter` 允许在自动可伸缩项上放置下标或上标，而 `Udelimiterunder` 和 `Udelimiterover` 允许将自动可伸缩项作为下标或上标放置在核上。输入：

```
$\Uoverdelimiter  0 "2194 {\hbox{\strut  overdelimiter}}$
$\Uunderdelimiter 0 "2194 {\hbox{\strut underdelimiter}}$
$\Udelimiterover  0 "2194 {\hbox{\strut  delimiterover}}$
$\Udelimiterunder 0 "2194 {\hbox{\strut delimiterunder}}$
```

渲染结果如下（演示输出已省略）。

将返回：

```
[2] [3] [4]
```

这些命令是为方便而提供的。在它们可用之前，你可以这样做：

```
\def\Umathcharclass{\directlua{tex.print(tex.getmathcode(token.scan_int())[1])}}
\def\Umathcharfam  {\directlua{tex.print(tex.getmathcode(token.scan_int())[2])}}
\def\Umathcharslot {\directlua{tex.print(tex.getmathcode(token.scan_int())[3])}}
```

8.6.5　末行

有一个新原语用于控制段落中间展示式数学中上一行计算中的超调。默认值为当前字体 em 宽度的 2 倍：

```
\predisplaygapfactor=2000
```

如果你希望末行长度独立于数学，即不想回到插入伪展示式数学公式以获取末行长度的 hack，以下方法通常也可行：

```
\def\lastlinelength{\dimexpr
    \directlua {tex.sprint (
        (nodes.dimensions(node.tail(tex.lists.page_head).list))
    )}sp
\relax}
```

8.7　数学模式

8.7.1　单字符数学命令的详细版本

LuaTEX 定义了六个新原语，它们与 ^、_、$ 和 $$ 功能相同：

primitive	explanation
Usuperscript	复制 ^ 的功能
Usubscript	复制 _ 的功能
Ustartmath	在非数学模式下使用时复制 $ 的功能
Ustopmath	在行间数学模式下使用时复制 $ 的功能
Ustartdisplaymath	在非数学模式下使用时复制 $$ 的功能
Ustopdisplaymath	在展示式数学模式下使用时复制 $$ 的功能

Ustopmath 和 Ustopdisplaymath 原语会检查当前数学模式是否正确（行间 vs. 展示），但你可以自由地将四个 mathon/mathoff 命令与显式美元符号混用。

8.7.2　脚本命令 Unosuperscript 和 Unosubscript

这两个命令产生上标和下标，但使用当前样式（渲染时的样式）。因此，

```
$
    x\Usuperscript  {1}\Usubscript  {2} =
    x\Unosuperscript{1}\Unosubscript{2} =
    x\Usuperscript  {1}\Unosubscript{2} =
    x\Unosuperscript{1}\Usubscript  {2}
$
```

结果为 script]。

8.7.3　非数学模式中允许的数学命令

命令 mathchar、Umathchar 以及由 mathchardef 或 Umathchardef 产生的控制序列在水平和垂直模式下也可接受。在这些情况下，使用所请求数学字族的 textfont。

8.8　附加特性

8.8.1　已废弃

自 TL 2026 起,以下原语被“忽略。二十年来它们未被使用,除了 ConTEXt 用于探索变体之外,但我们现在可以用不同方式做这件事。触发警告的废弃命令:`\mathoption` 、`\mathitalicssmode` 、`\mathnolimitsmode` 、`\mathscriptboxmode` 、`\mathscriptcharmode` 、`\mathdefaultsmode` 、`\mathrulethicknessmode` 、`\mathdelimitersmode` 。也不再使用的参数有 `\Umathnolimitsupfactor` 和 `\Umathnolimitsubfactor` 。

8.8.2　扁平化：`mathflattenmode`

TEX 数学引擎会折叠没有上下标且核为字符的 `ord` noad，这在 OpenType 模式下有一个副作用：会应用斜体修正（前提是它们已被启用）。

```
$V \mathbin{\mathbin{v}} V$\par
$V \mathord{\mathord{v}} V$\par
```

渲染结果为:

（演示输出已省略。）当我们将 `mathflattenmode` 设为 31 时得到:

（演示输出已省略。）如果你看不出区别，那么字体可能具有正确的字符尺寸，不需要斜体修正。对于 Latin Modern（至少到 2018 年），存在视觉差异。在这方面，此参数并不总是需要的,除非你无论如何都想要高效的数学表。

你可以通过将适当的数字加到模式参数值来影响扁平化。默认值为 1。

mode	class
1	ord
2	bin
4	rel
8	punct
16	inner

8.8.3　减少追踪

由于有相当多的数学相关参数和值,可以限制追踪。仅当 `tracingassigns` 和/或 `tracingrestores` 设为 2 或更高时才会追踪它们。

第 9 章 节点

9.1 Lua 节点表示

TEX 的节点在 Lua 中表示为带有可变字段集合的 userdata 对象。在下面的语法表中，此类 userdata 对象的类型表示为 <node>。

node.types() 当前的返回值为：.

lastnodetype 原语符合 e-TEX 规范。有效范围仍为 $[-1, 15]$，glyph 节点（以前称为 char 节点）编号为 0，而 ligature 节点映射为 7。这样宏包就可以使用与传统 e-TEX 中相同的符号名称。请注意，这些 e-TEX 节点编号与真实的内部编号不同，而且 e-TEX 节点类型多于 15 种。

你可以用 node.fields 请求字段列表，用 node.subtypes 请求有效的子类型。node.values 函数报告一些已使用的值。有效的参数有 dir、direction、glue、pdf_literal、pdf_action、pdf_window 和 color_stack。请注意，设置器通常期望传入一个数字，但这个辅助函数会给出一份哪些数字有意义的清单。出于实用考虑，pagestate 的值也会通过这个辅助函数报告。

9.2 主要文本节点

这些是构成实际排版命令的节点。无论节点类型如何，所有节点都包含少数几个共同字段，它们是：

字段	类型	说明
next	node	列表中的下一个节点，或 nil
id	number	该节点的类型（id）编号
subtype	number	该节点的 subtype 标识符

subtype 有时只是一个占位条目，因为并非所有节点都实际使用 subtype，但这样一来你可以确信所有节点都接受它作为合法字段名，这在节点列表遍历中通常很方便。在下面的表中，next 和 id 不再显式列出。

除了这三个字段外，几乎所有节点还有一个 attr 字段，另外还有一个名为 prev 的字段。最后这个字段始终存在，但只在显式请求时才初始化：当调用 node.slide() 函数时，它会在参数节点列表中把 prev 字段设置为反向指针。到目前为止，TEX 的大多数节点处理都会确保 prev 节点有效，但也可能有例外，尤其是当内部机制使用一个前置 temp 节点来临时存储状态时。

9.2.1 hlist 节点

字段	类型	说明
subtype	number	[字段列表]
attr	node	属性列表
width	number	盒子的宽度
height	number	盒子的高度
depth	number	盒子的深度
shift	number	垂直于字符前进方向的位移
glue_order	number	取值范围在 $[0, 4]$ 内的数字，表示弹性间距的次序
glue_set	number	计算出的弹性间距伸缩比
glue_sign	number	0 = normal，1 = stretching，2 = shrinking
head/list	node	该列表主体的第一个节点
dir	string	该盒子的方向，见 9.2.15

警告：切勿将节点列表赋给 head 字段，除非你确信其内部链接结构正确，否则可能导致错误。

注意：字段名 head 和 list 都有效。有时用 head 指代一个列表更自然，有时用 list 更自然。

9.2.2　vlist 节点

该节点与 hlist 类似，区别在于 “shift 是垂直于行前进方向的位移，且 “subtype 只有 0、4 和 5 三个值。

9.2.3　rule 节点

与传统 TEX 不同，LuaTEX 有更多 rule 子类型，因为我们还使用规则来存储可复用对象和图像。用户节点不可见，可以通过回调拦截。

字段	类型	说明
subtype	number	[字段列表]
attr	node	属性列表
width	number	规则的宽度，其中特殊值 −1073741824 用于 “running 弹性间距尺寸
height	number	规则的高度（可为负）
depth	number	规则的深度（可为负）
left	number	左端的位移（也从宽度中减去）
right	number	（从宽度中减去）

dir	string	该规则的方向，见 9.2.15
index	number	一个可选的索引，可被引用
transform	number	一个私有变量（也用于指定轮廓宽度）

left 和 right 键有些特殊（且属于实验性质）。当规则自动适配周围盒子宽度时，你可以通过设置 left 强制向右偏移。该值也会从宽度中减去，而宽度可能是引擎自身设置的值，并不完全受用户控制。right 同样从宽度中减去。这一切都在后端完成，因此它们不影响前端的计算（实际上自动设置也在后端完成）。对于垂直规则，left 影响高度，right 影响深度。在 TEX 端没有对应的接口（尽管我们可以为规则增加更多关键字，但那会使事情复杂化并带来速度损失）。不过，你完全可以用 Lua 构造一个规则节点，再写入 TEX 输入。outline 子类型只是一个方便的变体，transform 字段指定轮廓的宽度。

9.2.4　ins 节点

该节点与 insert 原语相关。

字段	类型	说明
subtype	number	插入类别
attr	node	属性列表
cost	number	与该插入相关的惩罚值
height	number	插入物的高度
depth	number	插入物的深度
head/list	node	该插入物主体的第一个节点

还有一组额外字段与相关的弹性间距有关：width、stretch、stretch_order、shrink 和 shrink_order。它们都是数字。

警告：切勿将节点列表赋给 head 字段，除非你确信其内部链接结构正确，否则可能导致错误。你可以改用 list（在函数中你常希望使用名称相近的局部变量，两个名字同样合理）。

9.2.5　mark 节点

该节点与 mark 原语相关。

字段	类型	说明
subtype	number	未使用
attr	node	属性列表
class	number	标记类别
mark	table	表示记号列表的表

9.2.6　adjust 节点

该节点来自 vadjust 原语。

字段	类型	说明
subtype	number	[字段列表]
attr	node	属性列表
head/list	node	调整后的材料

警告：切勿将节点列表赋给 head 字段，除非你确信其内部链接结构正确，否则可能导致错误。

9.2.7　disc 节点

discretionary 和 -、- 字符，以及断字机制都会产生这类节点。

字段	类型	说明
subtype	number	[字段列表]
attr	node	属性列表
pre	node	指向断字前文本的指针
post	node	指向断字后文本的指针
replace	node	指向不断字文本的指针
penalty	number	与该断行相关的惩罚值，通常为 hyphenpenalty 或 exhyphenpenalty

子类型编号 4 和 5 属于别处给出的 “of-f-ice 说明。这些 disc 节点有些特殊，因为在某个时刻它们还保存着关于断点和嵌套连字的信息。

pre、post 和 replace 字段在 Lua 端实际上是间接访问的，并且带有一个不为 nil 的 prev 指针。这意味着，当你改动这三个列表之首时，必须重新赋值，因为那样才能恢复正确的 prev 指针，即：

```
pre = d.pre
-- change the list starting with pre
d.pre = pre
```

否则你可能得到一个与内部现实不符的无效状态，LuaTEX 甚至可能直接崩溃。这也意味着，向前遍历例如 pre 是可以的，但向后遍历时你必须在 pre 处停下。而且你绝对不能去动 prev 指向的那个节点，原因很简单：它其实不是一个真正的节点，而是 disc 数据结构的一部分（所以再次释放它可能会让 LuaTEX 崩溃）。

9.2.8　math 节点

数学节点表示一个数学公式的边界，通常被包裹在 $ 之中。

字段	类型	说明
subtype	number	[字段列表]
attr	node	属性列表
surround	number	mathsurround 紧排的宽度

还有一组额外字段与相关的弹性间距有关：width、stretch、stretch_order、shrink 和 shrink_order。它们都是数字。

9.2.9　glue 节点

在传统 TEX 中，弹性间距（skip）几乎是唯一一种不是简单值的数据对象。当 TEX 在文本流中遇到空格时会插入它，hskip 和 vskip 也会插入它。表示一个 skip 弹性间距各组成部分的结构称为 glue_spec，它具有以下可访问字段：

字段	类型	说明
width	number	水平或垂直位移
stretch	number	额外的（正向）位移或伸长量
stretch_order	number	作用于伸长量的系数
shrink	number	额外的（负向）位移或收缩量
shrink_order	number	作用于收缩量的系数

某些弹性间距子类型的有效宽度取决于将外层盒子撑满其尺寸所需的伸长或收缩。例如在一段落中，行内通常有表示空格的弹性间距，它们通过伸长或收缩使内容恰好填满可用空间。effective_glue 函数接受一个 glue 节点和一个父节点（hlist 或 vlist），返回该弹性间距项的有效宽度。当你传入第三个参数 true 时，该值会被取整。

glue_spec 节点是一种特殊节点，用于在寄存器中存储一组弹性间距值。最初它们还被用于存储 glue 节点的属性（使用引用计数机制），但现在我们把这些属性直接保存在 glue 节点自身中，这给 Lua 提供了更干净的接口。

间接 spec 方式实际上是原始 TEX 代码中的一项优化。首先，它可以节省不少内存，因为所有这些变成 glue 的空格现在共享同一份规格（只需递增引用计数），而且零值判断也稍快一些，因为只需检查指针即可（对于实现了例如凸出排版的引擎来说这不再成立，因为在进行边界判断时我们确实需要确保零就是零）。另一个副作用是弹性间距规格是只读的，所以最终在 Lua 中使用它们时必须复制（对字段的每次赋值都可能产生一份新副本）。因此，共享带来的优势其实并不高（而且如今内存已不再是大问题，何况一个 glue 节点只比一个 spec 多几个内存字）。

字段	类型	说明
subtype	number	[字段列表]

attr	node	属性列表
leader	node	指向引导线所用盒子或规则的指针

此外还有 width、stretch、stretch_order、shrink 和 shrink_order 字段。注意，我们在水平和垂直弹性间距中都使用键名 width。这与 TEX 内部实现很契合，所以我们决定沿用这个命名。

一个普通的词间空格也会产生一个 spaceskip 子类型(它过去是 subtype 为零的 userskip)。

9.2.10　kern 节点

kern 命令会创建这类节点，但例如字体和数学机制也可能添加它们。

字段	类型	说明
subtype	number	[字段列表]
attr	node	属性列表
kern	number	固定的水平或垂直前进量

9.2.11　penalty 节点

penalty 命令就是生成这类节点的命令之一。

字段	类型	说明
subtype	number	[字段列表]
attr	node	属性列表
penalty	number	惩罚值

子类型仅作信息用途，TEX 本身并不使用它们。当你遇到一个 linebreakpenalty 时，需要记住它是 club、widow 以及其他相关惩罚值的累加结果。

9.2.12　glyph 节点

这些大概是最常用的节点，虽然你可以用例如 char 把它们压入当前列表，但当 TEX 认为某些输入是文本时，通常会替你完成这一工作。

字段	类型	说明
subtype	number	位域
attr	node	属性列表
char	number	该字符在字体中的索引
font	number	字体标识符
lang	number	语言标识符

left	number	冻结的 \lefthyphenmnin 值
right	number	冻结的 \righthyphenmnin 值
uchyph	boolean	冻结的 uchyph 值
components	node	指向连字组成部分的指针
xoffset	number	水平方向上的虚拟位移
yoffset	number	垂直方向上的虚拟位移
width	number	字符的（原始）宽度
height	number	字符的（原始）高度
depth	number	字符的（原始）深度
expansion_factor	number	待应用的扩展系数
data	number	供用户使用的通用字段（我们正好有这个空间）

width、height 和 depth 的值是只读的。expansion_factor 在段落构造器中赋值，在后端使用。

警告：切勿将节点列表赋给 components 字段，除非你确信其内部链接结构正确，否则可能导致错误。subtype 字段的有效位如下：

位	含义
0	character
1	ligature
2	ghost
3	left
4	right

关于 subtype 字段的详细说明，见 6.2。

expansion_factor 是作为字体与后端分离的一部分引入的。它是对扩展机制更高效实现进行大量实验的结果。早期版本的 LuaTEX 已经在后端通过缩放替换了多个字体实例，但与 pdfTEX 不同，在 LuaTEX 中我们现在还摆脱了前端的字体副本，代之以随 glyph 节点携带的扩展系数。除了更干净的实现方式外，这也是朝更好地分离前端与后端迈出的一步。

is_char 函数检查一个节点是否为 subtype 仍小于 256 的 glyph 节点。该函数可用于判断对一个 glyph 节点应用字体逻辑是否有意义。当节点不是 glyph 时返回 nil；当节点仍被标记为字符时返回一个字符编号；否则返回 false。当返回 nil 时，同时还会返回 id。is_glyph 变体不检查 subtype 是否小于 256，因此它要么返回字符值，要么返回 nil 加上 id。这些辅助函数并不总是比单独调用更快，但有时能让判断更可读。uses_font 辅助函数接受一个节点和一个字体 id，当 glyph 或 disc 节点引用该字体时返回 true。

9.2.13　boundary 节点

该节点与 noboundary、boundary、protrusionboundary 和 wordboundary 等原语相关。

字段	类型	说明
subtype	number	[字段列表]
attr	node	属性列表
value	number	0–255 为保留值

9.2.14　local_par 节点

该节点插入在段落开头。你不要对它做太多改动。

字段	类型	说明
attr	node	属性列表
pen_inter	number	局部行间惩罚值（来自 localinterlinepenalty）
pen_broken	number	局部断行惩罚值（来自 localbrokenpenalty）
dir	string	本段落的方向，见 9.2.15
box_left	node	localleftbox
box_left_width	number	localleftbox 的宽度
box_right	node	localrightbox
box_right_width	number	localrightbox 的宽度

警告：切勿将节点列表赋给 box_left 或 box_right 字段，除非你确信其内部链接结构正确，否则可能导致错误。

9.2.15　dir 节点

方向节点标记需要改变方向的正文片段，textdir 命令会生成它们。

字段	类型	说明
attr	node	属性列表
dir	string	方向（但见下文）
level	number	该方向 whatsit 的嵌套层级

方向说明符是由 T、B、R 和 L 组成的三个字母的组合。它们由三个独立项构成：

- 第一个是段落 “top（顶部）的方向
- 第二个是行 “start（起始端）的方向
- 第三个是 glyph “top（顶部）的方向

不过，只有四种组合被接受：TLT、TRT、RTT 和 LTL。在实际的 dir 节点内部，dir 的表示不是三个字母，而是数字的组合。打印时，方向用 + 或 - 表示，指示该值是从方向栈中压入还是弹出。

9.2.16 marginkern 节点

边距紧排由凸出排版产生。

字段	类型	说明
subtype	number	[字段列表]
attr	node	属性列表
width	number	紧排的前进量
glyph	node	所用的字形

9.3 数学 noad

这些就是所谓的 “noad，即专门与数学处理相关的节点。这些节点大多数都包含子节点，因此可能的字段列表实际上相当小。首先是子节点：

9.3.1 数学核心子节点

数学模式中的许多对象字段要么是特定族中的简单字符，要么是数学列表或节点列表。有四个相关的子节点表示这些情况（在下面的节点描述中，它们用 <kernel> 一词表示）。

这些子节点的 next 和 prev 字段未被使用。

9.3.2 math_char 与 math_text_char 子节点

字段	类型	说明
attr	node	属性列表
char	number	字符索引
fam	number	族号

math_char 是最简单的子节点字段，它包含单个字形对象的字符和族。math_text_char 是一个特殊情况，你通常不会遇到它，它在数学列表转换过程中临时出现（其唯一作用是抑制紧随其后的斜体修正量）。

9.3.3 sub_box 与 sub_mlist 子节点

字段	类型	说明
attr	node	属性列表

head/list	node	节点列表

这两种子节点类型用于附属列表项。对于 sub_box，head 指向一个 “普通的 vbox 或 hbox。对于 sub_mlist，head 指向一个尚未转换的数学列表。

警告：切勿将节点列表赋给 head 字段，除非你确信其内部链接结构正确，否则会触发错误。

9.3.4 delim 子节点

还有第五种子节点类型，专门用于分隔符字段。与前面一样，next 和 prev 字段未被使用。

字段	类型	说明
attr	node	属性列表
small_char	number	基础字符的字符索引
small_fam	number	基础字符的族号
large_char	number	下一个更大字符的字符索引
large_fam	number	下一个更大字符的族号

large_char 和 large_fam 字段可以为零，在那种情况下，为 small_fam 设置的字体应当通过将 small_char 扩展来提供大号版本。

9.3.5 数学核心节点

首先，是与简单数学对象相关的对象（TEX 书称之为 “atoms）：ord、op、bin、rel、open、close、punct、inner、over、under、vcent。它们都具有相同的字段，被合并为单一节点类型，通过不同的子类型加以区分。

有些 noad 带有一个选项字段。这个位集中的常用值为：

含义	位
set	0x08
internal	0x00 + 0x08
internal	0x01 + 0x08
axis	0x02 + 0x08
no axis	0x04 + 0x08
exact	0x10 + 0x08
left	0x11 + 0x08
middle	0x12 + 0x08
right	0x14 + 0x08
no sub script	0x21 + 0x08
no super script	0x22 + 0x08
no script	0x23 + 0x08

9.3.6　简单 noad 节点

字段	类型	说明
subtype	number	[字段列表]
attr	node	属性列表
nucleus	kernel node	基座
sub	kernel node	下标
sup	kernel node	上标
options	number	渲染选项位集

9.3.7　accent 节点

字段	类型	说明
subtype	number	[字段列表]
nucleus	kernel node	基座
sub	kernel node	下标
sup	kernel node	上标
accent	kernel node	上方重音
bot_accent	kernel node	下方重音
fraction	number	更大的步进判据（除以 1000）

9.3.8　style 节点

字段	类型	说明
style	string	包含所用样式

该字符串值有八种可能：display、text、script 或 scriptscript 之一。每一种都可以冠以 cramped 前缀。

9.3.9　choice 节点

字段	类型	说明
attr	node	属性列表
display	node	展示尺寸备选列表
text	node	文本尺寸备选列表
script	node	下标尺寸备选列表
scriptscript	node	双下标尺寸备选列表

警告：切勿将节点列表赋给 display、text、script 或 scriptscript 字段，除非你确信其内部链接结构正确，否则可能发生错误。

9.3.10　radical 节点

字段	类型	说明
subtype	number	[字段列表]
attr	node	属性列表
nucleus	kernel node	基座
sub	kernel node	下标
sup	kernel node	上标
left	delimiter node	
degree	kernel node	仅由 Uroot 设置
width	number	所需宽度
options	number	渲染选项位集

警告：切勿将节点列表赋给 nucleus、sub、sup、left 或 degree 字段，除非你确信其内部链接结构正确，否则可能触发错误。

9.3.11　fraction 节点

字段	类型	说明
attr	node	属性列表
width	number	（可选）分数线的宽度
num	kernel node	分子
denom	kernel node	分母
left	delimiter node	左侧符号
right	delimiter node	右侧符号
middle	delimiter node	中间符号
options	number	渲染选项位集

警告：切勿将节点列表赋给 num 或 denom 字段，除非你确信其内部链接结构正确，否则可能导致错误。

9.3.12　fence 节点

字段	类型	说明
subtype	number	[字段列表]
attr	node	属性列表

delim	delimiter node	分隔符规格
italic	number	斜体修正量
height	number	所需高度
depth	number	所需深度
options	number	渲染选项位集
class	number	与间距相关的类别

警告：其中一些字段会被渲染器使用，并可能在过程中被修改。

9.4　前端 whatsit

whatsit 节点有许多子类型，你可以通过运行 node.whatsits 来查询：.

其中一些是通用的，与输出模式无关，另一些则与所选后端 DVI 或 PDF 相关。这里我们讨论通用的前端节点。

9.4.1　open

字段	类型	说明
attr	node	属性列表
stream	number	TEX 的流 id 编号
name	string	文件名
ext	string	文件扩展名
area	string	文件所在区域（以后可能弃用）

9.4.2　write

字段	类型	说明
attr	node	属性列表
stream	number	TEX 的流 id 编号
data	string	表示待写出记号列表的字符串
value	table	表示待写出记号列表的表

9.4.3　close

字段	类型	说明
attr	node	属性列表
stream	number	TEX 的流 id 编号

9.4.4 user_defined

用户定义的 whatsit 节点只能由 Lua 代码创建和处理。实际上，它们是扩展机制的扩展。LuaTEX 引擎会直接跳过这类 whatsit，根本不会查看其内容。

字段	类型	说明
attr	node	属性列表
user_id	number	id 编号
type	number	值的类型
value	number	一个 Lua 数字
	node	一个节点列表
	string	一个 Lua 字符串
	table	一个 Lua 表

type 可以取六个不同值之一。该数字是类型名首字符的 ASCII 码值(所以你可以用 string.byte("l") 代替 108)。

值	含义	说明
97	a	属性列表（一个节点列表）
100	d	一个 Lua 数字
108	l	一个 Lua 值（表、数字、布尔值等）
110	n	一个节点列表
115	s	一个 Lua 字符串
116	t	以 Lua 表形式表示的 Lua 记号列表（三元组列表）

9.4.5 save_pos

字段	类型	说明
attr	node	属性列表

9.4.6 late_lua

字段	类型	说明
attr	node	属性列表
data	string or function	以 Lua 值形式存储的待写出信息

token	string	以记号列表形式存储的待写出信息
name	string	用于 Lua 错误报告的名称

data 与 string 的区别在于，在赋值时 data 字段会被转换为记号列表，参见作为 latelua 的用法。string 版本则被视为字面字符串。

当使用函数时，它会被调用，第一个参数是触发该调用的节点。

9.5　DVI 后端 whatsit

9.5.1　special

DVI 后端只有一种 whatsit，它只是把内容直接冲刷到输出文件。

字段	类型	说明
attr	node	属性列表
data	string	special 信息

9.6　PDF 后端 whatsit

9.6.1　pdf_literal 与 pdf_late_literal

字段	类型	说明
attr	node	属性列表
mode	number	该字面量的 “mode 设置
data	string	以 Lua 字符串形式存储的待写出信息
token	string	以记号列表形式存储的待写出信息

可能的 mode 值为：

值	关键字
0	origin
1	page
2	direct
3	raw
4	text

数字越高，检查越少，你就越容易遇到麻烦。尤其是 raw 变体可能产生糟糕的 PDF，所以你最好检查自己生成的内容。

9.6.2 pdf_refobj

字段	类型	说明
attr	node	属性列表
objnum	number	所引用的 PDF 对象编号

9.6.3 pdf_annot

字段	类型	说明
attr	node	属性列表
width	number	宽度（不用于计算）
height	number	高度（不用于计算）
depth	number	深度（不用于计算）
objnum	number	所引用的 PDF 对象编号
data	string	注释数据

9.6.4 pdf_start_link

字段	类型	说明
attr	node	属性列表
width	number	宽度（不用于计算）
height	number	高度（不用于计算）
depth	number	深度（不用于计算）
objnum	number	所引用的 PDF 对象编号
link_attr	table	链接属性记号列表
action	node	要执行的动作

9.6.5 pdf_end_link

字段	类型	说明
attr	node	

9.6.6 pdf_dest

字段	类型	说明
attr	node	属性列表

width	number	宽度（不用于计算）
height	number	高度（不用于计算）
depth	number	深度（不用于计算）
named_id	number	dest_id 是否为字符串值?
dest_id	number	目标 id
	string	目标名称
dest_type	number	目标类型
xyz_zoom	number	缩放系数（乘以 1000）
objnum	number	PDF 对象编号;对于结构引用，则为所链接结构元素的 PDF 对象编号

9.6.7 pdf_action

这是一类特殊项，只出现在 PDF start link 对象内部。

字段	类型	说明
action_type	number	所涉及动作的种类
action_id	number or string	记号列表引用或字符串
named_id	number	dest_id 和 struct_id 是否为字符串值?
file	string	目标文件名
new_window	number	目标的窗口状态
data	string	目标的名称
struct_id	nil	该动作不引用结构目标
	number	所引用结构目标的 id
	string	所引用结构目标的名称

有效的动作类型为:

值	含义
0	page
1	goto
2	thread
3	user

有效的窗口类型为:

值	含义
0	notset

1	new
2	nonew

9.6.8 pdf_thread

字段	类型	说明
attr	node	属性列表
width	number	宽度（不用于计算）
height	number	高度（不用于计算）
depth	number	深度（不用于计算）
named_id	number	tread_id 是否为字符串值?
tread_id	number	线程 id
	string	线程名称
thread_attr	number	额外的线程信息

9.6.9 pdf_start_thread

字段	类型	说明
attr	node	属性列表
width	number	宽度（不用于计算）
height	number	高度（不用于计算）
depth	number	深度（不用于计算）
named_id	number	tread_id 是否为字符串值?
tread_id	number	线程 id
	string	线程名称
thread_attr	number	额外的线程信息

9.6.10 pdf_end_thread

字段	类型	说明
attr	node	

9.6.11 pdf_colorstack

字段	类型	说明
attr	node	属性列表
stack	number	颜色栈 id 编号

command	number	要执行的命令
data	string	数据

9.6.12 pdf_setmatrix

字段	类型	说明
attr	node	属性列表
data	string	数据

9.6.13 pdf_save

字段	类型	说明
attr	node	属性列表

9.6.14 pdf_restore

字段	类型	说明
attr	node	属性列表

9.7 node

9.7.1 引言

node 库包含便于处理（节点）列表及其值的函数。它们允许你创建、修改、复制、删除和插入 LuaTEX 节点对象，即排版器内部的核心对象。

LuaTEX 节点在 Lua 中表示为 userdata，其元数据类型为 luatex.node。节点的各个组成部分可以通过命名字段访问。

每个节点至少有 next、id 和 subtype 三个字段：

- next 字段返回节点链表中下一个节点的 userdata 对象；如果没有下一个节点，则返回 nil。
- id 指示 TEX 的 “节点类型。出于效率考虑，id 字段取数值，但某些库函数也接受字符串值代替 id。
- subtype 是另一个数字。它通常给出关于特定 id 节点的进一步信息，但在处理 “whatsit 时最为重要，因为它们仅根据 subtype 加以区分。

其他可用字段取决于节点的 id（对于 “whatsit，则取决于 subtype）。

对 unset（对齐）节点的支持是部分的：它们可以从 Lua 代码中查询和修改，但不能创建。

节点之间可以相互比较，但：你实际上比较的是节点内存中的索引。这意味着相等性测试只在非常有限的条件下才可信。当两个节点之一已被释放和/或重新分配时，它将无法正确工作：在那种情况下会出现误判。

目前，节点的内存管理仍应由用户显式完成。节点不被 Lua 垃圾回收器“看见，所以当你不再需要某个节点（列表）时，必须自行调用节点释放函数。节点构成不带引用计数的链表，因此你必须小心：当控制返回到 LuaTEX 自身时，你没有删除仍被其他地方的 next 指针引用的节点，也没有创建被多次引用的节点。通常设置器和获取器会替你处理这些。

关于已分配的节点内存有一些统计信息，便于跟踪调试。

9.7.2　is_node

```
<boolean|integer> t =
    node.is_node(<any> item)
```

如果参数是 <node> 类型的 userdata 对象，该函数返回一个数字（节点的内部索引）；若未传入节点则返回 false。

9.7.3　types 与 whatsits

该函数返回一个数组，将节点 id 编号映射到节点类型字符串，给出可能的顶层 id 类型概览。

```
<table> t =
    node.types()
```

TEX 的“whatsit 都具有相同的 id。各种子类型由其 subtype 字段定义。该函数与 types 很相似，只是它提供的是 subtype 映射数组。

```
<table> t =
    node.whatsits()
```

9.7.4　id

该函数将单个类型名转换为其内部数值表示。

```
<number> id =
    node.id(<string> type)
```

9.7.5　type 与 subtype

如果参数是数字，则下一个函数将内部数值表示转换为外部字符串表示。否则，如果该对象表示一个节点，它将返回字符串 node；否则返回 nil。

```
<string> type =
    node.type(<any> n)
```

下一个函数将单个 whatsit 名称转换为其内部数值表示（subtype）。

```
<number> subtype =
    node.subtype(<string> type)
```

9.7.6 fields

该函数返回某个特定类型节点的合法字段名数组。如果你想获取 “whatsit 的合法字段，则必须同时提供第二个参数。在其他情况下，任何给出的第二个参数都会被静默忽略。

```
<table> t =
    node.fields(<number> id)
<table> t =
    node.fields(<number> id, <number> subtype)
```

该函数也接受字符串 id 和 subtype 值。

9.7.7 has_field

该函数返回一个布尔值，只有当 n 确实是一个节点、并且它具有该字段时才为真。

```
<boolean> t =
    node.has_field(<node> n, <string> field)
```

9.7.8 new

new 函数创建一个新节点。除 id 和 subtype 外，其所有字段都被初始化为零或 nil。你也可以使用字符串（名称）代替数字。如果创建新的 whatsit 节点，第二个参数是必需的。与所有节点函数一样，该函数在 TEX 层创建节点。

```
<node> n =
    node.new(<number> id)
<node> n =
    node.new(<number> id, <number> subtype)
```

9.7.9 free、flush_node 与 flush_list

下一个函数将节点 n 从 TEX 的内存中移除。注意：它不检查该节点是否仍被某个寄存器或某个 next 字段指向：确保内部数据结构保持正确取决于你。

```
<node> next =
    node.free(<node> n)
flush_node(<node> n)
```

free 函数返回被释放节点的 next 字段，而 flush_node 变体不返回任何内容。

以节点 n 开头的列表也可以从 TEX 的内存中冲刷掉。注意：它不检查这些节点中是否仍有任何一个被某个寄存器或某个 next 字段指向：确保内部数据结构保持正确取决于你。

```
node.flush_list(<node> n)
```

9.7.10　copy 与 copy_list

该函数创建节点 n 的深拷贝，包括所有嵌套列表（如 hlist 或 vlist 节点的情况）。只有 next 字段不被复制。

```
<node> m =
    node.copy(<node> n)
```

也可以创建从 n 开始的节点列表的深拷贝。如果同时给出 m，则拷贝在节点 m 之前停止。

```
<node> m =
    node.copy_list(<node> n)
<node> m =
    node.copy_list(<node> n, <node> m)
```

注意，你不能用这种方式复制属性列表。不过，通常没有必要复制属性列表，因为当你对 attr 字段赋值或修改特定属性时，所需的复制和释放会自动进行。

9.7.11　prev 与 next

这两个函数返回给定节点之前或之后的节点，如果不存在这样的节点则返回 nil。

```
<node> m =
    node.next(<node> n)
<node> m =
    node.prev(<node> n)
```

9.7.12　current_attr

如果存在当前活动的属性列表，该函数返回它。

```
<node> m =
    node.current_attr()
```

current_attr 的预期用法如下：

```
local x1 = node.new("glyph")
x1.attr = node.current_attr()
local x2 = node.new("glyph")
x2.attr = node.current_attr()
```

或者：

```
local x1 = node.new("glyph")
local x2 = node.new("glyph")
local ca = node.current_attr()
x1.attr = ca
x2.attr = ca
```

属性列表是引用计数的，赋值会负责递增引用计数。当你赋值属性（使用 tex.setattribute）或控制已交回给 TEX 时，不能再指望值 ca 仍然有效。

注意：该函数有些实验性质，它返回的是实际的属性列表本身，而不是其副本。因此，修改列表中任何属性都会改变所有被赋了当前属性列表的节点的这些值。

9.7.13　hpack

该函数通过将从节点 n 开始的列表打包成水平盒子来创建新的 hlist。只带一个参数时，该盒子使用其各组成部分的自然宽度创建。在三参数形式中，info 必须是 additional 或 exactly，而 w 是所用的附加（\hbox spread）或精确（\hbox to）宽度。第二个返回值是生成盒子的坏度。

```
<node> h, <number> b =
    node.hpack(<node> n)
<node> h, <number> b =
    node.hpack(<node> n, <number> w, <string> info)
<node> h, <number> b =
    node.hpack(<node> n, <number> w, <string> info, <string> dir)
```

注意：该函数可能有意外副作用，比如更新某些 marks 和 \inserts 。还要注意，h 的内容就是原始节点列表 n：如果你调用 node.free(h)，也会释放该节点列表本身，除非你事先显式将 list 字段设为 nil。同样地，调用 node.free(n) 也会使 h 失效！

9.7.14　vpack

该函数通过将从节点 n 开始的列表打包成垂直盒子来创建新的 vlist。只带一个参数时，该盒子使用其各组成部分的自然高度创建。在三参数形式中，info 必须是 additional 或 exactly，而 w 是所用的附加（\vbox spread）或精确（\vbox to）高度。

```
<node> h, <number> b =
    node.vpack(<node> n)
<node> h, <number> b =
    node.vpack(<node> n, <number> w, <string> info)
<node> h, <number> b =
    node.vpack(<node> n, <number> w, <string> info, <string> dir)
```

第二个返回值是生成盒子的坏度。关于一些内存分配注意事项，见 hpack 的描述。

9.7.15　prepend_prevdepth

该函数有些特殊，它是一个实验性辅助函数，将行距加到一行上，同时把 baselineskip 和 lineskip 考虑在内。

```
<node> n, <number> delta =
    node.prepend_prevdepth(<node> n,<number> prevdepth)
```

9.7.16　dimensions 与 rangedimensions

```
<number> w, <number> h, <number> d  =
    node.dimensions(<node> n)
<number> w, <number> h, <number> d  =
    node.dimensions(<node> n, <string> dir)
<number> w, <number> h, <number> d  =
    node.dimensions(<node> n, <node> t)
<number> w, <number> h, <number> d  =
    node.dimensions(<node> n, <node> t, <string> dir)
```

该函数计算从节点 n 开始、到节点 t 之前结束的节点列表的自然行内尺寸（如果没有第二个参数，则到列表末尾）。返回值为缩放点。也有一种以前三个参数为弹性间距参数的替代形式：

```
<number> w, <number> h, <number> d  =
    node.dimensions(<number> glue_set, <number> glue_sign, <number> glue_order,
        <node> n)
<number> w, <number> h, <number> d  =
    node.dimensions(<number> glue_set, <number> glue_sign, <number> glue_order,
        <node> n, <string> dir)
<number> w, <number> h, <number> d  =
    node.dimensions(<number> glue_set, <number> glue_sign, <number> glue_order,
        <node> n, <node> t)
<number> w, <number> h, <number> d  =
    node.dimensions(<number> glue_set, <number> glue_sign, <number> glue_order,
        <node> n, <node> t, <string> dir)
```

这种调用方式会把弹性间距设置考虑在内，对于查找已装盒节点子列表的实际宽度特别有用，例如在如下代码中，它打印 a 与 b 之间空格的宽度，就像 \box 0 原样使用时那样：

```
\setbox0 = \hbox to 20pt {a b}

\directlua{print (node.dimensions(
    tex.box[0].glue_set,
    tex.box[0].glue_sign,
    tex.box[0].glue_order,
    tex.box[0].head.next,
    node.tail(tex.box[0].head)
)) }
```

你需要记住,这是 TEX 中少数使用浮点数的地方之一,这意味着当你比较 hpack 与 dimensions 报告的宽度时，可能会得到微小的取整差异。

第二种替代形式省去了几次查找，在某些情况下更方便：

```
<number> w, <number> h, <number> d  =
    node.rangedimensions(<node> parent, <node> first)
<number> w, <number> h, <number> d  =
    node.rangedimensions(<node> parent, <node> first, <node> last)
```

9.7.17　mlist_to_hlist

```
<node> h =
    node.mlist_to_hlist(<node> n, <string> display_type, <boolean> penalties)
```

该函数运行内部的 mlist 到 hlist 转换，将 n 中的数学列表转换为水平列表 h。其接口与回调 mlist_to_hlist 完全相同。

9.7.18　slide

```
<node> m =
    node.slide(<node> n)
```

返回从 n 开始的节点列表的最后一个节点。作为副作用，它还会在节点之间创建一条由 prev 指针构成的反向链。

9.7.19　tail

```
<node> m =
    node.tail(<node> n)
```

返回从 n 开始的节点列表的最后一个节点。

9.7.20　length 与 count

```
<number> i =
    node.length(<node> n)
<number> i =
    node.length(<node> n, <node> m)
```

返回从 n 开始的节点列表所包含的节点数。如果同时给出 m，则在 m 处停止，而不是到列表末尾。节点 m 不计入。

```
<number> i =
    node.count(<number> id, <node> n)
<number> i =
    node.count(<number> id, <node> n, <node> m)
```

返回从 n 开始的节点列表中、具有匹配 id 字段的节点数。如果同时给出 m，则计数在 m 处停止，而不是到列表末尾。节点 m 不计入。该函数也接受字符串 id。

9.7.21　is_char 与 is_glyph

glyph 节点的子类型指示该字形是否已经被转换为字符引用。

```
<boolean> b =
    node.is_char(<node> n)
<boolean> b =
    node.is_glyph(<node> n)
```

9.7.22　traverse

```
<node> t, id, subtype =
    node.traverse(<node> n)
```

这是一个 Lua 迭代器，遍历从 n 开始的节点列表。典型代码如下所示：

```
for n in node.traverse(head) do
   ...
end
```

在功能上等价于：

```
do
  local n
  local function f (head,var)
    local t
    if var == nil then
```

```
        t = head
      else
        t = var.next
      end
      return t
    end
    while true do
      n = f (head, n)
      if n == nil then break end
      ...
    end
  end
```

从函数 f 的定义可以清楚地看出，尽管在遍历过程中可以向节点列表添加或移除节点，但你必须非常小心，确保所有 next（以及 prev）指针保持有效。

如果你对上面的内容不清楚，请参阅 Lua 参考手册中的 “For Statement 一节。

9.7.23　traverse_id

```
<node> t, subtype =
    node.traverse_id(<number> id, <node> n)
```

这是一个迭代器，遍历从 n 开始的列表中所有具有匹配 id 字段的节点。

详见上一节。区别在于局部函数 f，它现在多了一个 while 循环，对照上值 id 进行检查：

```
 local function f(head,var)
   local t
   if var == nil then
      t = head
   else
      t = var.next
   end
   while not t.id == id do
      t = t.next
   end
   return t
 end
```

9.7.24　traverse_char 与 traverse_glyph

traverse_char 迭代器遍历列表中的 glyph 节点。只看子类型小于 256 的节点。

```
<node> n, char, font =
    node.traverse_char(<node> n)
```

traverse_glyph 迭代器遍历一个列表，返回列表并过滤所有字形：

```
<node> n, char, font =
    node.traverse_glyph(<node> n)
```

9.7.25　traverse_list

该迭代器遍历列表中的 hlist 和 vlist 节点。

```
<node> n, id, subtype, list =
    node.traverse_list(<node> n)
```

与逐个获取这些字段相比，这四个返回值可以节省一些时间，但实际上你很少需要它们全部。所以把它看作一项（实验性的副作用带来的）便利。

9.7.26　has_glyph

该函数返回给定列表中的第一个 glyph 或 disc 节点：

```
<node> n =
    node.has_glyph(<node> n)
```

9.7.27　end_of_math

```
<node> t =
    node.end_of_math(<node> start)
```

查找并返回 start 之后的下一个 math_node。如果给定节点是一个数学结束节点，该辅助函数返回该节点；否则它沿列表前进，返回下一个数学结束节点。如果未找到这样的节点，则返回 nil。

9.7.28　remove

```
<node> head, current =
    node.remove(<node> head, <node> current)
```

该函数从 head 之后的列表中移除节点 current。确保它确实是该列表的一部分取决于你。返回值是新的 head 和 current 节点。返回的 current 是调用参数中 current 之后的节点，传回来只是为了方便（如果没有这样的节点，则为 nil）。返回的 head 更为重要，因为如果调用时 current 等于 head，它会被改变。

9.7.29　insert_before

```
<node> head, new =
    node.insert_before(<node> head, <node> current, <node> new)
```

该函数将节点 new 插入到 head 之后的列表中、current 之前。确保 current 确实是该列表的一部分取决于你。返回值是（可能已改变的）head 和节点 new，后者已被设置为列表的一部分（带有正确的 next 字段）。如果 head 初始为 nil，它将变为 new。

9.7.30　insert_after

```
<node> head, new =
    node.insert_after(<node> head, <node> current, <node> new)
```

该函数将节点 new 插入到 head 之后的列表中、current 之后。确保 current 确实是该列表的一部分取决于你。返回值是 head 和节点 new，后者已被设置为列表的一部分（带有正确的 next 字段）。如果 head 初始为 nil，它将变为 new。

9.7.31　first_glyph

```
<node> n =
    node.first_glyph(<node> n)
<node> n =
    node.first_glyph(<node> n, <node> m)
```

返回从 n 开始的列表中、第一个子类型指示为字形的 glyph 节点，或 nil。如果给出 m，处理在该节点处停止（含该节点），否则处理在列表末尾停止。

9.7.32　ligaturing

```
<node> h, <node> t, <boolean> success =
    node.ligaturing(<node> n)
<node> h, <node> t, <boolean> success =
    node.ligaturing(<node> n, <node> m)
```

对指定节点列表应用 TEX 风格的连字处理。尾节点 m 是可选的。返回的两个节点 h 和 t 是新的头和尾（n 和 m 都可能变成一个新的连字）。

9.7.33　kerning

```
<node> h, <node> t, <boolean> success =
    node.kerning(<node> n)
<node> h, <node> t, <boolean> success =
    node.kerning(<node> n, <node> m)
```

对指定节点列表应用 TEX 风格的紧排。尾节点 m 是可选的。返回的两个节点 h 和 t 是头和尾（其中任何一个都可能是插入的 kern 节点，因为在词边界处可能有特殊紧排）。

9.7.34　unprotect_glyph[s]

```
node.unprotect_glyph(<node> n)
node.unprotect_glyphs(<node> n,[<node> ,m])
```

从所有 glyph 节点的子类型中减去 256。这个函数和下一个函数是在节点处理过程中从 characters 转换为 glyphs 的辅助函数。第二个参数是可选的，指示一个范围的终点。

9.7.35　protect_glyph[s]

```
node.protect_glyph(<node> n)
node.protect_glyphs(<node> n,[<node> m])
```

从 n 开始的节点列表中，给所有 glyph 节点的子类型加 256，但如果值为 1，则只加 255。对 1 的特殊处理意味着，characters 在减去 256 后会变成 glyphs。单个字符可以用单数形式调用标记。第二个参数是可选的，指示一个范围的终点。

9.7.36　last_node

```
<node> n =
    node.last_node()
```

该函数从 TEX 的 “current list 中弹出最后一个节点。它返回该节点；如果当前列表为空，则返回 nil。

9.7.37　write

```
node.write(<node> n)
```

该函数将一个节点列表附加到 TEX 的 “current list。该节点列表不进行深拷贝！也没有错误检查！你可能需要强制进入水平模式，才能让它按预期工作。

9.7.38　protrusion_skippable

```
<boolean> skippable =
    node.protrusion_skippable(<node> n)
```

当字符凸出排版启用时，为了行边界查找的目的，如果该节点可以被跳过，则返回 true。

9.8　弹性间距处理

9.8.1　setglue

你可以一次性设置一个弹性间距的五个属性。非数字值等价于零，并重置一个属性。

```
node.setglue(<node> n)
node.setglue(<node> n,width,stretch,shrink,stretch_order,shrink_order)
```

当你传入值时，只有数字类型的参数会被赋值，所以

```
node.setglue(n,655360,false,65536)
```

将只调整 width 和 shrink。
当传入一个列表节点时，你改为设置它的弹性间距、次序和符号。

9.8.2 getglue

下一个调用会返回 5 个值；如果未传入弹性间距，则什么也不返回。

```
<integer> width, <integer> stretch, <integer> shrink, <integer> stretch_order,
    <integer> shrink_order = node.getglue(<node> n)
```

当第二个参数为 false 时，只返回 width（这与 tex.get 一致）。
当传入一个列表节点时，你取回的是所设置的弹性间距、该弹性间距的次序以及符号。

9.8.3 is_zero_glue

当 width、stretch 和 shrink 属性都为零时，该函数返回 true。

```
<boolean> isglue =
    node.is_zero_glue(<node> n)
```

9.9 属性处理

9.9.1 属性

新引入的属性寄存器并不简单，因为附在节点上的值本质上是一个键值对的稀疏数组。通常最容易的做法是使用 node 库中的专用函数来处理属性列表和属性，但为完整起见，这里给出底层接口。

属性在单个节点的 attr 字段中以 userdata 对象链表的形式出现。它们可以被单独处理，但使用与之相关的专用函数要安全和高效得多。

9.9.2 attribute_list 节点

一个 attribute_list 项用作属性项列表的头指针。它只有一个用户可见字段：

字段	类型	说明
next	node	指向第一个属性的指针

9.9.3 attr 节点

普通节点的属性字段将指向一个 attribute_list 类型的项，而该项的 next 字段指向第一个已定义的 “attribute 项，后者的 next 指向第二个 “attribute 项，依此类推。

字段	类型	说明
next	node	指向下一个属性的指针
number	number	属性类型 id
value	number	属性值

如前所述，最好使用官方辅助函数，而不是直接编辑这些字段。例如，prev 字段被用于其他目的，这里没有双向链表。

9.9.4　has_attribute

```
<number> v =
    node.has_attribute(<node> n, <number> id)
<number> v =
    node.has_attribute(<node> n, <number> id, <number> val)
```

测试一个节点是否设置了编号为 id 的属性。如果同时给出 val，还测试值是否与 val 匹配。它返回该值；如果未找到匹配，则返回 nil。

9.9.5　get_attribute

```
<number> v =
    node.get_attribute(<node> n, <number> id)
```

测试一个节点是否设置了编号为 id 的属性。它返回该值；如果未找到匹配，则返回 nil。如果未给出 id，则假定为零号属性。

9.9.6　find_attribute

```
<number> v, <node> n =
    node.find_attribute(<node> n, <number> id)
```

查找第一个设置了编号为 id 属性的节点。如果有匹配，它返回该值和节点；否则什么也不返回。

9.9.7　set_attribute

```
node.set_attribute(<node> n, <number> id, <number> val)
```

将编号为 id 的属性设置为值 val。重复赋值会被忽略。

9.9.8　unset_attribute

```
<number> v =
    node.unset_attribute(<node> n, <number> id)
<number> v =
    node.unset_attribute(<node> n, <number> id, <number> val)
```

取消设置编号为 id 的属性。如果同时给出 val，则只有当值与 val 匹配时才执行此操作。缺失的属性或属性值对会被忽略。

如果该属性确实被删除，返回其旧值。否则返回 nil。

9.9.9　slide

该辅助函数确保节点列表是双向链接的，并返回找到的尾节点。

```
<node> tail =
    node.slide(<node> n)
```

在某些回调之后会自动进行滑动。可以用 `node.fix_node_lists(false)` 关闭此功能，但那样你最好确保自己没有搞乱列表。在大多数情况下，TEX 本身只使用 `next` 指针，但你的其他回调可能也期望正确的 `prev` 指针。未来的 LuaTEX 版本可以增加更多检查，但这不会影响用法。

9.9.10　check_discretionary、check_discretionaries

当你随意摆弄 disc 节点时，需要注意它们有一种特殊的内部数据结构。只要你在扩展列表后重新赋值这些字段就没问题，因为那时尾指针会被更新；但如果你在没有重新赋值的情况下向列表添加内容，当断行例程介入时，就可能陷入麻烦。你可以调用该函数检查列表中 disc 节点的问题。

```
node.check_discretionary(<node> n)
node.check_discretionaries(<node> head)
```

复数形式遍历一个列表中的所有 disc 节点，单数形式只检查一个节点（它还会检查该节点是否为 disc 节点）。

9.9.11　flatten_discretionaries

该函数会移除列表中的断字点，并在 replace 字段被设置时注入它。

```
<node> head, count = node.flatten_discretionaries(<node> n)
```

9.9.12　family_font

当你传入一个正确的族标识符时，下一个辅助函数会返回当前与它关联的字体。你通常也可以用普通的字体字段或获取器访问该字体，因为对于 noad，它会自动解析族。

```
<integer> id =
    node.family_font(<integer> fam)
```

9.10　两种访问模型

在 TEX 的底层，一个节点有一个数字，它是内存表中的一个数值项。实际上，这种由 TEX 管理内存的模型非常快，这也是为什么在节点上操作的回调插进来也相当快的原因之一。每个节点都得到一个数字，它实际上是内存表中的索引，当你打印节点相关信息时，经常会报告这个数字。你可以在 userdata 节点和它们的数值引用之间双向转换：

```
<integer> d = node.direct.todirect(<node> n))
<node> n = node.direct.tonode(<integer> d))
```

userdata 模型相当健壮，因为它是一个带有额外检查的虚拟接口，而更直接的访问则直接使用节点编号。不过，即使用 userdata，当你释放已不再分配的节点或搞乱列表时，也会遇到麻烦。如果你对节点应用 `tostring`，就会看到它的内部（直接）编号和 id。

第一种模型提供基于键的访问，而第二种总是通过函数访问字段：

```
nodeobject.char
getfield(nodenumber,"char")
```

如果你使用直接模型，即使知道自己处理的是数字，也不应依赖这一性质，而应像对待传统节点一样把它当作一种抽象。实际上，我们使用简单基本数据类型的代价是能做的检查更少，但检查更少也是它稍快的原因。一个重要的方面是，两种方法不能混用，但你可以对两种模型进行强制转换。所以，把一个节点编号相乘没有任何意义。

因此我们的建议是：尽可能使用索引（表）方式，只有在速度确实成为问题时才研究直接方式。出于这个原因，LuaTEX 还在顶层 node 命名空间中提供了 `get*` 和 `set*` 函数。获取器数量有限。在实现这种直接方式时，按键索引的常规变体也得到了优化，所以直接访问只有在节点被访问数百万次时才有意义（例如在某些字体处理中会发生）。

我们主要谈获取器，因为设置器不那么重要。文档中与内容相关的节点没有那么多，设置成千上万属性与数百万次查询相比几乎不算负担。

通常你会这样访问节点：

```
local next = current.next
if next then
    -- do something
end
```

这里 `next` 不是真正的字段，而是一个虚拟字段。访问它会导致调用一个元表方法。实际上它归结为查找节点类型，然后根据节点类型检查字段名。最坏情况下，你会遇到一个排在查找表末尾的节点类型，以及一个排在查找链最后的字段。不过，在 LuaTEX 的后续版本中，这些查找已经得到优化，最常访问的节点和字段具有更高优先级。

因为在实践中 `next` 访问器会导致一次函数调用，所以有一些开销。下面这段代码做同样的事，并且稍快一点点（但快不了多少，因为它仍然是一次函数调用，只不过这一次它知道要查找什么）。

```
local next = node.next(current)
if next then
    -- do something
end
```

有些访问器使用频繁，我们为它们提供了更高效的辅助函数：

函数	说明
`getnext`	解析节点列表总是涉及它
`getprev`	使用较少，但作为 `getnext` 的逻辑配对
`getboth`	返回一个节点的 next 和 prev 指针
`getid`	被频繁查询
`getsubtype`	查询较少，但也名列前茅

getfont	在 OpenType 处理中大量使用（glyph 节点被频繁查询）
getchar	同上，在其他地方也用
getwhd	返回列表、规则或（未展开）字形以及弹性间距的 width、height 和 depth（其 spec 会被查看），还有 unset 节点
getdisc	返回 pre、post 和 replace 字段，可选地在传入 true 时也返回尾字段
getlist	我们经常解析嵌套列表，所以这也很方便
getleader	与 list 类似，在 TEX 中很少用（但像列表一样需要频繁查询；引导线本可以做成一种专用节点类型）
getfield	通用获取器，其余字段足够使用（其他字段名常常是共享的，所以专门的获取器就没意义了）
getbox	获取给定的盒子（一个列表节点）
getoffsets	获取字形的 xoffset 和 yoffset，或规则的 left 和 right 值

在直接命名空间中还有更多这类辅助函数，其中大多数都配有对应的设置器。获取器和设置器足够聪明，能判断指的是哪个节点。我们不处理 whatsit 节点：它们的字段总是按名字访问。为所有字段添加获取器没有意义，我们只识别最可能用到的候选者。在复杂文档中，许多节点和字段类型根本不会被看到，或只被看到几次，但例如字形就是这种优化的候选者。node.direct 接口还有一些辅助函数。[1]

setdisc 辅助函数接受三个（可选）参数，再加一个可选的第四个参数指示子类型。其对应的 getdisc 接受一个可选布尔值；当值为 true 时，也会返回尾节点。setfont 辅助函数接受一个可选的第二个参数，即字符。直接模式设置器 setlink 接受一个节点列表并将它们链接起来，同时忽略 nil 项。返回第一个有效节点（注意：出于充分理由，它假定是单节点）。对于很少使用的字段，不提供辅助函数；有少数几个可能也很少用，但为了一致性而加入了。你当然随时可以用 getfield 和 setfield 定义额外的访问器，开销很小。当 setattributelist 的第二个参数为 true 时，假定使用当前属性列表。

[1]我们可以在 node 命名空间中用 getfield 定义这些辅助函数，效率差不多，所以将来某个时候我们可能把它作为模块提供。

函数	node	direct
check_discretionaries	+	+
check_discretionary	+	+
copy_list	+	+
copy	+	+
count	+	+
current_attr	+	+
dimensions	+	+
effective_glue	+	+
end_of_math	+	+
family_font	+	−
fields	+	−
find_attribute	+	+
first_glyph	+	+
fix_node_lists	+	−
flatten_discretionaries	+	+
flush_list	+	+
flush_node	+	+
flush_properties_table	+	+
free	+	+
get_attribute	+	+
get_properties_table	+	+
get_synctex_fields	−	+
getattributelist	−	+
getboth	+	+
getbox	−	+
getchar	+	+
getcomponents	−	+
getdata	−	+
getdepth	−	+
getdirection	−	+
getdir	−	+
getdisc	+	+
getexpansion	−	+
getfam	−	+
getfield	+	+
getfont	+	+
getglue	+	+

getheight	−	+
getid	+	+
getkern	−	+
getlang	−	+
getleader	+	+
getlist	+	+
getnext	+	+
getnucleus	−	+
getoffsets	−	+
getpenalty	−	+
getprev	+	+
getproperty	+	+
getshift	−	+
getsubtype	+	+
getsub	−	+
getsup	−	+
getwhd	+	+
getwidth	−	+
has_attribute	+	+
has_field	+	+
has_glyph	+	+
hpack	+	+
hyphenating	−	+
id	+	−
insert_after	+	+
insert_before	+	+
is_char	+	+
is_direct	−	+
is_glyph	+	+
is_node	+	+
is_zero_glue	+	+
kerning	+	+
last_node	+	+
length	+	+
ligaturing	+	+
make_extensible	+	−
mlist_to_hlist	+	−
new	+	+
next	+	−
prepend_prevdepth	+	+

prev	+	−
protect_glyphs	+	+
protect_glyph	+	+
protrusion_skippable	+	+
rangedimensions	+	+
remove	+	+
set_attribute	+	+
set_properties_mode	+	+
set_synctex_fields	−	+
setattributelist	−	+
setboth	−	+
setbox	−	+
setchar	−	+
setcomponents	−	+
setdata	−	+
setdepth	−	+
setdirection	−	+
setdir	−	+
setdisc	−	+
setexpansion	−	+
setfam	−	+
setfield	+	+
setfont	−	+
setglue	+	+
setheight	−	+
setkern	−	+
setlang	−	+
setleader	−	+
setlink	−	+
setlist	−	+
setnext	−	+
setnucleus	−	+
setoffsets	−	+
setpenalty	−	+
setprev	−	+
setproperty	+	+
setshift	−	+
setsplit	−	+
setsubtype	−	+
setsub	−	+

setsup	−	+
setwhd	−	+
setwidth	−	+
slide	+	+
subtypes	+	−
subtype	+	−
tail	+	+
todirect	−	+
tonode	−	+
tostring	+	+
traverse_char	+	+
traverse_glyph	+	+
traverse_id	+	+
traverse_list	−	+
traverse	+	+
types	+	−
type	+	−
unprotect_glyphs	+	+
unprotect_glyph	+	+
unset_attribute	+	+
usedlist	+	+
uses_font	+	+
values	+	−
vpack	+	+
whatsits	+	−
write	+	+

node.next 和 node.prev 函数会保留，但为了一致性，有分别称为 getnext 和 getprev 的变体。我们不得不用 get，因为 node.id 和 node.subtype 已经被占用来提供节点的元信息。注意：获取器只对合法键做基本检查。你应当只坚持使用描述节点属性的各节中提到的键。

鉴于字段相关，有些获取器和设置器处理多种节点类型。在那种情况下，某些字段名被认为是相似的（如 kern 与 width，或 data 与 value。回过头看，我们本可以把字段名规范化得更好，但我们决定尽可能沿用原来的（内部）名称。毕竟在 Lua 端，很容易创建同义词。

在 setfield 和 getfield 中，special 和 write 把 data 和 value 当作等价键接受。对于 write whatsit，获取 value 会以表的形式返回记号列表，而 data 则以字符串形式返回展开结果（版本 1.23.2，2025）。

有些节点带有间接引用。例如一个数学字符引用的是一个族而不是字体。在那种情况下，我们提供一个虚拟字体字段作为访问器。所以，getfont 和.font 可以用于它们。弹性间距节点的 width、height 和 depth 也是如此。它们实际上访问的是 spec 节点属性，在这里我们既可以设置也可以获取这些值。

在某些地方，LuaTEX 可以对合法节点列表做一些额外检查，你可以用下面的方式启用：

```
node.fix_node_lists(<boolean> b)
```

你可以为弹性间距、kern、hlist、vlist、rule 和 math 节点以及 glyph 节点设置和查询 SyncTEX 字段（一个文件编号即 tag，和一个行号）（不过最后这一种在原生 SyncTEX 中不使用）。

```
node.direct.set_synctex_fields(<integer> n, <integer> f, <integer> l)
<integer> f, <integer> l =
    node.direct.get_synctex_fields(<integer> n)
```

当然，你需要知道自己在做什么，因为不对合理的值做检查。而且，编辑器中使用的 synctex 解释器相当古怪，有一些假设（启发式规则）。

9.11　属性表（properties）

属性是将额外信息关联到节点的一种便捷方式。你可以在 TEX 端以及 Lua 端给它们赋值，并在 Lua 端查询。一大优点是它们遵守分组。它们是链表，通常检查它们相当高效，即使你使用大量属性。宏包必须在 TEX 端提供某种管理这些属性的方式，否则它们的使用可能发生冲突。

每个节点还可以有一个属性表，你可以用 `setproperty` 函数给这个表赋值，用 `getproperty` 函数获取属性。管理 properties 比管理属性要求高得多。

看下面的例子：

```
\directlua {
    local n = node.new("glyph")

    node.setproperty(n,"foo")
    print(node.getproperty(n))

    node.setproperty(n,"bar")
    print(node.getproperty(n))

    node.free(n)
}
```

这会打印 `foo` 和 `bar`，当多种机制都想使用这一特性时，它本身并不那么有用。另一种变体是：

```
\directlua {
    local n = node.new("glyph")

    node.setproperty(n,{ one = "foo", two = "bar" })
    print(node.getproperty(n).one)
    print(node.getproperty(n).two)

    node.free(n)
}
```

这一次我们在节点上存储了两个属性。用一个表作为属性确实很有意义，因为那样我们就能存更多东西。但为了让它正常工作，你需要这样做：

```
\directlua {
    local n = node.new("glyph")

    local t = node.getproperty(n)

    if not t then
        t = { }
        node.setproperty(n,t)
    end
    t.one = "foo"
    t.two = "bar"

    print(node.getproperty(n).one)
    print(node.getproperty(n).two)

    node.free(n)
}
```

这样我们自己的属性就不会覆盖其他用户的属性，当然，除非他们使用相同的键。所以最终你会得到类似这样的东西：

```
\directlua {
    local n = node.new("glyph")

    local t = node.getproperty(n)

    if not t then
        t = { }
        node.setproperty(n,t)
    end
    t.myself = { one = "foo", two = "bar" }

    print(node.getproperty(n).myself.one)
    print(node.getproperty(n).myself.two)

    node.free(n)
}
```

这假定只有你使用 `myself` 作为子表。可能性是无穷的，但需要小心。例如，随 ConTEXt 一起发布的通用字体处理器使用 `injections` 子表，你不应该去动它！

有几个辅助函数，作为用户你通常不应碰它们：`flush_properties_table` 会清空整个表（通常是个坏主意），`get_properties_table` 会给出存储属性的表（使用直接条目），你最好也不要对它做太多改动，因为 LuaTEX 自身会确保与节点相关的条目在节点被释放时被清除，从而让 Lua 垃圾回收器能正常工作。实际上，我们之所以有这套机制，主要是为了替用户（或宏包）省去一些工作。你可以很容易地在 Lua 中写一套属性机制，在 shipout 后清理属性，但要确保每个被释放的节点的属性也随之释放并不那么简单，因为可能有一些节点遗留到下一页。而给节点释放器绑定一个回调会带来太多开销。

在节点（释放）分配函数中管理 properties 默认是关闭的，可通过下面的方式启用：

```
node.set_properties_mode(true)
```

当我们复制一个带有表属性的节点列表时，有几种可能：像对待新节点一样处理；把该条目复制到 properties 中的表里（一个引用）；对 properties 中的表做深拷贝；创建一个新表，并把原表作为它的元表。在对这些方案做了一些实验（也包括计时）之后，我们决定深拷贝没意义，置空也没意义。最终，浅拷贝和元表变体都可以接受，尽管第二种更慢。需要记住的最重要一点是，properties 中对其他节点的引用，对那份拷贝来说可能不再有效。我们本可以用两个表（一个唯一、一个共享）或元表，但那只会让事情更复杂。

在定义新节点时，我们本可以立刻分配一个表，但在 lua 端做这件事相当容易，例如用元表 `__index` 方法。这样它就处于宏包的控制之下。删除节点时，我们本可以保留这个槽（例如把它设为 false），但当我们有临时的大型节点列表、之后又只有小列表时，这可能导致内存消耗不必要地上升。两种做法都没有采用。

所以最终现在是这样：当一个节点被复制、且它有一个表属性时，新节点会共享那个表。如果 `set_properties_mode` 的第二个参数为 `true`，则选择元表方式：拷贝得到自己的表，以原表作为元表。如果你使用通用字体加载器，模式就是以这种方式启用的。

还做了一些其他实验。例如：把属性复制到 properties 中，以便在 Lua 端快速访问。最终，速度和便利所带来的收益抵不过开销；实际上，访问属性并没有那么慢。所以这一方案被否决。

另一个实验涉及在节点中放一个位集，但同样，与属性相比收益微乎其微，而且可用位数量很少，这还要求就哪个位代表什么达成相当强的一致，而这在 TEX 社区中不太可能成功。它不值得。

如果有人想知道为什么 properties 有意义：我们得到的与其说是速度，不如说是便利：把各种（临时）数据存在属性里并不愉快，而这套机制确保节点被释放时属性会被清理。此外，拥有一个或多或少全局的 properties 表的优点是，我们待在 Lua 端。另一种做法是在节点自身中存一个引用，但这会因为寄存器有一些限制（不能用数字键）而变得复杂，而且我们也不想过多地动它。

第 10 章 Lua 回调

10.1 注册回调

本库包含用于注册、查找和列出回调的函数。回调是在明确定义的位置被调用的 Lua 函数。回调分为两类：一类与现有功能混合，另一类（启用后）替换功能。在大多数情况下，第二类应与内置功能表现相似，因为下一步需要特定的数据。例如，你可以替换断字例程。该函数接收一个可以（或不可以）断字的列表。最终列表必须有效，并且（通常）用于构造段落。另一个函数可以替换连字构建器和 |/| 或字距调整器。做其他事情也是可能的，但最终可能不会给用户带来预期的结果。

你需要做的第一件事是注册一个回调：

```
id, error =
    callback.register(<string> callback_name, <function> func)
id, error =
    callback.register(<string> callback_name, nil)
id, error =
    callback.register(<string> callback_name, false)
```

这里的 `callback_name` 是预定义的回调名称，见下文。该函数返回回调的内部 `id`；如果回调无法注册，则返回 `nil`。在后一种情况下，`error` 包含错误消息，否则为 `nil`。

LuaTEX 会将回调函数内部化，因此即使你意外地重新定义了一个函数也没有关系。

回调赋值始终是全局的。你可以使用特殊值 `nil` 代替函数来清除回调。

为了获得少量速度提升，你可以将布尔值 `false` 赋给与文件无关的回调，这样做将阻止 LuaTEX 执行其默认情况下会执行的任何操作（当完全没有注册回调函数时）。请注意：除非你 N.B. 确切地知道自己在做什么，否则这可能会导致各种问题！

```
<table> info =
    callback.list()
```

表中的键是已知的回调名称，值是一个布尔值，其中 `true` 表示该回调当前已设置（处于活动状态）。

```
<function> f = callback.find(callback_name)
```

如果回调未设置，`find` 返回 `nil`。

10.2 文件发现回调

本小节中记录的行为被认为是稳定的，即不会再有向后不兼容的更改。

10.2.1　find_read_file 与 find_write_file

你的回调函数应遵循以下约定：

```
<string> actual_name =
    function (<number> id_number, <string> asked_name)
```

参数：

id_number 对于日志文件或 input 文件，此数字为零。对于 TEX 的 read 或 write，该数字加一，因此 \read 0 变为 1。

asked_name 这是用户提供的文件名，由 input、openin 或 openout 找到。

返回值：

actual_name 这是实际使用的文件名。对于 TEX 读入的第一个文件，你必须确保返回的 actual_name 带有扩展名，并且适合用作 jobname。否则，你将不得不在 LuaTEX 结束后手动修复日志文件和输出文件的名称，并且最终的格式文件名也会变得混乱。这是因为这些文件名依赖于 jobname。

如果找不到文件，你必须返回 nil。

10.2.2　find_font_file

你的回调函数应遵循以下约定：

```
<string> actual_name =
    function (<string> asked_name)
```

asked_name 是一个 OTF 或 TFM 字体度量文件。

如果找不到文件，返回 nil。

10.2.3　find_output_file

你的回调函数应遵循以下约定：

```
<string> actual_name =
    function (<string> asked_name)
```

asked_name 是用于写入的 PDF 或 DVI 文件。

10.2.4　find_format_file

你的回调函数应遵循以下约定：

```
<string> actual_name =
    function (<string> asked_name)
```

asked_name 是用于读取的格式文件（用于写入的格式文件始终在当前目录中打开）。

10.2.5　find_vf_file

类似于 find_font_file，但用于虚拟字体。这同时适用于 Aleph 的 OVF 文件和传统的 Knuth 式 VF 文件。

10.2.6　find_map_file

类似于 find_font_file，但用于 map 文件。

10.2.7　find_enc_file

类似于 find_font_file，但用于 enc 文件。

10.2.8　find_pk_file

类似于 find_font_file，但用于 pk 位图文件。此回调接受两个参数：name 和 dpi。在你的回调中，你可以决定查找：

```
<base res>dpi/<fontname>.<actual res>pk
```

但其他策略也是可行的。由你来找到一个与该规格匹配的“合理位图文件。

10.2.9　find_data_file

类似于 find_font_file，但用于嵌入文件（\pdfobj file '...'）。

10.2.10　find_opentype_file

类似于 find_font_file，但用于 OpenType 字体文件。

10.2.11　find_truetype_file 与 find_type1_file

你的回调函数应遵循以下约定：

```
<string> actual_name =
    function (<string> asked_name)
```

asked_name 是一个字体文件。此回调在 LuaTEX 构建其所需字体文件的内部列表时被调用，因此实际的调用时机可能会让你感到意外。你的返回值稍后会被反馈到匹配的 read_file 回调中。

奇怪的是，find_type1_file 也用于 OpenType（OTF）字体。

10.2.12　find_image_file

你的回调函数应遵循以下约定：

```
<string> actual_name =
    function (<string> asked_name)
```

asked_name 是一个图像文件。你的返回值用于从硬盘打开文件，因此请确保返回的内容是你的操作系统认为的有效文件名。

10.3　文件读取回调

本小节中记录的行为被认为是稳定的，即不会再有向后不兼容的更改。

10.3.1　open_read_file

你的回调函数应遵循以下约定：

```
<table> env =
    function (<string> file_name)
```

参数：

tentative_name 由前一个 find_read_file 返回的文件名；如果没有定义此类回调，则为 kpse.find_file() 的返回值。

返回值：

env 这是一个表，其中至少包含一个必需的和一个可选的回调函数，用于此文件。必需的字段是 reader，关联的函数将在每读取一行新内容时被调用一次；可选的字段是 close，当 LuaTEX 处理完该文件时将被调用一次。LuaTEX 从不查看表的其余部分，因此你可以用它来存储你私有的逐文件数据。这两个回调函数都将该表作为它们唯一的参数。

reader

每当 LuaTEX 需要从文件中读取新的输入行时，就会运行此函数。

```
function(<table> env)
    return <string> line
end
```

你的函数应返回一个字符串或 nil。值 nil 表示已到达文件末尾，并将使 TEX 接下来调用可选的 close 函数。

close

当 LuaTEX 决定关闭文件时，将运行此可选函数。

```
function(<table> env)
end
```

你的函数不应返回任何值。

10.3.2　通用文件读取器

有一组回调用于加载二进制数据文件。它们都使用相同的接口：

```
function(<string> name)
    return <boolean> success, <string> data, <number> data_size
end
```

name 通常是完整路径名，由文件发现回调之一或 kpse.find_file() 的内部版本返回。

success 当发生致命错误时返回 false（例如，当文件终究找不到时）。

data 构成该文件的字节。

tdata的长度，size 以字节为单位。

如果找到了文件但读取时出现问题，则返回空字符串和零。

函数列表如下：

函数	用途
read_font_file	ofm 或 tfm 文件
read_vf_file	虚拟字体
read_map_file	map 文件
read_enc_file	编码文件
read_pk_file	pk 位图文件
read_data_file	嵌入文件（如 PDF 对象所支持的）
read_truetype_file	TrueType 字体文件
read_type1_file	Type 1 字体文件
read_opentype_file	OpenType 字体文件

10.4 数据处理回调

10.4.1 process_input_buffer

此回调允许你在 LuaTEX 实际开始查看行输入缓冲区之前更改其内容。

```
function(<string> buffer)
    return <string> adjusted_buffer
end
```

如果你返回 nil，LuaTEX 将假装你的回调从未发生。你可以从中获得少量处理时间。此回调不替换任何内部代码。

10.4.2 process_output_buffer

此回调允许你在 LuaTEX 因 write 命令而实际开始将行输出缓冲区写入文件之前更改其内容。它仅在输出到实际文件时被调用（即不包括日志、终端以及所谓的 write 18 调用）。

```
function(<string> buffer)
    return <string> adjusted_buffer
end
```

如果你返回 nil，LuaTEX 将假装你的回调从未发生。你可以从中获得少量处理时间。此回调不替换任何内部代码。

10.4.3 process_jobname

此回调允许你更改 TEX 中由 jobname 给出、以及 Lua 中由 tex.jobname 给出的作业名。它不影响内部作业名，也不影响输出文件或日志文件的名称。

```
function(<string> jobname)
    return <string> adjusted_jobname
end
```

唯一的参数是实际的作业名；你不应在此函数内使用 tex.jobname，否则可能发生无限递归。如果你返回 nil，LuaTEX 将假装你的回调从未发生。此回调不替换任何内部代码。

10.5 节点列表处理回调

节点和节点列表的描述见 9。

10.5.1 contribute_filter

当 LuaTEX 向列表添加内容时，会调用此回调：

```
function(<string> extrainfo)
end
```

该字符串报告组代码。由此你可以推断出你可以处理哪个列表。

值	说明
pre_box	正在添加行间材料
pre_adjust	正在添加 vadjust 材料
box	正在添加一个排版好的盒子（始终被调用）
adjust	正在添加 vadjust 材料

10.5.2 buildpage_filter

每当 LuaTEX 准备好将内容移至主垂直列表时，就会调用此回调。你可以使用此回调对页面构建阶段进行特殊处理，例如拼版或栏平衡。

```
function(<string> extrainfo)
end
```

字符串 extrainfo 提供有关 TEX 相对于“当前页面的状态的一些额外信息。buildpage_filter 回调的可能值为：

值	说明
alignment	正在添加一个（部分）对齐

after_output	输出例程刚刚结束
new_graf	新段落的开始
vmode_par	在垂直模式下遇到了 par
hmode_par	在水平模式下遇到了 par
insert	添加了一个插入
penalty	一个处罚点（在垂直模式下）
before_display	就在显示公式开始之前
after_display	显示公式结束
end	LuaTEX 正在终止（一切都结束了）

10.5.3 build_page_insert

当页面构建器添加插入时，会调用此回调。对这个机制没有太多控制权，但此回调允许在插入之前对间距进行一些最后一刻的调整，例如当连续追加多个插入（类型）时，这可能会很方便。

```
function(<number> n, <number> i)
    return <number> register
end
```

其中

值	说明
n	插入类别
i	插入的顺序

返回值是一个数字，指示用于前置间距的 skip 寄存器。例如，这允许使用不同的顶部间距（当 i 等于一时）和中间间距（当 i 大于一时）。当然，你可以随意处理插入盒子，但你需要确保 LuaTEX 之后能正常工作。

10.5.4 pre_linebreak_filter

此回调在 LuaTEX 开始将节点列表转换为 hbox 堆栈之前、即添加 parfillskip 之后被调用。

```
function(<node> head, <string> groupcode)
    return true | false | <node> newhead
end
```

名为 groupcode 的字符串标识节点列表在 TEX 处理过程中的上下文。可能的取值范围在下表中给出，但并非所有这些值都实际出现在 pre_linebreak_filter 中，其中一些属于 hpack_filter 和 vpack_filter 回调，将在接下来的两段中解释。

值	说明
<empty>	主垂直列表
hbox	水平模式下的 hbox
adjusted_hbox	垂直模式下的 hbox
vbox	vbox
vtop	vtop
align	halign 或 valign
disc	断点
insert	打包一个插入
vcenter	vcenter
local_box	localleftbox 或 localrightbox
split_off	vsplit 的顶部
split_keep	vsplit 的剩余部分
align_set	对齐单元
fin_row	对齐行

与所有处理节点的回调一样，返回值可以是以下三种之一：

- 布尔值 true 表示处理成功
- <node> 表示应将 “head 节点替换为返回的节点
- 布尔值 false 表示应忽略 “head 节点列表并将其从内存中清除

此回调不替换任何内部代码。

10.5.5 linebreak_filter

此回调替换 LuaTEX 的断行算法。

```
function(<node> head, <boolean> is_display)
    return <node> newhead
end
```

返回的节点是将被添加到主垂直列表的列表头；如果此段落被后续的数学显示公式打断，则布尔参数为 true。

如果你返回的不是 <node>，LuaTEX 将对从 <head> 开始的列表应用内部断行算法。否则，你返回的 <node> 应该是一个节点列表的表头，这些节点都在垂直模式下允许，并且其中至少有一个必须表示一个 hbox。否则将导致致命错误。

将此回调设置为 false 是可能的，但很危险，因为你可能最终陷入一个无法修复的 “死循环（deadcycles loop）。

10.5.6 append_to_vlist_filter

每当 LuaTEX 向垂直列表添加盒子时，就会调用此回调：

```
function (<node> box, <string> locationcode, <number> prevdepth, <boolean> mirrored)
    return list, prevdepth
end
```

不返回任何值也是可以的，在这种情况下你还需要清除该盒子或自行处理它。prevdepth 也是可选的。位置有 box、alignment、equation、equation_number 和 post_linebreak。你可以传递 nil 而不是节点。

10.5.7 post_linebreak_filter

此回调在 LuaTEX 将节点列表转换为 hbox 堆栈之后立即被调用。

```
function(<node> head, <string> groupcode)
    return true | false | <node> newhead
end
```

此回调不替换任何内部代码。

10.5.8 hpack_filter

当 TEX 准备好开始将某些水平模式材料打包成盒子时，会调用此回调。目前数学项和行盒子被忽略。

```
function(<node> head, <string> groupcode, <number> size,
         <string> packtype [, <string> direction] [, <node> attributelist])
    return true | false | <node> newhead
end
```

packtype 为 additional 或 exactly。如果是 additional，则 size 是一个 \hbox spread ... 参数。如果是 exactly，则 size 是一个 \hbox to ...。在两种情况下，该数字都以缩放点为单位。

direction 是三个字母的方向指示符字符串之一，或者是 nil。

此回调不替换任何内部代码。

10.5.9 vpack_filter

当 TEX 准备好开始将某些垂直模式材料打包成盒子时，会调用此回调。目前数学显示公式被忽略。

此函数与 hpack_filter 非常相似。除了调用时机不同之外，还有一个额外的变量，对应于 TEX 的 maxdepth 设置。

```
function(<node> head, <string> groupcode, <number> size, <string> packtype,
        <number> maxdepth [, <string> direction] [, <node> attributelist])
    return true | false | <node> newhead
end
```

此回调不替换任何内部代码。

10.5.10 hpack_quality

此回调可用于拦截因打包水平列表（如段落构建器中发生的）而可能产生的 overfull 消息。该函数接受几个参数：

```
function(<string> incident, <number> detail, <node> head, <number> first,
         <number> last)
    return <node> whatever
end
```

incident 是 overfull、underfull、loose 或 tight 之一。detail 要么是 overfull 时的溢出量，要么是其他情况下的坏度（badness）。head 是所构建的列表（当启用突出或扩展时，这是一个中间列表）。你可以选择返回一个节点，例如 overfull 规则指示器。该节点将被追加到列表中（就像 TEX 自己的规则一样）。

10.5.11 vpack_quality

此回调可用于拦截因打包垂直列表（如页面构建器中发生的）而可能产生的 overfull 消息。该函数接受几个参数：

```
function(<string> incident, <number> detail, <node> head, <number> first,
         <number> last)
end
```

incident 是 overfull、underfull、loose 或 tight 之一。detail 要么是 overfull 时的溢出量，要么是其他情况下的坏度。head 是所构建的列表。

10.5.12 process_rule

这是一个实验性回调。它可用于子类型 4（用户）的规则。该回调接收三个参数：节点、宽度和高度。该回调可以使用 pdf.print 向 PDF 文件写入代码，但请注意不要破坏最终结果。不进行任何检查。

10.5.13 pre_output_filter

当 TEX 准备好开始为 output 打包盒子 255 时，会调用此回调。

```
function(<node> head, <string> groupcode, <number> size, <string> packtype,
        <number> maxdepth [, <string> direction])
    return true | false | <node> newhead
end
```

此回调不替换任何内部代码。

10.5.14 hyphenate

```
function(<node> head, <node> tail)
end
```

无返回值。此回调必须在其接收到的节点列表中插入可选断点节点。

将此回调设置为 false 将阻止内部的可选断点插入过程。

10.5.15 ligaturing

```
function(<node> head, <node> tail)
end
```

无返回值。此回调必须对其接收到的节点列表应用连字处理。

你不必担心返回值，因为传递给回调的 head 节点保证不是 glyph_node （如有需要，会前置一个临时节点），因此它不会受到所发生变更的影响。回调之后，“列表尾部的内部值将被重新计算。

head 的 next 保证非 nil。

tail 的 next 保证为 nil，因此第二个回调参数通常可以忽略。提供它是为了保持一致性，并且在需要进行特殊处理时有时会很方便。

将此回调设置为 false 将阻止内部的连字创建过程。

你不得破坏节点列表。例如，head 通常是一个 local par 节点，而 tail 是一个 glue。过度破坏可能会使 LuaTEX 进入恐慌模式。

10.5.16 kerning

```
function(<node> head, <node> tail)
end
```

无返回值。此回调必须在其接收到的节点列表中的节点之间应用字距调整。调用约定参见 ligaturing。

将此回调设置为 false 将阻止内部的 kern 插入过程。

你不得破坏节点列表。例如，head 通常是一个 local par 节点，而 tail 是一个 glue。过度破坏可能会使 LuaTEX 进入恐慌模式。

10.5.17 insert_local_par

每个段落都以一个 local par 节点开始，该节点用于跟踪例如方向等信息。你可以在创建过程中挂接一个回调：

```
function(<node> local_par, <string> location)
end
```

没有返回值，你应该确保节点保持有效，否则 TEX 可能会混乱。

10.5.18 mlist_to_hlist

此回调替换 LuaTEX 的数学列表到节点列表的转换算法。

```
function(<node> head, <string> display_type, <boolean> need_penalties)
    return <node> newhead
end
```

返回的节点是将被添加到垂直或水平列表的列表头；字符串参数根据当前数学模式为 “text 或 “display；如果需要在此列表中插入处罚点，布尔参数为 true，否则为 false。

将此回调设置为 false 很糟糕，几乎肯定会导致死循环。

10.6 信息报告回调

10.6.1 pre_dump

```
function()
end
```

此函数在开始转储到格式文件之前被调用。它不替换任何代码，既没有参数也没有返回值。

10.6.2 start_run

```
function()
end
```

此回调替换打印 LuaTEX 横幅的代码。请注意，要成功使用，此回调必须在 Lua 初始化脚本中设置，否则它只会在运行已经开始后才被看到。

10.6.3 stop_run

```
function()
end
```

此回调替换打印 LuaTEX 统计信息和 “output written to 消息的代码。引擎仍可能进行内部整理，因此你不应依赖此挂钩来后处理 PDF 或日志文件。

10.6.4 start_page_number

```
function()
end
```

替换在 shipout 开始时打印 [和页码的代码。此回调还将覆盖通常在 tracingoutput 为正时进行的盒子信息打印。

10.6.5 stop_page_number

```
function()
end
```

替换在 shipout 结束时打印] 的代码。

10.6.6 show_error_hook

```
function()
end
```

此回调从 TEX 错误函数内部运行，其目的是允许你在 TEX 已有的操作之上进行一些额外的报告（不会移除任何正常操作）。你可能会发现 status 表中的一些值很有用。此回调不替换任何内部代码。

10.6.7　show_error_message

```
function()
end
```

此回调替换打印错误消息的代码。消息之后的常规交互不受影响。

10.6.8　show_ignored_error_message

```
function()
end
```

此回调替换在启用 ignoreprimitiveerror 时打印错误消息的代码。与之前一样，消息之后的常规交互不受影响。

10.6.9　show_lua_error_hook

```
function()
end
```

此回调替换打印额外 Lua 错误消息的代码。

10.6.10　start_file

```
function(category,filename)
end
```

此回调替换在打开文件时打印 LuaTEX 标记的代码，例如对于普通文件打印 (filename。category 是一个数字：

值	含义
1	普通数据文件，如 TEX 源文件
2	字体映射，将字体名称关联到资源
3	图像文件（png、pdf 等）
4	嵌入的字体子集
5	完全嵌入的字体

10.6.11　stop_file

```
function(category)
end
```

此回调替换在关闭文件时打印 LuaTEX 标记的代码，例如对于普通文件打印)。

10.6.12　call_edit

```
function(filename,linenumber)
end
```

当在错误消息提示下按下 “E 时，此回调替换对外部编辑器的调用。回调将控制权返回给主程序后，处理将立即结束。

10.6.13　finish_synctex

此回调可用于完成替代的 synctex 方法。它在正常的 synctex 终结器之后启动（当未启用原生 synctex 时，该终结器会在一次运行后删除 synctex 文件）。

10.6.14　wrapup_run

此回调在 PDF 和日志文件关闭后被调用。使用风险自负。

10.7　与 PDF 相关的回调

10.7.1　finish_pdffile

```
function()
end
```

当所有文档页面已写入 PDF 文件，且 LuaTEX 即将完成输出文档结构时，会调用此回调。其预期用途是最终更新 PDF 字典，例如 /Catalog 或 /Info。此回调不替换任何代码。既没有参数也没有返回值。

10.7.2　finish_pdfpage

```
function(shippingout)
end
```

此回调在 PDF 页面流组装完成之后、页面对象最终确定之前被调用。

10.7.3　page_order_index

这是供专家使用的一个回调，用来操纵页面树——一种决定 PDF 文件中页面顺序的数据结构：

```
function(pagenumber)
  return pagenumber
end
```

假设我们有 12 页，那么我们可以这样做：

```
callback.register("page_order_index",function(page)
        if page ==  1 then return 12
    elseif page ==  2 then return 11
    elseif page == 11 then return  2
    elseif page == 12 then return  1
    else                   return page
    end
end)
```

这将交换前两页和后两页。你需要知道页数，这是实现的一个副作用。当你搞砸了的时候，不要抱怨。

10.7.4 process_pdf_image_content

当嵌入 PDF 文件中的一页时，其页面流以及相关对象会被复制到目标文件中。但是，页面流中可能包含假定有额外资源的运算符，例如标记文本。你可以决定对其进行过滤，为此 LuaTEX 提供了一个回调。下面是一个简单的使用示例：

```
pdf.setrecompress(1)

callback.register("process_pdf_image_content",function(s)
    print(s)
    return s
end)
```

你需要显式启用重新压缩，否则内容流将仅以其原始压缩形式传递。

10.8 与字体相关的回调

10.8.1 define_font

```
function(<string> name, <number> size, <number> id)
    return <table> font | <number> id
end
```

字符串 name 是字体规范中的文件名部分，由用户给出。

数字 size 有点特殊：

- 如果为正数，则以缩放点指定一个 “at size（指定字号）。
- 如果为负数，其绝对值表示相对于字体设计字号的 “scaled （缩放）设置。

id 是分配给字体的内部编号。

要返回的 font 表的内部结构在 7 中解释。该表在内部保存，因此你可以在表中放置额外字段供你后续的 Lua 代码使用。或者，retval 可以是先前定义的 fontid。如果可以重用先前的定义而不是创建全新的字体结构，这会很有用。

将此回调设置为 false 没有意义，因为它会完全阻止字体加载，但仍然会生成错误。

10.8.2 glyph_not_found 与 glyph_info

glyph_not_found 回调在设置后，会在后端无法插入字形时启动。如果没有定义回调，则会向日志写入一条消息。

```
function(<number> id, <number> char)
    -- do something with font id and char code
end
```

可以设置 glyph_info 回调来报告字形的有用表示。

```
function(<node> g)
    -- return a string or nil
end
```

当返回 nil 时，打印字符代码；否则使用返回的字符串。默认显示 UTF 表示，这不总是那么有用，尤其是在没有实际表示时。请记住，设置此回调可能会以不兼容的方式更改日志。

10.8.3　provide_charproc_data

当后端正在写出用户定义的 Type 3 字体时，会触发 provide_charproc_data 回调：

```
function(<number> mode, <number> id, <number> char)
    if mode == 1 then
        -- preroll
        return
    elseif mode == 2 then
        -- write out the glyph contents
        return <number> stream, <number> width
    elseif mode == 3 then
        -- the overall font scale
        return <number> scale
    end
end
```

要创建用户定义的 Type 3 字体，你需要在定义字体时在顶层设置 encodingbytes = 0 和 psname = "none"。从 glyph_not_found 回调中，你应在 mode 3 时返回整体字体缩放比例（通常为 0.001），在 mode 2 时返回 PDF 流的索引（其中第一个运算符为 d0 或 d1）以及字形的宽度（以 sp 为单位）。通常可以忽略 mode 1。

第 11 章 与 TEX 相关的库

11.1 lua

11.1.1 版本信息

此库包含一个只读项目：

```
<string> s = lua.version
```

这会返回 Lua 版本标识字符串。当前值为。

11.2 status

此库包含一些运行时配置项，你可能会发现它们在消息报告中很有用；以及一个迭代器函数，它以 table 形式获取所有名称和值。

```
<table> info = status.list()
```

table 中的键是已知项目，值为当前值。status 中几乎所有值都是在运行时通过元表在访问时获取的，因此你不能对 status 使用 pairs，但你可以对 info 使用 pairs。如果你不需要完整列表，也可以通过将名称作为 status 的索引来请求单个项目。当前列表如下：

键	说明
banner	终端显示横幅
best_page_break	当前最佳断点（一个节点）
buf_size	当前分配的行缓冲区大小
callbacks	迄今为止执行的回调总数
cs_count	控制序列数量
dest_names_size	PDF 目标表大小
dvi_gone	已写入的 DVI 字节数
dvi_ptr	尚未写入的 DVI 字节数
dyn_used	正在使用的令牌（多字）内存
filename	当前输入文件名
fix_mem_end	已使用令牌的最大数量
fix_mem_min	为令牌分配的最小字数
fix_mem_max	为令牌分配的最大字数

font_ptr	活动字体数量
hash_extra	额外允许的哈希量
hash_size	哈希表大小
indirect_callbacks	其中本身是其他回调（例如文件读取器）结果的数量
ini_version	如果这是一次 initex 运行则为 true
init_pool_ptr	initex 字符串池索引
init_str_ptr	initex 字符串数量
input_ptr	我们当前所在的输入层级
inputid	当前输入的数字 id
largest_used_mark	最大引用的 marks 类
lasterrorcontext	最后一个错误上下文字符串（含换行）
lasterrorstring	最后一个 TEX 错误字符串
lastluaerrorstring	最后一个 Lua 错误字符串
lastwarningstring	最后一个警告标签，通常指示位于哪个部分
lastwarningtag	最后一个警告字符串
linenumber	当前输入文件中的位置
log_name	日志文件名
luabytecode_bytes	Lua 字节码寄存器中的字节数
luabytecodes	活动 Lua 字节码寄存器数量
luastate_bytes	Lua 解释器正在使用的字节数
luatex_engine	LuaTEX 引擎标识符
luatex_hashchars	Lua 对字符串做哈希的长度（2^n）
luatex_hashtype	使用的哈希方法（在 LuaJITTeX 中）
luatex_version	LuaTEX 版本号
luatex_revision	LuaTEX 修订字符串
max_buf_stack	最大已用缓冲区位置
max_in_stack	最大已用输入栈条目
max_nest_stack	最大已用嵌套栈条目
max_param_stack	最大已用参数栈条目
max_save_stack	最大已用保存栈条目
max_strings	最大允许字符串数
nest_size	嵌套栈大小
node_mem_usage	一个字符串，提供当前使用节点的洞察

obj_ptr	最大 PDF 对象指针
obj_tab_size	PDF 对象表大小
output_active	如果 output 例程处于活动状态则为 true
output_file_name	PDF 或 DVI 文件名
param_size	参数栈大小
pdf_dest_names_ptr	最大 PDF 目标指针
pdf_gone	已写入的 PDF 字节数
pdf_mem_ptr	最大已用 PDF 内存
pdf_mem_size	PDF 内存大小
pdf_os_cntr	最大 PDF 对象流指针
pdf_os_objidx	PDF 对象流索引
pdf_ptr	尚未写入的 PDF 字节数
pool_ptr	字符串池索引
pool_size	当前为字符串字符分配的大小
save_size	保存栈大小
shell_escape	0 表示禁用，1 表示允许任何操作，2 为受限
safer_option	1 表示强制执行更安全模式
luadebug_option	如果在命令行启用了 debug 库则为 true
output_directory	输出目录的值
kpse_used	1 表示使用 kpse
stack_size	输入栈大小
str_ptr	字符串数量
total_pages	已写入的页数
var_mem_max	为节点分配的字数
var_used	正在使用的变量（单字）内存
lc_collate	启动时 LC_COLLATE 的值（启动时变为 C）
lc_ctype	启动时 LC_CTYPE 的值（启动时变为 C）
lc_numeric	启动时 LC_NUMERIC 的值

错误和警告消息可以通过 resetmessages 函数清除。可以使用 setexitcode 设置返回值。

11.3 tex

11.3.1　简介

tex table 包含大量虚拟内部 TEX 参数，其中部分可写。

“虚拟这一称谓意味着这些项目并非在 Lua 中正式定义，而只是由元表处理实际 TEX 值的前端。因此，大多数 Luatable 操作符（如 pairs 和 ##）对此类项目不起作用。

目前，几乎可以访问所有你能在 the 之后使用的参数，即单个令牌或在 TEX 中具有特殊性质的参数。这不包括需要额外参数的参数，如 \the \scriptfont 。由简单整数和尺寸寄存器组成的子集既可读又可写（如 tracingcommands 和 parindent）。

11.3.2　内部参数值：set

对于本节中的所有参数，可以直接使用其名称作为 tex table 中的索引来访问，或者使用 tex.get 和 tex.set 函数之一。

确切的参数和返回值因实际参数而异，tex.set 是否有作用也是如此。对于可以设置的参数，可以使用 global 作为 tex.set 的第一个参数；这使赋值变为全局而非局部。

```
tex.set (["global",] <string> n, ...)
... = tex.get (<string> n)
```

黏胶有点特殊，因为涉及五个值。返回值是一个 glue_spec 节点，但当你向 tex.get 传递 false 作为最后一个参数时，你得到黏胶的宽度；传递 true 则得到全部五个值。否则你得到一个节点，它是内部值的副本，因此你有责任在 Lua 端释放它。设置黏胶量时，你可以传递一个 glue_spec 或最多五个数字。向 get 传递 true 时，黏胶返回 5 个值；传递 false 时仅返回宽度。

整数参数

整数参数接受并返回 Lua 数字。以下为可读写的:

```
tex.adjdemerits
tex.binoppenalty
tex.brokenpenalty
tex.catcodetable
tex.clubpenalty
tex.day
tex.defaulthyphenchar
tex.defaultskewchar
tex.delimiterfactor
tex.displaywidowpenalty
tex.doublehyphendemerits
tex.endlinechar
tex.errorcontextlines
tex.escapechar
tex.exhyphenpenalty
tex.fam
tex.finalhyphendemerits
tex.floatingpenalty
tex.globaldefs
```

```
tex.hangafter
tex.hbadness
tex.holdinginserts
tex.hyphenpenalty
tex.interlinepenalty
tex.language
tex.lastlinefit
tex.lefthyphenmin
tex.linepenalty
tex.localbrokenpenalty
tex.localinterlinepenalty
tex.looseness
tex.mag
tex.maxdeadcycles
tex.month
tex.newlinechar
tex.outputpenalty
tex.pausing
tex.postdisplaypenalty
tex.predisplaydirection
tex.predisplaypenalty
tex.pretolerance
tex.relpenalty
tex.righthyphenmin
tex.savinghyphcodes
tex.savingvdiscards
tex.showboxbreadth
tex.showboxdepth
tex.time
tex.tolerance
tex.tracingassigns
tex.tracingcommands
tex.tracinggroups
tex.tracingifs
tex.tracinglostchars
tex.tracingmacros
tex.tracingnesting
tex.tracingonline
tex.tracingoutput
tex.tracingpages
tex.tracingparagraphs
tex.tracingrestores
tex.tracingscantokens
tex.tracingstats
tex.uchyph
tex.vbadness
tex.widowpenalty
tex.year
```

以下为只读的:

```
tex.deadcycles
tex.insertpenalties
```

```
tex.parshape
tex.interlinepenalties
tex.clubpenalties
tex.widowpenalties
tex.displaywidowpenalties
tex.prevgraf
tex.spacefactor
```

尺寸参数

尺寸参数接受 Lua 数字（表示缩放点）或字符串（含尺寸单位）。结果始终为以缩放点为单位的数字。以下为可读写的：

```
tex.boxmaxdepth
tex.delimitershortfall
tex.displayindent
tex.displaywidth
tex.emergencystretch
tex.hangindent
tex.hfuzz
tex.hoffset
tex.hsize
tex.lineskiplimit
tex.mathsurround
tex.maxdepth
tex.nulldelimiterspace
tex.overfullrule
tex.pagebottomoffset
tex.pageheight
tex.pageleftoffset
tex.pagerightoffset
tex.pagetopoffset
tex.pagewidth
tex.parindent
tex.predisplaysize
tex.scriptspace
tex.splitmaxdepth
tex.vfuzz
tex.voffset
tex.vsize
tex.prevdepth
tex.prevgraf
tex.spacefactor
```

以下为只读的：

```
tex.pagedepth
tex.pagefilllstretch
tex.pagefillstretch
tex.pagefilstretch
tex.pagegoal
tex.pageshrink
```

```
tex.pagestretch
tex.pagetotal
```

注意：与所有 Luatable 一样，你可以向其中添加值。因此，以下是有效的：

```
tex.foo = 123
```

当你访问一个 TEX 参数时，会进行查找。对于只读变量，这意味着你会得到返回值，但当你设置它们时，会在 table 中创建一个新条目，从而使原始条目不可见。

有几个特殊情况我们做了例外处理：`prevdepth`、`prevgraf` 和 `spacefactor`。通常它们通过 `tex.nest` table 访问：

```
tex.nest[tex.nest.ptr].prevdepth   = p
tex.nest[tex.nest.ptr].spacefactor = s
```

不过，以下方式也有效：

```
tex.prevdepth   = p
tex.spacefactor = s
```

请记住，当你在 Lua 端直接操作节点列表时，可能需要显式更新嵌套栈顶的 `prevdepth`，因为 LuaTEX 无法猜测你的意图。通过使用 `tex` table 中的访问器，你可以获取和设置嵌套栈顶的值。

方向参数

方向参数为只读，返回一个 Lua 字符串。

```
tex.bodydir
tex.mathdir
tex.pagedir
tex.pardir
tex.textdir
```

黏胶参数

黏胶参数接受并返回一个表示 `glue_spec` 节点的 userdata 对象。

```
tex.abovedisplayshortskip
tex.abovedisplayskip
tex.baselineskip
tex.belowdisplayshortskip
tex.belowdisplayskip
tex.leftskip
tex.lineskip
tex.parfillskip
tex.parskip
tex.rightskip
tex.spaceskip
tex.splittopskip
tex.tabskip
tex.topskip
tex.xspaceskip
```

muglue 参数

所有 muglue 参数均为只读，返回一个 Lua 字符串。

```
tex.medmuskip
tex.thickmuskip
tex.thinmuskip
```

令牌列表参数

令牌列表参数接受并返回 Lua 字符串。Lua 字符串使用 `the toks` 式展开与令牌列表相互转换：所有类别码要么是空格（10），要么是其他（12）。因此，对其中一些参数（如 “tex.output）赋值实际上没有用处，但考虑到未来将支持完整令牌字符串的扩展，做例外处理并不合适。

```
tex.errhelp
tex.everycr
tex.everydisplay
tex.everyeof
tex.everyhbox
tex.everyjob
tex.everymath
tex.everypar
tex.everyvbox
tex.output
```

11.3.3　转换命令

所有 “convert 命令均为只读，返回一个 Lua 字符串。当前支持的命令有：

```
tex.eTeXVersion
tex.eTeXrevision
tex.formatname
tex.jobname
tex.luatexbanner
tex.luatexrevision
tex.fontname(number)
tex.uniformdeviate(number)
tex.number(number)
tex.romannumeral(number)
tex.fontidentifier(number)
```

如果你想知道为什么这个列表看起来杂乱无章；这些都是不需要参数的 “convert 内部命令的所有情况，以及只需要简单数值的命令。特殊的（仅限 Lua）`tex.fontidentifier` 情况返回与字体 id 编号匹配的 `csname` 字符串（如果存在的话）。

11.3.4　最后一项命令

所有 “last item 命令均为只读，返回一个数字。当前支持的命令有：

```
tex.lastpenalty
tex.lastkern
```

```
tex.lastskip
tex.lastnodetype
tex.inputlineno
tex.lastxpos
tex.lastypos
tex.randomseed
tex.luatexversion
tex.eTeXminorversion
tex.eTeXversion
tex.currentgrouplevel
tex.currentgrouptype
tex.currentiflevel
tex.currentiftype
tex.currentifbranch
```

11.3.5　访问寄存器：set*

TEX 的属性（attribute）、计数器（count）、尺寸（dimen）、skip（skip、muskip）和令牌（toks）寄存器可以通过 tex table 的两组各五个虚拟子表来访问和写入：

```
tex.attribute
tex.count
tex.dimen
tex.skip
tex.glue
tex.muskip
tex.muglue
tex.toks
```

可以将相关的 attributedef、countdef、dimendef、skipdef 或 toksdef 控制序列的名称用作这些 table 的索引：

```
tex.count.scratchcounter = 0
enormous = tex.dimen['maxdimen']
```

在这种情况下，LuaTEX 会即时为你查找值。你必须使用有效的 countdef（或 attributedef、dimendef、skipdef、toksdef），其他任何内容都会产生错误（其意图是最终也允许 <chardef tokens> 甚至展开为数字的宏）。

- count 寄存器接受并返回 Lua 数字。
- 尺寸寄存器接受 Lua 数字（以缩放点为单位）或字符串（含绝对尺寸；em、ex 和 px 不允许）。结果始终为以缩放点为单位的数字。
- 令牌寄存器接受并返回 Lua 字符串。Lua 字符串使用 the toks 式展开与令牌列表相互转换：所有类别码要么是空格（10），要么是其他（12）。
- skip 寄存器接受并返回 glue_spec userdata 节点对象（参见本手册其他部分对节点接口的描述）。
- glue 寄存器就是 skip 寄存器，但不是 userdata 形式而是详细表示形式。
- 与 count 一样，属性寄存器接受并返回 Lua 数字。

除了数组寻址之外，还为所有情况定义了访问器函数，例如，以下是 skip 寄存器的一组可能函数：

```
tex.setskip (["global",] <number> n, <node> s)
tex.setskip (["global",] <string> s, <node> s)
<node> s = tex.getskip (<number> n)
<node> s = tex.getskip (<string> s)
```

我们对 count、dimen、muskip 和 toks 有类似的设置函数。计数器和尺寸以数字表示，skip 和 muskip 以节点表示，toks 以字符串表示。

同样，glue 变体不使用 glue_spec userdata 节点。setglue 函数最多接受 5 个参数：宽度、伸展、收缩、伸展阶和收缩阶。如果你不传递值或某个值不是数字，相应属性将变为零。getglue 函数报告所有属性，除非第二个参数为 false，此时仅返回宽度。

以下是一个使用三元组的示例：

```
local d = tex.getdimen("foo")
if tex.isdimen("bar") then
    tex.setdimen("bar",d)
end
```

有六个额外的 skip（glue）相关辅助函数：

```
tex.setglue (["global"], <number> n,
    width, stretch, shrink, stretch_order, shrink_order)
tex.setglue (["global"], <string> s,
    width, stretch, shrink, stretch_order, shrink_order)
width, stretch, shrink, stretch_order, shrink_order =
    tex.getglue (<number> n)
width, stretch, shrink, stretch_order, shrink_order =
    tex.getglue (<string> s)
```

另外两个是 tex.setmuglue 和 tex.getmuglue。

dimen、count、skip、muskip、box 和 attribute 寄存器都有这样的辅助函数，但 glue 的比较特殊，因为它们需要处理更多属性。

与之前讨论的通用 get 和 set 函数一样，对于 skip 寄存器，getskip 返回节点，getglue 返回数字；而 setskip 接受节点，setglue 期望最多五个数字。同样，当你向 getglue 传递 false 作为第二个参数时，仅返回宽度。getmuskip、setmuskip 的 mu 变体也是如此。

对于令牌寄存器，我们有一个备选方案，可以指定类别码表：

```
tex.scantoks(0,3,"$e=mc^2$")
tex.scantoks("global",0,"$\int\limits^1_2$")
```

在基于函数的接口中，可以通过使用字符串 global 作为第一个函数参数来全局定义值。

marks 有一个专用的获取函数：getmark，它接受两个参数。第一个参数是 top、bottom、first、splitbottom 或 splitfirst 之一，第二个参数是 marks 类编号。当不提供参数时，返回当前最大类数。

11.3.6　字符码寄存器：[get|set*code[s]

]

TEX 的字符码表（lccode、uccode、sfcode、catcode、mathcode、delcode）可以通过 tex table 的六个虚拟子表来访问和写入

```
tex.lccode
tex.uccode
tex.sfcode
tex.catcode
tex.mathcode
tex.delcode
```

函数调用接口大致如上，但有一些曲折。sfcode 是最简单的：

```
tex.setsfcode (["global",] <number> n, <number> s)
<number> s = tex.getsfcode (<number> n)
```

lccode 和 uccode 的函数调用接口还允许你同时设置关联的兄弟码：

```
tex.setlccode (["global"], <number> n, <number> lc)
tex.setlccode (["global"], <number> n, <number> lc, <number> uc)
<number> lc = tex.getlccode (<number> n)
tex.setuccode (["global"], <number> n, <number> uc)
tex.setuccode (["global"], <number> n, <number> uc, <number> lc)
<number> uc = tex.getuccode (<number> n)
```

catcode 的函数调用接口还允许你在赋值或查询时指定要使用的类别表（两种情况的默认值都是当前表）：

```
tex.setcatcode (["global"], <number> n, <number> c)
tex.setcatcode (["global"], <number> cattable, <number> n, <number> c)
<number> lc = tex.getcatcode (<number> n)
<number> lc = tex.getcatcode (<number> cattable, <number> n)
```

delcode 和 mathcode 的接口使用小数组 table 来设置和检索值：

```
tex.setmathcode (["global"], <number> n, <table> mval )
<table> mval = tex.getmathcode (<number> n)
tex.setdelcode (["global"], <number> n, <table> dval )
<table> dval = tex.getdelcode (<number> n)
```

其中 mathcode 的 table 是一个包含 3 个数字的数组，如下所示：

```
{
    <number> class,
    <number> family,
    <number> character
}
```

而 delcode 的 table 是一个包含 4 个数字的数组，如下所示：

```
{
    <number> small_fam,
    <number> small_char,
    <number> large_fam,
    <number> large_char
}
```

你也可以避免使用 table:

```
tex.setmathcode (["global"], <number> n, <number> class,
    <number> family, <number> character)
class, family, char =
    tex.getmathcodes (<number> n)
tex.setdelcode (["global"], <number> n, <number> smallfam,
    <number> smallchar, <number> largefam, <number> largechar)
smallfam, smallchar, largefam, largechar =
    tex.getdelcodes (<number> n)
```

通常，根据 Udelcode 的用法，分隔符码赋值中的第三个和第四个值为零，但返回的 table 中这些位置可能有值（例如，如果分隔符码是使用 delcode 设置的）。未设置的 delcode 可以通过 dval[1] 为 −1 来识别。

11.3.7　Box 寄存器：[get|setbox

]

可以使用 node 库中定义的节点接口来设置和查询实际的 box，例如来自 hbox、vbox 或 vtop 的 box:

```
tex.box
```

用于数组访问，或

```
tex.setbox(["global",] <number> n, <node> s)
tex.setbox(["global",] <string> cs, <node> s)
<node> n = tex.getbox(<number> n)
<node> n = tex.getbox(<string> cs)
```

用于基于函数的访问。在基于函数的接口中，可以通过使用字符串 global 作为第一个函数参数来全局定义值。

请注意，如下赋值

```
tex.box[0] = tex.box[2]
```

不会复制节点列表，它只是复制了一个节点指针。如果 \box 2 之后被 TEX 命令清除，\box 0 的内容也会变为无效。为防止这种情况发生，始终使用 node.copy_list，除非你赋值给临时变量:

```
tex.box[0] = node.copy_list(tex.box[2])
```

11.3.8　重用 box：[use|saveboxresource 与 getboxresourcedimensions

]

以下函数将注册一个 box 以供重用（这是仿照 PDF 中所谓的 xforms 建模的）。你可以通过 useboxresource 或创建子类型为 2 的规则节点来（重）使用该 box。

```
local index = tex.saveboxresource(n,attributes,resources,immediate,type,margin)
```

可选的第二个和第三个参数是字符串，第四个是布尔值。第五个参数是类型。设置为非零时省略 /Type 条目。值为 1 或 3 仍会写入 /BBox，而 2 或 3 会写入 /Matrix。第六个参数是（虚拟）边距，它超出 TEX 所见的有效包围盒。除了 box 编号之外，还可以传递一个 [h|v]list 节点。

你可以通过以下方式生成引用（规则类型）：

```
local reused = tex.useboxresource(n,wd,ht,dp)
```

尺寸是可选的，最终尺寸作为额外值返回。以下只是一个附加功能（不返回尺寸意味着该资源未知）：

```
local w, h, d, m = tex.getboxresourcedimensions(n)
```

这会返回资源的宽度、高度、深度和边距。

11.3.9　triggerbuildpage

你不应指望 triggerbuildpage 辅助函数有太多作为，因为通常当 TEX 认为没有什么需要做时就不会做太多，但它可能对某些应用程序有用。它只是按照其名称所做的那样——调用构建页面的内部函数，前提是有东西需要构建。

11.3.10　splitbox

你可以拆分一个 box：

```
local vlist = tex.splitbox(n,height,mode)
```

余数保留在原始 box 中，并返回一个打包好的 vlist。此操作类似于 vsplit 操作。mode 可以是 additional 或 exactly，涉及被拆分出去的 box。

11.3.11　访问数学参数：[get|setmath

]

可以使用以下方式设置和查询内部数学参数：

```
tex.setmath(["global",] <string> n, <string> t, <number> n)
<number> n = tex.getmath(<string> n, <string> t)
```

与之前一样，可选的第一个参数 global 表示全局赋值。

第一个字符串是去掉前导 “Umath 的参数名，第二个字符串是去掉尾部 “style 的样式名。为完整起见，数学参数名的值如下：

```
quad                 axis                 operatorsize
overbarkern          overbarrule          overbarvgap
underbarkern         underbarrule         underbarvgap
radicalkern          radicalrule          radicalvgap
radicaldegreebefore  radicaldegreeafter   radicaldegreeraise
stackvgap            stacknumup           stackdenomdown
fractionrule         fractionnumvgap      fractionnumup
fractiondenomvgap    fractiondenomdown    fractiondelsize
limitabovevgap       limitabovebgap       limitabovekern
limitbelowvgap       limitbelowbgap       limitbelowkern
underdelimitervgap   underdelimiterbgap
overdelimitervgap    overdelimiterbgap
subshiftdrop         supshiftdrop         subshiftdown
subsupshiftdown      subtopmax            supshiftup
supbottommin         supsubbottommax      subsupvgap
spaceafterscript     connectoroverlapmin
ordordspacing        ordopspacing         ordbinspacing      ordrelspacing
ordopenspacing       ordclosespacing      ordpunctspacing    ordinnerspacing
opordspacing         opopspacing          opbinspacing       oprelspacing
opopenspacing        opclosespacing       oppunctspacing     opinnerspacing
binordspacing        binopspacing         binbinspacing      binrelspacing
binopenspacing       binclosespacing      binpunctspacing    bininnerspacing
relordspacing        relopspacing         relbinspacing      relrelspacing
relopenspacing       relclosespacing      relpunctspacing    relinnerspacing
openordspacing       openopspacing        openbinspacing     openrelspacing
openopenspacing      openclosespacing     openpunctspacing   openinnerspacing
closeordspacing      closeopspacing       closebinspacing    closerelspacing
closeopenspacing     closeclosespacing    closepunctspacing  closeinnerspacing
punctordspacing      punctopspacing       punctbinspacing    punctrelspacing
punctopenspacing     punctclosespacing    punctpunctspacing  punctinnerspacing
innerordspacing      inneropspacing       innerbinspacing    innerrelspacing
inneropenspacing     innerclosespacing    innerpunctspacing  innerinnerspacing
```

样式参数的值为：

```
display       crampeddisplay
text          crampedtext
script        crampedscript
scriptscript  crampedscriptscript
```

值要么是一个数字（表示尺寸或数值），要么是表示 `ordordspacing` 及类似间距参数的 muskip 的黏胶规格节点。

11.3.12　特殊列表头：[get|setlist

]

虚拟 table `tex.lists` 包含跟踪构建页面列表的内部寄存器集合。

字段	说明
`page_ins_head`	待处理插入项的循环链表

contrib_head	最近的贡献内容
page_head	当前页面内容
hold_head	用于为下一页保留的项目
adjust_head	当前 vadjust 列表的头
pre_adjust_head	当前 \vadjust pre 列表的头
page_discards_head	分页断开时被丢弃项目的头
split_discards_head	vsplit 中被丢弃项目的头

获取和设置函数是 getlist 和 setlist。你必须小心设置的内容，因为 TEX 可能对列表的构建方式或其所处的状态有期望。

11.3.13　语义嵌套层级：getnest

虚拟 table nest 包含当前活动的语义嵌套状态。它有两个主要部分：一个以零为基的 user-data 数组，表示语义嵌套本身；以及数值 ptr，给出最高可用索引。nest[] 中的数组项和 ptr 都不能赋值（因为这会使排版引擎陷入无法修复的混乱），但你可以对数组项内部的各个值赋值，例如 tex.nest[tex.nest.ptr].prevdepth。

tex.nest[tex.nest.ptr] 是当前嵌套状态，nest[0] 是最外层（主垂直列表）层级。获取函数是 getnest。你可以传递一个数字（返回一个列表）、不传递参数、或 top（返回最顶层列表），或字符串 ptr（返回最顶层列表的索引）。

已知字段有：

键	类型	模式	说明
mode	number	所有	这些数字的含义取决于引擎有时甚至取决于版本；你可以使用 tex.getmodevalues() 获取映射：正值表示垂直、水平和数学模式，负值表示内部和行内变体
modeline	number	所有	进入此模式时的源输入行，输出例程内部为负值
head	node	所有	当前列表的头
tail	node	所有	当前列表的尾
prevgraf	number	vmode	上一段落的行数
prevdepth	number	vmode	上一段落的深度
spacefactor	number	hmode	当前间距因子
dirs	node	hmode	供换行算法临时存储使用
noad	node	mmode	用于临时存储待处理的分数分子，用于 over 等

delimptr	node	mmode	用于临时存储前一个数学分隔符，用于 middle
mathdir	boolean	mmode	在数学处理期间 mathdir 与周围的 textdir 不同时为 true
mathstyle	number	mmode	当前 mathstyle

11.3.14　打印函数

tex table 还包含三个打印函数，它们是从 Lua 脚本到 TEX 的主要接口。这三个函数的参数都存储在一个内存虚拟文件中，该文件作为 directlua 展开的结果馈送给 TEX 扫描器。

一个 directlua 命令可返回的文本总量仅受可用系统 RAM 的限制。不过，每个单独的打印字符串必须完全适合 TEX 的输入缓冲区。目前在回调内部使用这些函数的结果是未定义的。

print

```
tex.print(<string> s, ...)
tex.print(<number> n, <string> s, ...)
tex.print(<table> t)
tex.print(<number> n, <table> t)
```

每个字符串参数都被 TEX 视为独立的输入行。如果是 table 参数而不是字符串列表，则必须是要打印的连续字符串数组（第一个非字符串值将停止打印过程）。

可选参数可用于使用由 catcodetable n 定义的类别码规则打印字符串。如果 n 为 −1，则使用当前活动的类别码规则。如果 n 为 −2，则生成的类别码是 the toks 的结果：所有类别码为 12（其他），除了空格字符的类别码为 10（空格）。否则，如果 n 不是有效的类别码表，则被忽略，改为使用当前活动的类别码规则。

在一个 directlua 中，最后一个 tex.print 命令的最后一个字符串不会附加 endlinechar，其他所有字符串都会附加。

sprint

```
tex.sprint(<string> s, ...)
tex.sprint(<number> n, <string> s, ...)
tex.sprint(<table> t)
tex.sprint(<number> n, <table> t)
```

每个字符串参数都被 TEX 视为一种特殊的输入行，使其适合用作部分行输入机制：

- TEX 不会切换到 “新行状态，因此前导空格不会被忽略。
- 不插入 endlinechar。
- 尾部空格不会被移除。注意，这并不妨碍 TEX 本身在解释行时吞掉空格。例如，在

```
before\directlua{tex.sprint("\\relax")tex.sprint(" inbetween")}after
```

中，in between 之前的空格会因 relax 的 “正常扫描而被吞掉。

如果是 table 参数而不是字符串列表，则必须是要打印的连续字符串数组（第一个非字符串值将停止打印过程）。

可选参数设置类别码规则，与 tex.print 一样。这会影响字符串参数（或转换为字符串的数字）。

虽然这需要谨慎使用，但你也可以传递令牌或节点 userdata 对象。这些会被注入流中。令牌最好是有效的令牌，而节点在被注入时必须存在。因此重要的是要认识到以下几点：

- 当你注入一个令牌时，你需要传递一个有效的令牌 userdata 对象。这个对象在不再被引用时会被 Lua 回收。当它被打印到 TEX 时，令牌本身会被复制，因此不会与 Lua 垃圾回收产生干扰。你自己管理该对象。由于令牌实际上只是数字，在 TEX 端没有真正的额外开销。
- 当你注入一个节点时，你需要传递一个有效的节点 userdata 对象。与该对象关联的节点在不再被引用时不会被 Lua 回收。它在 TEX 端自己的内存空间中继续存在。当它被打印到 TEX 时，节点引用被使用，假设该节点仍然存在。不涉及 Lua 垃圾回收。同样，你自己管理该对象。节点本身在 TEX 用完后被释放。

如果你考虑最后一点，可能会意识到当打印的字符串、令牌和节点的混合体被重用时我们会遇到问题。在 TEX 内部，序列变为输入缓冲区的链表。因此，"123" 或"\foo {123}" 会被即时读取和解析，而 <token userdata> 已经是令牌化的，实际上现在是一个令牌列表。<node userdata> 也被令牌化为令牌列表，但它有一个对真实节点的引用。通常这没问题。但现在假设你将整个内容存储在一个宏中：在这种情况下，令牌化节点可以被多次刷新。但是，在第一次这样的刷新后，节点被使用且其内存被释放。你可以通过使用副本来防止这种情况，这通过将 luacopyinputnodes 设置为非零值来控制。如果你真的要处理这些底层问题，这是你必须接受的模糊区域之一。

tprint

```
tex.tprint({<number> n, <string> s, ...}, {...})
```

此函数基本上是重复调用 tex.sprint(<number> n, <string> s, ...) 的快捷方式，对每个提供的参数 table 调用一次。

cprint

此函数接受一个表示要使用的类别码的数字，以及一个字符串 table 或将被推入输入流的字符串参数列表。

```
tex.cprint( 1," 1: $&{\\foo}") tex.print("\\par") -- a lot of \{ s
tex.cprint( 2," 2: $&{\\foo}") tex.print("\\par") -- matching \} s
tex.cprint( 9," 9: $&{\\foo}") tex.print("\\par") -- all get ignored
tex.cprint(10,"10: $&{\\foo}") tex.print("\\par") -- all become spaces
tex.cprint(11,"11: $&{\\foo}") tex.print("\\par") -- letters
tex.cprint(12,"12: $&{\\foo}") tex.print("\\par") -- other characters
tex.cprint(14,"12: $&{\\foo}") tex.print("\\par") -- comment triggers
```

write

```
tex.write(<string> s, ...)
tex.write(<table> t)
```

每个字符串参数都被 TEX 视为一种特殊的输入行，使其适合用作快速转储信息的方式：

- 该行上的所有类别码要么是 “空格（用于’ ’）要么是 “字符（用于其他所有字符）。
- 不附加 `endlinechar`。

如果是 table 参数而不是字符串列表，则必须是要打印的连续字符串数组（第一个非字符串值将停止打印过程）。

11.3.15　辅助函数

round

```
<number> n = tex.round(<number> o)
```

对 Lua 数字 `o` 取整，返回一个在有效 TEX 寄存器值范围内的数字。如果数字初始超出范围，还会产生 “number too big 错误。

scale

```
<number> n = tex.scale(<number> o, <number> delta)
<table> n = tex.scale(table o, <number> delta)
```

将 Lua 数字 `o` 和 `delta` 相乘，返回一个在有效 TEX 寄存器值范围内的取整数字。在 table 版本中，它创建一个 table 的副本，其中所有顶层数值都按此方式缩放。如果相乘后的数字超出范围，还会产生 “number too big 错误。

注意：此函数输出的精度取决于你计算机的体系结构和操作系统，因此请谨慎使用！稍后将添加一个与 LuaTEX 内部 100% 可移植缩放函数的接口。

number 与 romannumeral

这是原语 `number` 和 `romannumeral` 的配套函数。它们可以这样使用：

```
tex.print(tex.romannumeral(123))
```

fontidentifier 与 fontname

第一个仅返回名称，第二个还报告字号。

```
tex.print(tex.fontidentifier(1))
tex.print(tex.fontname(1))
```

sp

```
<number> n = tex.sp(<number> o)
<number> n = tex.sp(<string> s)
```

将数字 o 或表示显式尺寸的字符串 s 转换为缩放点的整数。

对于解析字符串，使用与 LuaTEX 在其类 TEX 输入语言中扫描尺寸说明符时相同的扫描和转换规则（包括为错误值生成错误），但以下情况除外：

- 仅允许显式值，不处理控制序列
- 无限尺寸单位（fil...）不允许
- mu 单位不产生错误（但可能也没有用处）

error 与 show_context

```
tex.error(<string> s)
tex.error(<string> s, <table> help)
```

这会创建一个错误，类似于 errhelp 和 errmessage 的组合。在此错误期间，删除操作被禁用。

help table 的数组部分必须包含字符串，每行错误帮助一个字符串。

在发生错误时，show_context 函数将显示我们当前所处的上下文（在展开中）。

run、finish

这两个函数启动解释并强制结束。一次运行通常归结为 TEX 进入所谓的主循环。获取一个令牌，根据其当前含义执行某些操作。有时操作立即发生，有时需要更多扫描。很多时候令牌被推回输入中。这一切意味着 TEX 扫描器不断推送和弹出输入状态，但最终在所有操作完成后返回主循环。

runtoks

由于 TEX 处于展开、处理字体、排版段落、处理 box、构建页面等复杂舞蹈中，你无法轻易运行嵌套的 TEX 运行（即嵌套主循环）。不过，有一个选项可以通过 runtoks 强制进行局部运行。给定令牌列表寄存器的内容在本地展开，然后我们返回到在 Lua 端触发此展开的位置。也可以传递一个执行某些工作的函数。你必须确保最后 TEX 处于健全状态，而这并不总是容易的。更复杂的机制会使 TEX 本身复杂化（可能也会损害性能），因此这个简单的局部展开循环只能将就。

```
tex.runtoks(<token register>)
tex.runtoks(<lua function>)
```

当 tracingnesting 参数设置为大于 2 的值时，会报告一些关于局部循环状态的信息。

当传递令牌寄存器时，此函数有两个可选参数：

```
tex.runtoks(<token register>,force,grouped)
```

在例如 \edef 内部，runtoks 函数的行为（至少尝试）就像它是一个 \the 。这防止了不良副作用：通常在这样的定义中令牌保持为令牌，（例如）字符不会变成节点。使用第二个参数，你可以强制局部主循环，无论如何。第三个参数添加一层分组。

你可以使用 \endlocalcontrol 或从 Lua 端使用 tex.quittoks 退出局部循环。在这种情况下，你结束一层！当然，最终这可能意味着你到达主层级，在这种情况下额外的结束将触发冗余警告（而不是中止！）。

forcehmode

一个（可能触发错误的）复杂情况的例子是，TEX 期望处于某种状态，比如水平模式，你必须确保在从 Lua 向 TEX 反馈内容时它确实处于该状态。通常用户不会遇到问题，但当你开始写入令牌或节点或进行嵌套运行时，可能会出现需要运行 forcehmode 的情况。对此没有现成方案，拦截可能的情况会削弱 LuaTEX 的灵活性。

hashtokens

```
for i,v in pairs (tex.hashtokens()) do ... end
```

返回名称列表。这对调试很有用，但注意这也会报告可能因局部重定义而在此刻无法访问的控制序列：它严格是哈希表的转储。你可以使用 token.create 检查属性，例如当创建的 table 中的 command 键等于 123 时，你的 cmdname 值为 undefined_cs。

definefont

```
tex.definefont(<string> csname, <number> fontid)
tex.definefont(<boolean> global, <string> csname, <number> fontid)
```

将 csname 与内部字体编号 fontid 关联。当且仅当指定了 global 且为 true 时，定义是全局的（不考虑 globaldefs 的设置）。

11.3.16　处理原语的函数

enableprimitives

```
tex.enableprimitives(<string> prefix, <table> primitive names)
```

此函数接受一个前缀字符串和一个原语名称数组。对于 “prefix 和 “name 的每个组合，tex.enableprimitive
首先验证 “name 是否是实际的原语（它必须由下面解释的 tex.extraprimitives 调用之一返回，或是 TEX82 的一部分，或是 directlua）。如果不是，tex.enableprimitives 不做任何操作，跳到下一对。

但如果是，它会通过连接 “prefix 和 “name 构造一个 csname 变量，除非 “prefix 已经是 “name 的实际前缀。在后一种情况下，它会丢弃 “prefix，直接使用 “name。

然后它会检查构造的 csname 是否存在。如果该 csname 当前未定义（注意：这与 relax 不同），它将全局定义该 csname 使其具有以下含义：运行属于原语 “name 的代码。如果由于某种原因 csname 已定义，它不做任何操作，尝试下一对。

示例：

```
tex.enableprimitives('LuaTeX', {'formatname'})
```

将定义 \LuaTeXformatname ，其内在含义与文档中说明的原语 formatname 相同，前提是控制序列 \LuaTeXformatname 当前未定义。

当 LuaTEX 以 --ini 运行时，只有 TEX82 原语和 directlua 可用，因此没有任何额外原语**可用**。

如果你想让所有新功能以其默认名称可用（就像现在这样），你必须在格式生成文件的开头附近添加

```
\ifx\directlua\undefined \else
    \directlua {tex.enableprimitives('',tex.extraprimitives ())}
\fi
```

或者你可以根据需要为不同子集选择不同的前缀。

不过，调用某种形式的 tex.enableprimitives 非常重要，因为如果不这样做，你最终会得到一个可以运行 Lua 代码但做不了多少其他事情的 TEX82 仿制品。定义的 csname（当然）保存在格式中，并在运行时可用。

extraprimitives

```
<table> t = tex.extraprimitives(<string> s, ...)
```

此函数返回源自所请求字符串值所给定引擎的原语列表。可能的值及其（当前）返回值在下表中给出。此外，还定义了一些特殊的原语 “ 、“/ 和 “-。

名称	值
tex	[Lua 输出]
core	[Lua 输出]
etex	[Lua 输出]
luatex	[Lua 输出]

注意，luatex 不包含 directlua，因为它被认为是核心原语，与所有 TEX82 原语一起，因此它是从'core' 返回的列表的一部分。

运行 tex.extraprimitives 会给你 -ini 启动时的完整原语列表。它完全等同于 tex.extraprimitives("e

primitives

```
<table> t = tex.primitives()
```

此函数返回 LuaTEX 已知的所有原语列表。

11.3.17　核心功能接口

badness

```
<number> b = tex.badness(<number> t, <number> s)
```

此辅助函数在换行计算中很有用。t 和 s 是缩放值；函数返回当总 t 应由总和为 s 的量构成时的不良度。返回的数字是 $100(t/s)^3$ 的合理近似。

tex.resetparagraph

此函数重置 TEX 在遇到新段落时通常会重置的参数。

linebreak

```
local <node> nodelist, <table> info =
    tex.linebreak(<node> listhead, <table> parameters)
```

可识别的参数如下：

名称	类型	说明
pardir	string	
pretolerance	number	
tracingparagraphs	number	
tolerance	number	
looseness	number	
hyphenpenalty	number	
exhyphenpenalty	number	
pdfadjustspacing	number	
adjdemerits	number	
pdfprotrudechars	number	
linepenalty	number	
lastlinefit	number	
doublehyphendemerits	number	
finalhyphendemerits	number	
hangafter	number	
interlinepenalty	number or table	如果是 table，则为类似 interlinepenalties 的数组
clubpenalty	number or table	如果是 table，则为类似 clubpenalties 的数组
widowpenalty	number or table	如果是 table，则为类似 widowpenalties 的数组
brokenpenalty	number	
emergencystretch	number	以缩放点为单位
hangindent	number	以缩放点为单位
hsize	number	以缩放点为单位
leftskip	glue_spec 节点	
rightskip	glue_spec 节点	

parshape	table

注意，displaywidowpenalties 没有接口，你必须自己为 widowpenalties 传递正确的选择。

确保 listhead 是一个正确的段落列表是你自己的责任：此函数不会向其添加任何节点。确切地说，如果你想替换核心换行，你可能需要执行以下操作（当你不在 pre_linebreak_filter 或 linebreak_filter 回调中工作时，或者当从 listhead 开始的原始列表是在水平模式下生成时）：

- 在开头添加一个“缩进 box，可能还有一个 local_par 节点（仅当你需要时）
- 将找到的任何最终黏胶替换为无限惩罚（或者如果最后一个节点不是黏胶，则添加这样的惩罚）
- 在该惩罚节点之后为 parfillskip 添加一个黏胶节点
- 确保所有 prev 指针正确

结果是一个节点列表，如果你想将其赋值给 vbox，它仍然需要 vpack。返回的 info table 包含四个值，全部为数字：

名称	说明
prevdepth	断开段落中最后一行的深度
prevgraf	断开段落中的行数
looseness	断开段落中的实际 looseness 值
demerits	所选方案的总不良度

注意，有几件事你无法通过此函数进行接口：除了通过 pdfadjustspacing 之外，你无法影响字体扩展，因为其设置在其他地方进行。hbadness 和 hfuzz 等也是如此。所有这些都在 hpack 例程中，它通过全局变量获取自己的变量。

shipout

```
tex.shipout(<number> n)
```

将 box 编号 n 发货到输出文件，并清除该 box 寄存器。

getpagestate

此辅助函数报告当前页面状态：empty、box_there 或 inserts_only，以整数值表示。

getlocallevel

此整数报告局部循环的当前层级。它仅用于调试，（相对状态）数字可能随实现而变化。

11.3.18　随机数生成器

由于实际原因,LuaTEX 有自己的随机数生成器。原始的 Lua 随机函数以 tex.lua_math_random 可用。你可以使用新种子通过 init_rand 初始化（lua_math_randomseed 与此等价）。

有三个生成器：normal_rand（不使用参数）、uniform_rand （接受一个在使用前会被取整的数字）和 uniformdeviate，其行为类似于原语，期望一个缩放整数，因此

```
tex.print(tex.uniformdeviate(65536)/65536)
```

将给出一个介于零和一之间的随机数。

11.3.19　与 synctex 相关的函数

以下辅助函数仅在你实现自己的 synctex 逻辑时才有意义。请记住，编辑器中使用的库假定某种逻辑，并且面向 plain 和 LATEX，因此经过十年，用户期望某种行为。

名称	说明
set_synctex_mode	0 为默认值，使用正常 synctex 逻辑，1 使用以下辅助函数设置的值，而 2 还为字形节点设置这些值；3 设置字形和黏胶，4 仅设置字形
set_synctex_tag	设置当前标签（文件）值（遵循保存栈）
set_synctex_line	设置当前行值（遵循保存栈）
set_synctex_no_files	禁用 synctex 文件日志记录
get_synctex_mode	返回当前模式（值见上文）
get_synctex_tag	获取当前设置的标签(文件)值
get_synctex_line	获取当前设置的行值
force_synctex_tag	覆盖标签（文件）值（0 重置）
force_synctex_line	覆盖行值（0 重置）

最后一个有些特殊。由于 SyncTEX 中文件的注册方式，如果我们提供自己的替代方案且想避免这种开销，我们需要显式禁用该功能。传递值 1 将禁用注册。

11.4　texconfig

这是一个创建为空的 table。启动 Lua 脚本可以用一些设置填充此 table，这些设置在加载和执行启动文件后由可执行文件读取。

键	类型	默认值	说明

kpse_init	boolean	true	false 完全禁用 kpathsea 初始化, 并启用以下数值键-值对的解释。（仅当你实现了所有文件查找回调时才取消设置此项!）
shell_escape	string	'f'	使用'y' 或't' 或'1' 无条件启用 \write 18, 'p' 启用 shell_escape_commands 中列出的命令
shell_escape_commands	string		以逗号分隔的命令名称列表，即使 shell_escape 设置为'p' 时也可由 \write 18 执行。逗号周围不要使用空格，任何所需的命令参数用空格分隔，对任何需要的参数或路径引用使用 ASCII 双引号 (")
string_vacancies	number	75000	参见 web2c 文档
pool_free	number	5000	参见 web2c 文档
max_strings	number	15000	参见 web2c 文档
strings_free	number	100	参见 web2c 文档
nest_size	number	50	参见 web2c 文档
max_in_open	number	15	参见 web2c 文档
param_size	number	60	参见 web2c 文档
save_size	number	4000	参见 web2c 文档
stack_size	number	300	参见 web2c 文档
dvi_buf_size	number	16384	参见 web2c 文档
error_line	number	79	参见 web2c 文档
half_error_line	number	50	参见 web2c 文档
max_print_line	number	79	参见 web2c 文档
hash_extra	number	0	参见 web2c 文档
pk_dpi	number	72	参见 web2c 文档
trace_file_names	boolean	true	false 禁用 TEX 正常的文件打开-关闭反馈（假设回调会处理这些）
trace_extra_newline	boolean	false	在 \tracingmacros 中添加额外换行
file_line_error	boolean	false	生成 file:line 样式的错误消息

halt_on_error	boolean	false	在遇到第一个错误时中止运行
formatname	string		如果命令行上未给出格式名称，则将首先测试此键，而不是直接退出
jobname	string		如果命令行上未给出输入文件名，则将首先测试此键，而不是直接放弃
level_chr	number		放在被跟踪宏前面的字符（见下一个值）
level_max	number		当大于零时，设置 \tracingmacros 时将显示输入嵌套层级；超过此值的层级将被裁剪，并在前面显示层级
check_dvi_total_pages	boolean		在 DVI 输出模式下，如果为 true，当页数超过 65535 时中止运行。这是默认行为。如果为 false，运行将像在 TEX 中一样继续。
texlua_img	boolean		如果为 true, 允许在 texlua 模式下访问 img 库。如果 false（默认），img 库在 texlua 模式下不可用（在 1.22.0 之前的 LuaTEX 版本中是无条件可用的）。注意，此设置为**实验性**，可能随时被移除，恕不另行通知。

注意：与 web2c 参数匹配的数值仅在 kpse_init 显式设置为 false 时使用。在所有其他情况下，使用 texmf.cnf 中的正常值。

你可以启动自己的嵌套层级可视化器，例如：

```
callback.register("input_level_string",function(n)
 if tex.tracingmacros > 0 and tex.count.tracingstacklevels > 0 then
    if tex.tracingmacros > 1 then
        return "! " .. string.rep(">",n) .. " "
    end
  end
  return ""
end)
```

或者，与其他引擎同步（未检查）：

```
\directlua {
  callback.register("input_level_string", function(n)
    if tex.tracingmacros > 0 then
      local l = tex.count.tracingstacklevels
      if l > 0 then
        return string.rep("~",l) .. string.rep(".",n-l)
      end
    end
    return ""
  end)
}
```

11.5 texio

此库负责底层 I/O 接口：写入日志文件和/或控制台。

11.5.1 write

```
texio.write(<string> target, <string> s, ...)
texio.write(<string> s, ...)
```

不带 target 参数时,将所有给定字符串写入 TEX 此刻写入消息的相同位置。如果 batchmode 生效，则仅写入日志，否则写入日志和终端。可选的 target 可以是三种可能性之一：term、log 或 term and log。

注意：如果给定多个字符串，且第一个字符串是或可能是上述目标之一，则必须显式指定 target，以防止 Lua 将第一个字符串解释为目标。

11.5.2 write_nl

```
texio.write_nl(<string> target, <string> s, ...)
texio.write_nl(<string> s, ...)
```

此函数的行为类似于 texio.write，但确保给定字符串出现在新行的开头。如果你只想移到下一行，可以传递一个空字符串。

11.5.3 setescape

你可以通过传递零值来禁用控制字符的 \textasciicircum \textasciicircum {} 转义。

11.5.4 closeinput

此函数应谨慎使用。它的作用类似于 endinput，但在 Lua 端。你可以使用它（某种程度上）强制跳回 TEX。通常 Lua 只会收集打印内容，最后提升一个输入层级并刷新这些打印内容。此函数可以帮助你停留在当前层级，但你需要知道自己在做什么（或者更准确地说：TEX 对输入做了什么）。

11.6　token

11.6.1　扫描器

令牌库提供了在 Lua 级别拦截输入并处理它的手段。该库提供了一个基本扫描器基础设施，可用于编写接受各种参数的宏。此接口刻意保持通用，性能也相当不错。可以构建额外的解析器，而不会有太多开销。如何从中受益取决于宏包编写者，因为 LuaTEX 背后的主要原则是提供最小的工具集而非解决方案。扫描函数可能是最引人入胜的。

函数	参数	结果
`scan_keyword`	string	如果给定关键字被吞掉则返回 true；与常规 TEX 关键字扫描器一样，这是不区分大小写的（且基于 ASCII）
`scan_keyword_cs`	string	如果给定关键字被吞掉则返回 true；此变体区分大小写，也适用于 UTF8
`scan_int`		返回一个整数
`scan_real`		从例如 `1`、`1.1`、`.1` 返回一个数字，可带有可选的合并符号
`scan_float`		从例如 `1`、`1.1`、`.1`、`1.1E10`、`.1e-10` 返回一个数字，可带有可选的合并符号
`scan_dimen`	infinity、mu-units	返回一个表示尺寸的数字和/或两个表示填充和阶的数字
`scan_glue`	mu-units	返回一个黏胶规格节点
`scan_toks`	definer、expand	返回一个令牌 table
`scan_code`	bitset	如果字符的类别在给定的 bitset（表示类别码）中则返回该字符
`scan_string`		返回 `{}` 之间给出的字符串，作为 `\macro` 或作为类别码 11 或 12 的字符序列
`scan_argument`	boolean	此函数类似于 `scanstring`，但也接受 `\cs`
`scan_word`		返回类别码为 11 或 12 的字符序列作为字符串
`scan_csname`		扫描 `\foo` 后返回 `foo`

scan_list	获取一个 box 规范并返回一个 [h|v]list 节点

除了扫描令牌的那个之外，扫描器可以被认为是稳定的。scan_code 函数接受一个可选数字，keyword 函数接受一个正常的 Lua 字符串。infinity 布尔值表示我们还允许 fill 作为尺寸，mu-units 标志扫描器我们期望数学单位。扫描令牌时，我们可以指示正在定义一个宏，在这种情况下结果还会提供关于期望什么参数的信息，在结果中这通过一个分隔令牌与含义分开。expand 标志决定列表是否会被展开。

scan_argument 函数展开给定的参数。当扫描带花括号的参数时，可以通过传递 false 来禁止展开（默认为 true）。对于控制序列，传递 false 将导致一级展开（宏的含义）。

字符串扫描器扫描花括号之间的内容并在过程中展开，或者当看到控制序列时返回其含义。否则它会扫描类别码为 letter 或 other 的字符。因此，给定以下定义：

```
\def\bar{bar}
\def\foo{foo-\bar}
```

bar：bar foo：foo myfirstbar：myfirstbar

这是按以下方式输入的：

你看到报告的是 bar 而不是 myfirstbar 的原因是，\bar 已在之前的段落中使用过。

如果我们现在说：

```
\def\foo{}
```

如果我们说

```
\def\foo{\bar}
```

当从 Lua 扫描时，我们根本不处于定义（未定义）宏的模式。在那里我们只得到真正的原始未定义宏令牌。

```
\directlua{local t = token.get_next() tex.print(t.id.." "..t.tok)}\myfirstbar
\directlua{local t = token.get_next() tex.print(t.id.." "..t.tok)}\mysecondbar
\directlua{local t = token.get_next() tex.print(t.id.." "..t.tok)}\mythirdbar
```

因此，我们确实得到了一个唯一的令牌，因为毕竟我们需要某种可使用和垃圾回收的 Lua 对象，但它基本上是同一个，表示一个未定义的控制序列。

11.7　kpse

此库提供两个独立但几乎相同的 kpathsea 文件搜索功能接口：有一个“正常的过程式接口，与 LuaTEX 本身共享其 kpathsea 实例；还有一个完全面向对象的接口。

11.7.1 set_program_name

库查找变量的方式由 texmf.cmf 文件驱动，其中当前设置的程序名称用作过滤器。你可以通过 default_texmfcnf 检查使用了哪个文件。

在使用搜索库之前，必须先初始化其数据库。有三种可能性，其中两种属于过程式接口。

首先，当 LuaTEX 用于排版文档时，此初始化自动发生，kpathsea 可执行文件和程序名称被设置为 luatex（即，除非用户的启动脚本明确禁止。详见 5.1）。

其次, 在 texlua 模式下, 必须通过 kpse.set_program_name 函数显式初始化, 它设置 kpathsea 可执行文件（以及可选的程序）名称。

```
kpse.set_program_name(<string> name)
kpse.set_program_name(<string> name, <string> progname)
```

第二个参数控制 texmf.cnf 配置文件中 “点号值的使用，默认为第一个参数。

第三，如果你更喜欢面向对象的接口，你必须调用不同的函数。它有相同的参数，但返回一个 userdata 变量。

```
local kpathsea = kpse.new(<string> name)
local kpathsea = kpse.new(<string> name, <string> progname)
```

除了这两个函数之外，接口的调用约定完全相同。根据所选接口，你调用 kpse.find_file 或 kpathsea:find_file，具有相同的参数和返回值。

11.7.2 record_input_file

这两个函数可用于注册已使用的文件。因为回调可以自己加载文件，你可能需要这些辅助函数（如果你使用记录功能的话）。

```
kpse.record_input_file(<string> name)
kpse.record_output_file(<string> name)
```

11.7.3 find_file

库中最常用的函数是 find_file:

```
<string> f = kpse.find_file(<string> filename)
<string> f = kpse.find_file(<string> filename, <string> ftype)
<string> f = kpse.find_file(<string> filename, <boolean> mustexist)
<string> f = kpse.find_file(<string> filename, <string> ftype, <boolean> mustexist)
<string> f = kpse.find_file(<string> filename, <string> ftype, <number> dpi)
```

参数:

filename 你要查找的文件名，带或不带扩展名。

ftype 映射到 kpsewhich 的 -format 参数。支持的 ftype 值与独立 kpsewhich 程序支持的值相同: local list = ”afm”, ”base”, ”bib”, ”bitmap font”, ”bst”, ”cid maps”, ”clua”, ”cmap files”, ”cnf”, ”cweb”, ”dvips config”, ”enc files”, ”fmt”, ”font feature files”, ”gf”, ”graphic|/|figure”, ”ist”, ”lig files”, ”ls-R”, ”lua”, ”map”, ”mem”, ”MetaPost support”, ”mf”, ”mfpool”,

”mft”, ”misc fonts”, ”mlbib”, ”mlbst”, ”mp”, ”mppool”, ”ocp”, ”ofm”, ”opentype fonts”, ”opl”, ”other binary files”, ”other text files”, ”otp”, ”ovf”, ”ovp”, ”pdftex config”, ”pk”, ”PostScript header”, ”subfont definition files”, ”tex”, ”TeX system documentation”, ”TeX system sources”, ”texmfscripts”, ”texpool”, ”tfm”, ”Troff fonts”, ”truetype fonts”, ”type1 fonts”, ”type42 fonts”, ”vf”, ”web”, ”web2c files”, table.sort(list) context(”
tttf %, t”,list)

默认类型为 `tex`。注意：这与 kpsewhich 不同，后者尝试从提供的扩展名推断文件类型。

`mustexist` 类似于 kpsewhich 的 `-must-exist`，默认为 `false`。如果你指定 `true`（或非零整数），则 kpathsea 库将同时搜索磁盘和 `ls-R` 数据库。

`dpi` 这用于格式 `pk`、`gf` 和 `bitmap font` 的尺寸参数。

如果指定了 `--output-directory` 且值为相对路径名，则首先在此处搜索文件，如果失败，则在标准树中搜索。

11.7.4 lookup

还有一种更强大（但更慢）的通用文件查找方法。它为每个找到的文件返回一个字符串。

```
<string> f, ... = kpse.lookup(<string> filename, <table> options)
```

选项与 `kpsewhich` 的命令行参数对应：

键	类型	说明
`debug`	number	为此查找设置调试标志
`format`	string	使用特定文件类型（见上面的列表）
`dpi`	number	为此查找使用此分辨率；默认 600
`path`	string	在给定路径中搜索
`all`	boolean	输出所有匹配项，而不仅仅是第一个
`mustexist`	boolean	必要时同时搜索磁盘和 ls-R
`mktexpk`	boolean	为此查找禁用/启用 mktexpk 生成
`mktextex`	boolean	为此查找禁用/启用 mktextex 生成
`mktexmf`	boolean	为此查找禁用/启用 mktexmf 生成
`mktextfm`	boolean	为此查找禁用/启用 mktextfm 生成

subdir	string 或 table	仅输出目录部分以给定字符串结尾的匹配项

如果指定了 --output-directory 且值为相对路径名，则首先在此处搜索文件，然后在标准树中搜索。

11.7.5 init_prog

为需要生成位图字体的程序提供额外初始化。

```
kpse.init_prog(<string> prefix, <number> base_dpi, <string> mfmode)
kpse.init_prog(<string> prefix, <number> base_dpi, <string> mfmode, <string> fallback)
```

11.7.6 readable_file

测试（绝对）文件名是否为可读文件。

```
<string> f = kpse.readable_file(<string> name)
```

返回值是你应该使用的实际绝对文件名，因为磁盘名称并不总是与请求的名称相同，这是由于别名和例如 MSDOS 下的系统特定处理。如果文件不存在或不可读则返回 nil。

11.7.7 expand_path

类似于 kpsewhich 的 -expand-path：

```
<string> r = kpse.expand_path(<string> s)
```

11.7.8 expand_var

类似于 kpsewhich 的 -expand-var：

```
<string> r = kpse.expand_var(<string> s)
```

11.7.9 expand_braces

类似于 kpsewhich 的 -expand-braces：

```
<string> r = kpse.expand_braces(<string> s)
```

11.7.10 in_name_ok

如果 fname 可打开读取则返回 true，否则返回 false 并向标准错误写入一条消息。

```
<boolean> r = kpse.in_name_ok(<string> fname)
```

11.7.11 in_name_ok_silent_extended

```
<boolean> r = kpse.in_name_ok_silent_extended(<string> fname)
```

如果 fname 可打开读取则返回 true；还会检查 TEXMFVAR 和 TEXMFSYSVAR 的值是否为绝对文件名。否则返回 false，但不向标准错误写入消息。

11.7.12　out_name_ok

如果 fname 可打开写入则返回 true。

```
<boolean> r = kpse.out_name_ok(<string> fname)
```

11.7.13　out_name_ok_silent_extended

```
<boolean> r = kpse.out_name_ok_silent_extended(<string> fname)
```

如果 fname 可打开写入则返回 true；还会检查 TEXMFVAR 和 TEXMFSYSVAR 的值是否为绝对文件名。否则返回 false，但不向标准错误写入消息。

11.7.14　show_path

类似于 kpsewhich 的 -show-path：

```
<string> r = kpse.show_path(<string> ftype)
```

11.7.15　var_value

类似于 kpsewhich 的 -var-value：

```
<string> r = kpse.var_value(<string> s)
```

11.7.16　version

返回 kpathsea 版本字符串。

```
<string> r = kpse.version()
```

11.7.17　check_permission

检查 filename 是否可执行。如果可执行则返回 1、filename 或安全的 cmd 替代方案，否则返回 0 和错误消息。

```
<string> res, cmd = kpse.check_permission(<string> filename)
```

第 12 章 图形库

12.1 img

img 库可用作 pdfximage 和 pdfrefximage 的替代方案,以及相关“卫星命令(如 \pdfximagebbox ）的替代。图像对象也可在虚拟字体中通过 7.3 中列出的 image 命令使用。

12.1.1 new

```
<image> var = img.new()
<image> var = img.new(<table> image_spec)
```

此函数创建一个类型为 “image 的 userdata 对象。image_spec 参数是可选的。如果给出，它必须是一个 table，且该 table 必须包含 filename 键。其他一些键也可能有用，下面逐一说明。

你可以这样写

```
a = img.new()
```

然后

```
a.filename = "foo.png"
```

或者你可以直接将文件名（以及部分或全部其他键）放入一个 table，像这样：

```
a = img.new({filename='foo.pdf', page=1})
```

生成的 <image> userdata 对象允许访问一组用户指定的值，以及一组通常由 LuaTEX 自动填充和更新的值。其中一些来自实际的图像文件，另一些则更新以反映对象的 PDF 输出状态。

有一个必需的用户指定字段：文件名（filename）。它可以选择性地附加所请求的图像尺寸（width、depth、height）、用户指定的图像属性（attr）、所请求的 PDF 页标识（page）、所请求的包围盒（pagebox）用于 PDF 包含、所请求的色彩空间对象（colorspace）。

函数 img.new 不访问实际的图像文件，它只创建 <image> userdata 对象并初始化一些内存结构。<image> 对象及其内部结构会被自动垃圾回收。

一旦图像被扫描，<image> 中除 width、height 和 depth 之外的所有值都会冻结，你不能再更改它们。

你可以使用 pdf.setignoreunknownimages(1)(或在 TEX 端使用 pdfvariable ignoreunknownimages）来绕过未找到已知图像类型（根据名称或序言判断）时的退出。注意：这无法捕获无效图像，我们也不能保证没有副作用。零尺寸图像在请求时仍会被包含。不设置特殊标志。正确的工作流程不应依赖这种捕获，而应确保图像有效。

12.1.2 fields

```
<table> keys = img.fields()
```

此函数返回所有可能的 image_spec 键的列表，包括用户提供的和自动生成的。

字段名	类型	说明
attr	string	供 LuaTEX 使用的图像属性
bbox	table	包含 4 个包围盒尺寸 llx、lly、urx 和 ury 的 table，覆盖 pagebox 条目
colordepth	number	色彩空间使用的位数
colorspace	number	色彩空间对象编号
depth	number	供 LuaTEX 使用的图像深度
filename	string	图像文件名
filepath	string	图像的完整（展开后）文件名
height	number	供 LuaTEX 使用的图像高度
imagetype	string	以下之一：pdf、png、jpg、jp2 或 jbig2
index	number	PDF 图像名称后缀
objnum	number	PDF 图像对象编号
page	number	所请求图像页的标识
pagebox	string	所请求的包围盒，以下之一：none、media、crop、bleed、trim、art
pages	number	可用页数总数
rotation	number	来自所包含 PDF 文件的图像旋转，以 90 度的倍数计。
stream	string	/Xobject /Form 对象的原始流数据
transform	number	图像变换，整数 0..7
orientation	number	(jpeg) 图像方向，整数 1..8（0 表示未设置）
width	number	供 LuaTEX 使用的图像宽度
xres	number	水平自然图像分辨率（以 dpi 为单位）
xsize	number	自然图像宽度
yres	number	垂直自然图像分辨率（以 dpi 为单位）
ysize	number	自然图像高度

visiblefilename	string	设置后，此名称将以 PTEX 规范的形式出现在 PDF 文件中；当赋值为空字符串时不写入文件；否则使用自然文件名
userpassword	string	打开 PDF 文件所需的用户密码
ownerpassword	string	打开 PDF 文件所需的所有者密码
keepopen	boolean	保持 PDF 文件打开
nobbox	boolean	不为流添加包围盒规范
nolength	boolean	不为流添加 length 键也不压缩
nosize	boolean	不为流添加尺寸字段

width、height 或 depth 中的运行中（未定义）尺寸在 Lua 中表示为 nil，因此如果你想以“自然尺寸加载图像，不必指定这三个字段中的任何一个。

stream 参数允许从给出流内容的字符串构造一个 /XObject /Form 对象，例如一个填充矩形：

```
a.stream = "0 0 20 10 re f"
```

写入图像时，会创建一个 /Xobject /Form 对象，就像嵌入 PDF 文件写入时一样。该对象只写出一次。stream 键要求同时给出 bbox table。stream 键与 filename 键冲突。transform 键在使用 stream 时照常工作。

bbox 键需要一个包含四个包围盒值的 table，例如：

```
a.bbox = { "30bp", 0, "225bp", "200bp" }
```

这会替换并覆盖任何给定的 pagebox 值；给定 bbox 时，嵌入 PDF 文件附带的盒尺寸将被忽略。图像缩放时，xsize 和 ysize 尺寸会相应设置。对于非 PDF 图像，bbox 参数被忽略。

transform 允许以 90 度为步长镜像和旋转图像。默认值 0 给出未镜像、未旋转的图像。值 1 – 3 分别给出逆时针旋转 90、180 或 270 度，而值 4 – 7 则先镜像图像，再逆时针旋转 90、180 或 270 度。transform 操作给出与你用图形工具在外部预处理图像再由 LuaTEX 使用时相同的视觉结果。如果要嵌入的 PDF 文件已包含 /Rotate 规范，旋转结果是 /Rotate 旋转后再执行 transform 操作的组合。

12.1.3　scan

```
<image> var = img.scan(<image> var)
<image> var = img.scan(<table> image_spec)
```

当你对一个新图像执行 img.scan(a) 时，文件会被扫描，xsize、ysize、图像 type、pages 数量和分辨率等变量会被提取出来。width、height、depth 字段会根据图像尺寸设置，前提

是它们尚未被显式赋值。对于给定的图像变量，图像文件绝不会被扫描超过一次。之后所有的 img.scan(a) 调用只会重新设置尺寸（如果用户在此期间更改了它们）。

为了便于使用，你可以直接执行

```
<image> a = img.scan { filename = "foo.png" }
```

而无需事先 img.new。

此时还没有写入任何内容，所以你可以执行 a=img.scan，检索可用信息如图像宽度和高度，然后通过 a=nil 丢弃 a。在这种情况下，PDF 中不会保留任何图像对象，使用的内存也会被自动清理。

12.1.4　copy

```
<image> var = img.copy(<image> var)
<image> var = img.copy(<table> image_spec)
```

如果你执行 a = b，则两个变量指向同一个 <image> 对象。如果你想以不同尺寸写出一个图像，可以执行 b = img.copy(a)。

之后，a 和 b 仍引用同一个实际图像字典，但 b 的尺寸现在可以从刚从 a 复制来的初始值进行更改。

12.1.5　write、immediatewrite、immediatewriteobject

```
<image> var = img.write(<image> var)
<image> var = img.write(<table> image_spec)
```

通过 img.write(a) 分配一个 PDF 对象编号，并生成一个子类型为 image 的规则节点放入输出列表。由此图像 a 被放入页面流，当前页面发货完成后图像文件被写出为一个图像流对象。

同样你可以使用简洁的调用方式

```
img.write { filename = "foo.png" }
```

<image> 变量会被返回，以备后续处理使用。你也可以写出一个对象。

通过 img.immediatewrite(a) 分配一个 PDF 对象编号，图像 a 的图像文件立即作为图像流对象写入 PDF 文件（类似于 immediatepdfximage）。图像流字典的对象编号随后可通过 objnum 键获取。不生成 pdf_refximage whatsit 节点。你需要调用 img.write(a) 或 img.node(a) 才能让图像出现在页面上，或者通过其他方式引用它；否则你将在 PDF 文件中留下一个悬空的图像对象。

```
<image> var = img.immediatewrite(<image> var)
<image> var = img.immediatewrite(<table> image_spec)
```

这里同样可以使用简洁的调用方式

```
a = img.immediatewrite { filename = "foo.png" }
```

<image> 变量会被返回，你很可能需要它。

下一个函数有些特殊，它从一个（PDF）图像文件复制一个对象。此功能是实验性的，可能会消失。

```
<integer> objnum = img.immediatewriteobject(<image> var, <integer> objnum)
<integer> objnum = img.immediatewriteobject(<table> image_spec, <integer> objnum)
```

12.1.6 node

```
<node> n = img.node(<image> var)
<node> n = img.node(<table> image_spec)
```

此函数分配一个 PDF 对象编号并返回一个子类型为 pdf_refximage 的 whatsit 节点，填充有图像参数 width、height、depth 和 objnum。这里同样可以使用简洁的调用方式：

```
n = img.node ({ filename = "foo.png" })
```

以下示例输出一个图像：

```
node.write(img.node{filename="foo.png"})
```

12.1.7 types

```
<table> types = img.types()
```

此函数返回一个支持的图像文件类型名称列表，目前包括 pdf、png、jpg、jp2（JPEG 2000）和 jbig2。

12.1.8 boxes

```
<table> boxes = img.boxes()
```

此函数返回一个支持的 PDF 页面盒名称列表，目前包括 media、crop、bleed、trim 和 art，均为小写。

PDF 文件在确定其属性后保持打开状态。当引用该图像的页面被刷新时包含完成后，文件才关闭。这意味着如果一个页面上有数千张图像，你的操作系统可能会决定中止本次运行。当你从一个 PDF 文件中包含多个页面时，可以在分配图像对象时设置 keepopen 标志，或者在通过 useimageresource 引用图像时传递 keepopen 指令。这仅在嵌入多个页面时才有意义。应用于 saveimageresource 的 immediate 也会在包含后强制关闭。

```
\immediate\useimageresource{foo.pdf}%
          \saveimageresource         \lastsavedimageresourceindex % closed
          \useimageresource{foo.pdf}%
          \saveimageresource         \lastsavedimageresourceindex % kept open
          \useimageresource{foo.pdf}%
          \saveimageresource keepopen\lastsavedimageresourceindex % kept open

\directlua{img.write(img.scan{ file = "foo.pdf" })}                  % closed
\directlua{img.write(img.scan{ file = "foo.pdf", keepopen = true })} % kept open
```

12.2　mplib

MetaPost 库接口在 table mplib 中注册自身。它基于 MPlib 版本 [Lua 输出]。

12.2.1　new

要创建一个新的 MetaPost 实例，调用

```
<mpinstance> mp = mplib.new({...})
```

这会创建 mp 实例对象。参数 hash 可以有多个不同字段，如下：

名称	类型	说明	默认值
error_line	number	错误行宽	79
print_line	number	ps 输出中的行长度	100
random_seed	number	初始随机种子	随机值
math_mode	string	使用的数制：scaled、double 或 decimal	scaled
interaction	string	交互模式：batch、nonstop、scroll 或 errorstop	errorstop
job_name	string	--jobname	mpout
find_file	function	用于查找文件的函数	仅本地文件

在 MPlib 的 LuaTEX 版本中，二进制模式不再可用。它没有实际优势，反而带来大量具有平台特定属性的额外库，我们现在可以避免这些。我们可能会在某个时候引入高分辨率 scaled 变体，但前提是这在性能上有所回报。

find_file 函数应具有以下形式：

```
<string> found = finder (<string> name, <string> mode, <string> type)
```

其中：

名称	所请求的文件
mode	文件模式：r 或 w
type	文件种类，以下之一：mp、tfm、map、pfb、enc

返回找到文件的完整路径名，如果找不到文件则返回 nil。

注意，新版本的 MPlib 不再使用二进制 mem 文件，因此预加载一组宏的方法就是在第一次 execute 调用中以 input 命令开始。

当你处理一段以 btex 开头、以 etex 或 verbatimtex 结尾的文本片段时，MetaPost 的 texscriptmode 参数控制空格和换行如何被处理。默认值为 1。可能的值有：

名称	含义
0	无换行
1	verbatimtex 中的换行
2	verbatimtex 和 etex 中的换行
3	verbatimtex 中不去除首尾空白
4	verbatimtex 和 btex 中不去除首尾空白

这样 Lua 处理器（分配给 make_text）就可以按自己的意愿处理。etex 后面必须跟一个空格或;，或位于行尾；前面必须跟一个空格或位于行首。

12.2.2 statistics

你可以通过以下方式请求统计信息：

```
<table> stats = mp:statistics()
```

此函数返回 MPlib 实例的重要统计数据。共有四个字段，给出四个已分配对象类中每类使用项目的最大数量：

字段	类型	说明
memory	number	内存大小
hash	number	哈希表大小
params	number	同时存在的宏参数
open	number	输入文件嵌套层级

注意在新版本的 MPlib 中，这仅供参考。所有对象都是动态分配的，因此除非可用的系统内存耗尽，否则不会有空间不足的风险。

12.2.3 execute

你可以通过调用以下函数让 MetaPost 解释器运行一段代码

```
<table> rettable = execute(mp,"metapost code")
```

用于处理各种 MetaPost 语言输入。务必检查 rettable.status（见下文），因为当发生致命的 MetaPost 错误时，MPlib 实例此后将不可用。

一般来说，最好保持代码段较小，但请注意所有代码段都必须遵循正确的语法，就像每个代码段都是一个小文件一样。例如，你不能将一条语句拆分到多个代码段中。

与通常的独立 mpost 命令不同，第一个代码段开头 N.B. 没有隐含的 “input。

12.2.4　finish

```
<table> rettable = finish(mp)
```

如果由于某种原因你想在处理尚未实际完成时停止使用某个 MPlib 实例, 可以调用 finish。最终，使用的内存会被释放、打开的文件会被 Lua 垃圾回收器关闭，但显式的 finish 是捕获输出流最后部分的唯一方法。

12.2.5　结果表

execute 和 finish 的返回值是一个 table，包含几个可能的键（只有 status 保证一定存在)。

字段	类型	说明
log	string	输出到 “log 流
term	string	输出到 “term 流
error	string	输出到 “error 流（仅用于 “内存不足）
status	number	返回值：0 = 正常，1 = 警告，2 = 错误，3 = 致命错误
fig	table	生成的图形数组（如果有）

当 status 等于 3 时，你应立即停止使用此 MPlib 实例，它已不再能够处理输入。

如果存在，fig 数组中的每个条目都是一个表示图形对象的 userdata，每个对象都有一些可以调用的方法:

字段	类型	说明
boundingbox	function	返回包围盒，为 4 个值的数组
postscript	function	返回一个字符串，即 fig 的 ps 输出。此函数接受两个可选整数参数，分别用于指定 prologues（第一个参数）和 procset（第二个参数）的值
svg	function	返回一个字符串,即 fig 的 svg 输出。此函数接受一个可选整数参数，用于指定 prologues 的值
objects	function	返回此 fig 中图形对象的实际数组
copy_objects	function	返回此 fig 中图形对象数组的深拷贝

filename	function	独立模式下此 fig 的 PostScript 输出会写入的文件名
width	function	fontcharwd 值
height	function	fontcharht 值
depth	function	fontchardp 值
italcorr	function	fontcharit 值
charcode	function	（取整后的）charcode 值

注意：对于任何一个 fig 对象，你只能调用一次 fig:objects()！

当包围盒表示一个 “取反矩形时，即第一组坐标大于第二组坐标时，该图为空。

图形对象有多种类型，每种类型都有不同的可访问值列表。类型包括：fill、outline、text、start_clip、stop_clip、start_bounds、stop_bounds、special。

有一个辅助函数（mplib.fields(obj)）可用于获取特定对象的可访问值列表，但你也可以直接使用下面给出的 table。

所有图形对象都有一个 type 字段，以字符串值给出对象类型；下表中不再显式列出。在下文中，number 是 PostScript 点，以浮点数表示，除非另有说明。类型为 table 的字段值将在下一节说明。

fill

字段	类型	说明
path	table	节点列表
htap	table	反向轨迹的节点列表
pen	table	笔的节点
color	table	对象的颜色
linejoin	number	线连接样式（裸数字）
miterlimit	number	miterlimit
prescript	string	prescript 文本
postscript	string	postscript 文本

htap 和 pen 条目是可选的。

outline

字段	类型	说明
path	table	节点列表
pen	table	笔的节点
color	table	对象的颜色

linejoin	number	线连接样式（裸数字）
miterlimit	number	miterlimit
linecap	number	线端点样式（裸数字）
dash	table	虚线列表的表示
prescript	string	prescript 文本
postscript	string	postscript 文本

dash 条目是可选的。

text

字段	类型	说明
text	string	文本
font	string	字体 tfm 名称
dsize	number	字号
color	table	对象的颜色
width	number	
height	number	
depth	number	
transform	table	文本变换
prescript	string	prescript 文本
postscript	string	postscript 文本

special

字段	类型	说明
prescript	string	special 文本

start_bounds、start_clip

字段	类型	说明
path	table	节点列表

stop_bounds、stop_clip

此处没有可用字段。

12.2.6　辅助表格式

路径和笔

路径和笔（就 MPlib 而言，笔实际上只是一种特殊类型的路径）由一个数组表示，每个条目是一个表示节点的 table。

字段	类型	说明
`left_type`	string	存在时：端点，但通常不存在
`right_type`	string	同 `left_type`
`x_coord`	number	此节点的 X 坐标
`y_coord`	number	此节点的 Y 坐标
`left_x`	number	此节点前控点的 X 坐标
`left_y`	number	此节点前控点的 Y 坐标
`right_x`	number	此节点后控点的 X 坐标
`right_y`	number	此节点后控点的 Y 坐标

有一个特殊情况：（可能经过变换的）椭圆笔除了包含节点列表的数组部分外，还有一个值为 `elliptical` 的额外字符串键 `type`。

颜色

颜色是一个包含 0、1、3 或 4 个值的整数数组：

字段	类型	说明
`0`	仅标记	无值
`1`	灰度	一个在 $(0,1)$ 范围内的值，“黑为 0
`3`	RGB	三个在 $(0,1)$ 范围内的值，“黑为 $0,0,0$
`4`	CMYK	四个在 $(0,1)$ 范围内的值，“黑为 $0,0,0,1$

如果内部对象的颜色模型是 `uninitialized`，则它被初始化为在 `shipout` 时生效的色彩空间 `defaultcolormodel` 中表示“黑的值。

变换

每个变换是一个六元素数组。

索引	类型	说明
`1`	number	表示 x

2	number	表示 y
3	number	表示 xx
4	number	表示 yx
5	number	表示 xy
6	number	表示 yy

注意平移（索引 1 和 2）在前。这与 PostScript 中的顺序不同，在 PostScript 中平移在最后。

虚线

每个 dash 是一个双元素 hash，使用与 PostScript 相同的模型表示虚线列表。dashes 是一个由 “开和 “关值组成的数组，offset 是图案的相位。

字段	类型	说明
dashes	hash	开-关数字的数组
offset	number	起始偏移值

12.2.7　笔和 pen_info

有一个辅助函数（pen_info(obj)）返回一个 table，其中包含所用笔的一些重要特性（所有值都是浮点数）：

字段	类型	说明
width	number	笔的宽度
sx	number	x 比例
rx	number	xy 乘数
ry	number	yx 乘数
sy	number	y 比例
tx	number	x 偏移
ty	number	y 偏移

12.2.8　字符尺寸信息

这些函数查找指定字体中一个字形的尺寸。fontname 与 infont 的参数名称相同；char 是一个范围在 0 到 255 之间的字形 id；返回的 w 以 AFM 单位表示。

char_width

```
<number> w = char_width(mp,<string> fontname, <number> char)
```

char_height

```
<number> w = char_height(mp,<string> fontname, <number> char)
```

char_depth

```
<number> w = char_depth(mp,<string> fontname, <number> char)
```

get_[boolean|numeric|string|path]

当脚本调用将你从 MetaPost 运行（暂时）带回 Lua 时，你可以访问变量，但前提是它们是已知的（例如循环变量等匿名 capsule 不可访问）。

```
<boolean> w = get_boolean(mp,<string> name)
<number>  n = get_numeric(mp,<string> name)
<string>  s = get_string (mp,<string> name)
<table>   p = get_path   (mp,<string> name)
```

路径以一个包含子 table 的 table 返回，每个子 table 有六个数字：点坐标、前控点和后控点。`cycle` 字段指示路径是否是闭合的。

第 13 章 字体加载器

fontloader 库在某种程度上独立于其余部分：它可以把字体加载到一个 Lua 表中，然后再把该表转换成适合 TEX 使用的表。这个库改编自 FontForge 的一个子集，因此它以类似的视角呈现字体（这在调试时很有优势）。我们不会在这里详细讨论 OpenType，因为 Microsoft 网站已经提供了足够的相关信息。加载器返回的表与标准相去不远。我们没有扩展加载器的计划（它甚至可能在某个时候变成一个外部模块）。

13.1 快速获取字体信息

当你想按名字定位字体时，需要一些隐藏在字体文件中的基本信息。为此我们提供了一个高效的辅助函数，它无需加载整个字体就能获取基本信息。通常这个辅助函数用于创建字体（名称）数据库。

```
<table> info =
    fontloader.info(<string> filename)
```

此函数返回 `nil`、一个 `table`，或者一个由小表组成的数组（在 TrueType 字体集合的情况下）。返回的表中会包含文件所定义字体的一些相当有用的信息项：

键	类型	说明
`fontname`	string	字体的 PostScript 名字
`fullname`	string	字体的正式名称
`familyname`	string	此字体所属的族名
`weight`	string	表示字体粗细值的字符串
`version`	string	字体内部版本号
`italicangle`	float	倾斜角
`units_per_em`	number	基于 PostScript 的字体为 1000，TrueType 通常为 2048
`pfminfo`	table	（见 13.6.6）

通过此函数获取信息比正常加载字体高效得多（有时是数量级的提升），因此在你想根据目录内容创建可用字体词典时非常方便。

13.2　加载 OpenType 或 TrueType 文件

如果你想使用 OpenType 字体，就必须从某处获取度量信息。使用 `fontloader` 库，获取该信息最简单的方法如下：

```
function load_font (filename)
  local metrics = nil
  local font = fontloader.open(filename)
  if font then
     metrics = fontloader.to_table(font)
     fontloader.close(font)
  end
  return metrics
end

myfont = load_font('/opt/tex/texmf/fonts/data/arial.ttf')
```

主要的函数调用是

```
<userdata> f, <table> w = fontloader.open(<string> filename)
<userdata> f, <table> w = fontloader.open(<string> filename, <string> fontname)
```

第一个返回值是字体的 userdata 表示。第二个返回值是一个表，包含 fontloader 在打开字体时报告的所有警告和错误。在正常排版中，你可能会忽略第二个参数，但它在调试时很有用。

对于 TrueType 集合（文件名以'ttc' 结尾时）和 dfont 集合，必须使用第二个字符串参数来指定想从集合中取哪个字体。请使用 `fontloader.info` 返回的 `fontname` 字符串。

要把字体转成表，对 `fontloader.open` 返回的字体使用 `fontloader.to_table`。

```
<table> f = fontloader.to_table(<userdata> font)
```

这个表不能直接被 LuaTEX 使用，应该按照 7 中的描述把它转换成另一个表。不要忘记把 `fontname` 值存入要返回给 LuaTEX 的度量表的 `psname` 字段，否则字体嵌入后端将无法在集合中找到正确的字体。

关于 `fontloader.open` 返回的 userdata 对象以及 `fontloader.to_table` 返回的 `metrics` 表的结构，详见 13.5。

字体文件由来自 FontForge 的字体加载例程解析并做部分解释。文件格式可以是 OpenType、TrueType、TrueType Collection、CFF 或 Type 1。

与我们自己直接读取字体文件相比，这种方法有几点优势：

- 字体会被自动重编码，因此 TrueType 和 OpenType 字体的 `metrics` 表使用 Unicode 作为字符索引。
- 许多特性会被预处理成比原始表更容易处理的格式。
- 基于 PostScript 的 OpenType 字体不在字体文件中存储字符的高度和深度，因此字符包围盒必须以某种方式计算出来。

加载过的字体用以下函数释放：

```
fontloader.close(<userdata> font)
```

13.3　应用 “特性文件

你可以把一个 “特性文件应用到已加载的字体上：

```
<table> errors = fontloader.apply_featurefile(<userdata> font, <string> filename)
```

“特性文件是 OpenType 字体中各项特性的文本表示。参见

```
http://www.adobe.com/devnet/opentype/afdko/topic_feature_file_syntax.html
```

和

```
http://fontforge.sourceforge.net/featurefile.html
```

以获取关于特性文件的更详细说明。

如果函数失败，返回值是一个表，包含 fontloader 在应用特性文件时报告的错误。成功时返回 `nil`。

13.4　应用 “AFM 文件

你可以把一个 “AFM 文件应用到已加载的字体上：

```
<table> errors = fontloader.apply_afmfile(<userdata> font, <string> filename)
```

AFM 文件是 Type 1 字体中（部分）元信息的文本表示。参见

```
ftp://ftp.math.utah.edu/u/ma/hohn/linux/postscript/5004.AFM_Spec.pdf
```

以获取关于 AFM 文件的更多信息。

注意：如果你用 `fontloader.open` 打开名为 `font.pfb` 的 Type 1 文件，库会自动在与 `font.pfb` 相同的目录中查找 `font.afm`(如果存在)并应用它。那种情况下无需显式调用 `apply_afmfile()`。

如果函数失败，返回值是一个表，包含 fontloader 在应用 AFM 文件时报告的错误。成功时返回 `nil`。

13.5　fontloader 字体表

如前所述，`fontloader.open` 的返回值是一个 userdata 对象。访问实际度量的一种方法是对此对象调用 `fontloader.to_table`，它返回后面几节所说明的表结构。在后面几节中，我们不会详细解释每个字段。大多数字段顾名思义，更技术性的细节需要查阅相关的字体参考资料。

事实证明，`fontloader.to_table` 的结果有时需要非常大的内存（取决于字体的复杂度和大小），因此可以直接访问 userdata 对象。

- `to_table()` 会返回的所有顶层键也都可以直接访问。
- 顶层键 “glyphs 返回一个虚拟数组，允许使用从 `f.glyphmin` 到（`f.glyphmax`）的索引。
- 该虚拟数组中的各项（实际的字形）本身也是 userdata 对象，每个对象都具有对下面 “字形项一节所说明所有键的访问器。

- 顶层键 “subfonts 返回一个实际的 userdata 对象数组，每个子字体一个（如果没有子字体则为 nil）。

下面这个简短示例或许有帮助。这段代码打印出字体 PunkNova.kern.otf 中所有字形的名字：

```
local f = fontloader.open('PunkNova.kern.otf')
print (f.fontname)
local i = 0
if f.glyphcnt > 0 then
    for i=f.glyphmin,f.glyphmax do
       local g = f.glyphs[i]
       if g then
          print(g.name)
       end
       i = i + 1
    end
end
fontloader.close(f)
```

在这种情况下，测试计算机上 LuaTEX 的内存需求保持在 100MB 以下，而 to_table() 生成的内部结构需要超过 2GB 的内存（该字体本身在磁盘上为 6.9MB）。

只有顶层字体、子字体表条目和字形是虚拟对象，其余一切仍然产生普通的 Lua 值和表。

如果你想知道字体或字形结构中有哪些合法字段，请在特定类型（字形或字体）的对象上调用 fields 函数：

```
<table> fields = fontloader.fields(<userdata> font)
<table> fields = fontloader.fields(<userdata> font_glyph)
```

例如：

```
local fields = fontloader.fields(f)
local fields = fontloader.fields(f.glyphs[0])
```

13.6　表类型

13.6.1　主表

返回表中的顶层键如下（本文档这一部分的说明尚未写完）：

键	类型	说明
table_version	string	表示度量版本（当前为 0.3）
fontname	string	PostScript 字体名
fullname	string	官方（面向人阅读的）字体名
familyname	string	族名
weight	string	粗细指示
copyright	string	版权信息

filename	string	文件名
version	string	字体版本
italicangle	float	倾斜角
units_per_em	number	基于 PostScript 的字体为 1000，TrueType 通常为 2048
ascent	number	上升部高度，单位为 units_per_em
descent	number	下降部深度，单位为 units_per_em
upos	float	
uwidth	float	
uniqueid	number	
glyphs	array	
glyphcnt	number	包含的字形数量
glyphmax	number	glyphs 数组中已使用的最大索引
glyphmin	number	glyphs 数组中已使用的最小索引
notdef_loc	number	.notdef 字形的位置；不存在时为 -1
hasvmetrics	number	
onlybitmaps	number	
serifcheck	number	
isserif	number	
issans	number	
encodingchanged	number	
strokedfont	number	
use_typo_metrics	number	
weight_width_slope_only	number	
head_optimized_for_cleartype	number	
uni_interp	enum	unset, none, adobe, greek, japanese, trad_chinese, simp_chinese, korean, ams
origname	string	文件名，由用户提供
map	table	
private	table	
xuid	string	
pfminfo	table	
names	table	
cidinfo	table	

subfonts	array	
commments	string	
fontlog	string	
cvt_names	string	
anchor_classes	table	
ttf_tables	table	
ttf_tab_saved	table	
kerns	table	
vkerns	table	
texdata	table	
lookups	table	
gpos	table	
gsub	table	
mm	table	
chosenname	string	
macstyle	number	
fondname	string	
fontstyle_id	number	
fontstyle_name	table	
strokewidth	float	
mark_classes	table	
creationtime	number	
modificationtime	number	
os2_version	number	
math	table	
validation_state	table	
horiz_base	table	
vert_base	table	
extrema_bound	number	
truetype	boolean	表示这是一个 TrueType 字体

13.6.2 glyphs

glyphs 是一个数组，包含逐字符的信息（其中不少字段仅在非零时才存在）。

键	类型	说明
name	string	字形名
unicode	number	unicode 码点，或 -1
boundingbox	array	由四个数字组成的数组，见下面的说明

width	number	仅用于横排字体
vwidth	number	仅用于竖排字体
tsidebearing	number	仅用于竖排 ttf/otf 字体，且仅在非零时存在
lsidebearing	number	仅在非零且不等于 boundingbox[1] 时存在
class	string	取”none”、”base”、”ligature”、”mark”、”component” 之一（如果不存在，则字形类为 “自动）
kerns	array	仅用于横排字体，且在设置时存在
vkerns	array	仅用于竖排字体，且在设置时存在
dependents	array	字形名字符串的线性数组，仅在非空时存在
lookups	table	仅在非空时存在
ligatures	table	仅在非空时存在
anchors	table	仅在设置时存在
comment	string	仅在设置时存在
tex_height	number	仅在设置时存在
tex_depth	number	仅在设置时存在
italic_correction	number	仅在设置时存在
top_accent	number	仅在设置时存在
is_extended_shape	number	仅当此字符是数学扩展列表的一部分时存在
altuni	table	备用 Unicode 项
vert_variants	table	
horiz_variants	table	
mathkern	table	

关于 boundingbox：TrueType 字体和基于 TrueType 的 OTF 字体的包围盒信息直接从字体文件中读取。基于 PostScript 的字体没有这个信息，因此传统 PostScript 字体的包围盒是通过解释实际的贝塞尔曲线来精确计算的。这可能是一个缓慢的过程，所以基于 PostScript 的 OTF 字体（以及原始 CFF 字体）的包围盒是根据仅基于实际字形点的字形形状近似值计算的，而不考虑整条曲线。这意味着在极值处缺少点的字形会得到一个过紧的包围盒，但处理速度快得多，因此我们认为这种折中是值得的。

kerns 和 vkerns 是由小哈希组成的线性数组：

键	类型	说明
char	string	

off	number
lookup	string

lookups 是一个以 lookup 子表名为键的哈希，每个键的值是一个由小哈希组成的线性数组：

键	类型	说明
type	enum	position, pair, substitution, alternate, multiple, ligature, lcaret, kerning, vkerning, anchors, contextpos, contextsub, chainpos, chainsub, reversesub, max, kernback, vkernback
specification	table	额外数据

对于 type 的前七个取值，可以有额外的子信息，存储在子表 specification 中：

取值	类型	说明
position	table	一个 offset_specs 类型的表
pair	table	一个字符串：paired，以及由一两个 offset_specs 表组成的数组：offsets
substitution	table	一个字符串：variant
alternate	table	一个字符串：components
multiple	table	一个字符串：components
ligature	table	两个字符串：components, char
lcaret	array	由数字组成的线性数组

offset_specs 表最多含四个数值字段：x（水平偏移）、y（垂直偏移）、h（前进宽度修正）和 v（前进高度修正）。

ligatures 是由小哈希组成的线性数组：

键	类型	说明
lig	table	使用与上面说明的 lookups 表中单个项相同的子结构
char	string	
components	array	由具名组件组成的线性数组

ccnt	number

anchor 表以表示锚点类型的字符串为索引，类型取以下之一：

键	类型	说明
mark	table	放置标记
basechar	table	用于把组合附加符号附着到基础字符上的标记
baselig	table	用于把组合附加符号附着到连字上的标记
basemark	table	用于连接组合附加符号的通用标记
centry	table	草书连笔入口点
cexit	table	草书连笔出口点

它们的内容是一个由已定义锚点组成的短数组，条目的键是锚点名。除 baselig 外，值都是单个表，定义如下：

键	类型	说明
x	number	x 位置
y	number	y 位置
ttf_pt_index	number	truetype 点索引，仅在给出时存在

对于 baselig，值是一个由这类锚点集合组成的小数组，连字的每个组成项各有一个。

为清楚起见，一个锚点表例如可能像这样：

```
['anchor'] = {
    ['basemark'] = {
        ['Anchor-7'] = { ['x']=170, ['y']=1080 }
    },
    ['mark'] ={
        ['Anchor-1'] = { ['x']=160, ['y']=810 },
        ['Anchor-4'] = { ['x']=160, ['y']=800 }
    },
    ['baselig'] = {
        [1] = { ['Anchor-2'] = { ['x']=160, ['y']=650 } },
        [2] = { ['Anchor-2'] = { ['x']=460, ['y']=640 } }
        }
    }
```

注意：baselig 表可以是稀疏的！

13.6.3　map

顶层 map 是一个编码映射列表。每一项本身也是一个表。

键	类型	说明
enccount	number	
encmax	number	
backmax	number	
remap	table	
map	array	非线性的映射数组
backmap	array	非线性的反向映射数组
enc	table	

remap 表非常小：

键	类型	说明
firstenc	number	
lastenc	number	
infont	number	

enc 表要详尽一些：

键	类型	说明
enc_name	string	
char_cnt	number	
char_max	number	
unicode	array	由 Unicode 位置编号组成
psnames	array	由 PostScript 字形名组成
builtin	number	
hidden	number	
only_1byte	number	
has_1byte	number	
has_2byte	number	
is_unicodebmp	number	仅在非零时存在
is_unicodefull	number	仅在非零时存在
is_custom	number	仅在非零时存在
is_original	number	仅在非零时存在
is_compact	number	仅在非零时存在
is_japanese	number	仅在非零时存在
is_korean	number	仅在非零时存在

is_tradchinese	number	仅在非零时存在 [名字?]
is_simplechinese	number	仅在非零时存在
low_page	number	
high_page	number	
iconv_name	string	
iso_2022_escape	string	

13.6.4 private

这是字体的私有 PostScript 字典（如果有的话）。键和值都是字符串。

13.6.5 cidinfo

键	类型	说明
registry	string	
ordering	string	
supplement	number	
version	number	

13.6.6 pfminfo

pfminfo 表包含大部分 OS/2 信息：

键	类型	说明
pfmset	number	
winascent_add	number	
windescent_add	number	
hheadascent_add	number	
hheaddescent_add	number	
typoascent_add	number	
typodescent_add	number	
subsuper_set	number	
panose_set	number	
hheadset	number	
vheadset	number	
pfmfamily	number	
weight	number	
width	number	
avgwidth	number	
firstchar	number	

lastchar	number	
fstype	number	
linegap	number	
vlinegap	number	
hhead_ascent	number	
hhead_descent	number	
os2_typoascent	number	
os2_typodescent	number	
os2_typolinegap	number	
os2_winascent	number	
os2_windescent	number	
os2_subxsize	number	
os2_subysize	number	
os2_subxoff	number	
os2_subyoff	number	
os2_supxsize	number	
os2_supysize	number	
os2_supxoff	number	
os2_supyoff	number	
os2_strikeysize	number	
os2_strikeypos	number	
os2_family_class	number	
os2_xheight	number	
os2_capheight	number	
os2_defaultchar	number	
os2_breakchar	number	
os2_vendor	string	
codepages	table	由两个数字组成的已编码代码页数组
unicoderages	table	由四个数字组成的已编码 unicode 范围数组
panose	table	

panose 子表恰好有 10 个字符串键:

键	类型	说明
familytype	string	取值同 OpenType 字体规范: Any, No Fit, Text and Display, Script, Decorative, Pictorial

serifstyle	string	取值见 OpenType 字体规范
weight	string	同上
proportion	string	同上
contrast	string	同上
strokevariation	string	同上
armstyle	string	同上
letterform	string	同上
midline	string	同上
xheight	string	同上

13.6.7 names

每一项有两个顶层键：

键	类型	说明
lang	string	此条目所用的语言
names	table	

names 的键就是实际的 TrueType 名字字符串。可能的键有：copyright, family, subfamily, uniqueid, fullname, version, postscriptname, trademark, manufacturer, designer, descriptor, venderurl, designerurl, license, licenseurl, idontknow, preffamilyname, prefmodifiers, compatfull, sampletext, cidfindfontname, wwsfamily and wwssubfamily.

13.6.8 anchor_classes

anchor_classes 类：

键	类型	说明
name	string	此锚点类的描述性 id
lookup	string	
type	string	取 mark, mkmk, curs, mklg 之一

13.6.9 gpos

gpos 表对每个 lookup 有一个数组条目。（gpos_ 前缀有点多余。）

键	类型	说明
type	string	取 gpos_single, gpos_pair, gpos_cursive, gpos_mark2base,

gpos_mark2ligature, gpos_mark2mark, gpos_context, gpos_contextchain 之一	
flags	table
name	string
features	array
subtables	array

flags 表中，每个实际被设置的 lookup 标志对应一个 true 值：

键	类型	说明
r2l	boolean	
ignorebaseglyphs	boolean	
ignoreligatures	boolean	
ignorecombiningmarks	boolean	
mark_class	string	

gpos 的 features 子表项含有：

键	类型	说明
tag	string	
scripts	table	

features 中的 scripts 表含有：

键	类型	说明
script	string	
langs	array of strings	

subtables 表含有：

键	类型	说明
name	string	
suffix	string	（仅在使用时存在）
anchor_classes	number	（仅在使用时存在）
vertical_kerning	number	（仅在使用时存在）
kernclass	table	（仅在使用时存在）

带子表的 kernclass 表含有：

键	类型	说明
firsts	array of strings	
seconds	array of strings	
lookup	string or array	关联的 lookup
offsets	array of numbers	

注意：kernclass（就我们所见）总是只有一个条目，因此它本可以浅一层。另外，seconds 从 [2] 开始，这接近 fontforge 的内部结构，所以我们也保留了这一点。

13.6.10 gsub

它的布局与 gpos 表完全相同，只是 type 不同：

键	类型	说明
type	string	取 gsub_sing gsub_multiple, gsub_alternate, gsub_ligature, gsub_context, gsub_contextchain, gsub_reversecontextchain 之一

13.6.11 ttf_tables

键	类型	说明
tag	string	
len	number	
maxlen	number	
data	number	

13.6.12 mm

键	类型	说明
axes	table	轴名数组
instance_count	number	
positions	table	实例位置数组（#axes * 实例数）
defweights	table	各实例默认粗细的数组
cdv	string	
ndv	string	

axismaps	table

axismaps:

键	类型	说明
blends	table	混合点数组
designs	table	设计值数组
min	number	
def	number	
max	number	

13.6.13 mark_classes

此表中的键是标记类名，值是该类中字形名组成的、以空格分隔的字符串。

13.6.14 math

math 表含有在数学一章中也讨论过的那些变量:ScriptPercentScaleDown, ScriptScriptPercentScaleD
DelimitedSubFormulaMinHeight, DisplayOperatorMinHeight, MathLeading, AxisHeight, AccentBaseHeigh
FlattenedAccentBaseHeight, SubscriptShiftDown, SubscriptTopMax, SubscriptBaselineDropMin,
SuperscriptShiftUp, SuperscriptShiftUpCramped, SuperscriptBottomMin, SuperscriptBaselineDropMax
SubSuperscriptGapMin, SuperscriptBottomMaxWithSubscript, SpaceAfterScript, UpperLimitGapMin,
UpperLimitBaselineRiseMin, LowerLimitGapMin, LowerLimitBaselineDropMin, StackTopShiftUp,
StackTopDisplayStyleShiftUp, StackBottomShiftDown, StackBottomDisplayStyleShiftDown,
StackGapMin, StackDisplayStyleGapMin, StretchStackTopShiftUp, StretchStackBottomShiftDown,
StretchStackGapAboveMin, StretchStackGapBelowMin, FractionNumeratorShiftUp, FractionNumeratorDi
FractionDenominatorShiftDown, FractionDenominatorDisplayStyleShiftDown, FractionNumeratorGapMi
FractionNumeratorDisplayStyleGapMin FractionRuleThickness, FractionDenominatorGapMin,
FractionDenominatorDisplayStyleGapMin, SkewedFractionHorizontalGap, SkewedFractionVerticalGap,
OverbarVerticalGap, OverbarRuleThickness, OverbarExtraAscender, UnderbarVerticalGap,
UnderbarRuleThickness, UnderbarExtraDescender, RadicalVerticalGap, RadicalDisplayStyleVerticalG
RadicalRuleThickness, RadicalExtraAscender, RadicalKernBeforeDegree, RadicalKernAfterDegree,
RadicalDegreeBottomRaisePercent, MinConnectorOverlap, FractionDelimiterSize and FractionDelimit

13.6.15 validation_state

这只是一个附带的表,其键为:bad_ps_fontname, bad_glyph_table, bad_cff_table, bad_metrics_table,
bad_cmap_table, bad_bitmaps_table, bad_gx_table, bad_ot_table, bad_os2_version 和
bad_sfnt_header.

13.6.16 horiz_base

键	类型	说明
tags	table	书写系统列表标签的数组
scripts	table	

scripts 子表：

键	类型	说明
baseline	table	
default_baseline	number	
lang	table	

lang 子表：

键	类型	说明
tag	string	书写系统标签
ascent	number	
descent	number	
features	table	

features 指向一个表数组，这些表的布局相同，只是在这些嵌套表中，tag 表示的是语言。

13.6.17 altuni

备用 Unicode 值的数组。该数组内是含以下字段的哈希：

键	类型	说明
unicode	number	此字形也用于此 unicode
variant	number	该备用字形由此 unicode 选择符驱动

13.6.18 vert_variants

键	类型	说明
variants	string	
italic_correction	number	
parts	table	

parts 表是由更小的表组成的数组：

键	类型	说明
component	string	
extender	number	
start	number	
end	number	
advance	number	

13.6.19 mathkern

键	类型	说明
top_right	table	
bottom_right	table	
top_left	table	
bottom_left	table	

每个子表都是由含两个键的小哈希组成的数组：

键	类型	说明
height	number	
kern	number	

13.6.20 kerns

子结构与逐字形子表相同。

13.6.21 vkerns

子结构与逐字形子表相同。

13.6.22 texdata

键	类型	说明
type	string	unset, text, math, mathext
params	array	22 个字体数值参数

13.6.23 lookups

顶层 lookups 与字符级的 lookups 大不相同。这个哈希中的键是字符串，值是实际的 lookup，以字典表表示。

键	类型	说明
`type`	string	
`format`	enum	取 `glyphs`, `class`, `coverage`, `reversecoverage` 之一
`tag`	string	
`current_class`	array	
`before_class`	array	
`after_class`	array	
`rules`	array	规则项数组

规则项有一个公共项和一个专用项:

键	类型	说明
`lookups`	array	lookup 名的线性数组
`glyphs`	array	仅当父表的 format 为 `glyphs` 时存在
`class`	array	仅当父表的 format 为 `class` 时存在
`coverage`	array	仅当父表的 format 为 `coverage` 时存在
`reversecoverage`	array	仅当父表的 format 为 `reversecoverage` 时存在

一个 glyph 表为:

键	类型	说明
`names`	string	
`back`	string	
`fore`	string	

一个 class 表为:

键	类型	说明
`current`	array	由数字组成
`before`	array	由数字组成
`after`	array	由数字组成

对于 coverage:

键	类型	说明
`current`	array	由字符串组成
`before`	array	由字符串组成
`after`	array	由字符串组成

而对于 reverse coverage：

键	类型	说明
`current`	array	由字符串组成
`before`	array	由字符串组成
`after`	array	由字符串组成
`replacements`	string	

第 14 章 HarfBuzz 库

14.1 luaharfbuzz

本库的文档是对 https://harfbuzz.github.io/luaharfbuzz 处 API 的原始从 HTML 到 PDF 的渲染；示例几乎是 core_type.lua 的逐字副本。

14.1.1　示例

该示例(经过小幅修改)取自文件 core_types.lua, 它需要 harfbuzz.lua; 两者都在源代码的 luaharfbuzz 文件夹下。字体 notonastaliq.ttf 和 amiri-regular.ttf' 位于 luaharfbuzz/fonts 下。

```
local hb = require("luaharfbuzz")

-- special tags
hb.Tag.NONE = hb.Tag.new()

-- special script codes (ISO 15924)
hb.Script.COMMON    = hb.Script.new("Zyyy")
hb.Script.INHERITED = hb.Script.new("Zinh")
hb.Script.UNKNOWN   = hb.Script.new("Zzzz")
hb.Script.INVALID   = hb.Script.from_iso15924_tag(hb.Tag.NONE)

-- directions
hb.Direction.INVALID = hb.Direction.new("invalid")
hb.Direction.LTR = hb.Direction.new("ltr")
hb.Direction.RTL = hb.Direction.new("rtl")
hb.Direction.TTB = hb.Direction.new("ttb")
hb.Direction.BTT = hb.Direction.new("btt")

-- special languages
hb.Language.INVALID = hb.Language.new()

hb.shape = function(font, buf, options)
  options = options or { }

  -- Apply options to buffer if they are set.
  if options.language then buf:set_language(options.language) end
  if options.script then buf:set_script(options.script) end
  if options.direction then buf:set_direction(options.direction) end

  -- Guess segment properties, in case all steps above have failed
  buf:guess_segment_properties()

  local features = {}

  -- Parse features
  if type(options.features) == "string" then
    for fs in string.gmatch(options.features, '([^,]+)') do
      local feature = hb.Feature.new(fs)
      if feature then
        table.insert(features, hb.Feature.new(fs))
      else
        error(string.format("Invalid feature string: '%s'", fs))
      end
    end
  elseif type(options.features) == "table" then
    features = options.features
```

```
  elseif options.features then -- non-nil but not a string or table
    error("Invalid features option")
  end

  return hb.shape_full(font,buf,features,options.shapers or {})
end

-- For backward compatibility
hb.Buffer.HB_BUFFER_CLUSTER_LEVEL_MONOTONE_GRAPHEMES  = hb.Buffer.CLUSTER_LEVEL_MONOTONE_GRAPHEMES
hb.Buffer.HB_BUFFER_CLUSTER_LEVEL_MONOTONE_CHARACTERS = hb.Buffer.CLUSTER_LEVEL_MONOTONE_CHARACTERS
hb.Buffer.HB_BUFFER_CLUSTER_LEVEL_CHARACTERS          = hb.Buffer.CLUSTER_LEVEL_CHARACTERS
hb.Buffer.HB_BUFFER_CLUSTER_LEVEL_DEFAULT             = hb.Buffer.CLUSTER_LEVEL_DEFAULT

hb.Tag.HB_TAG_NONE = hb.Tag.NONE

hb.Script.HB_SCRIPT_COMMON    = hb.Script.COMMON
hb.Script.HB_SCRIPT_INHERITED = hb.Script.INHERITED
hb.Script.HB_SCRIPT_UNKNOWN   = hb.Script.UNKNOWN
hb.Script.HB_SCRIPT_INVALID   = hb.Script.INVALID

hb.Language.HB_LANGUAGE_INVALID = hb.Language.INVALID

hb.Direction.HB_DIRECTION_INVALID = hb.Direction.INVALID
hb.Direction.HB_DIRECTION_LTR = hb.Direction.LTR
hb.Direction.HB_DIRECTION_RTL = hb.Direction.RTL
hb.Direction.HB_DIRECTION_TTB = hb.Direction.TTB
hb.Direction.HB_DIRECTION_BTT = hb.Direction.BTT

hb.Direction.HB_DIRECTION_IS_VALID = hb.Direction.is_valid
hb.Direction.HB_DIRECTION_IS_HORIZONTAL = hb.Direction.is_horizontal
hb.Direction.HB_DIRECTION_IS_VERTICAL = hb.Direction.is_vertical
hb.Direction.HB_DIRECTION_IS_FORWARD = hb.Direction.is_forward
hb.Direction.HB_DIRECTION_IS_BACKWARD = hb.Direction.is_backward

hb.Buffer.get_glyph_infos_and_positions = hb.Buffer.get_glyphs

return hb
```

文件 harfbuzz.lua:

```
local harfbuzz = require('harfbuzz')

-- Harfbuzz API Version
print("Harfbuzz API version", harfbuzz.version())

-- Shapers available
print("Shapers:", harfbuzz.shapers())

-- harfbuzz.Face
local face = harfbuzz.Face.new('notonastaliq.ttf')
print('\nFace upem = '..face:get_upem())
```

```
-- harfbuzz.Font
local font = harfbuzz.Font.new(face)
local xs, xy = font:get_scale()
print("\nDefault font scale = X: "..xs.."", Y: "..xy)

-- harfbuzz.Buffer
local text = " " -- U+06CC U+06C1
local buf = harfbuzz.Buffer.new()
buf:add_utf8(text)

-- harfbuzz.shape (Shapes text)
print("\nShaping '"..text.."' set with Noto Nastaliq Urdu")
harfbuzz.shape(font, buf, { language = harfbuzz.Language.new("urd"),
script = harfbuzz.Script.new("Arab"), direction = harfbuzz.Direction.RTL})

local glyphs = buf:get_glyphs()
print("No. of glyphs", #glyphs)
for k,v in pairs(glyphs) do
  print(k)
  for k1,v1 in pairs(v) do
   print("",k1,v1)
  end
end

local opts = { language = harfbuzz.Language.new("eng"),
script = harfbuzz.Script.new("Latn"), direction = harfbuzz.Direction.LTR }

local amiri_face = harfbuzz.Face.new('amiri-regular.ttf')
local amiri_font = harfbuzz.Font.new(amiri_face)

-- shaping '123' w/o features
print("\nShaping '123' set with Amiri Regular and no features")
buf= harfbuzz.Buffer.new()
buf:add_utf8("123")
harfbuzz.shape(amiri_font, buf, opts)
glyphs = buf:get_glyphs()
for k,v in pairs(glyphs) do
  print(k)
  for k1,v1 in pairs(v) do
   print("",k1,v1)
  end
end

-- shaping '123' with '+numr' (numerators)
print("\nShaping '123' set with Amiri Regular with 'numr' feature turned on")
buf= harfbuzz.Buffer.new()
buf:add_utf8("123")
opts.features = "+numr"
harfbuzz.shape(amiri_font, buf, opts)
glyphs = buf:get_glyphs()
for k,v in pairs(glyphs) do
  print(k)
  for k1,v1 in pairs(v) do
```

```
   print("",k1,v1)
  end
end
```

示例:

```
Harfbuzz API version    12.2.0
Shapers:    graphite2    ot    fallback

Face upem = 2048

Default font scale = X: 2048, Y: 2048

Shaping  '' set with Noto Nastaliq Urdu
No. of glyphs    4
1
    codepoint    277
    y_advance    0.0
    cluster    2
    flags    1.0
    y_offset    0.0
    x_advance    472.0
    x_offset    0.0
2
    codepoint    19
    y_advance    0.0
    cluster    0
    y_offset    -383.0
    x_advance    0.0
    x_offset    310.0
3
    codepoint    985
    y_advance    0.0
    cluster    0
    y_offset    0.0
    x_advance    0.0
    x_offset    0.0
4
    codepoint    316
    y_advance    0.0
    cluster    0
    y_offset    -68.0
    x_advance    731.0
    x_offset    0.0

Shaping '123' set with Amiri Regular and no features
1
    codepoint    20
    y_advance    0.0
    cluster    0
    y_offset    0.0
    x_advance    1090.0
```

```
    x_offset    0.0
2
    codepoint    21
    y_advance    0.0
    cluster    1
    y_offset    0.0
    x_advance    1090.0
    x_offset    0.0
3
    codepoint    22
    y_advance    0.0
    cluster    2
    y_offset    0.0
    x_advance    1090.0
    x_offset    0.0

Shaping '123' set with Amiri Regular with 'numr' feature turned on
1
    codepoint    6673
    y_advance    0.0
    cluster    0
    y_offset    0.0
    x_advance    600.0
    x_offset    0.0
2
    codepoint    6674
    y_advance    0.0
    cluster    1
    y_offset    0.0
    x_advance    600.0
    x_offset    0.0
3
    codepoint    6675
    y_advance    0.0
    cluster    2
    y_offset    0.0
    x_advance    600.0
    x_offset    0.0
```

结果:

第 15 章 后端库

15.1 pdf

此库包含与 PDF 后端相关的变量和函数。有关设置器的预期值的更多详细信息，请参见 4.2。

15.1.1 mapfile

```
pdf.mapfile(<string> map file)
pdf.mapline(<string> map line)
```

这两个函数可用于替换从 pdfTEX 继承的原语 pdfmapfile 和 pdfmapline。它们接受一个字符串作为唯一参数，没有返回值。映射行中的第一个字符可以是 -、+ 或 =，分别表示删除、添加或替换该行。它们不是状态设置器，而是立即生效。

15.1.2 [set|get

]

这些函数补充了处理元数据的相应 PDF 后端令牌列表。值类型为字符串，它们在 TEX 端设置的令牌寄存器写入之后直接写入 PDF 文件。

15.1.3 [set|get

]

这些函数补充了处理页面资源的相应 PDF 后端令牌列表。这些变量与相应的 PDF 后端令牌寄存器没有交互。它们在 TEX 端设置的令牌寄存器写入之后直接写入 PDF 文件。

15.1.4 [set|get

]

这些函数补充了处理可复用盒子和图像的相应 PDF 后端令牌列表。这些变量与相应的 PDF 后端令牌寄存器没有交互。它们在 TEX 端设置的令牌寄存器写入之后直接写入 PDF 文件。

15.1.5 [set|getversion

]

你可以设置输出的主版本号和次版本号。主版本号通常为 1，但当设置为 2 时，为了符合标准，某些数据将不会写入文件。你设置哪个次版本号取决于你使用的 PDF 功能。这不在 LuaTEX 的控制范围内。

可以将主版本号设置为 2，但我们无法保证引擎能正确适配，因为据我们所知没有公开免费的规范。此外，用户构造的注释不会被检查，而是直接传递给文件。另一方面，引擎生成的 PDF 相当简单，对版本的依赖不大。

15.1.6　getcreationdate

此函数返回一个字符串，其中包含最终出现在 PDF 文件中的日期格式，本例中为：[□□□]。

15.1.7　[set|getinclusionerrorlevel 和 [set|get]ignoreunknownimages

]

这些变量控制如何处理嵌入图像中的错误。它们是仿照 pdfTEX 中的对应项建模的。

15.1.8　[set|get]suppressoptionalinfo, [set|get]trailerid, [set|get]omitcidset, [set|get]omitinfo, [set|get]omitmediabox, [set|get]omitprocset, [set|get]ptexprefix

可选信息位集（一个数字）确定要刷新哪些类型的信息。默认情况下我们刷新所有。更多详细信息请参见 4.2.2。

你可以设置自己的 trailer id。这必须是一个包含有效 PDF 数组内容和校验和的字符串。

cidset、charset 和 info 标志（数字）会禁用所谓的 `CIDSet` 和 `CharSet` 条目的包含，这在针对众多 PDF 子标准中的某些时可能很方便。`ProcSet` 和 `PTEX` 前缀也是如此，设置此标志将强制使用 _ 而不是 .。[1]

当省略它时，应通过页面属性选项提供 `MediaBox`，因为这是一个必需字段。不进行任何检查。

15.1.9　[set|getcompresslevel 和 [set|get]recompress

]

这些函数设置流压缩级别。启用对象压缩时，多个对象将被打包到一个压缩流中，从而节省空间。最小值为 0，最大值为 9。

当 recompression 设为 1 时，压缩对象将被解压；当 compresslevel 大于零时，它们将被重新压缩。这主要是一个调试功能，不应依赖。

15.1.10　[set|getgentounicode

]

此标志启用 tounicode 生成（如 pdfTEX 中那样）。通常这些值由字体加载器提供。

15.1.11　[set|getdecimaldigits

]

[1]在 info 字典中，句点在 2 版之前是有效的，但在其他地方必须使用下划线。此前缀始于 pdfTEX 的早期，当时使用句点被认为是可以接受的。后来的规范对此进行了澄清。

这两个函数设置写入 PDF 文件的浮点数精度。你可以设置任何值，但后端不会低于 3 或高于 6。

15.1.12　[set|getpkresolution

]

此设置器接受两个参数：分辨率和一个可选的零或一，指示这是否是固定的。获取器返回这两个值。

15.1.13　getlast[obj|link|annot 和 getretval

]

这些状态变量类似于传统上在 TEX 端后端接口中使用的变量。

15.1.14　getmaxobjnum

这些内省辅助函数主要在你自己构造 PDF 对象时使用，例如需要有关（将要）嵌入的字体的信息时。

15.1.15　[set|getorigin

]

此函数用于设置水平和 |/| 或垂直偏移，这是一个传统的后端属性。

```
pdf.setorigin() -- sets both to 0pt
pdf.setorigin(tex.sp("1in")) -- sets both to 1in
pdf.setorigin(tex.sp("1in"),tex.sp("1in"))
```

此函数的对应获取器返回两个值。

15.1.16　[set|getimageresolution

]

这两个函数与 imageresolution 相关，当图像本身未提供非零 x 或 y 分辨率时使用。

15.1.17　[set|getmargin

]

这些函数可用于设置和检索添加到相应对象自然边界框的边距。

15.1.18　get[pos|hpos|vpos

]

这些函数获取输出页面上的当前位置，从其左下角开始测量。返回值以缩放点为单位。

```
local h, v = pdf.getpos()
```

15.1.19　[has|getmatrix

]

当前矩阵变换可通过 getmatrix 命令获得，该命令返回 6 个值：sx、rx、ry、sy、tx 和 ty。当应用了矩阵时，hasmatrix 函数返回 true。

```
if pdf.hasmatrix() then
    local sx, rx, ry, sy, tx, ty = pdf.getmatrix()
    -- do something useful or not
end
```

15.1.20　print

你可以从 latelua 调用内向 PDF 文档打印一个字符串。此函数不应在 directlua 内部使用，除非你确切地知道自己在做什么。

```
pdf.print(<string> s)
pdf.print(<string> type, <string> s)
```

可选参数可用于模拟 PDF 字面量的行为：type 为 direct 或 page。

15.1.21　immediateobj

此函数创建一个 PDF 对象并立即将其写入 PDF 文件。它是仿照 pdfTEX 的 immediate pdfobj 原语建模的。所有函数变体都返回新生成对象的对象编号。

```
<number> n =
    pdf.immediateobj(<string> objtext)
<number> n =
    pdf.immediateobj("file", <string> filename)
<number> n =
    pdf.immediateobj("stream", <string> streamtext, <string> attrtext)
<number> n =
    pdf.immediateobj("streamfile", <string> filename, <string> attrtext)
```

第一个版本将 objtext 原样放入一个对象中。只有对象包装器是自动生成的，但任何内部结构（如 << >> 字典标记）都需要由用户提供。第二个版本以关键字 file 作为第一个参数，将名为 filename 的文件内容原样放入对象中。第三个版本以关键字 stream 创建一个流对象，并将 streamtext 原样放入流中。流长度自动计算。可选的 attrtext 放入该对象的字典中。第四个版本以关键字 streamfile 与第三个版本作用相同，只是从文件中原样读取流数据。

可以给出一个可选的第一个参数，使函数使用先前保留的 PDF 对象。

```
<number> n =
    pdf.immediateobj(<integer> n, <string> objtext)
<number> n =
    pdf.immediateobj(<integer> n, "file", <string> filename)
<number> n =
    pdf.immediateobj(<integer> n, "stream", <string> streamtext, <string> attrtext)
<number> n =
    pdf.immediateobj(<integer> n, "streamfile", <string> filename, <string> attrtext)
```

15.1.22　obj

此函数创建一个 PDF 对象，该对象仅在被引用时（例如通过 refobj()）才写入 PDF 文件。

所有函数变体都返回新生成对象的对象编号，并且有两种独立的调用模式。第一种模式是仿照 pdfTEX 的 pdfobj 原语建模的。

```
<number> n =
    pdf.obj(<string> objtext)
<number> n =
    pdf.obj("file", <string> filename)
<number> n =
    pdf.obj("stream", <string> streamtext, <string> attrtext)
<number> n =
    pdf.obj("streamfile", <string> filename, <string> attrtext)
```

可以给出一个可选的第一个参数，使函数使用先前保留的 PDF 对象。

```
<number> n =
    pdf.obj(<integer> n, <string> objtext)
<number> n =
    pdf.obj(<integer> n, "file", <string> filename)
<number> n =
    pdf.obj(<integer> n, "stream", <string> streamtext, <string> attrtext)
<number> n =
    pdf.obj(<integer> n, "streamfile", <string> filename, <string> attrtext)
```

第二种模式接受一个带有键值对的参数表。

```
<number> n = pdf.obj {
    type           = <string>,
    immediate      = <boolean>,
    objnum         = <number>,
    attr           = <string>,
    compresslevel  = <number>,
    objcompression = <boolean>,
    file           = <string>,
    string         = <string>,
    nolength       = <boolean>,
}
```

type 字段可以取 raw 和 stream 值，此字段为必填，其他字段为可选（在约束范围内）。设置 nolength 时，不会向字典中添加 /Length 条目。

注意：这种模式使 obj 看起来比实际更灵活：独立参数版本的约束仍然适用，例如你不能同时使用 string 和 file。

15.1.23　refobj

此函数是 pdfrefobj 原语的 Lua 版本，通过对象编号引用一个对象，以便将该对象写入 PDF 文件。

```
pdf.refobj(<integer> n)
```

此函数在 directlua 和 latelua 环境中都有效。在 directlua 内部，会创建一个新的 whatsit 节点 “pdf_refobj，该节点将在页面输出期间被标记为待刷新，然后该对象将在页面之后直接写入，此时资源对象也会被写入 PDF 文件。在 latelua 内部，该对象将被标记为待刷新。

此函数没有返回值。

15.1.24　reserveobj

此函数创建一个空的 PDF 对象并返回其编号。

```
<number> n = pdf.reserveobj()
<number> n = pdf.reserveobj("annot")
```

15.1.25　getpageref

可以通过此函数获取页面的对象编号。这可能是一个前向引用，因此当你请求未来的页面时，确实会得到一个编号。

```
<number> n = pdf.getpageref(123)
```

15.1.26　registerannot

此函数将一个对象编号添加到当前页面的 /Annots 数组中，不做其他任何事情。此函数只能在 latelua 内部使用。

```
pdf.registerannot(<number> objnum)
```

15.1.27　newcolorstack

此函数分配一个新的颜色堆栈并返回其 id。参数与类似的后端扩展原语相同。

```
pdf.newcolorstack("0 g","page",true) -- page|direct|origin
```

15.1.28　setfontattributes

此函数将强制向字体资源中添加一些额外代码。例如，它可用于向位图文件添加自定义 ToUnicode 向量。

```
pdf.setfontattributes(<number> font id, <string> pdf code)
```

15.2　pdfe

15.2.1　简介

pdfe 库取代了 epdf 库，提供了对 PDF 文件的接口。它使用与 PDF 图像嵌入相同的代码。由 Paweł Jackowski 编写的 pplib 库取代了 poppler（派生自 xpdf）库。

PDF 文件基本上是一棵对象树，通过字典（键/值）和数组（索引/值）进入树中。有几个从根开始的最顶层字典可以更直接地访问。

尽管 PDF 中的一切基本上都是对象，但我们只将其中一些封装为所谓的 userdata Lua 对象。

PDF	Lua
null	nil
boolean	boolean
integer	integer
float	number
name	string
string	string
array	array userdatum
dictionary	dictionary userdatum
stream	stream userdatum（带有相关字典）
reference	reference userdatum

常规获取器返回这些 Lua 数据类型，但也可以获得更详细的信息。

15.2.2 open

文档从文件或字符串加载

```
<pdfe document> = pdfe.open(filename)
<pdfe document> = pdfe.new(somestring,somelength)
```

这样的文档用以下方式关闭：

```
pdfe.close(<pdfe document>)
```

你可以通过以下方式检查文档是否成功打开：

```
pdfe.getstatus(<pdfe document>)
```

返回的代码为：

值	说明
-2	文档（仍然）受保护
-1	文档打开失败
0	文档未加密
1	文档已被解密

加密文档可以通过以下命令解密，其中两个密码参数都可以给出 `nil`：

```
pdfe.unencrypt(<pdfe document>,userpassword,ownerpassword)
```

15.2.3　getsize

成功打开的文档可以提供一些信息：

```
bytes = getsize(<pdfe document>
major, minor = getversion(<pdfe document>
n = getnofobjects(<pdfe document>
n = getnofpages(<pdfe document>
bytes, waste = getmemoryusage(<pdfe document>
```

15.2.4　get[catalog|trailer|info

]

要访问文档结构，你从所谓的 catalog（目录）开始，它是一个字典：

```
<pdfe dictionary> = pdfe.getcatalog(<pdfe document>)
```

另外两个根字典通过以下方式访问：

```
<pdfe dictionary> = pdfe.gettrailer(<pdfe document>)
<pdfe dictionary> = pdfe.getinfo(<pdfe document>)
```

15.2.5　getpage

可以通过以下命令方便地访问特定页面，该命令返回一个字典。第一个参数应是一个页面字典。

```
<pdfe dictionary> = pdfe.getpage(<pdfe document>,pagenumber)
```

另一个便捷命令为你提供（通常是页面的）（边界）框，该框可以从文档本身继承。有效框名的一个例子是 MediaBox。

```
box_dimens <array> = pdfe.getbox(<pdfe dictionary>,boxname)
```

15.2.6　get[string|integer|number|boolean|name, type

]

字典和数组中的常见值是字符串、整数、浮点数、布尔值和名称（它们也是字符串），这些也是普通的 Lua 对象：

```
s = getstring <pdfe array|dictionary>,index|key)
i = getinteger(<pdfe array|dictionary>,index|key)
n = getnumber <pdfe array|dictionary>,index|key)
b = getboolean(<pdfe array|dictionary>,index|key)
n = getname   <pdfe array|dictionary>,index|key)
s = type      <pdfe array|dictionary|document|reference|stream>
```

type 返回一个描述对象类型的字符串，例如”pdfe.array”、”pdfe.dictionary”、”pdfe”、”pdfe.reference”、”pdfe.stream”。

getstring 函数有两个额外的变体：

```
s, h = getstring <pdfe array|dictionary>,index|key,false)
s    = getstring <pdfe array|dictionary>,index|key,true)
```

第一次调用返回原始字符串加上一个布尔值，指示该字符串是否为十六进制编码。第二次调用返回未编码的字符串。

15.2.7　get[from

]

通常你会在数组中使用索引，在字典中使用键，但字典也接受索引。数组或字典的大小可以用通常的 ## 运算符获得。

```
<pdfe dictionary>   = getdictionary(<pdfe array|dictionary>,index|key)
<pdfe array>        = getarray     <pdfe array|dictionary>,index|key)
<pdfe stream>,
<pdfe dictionary>   = getstream    <pdfe array|dictionary>,index|key)
```

这些命令返回字典、数组和流，流是附加了一块数据的字典。

在介绍另一种访问模式之前，我们提到这些对象也以不同的方式提供访问，例如以下是有效的：

```
print(pdfe.open("foo.pdf").Catalog.Type)
```

在最顶层有 Catalog、Info、Trailer 和 Pages，所以以下也是可以的：

```
print(pdfe.open("foo.pdf").Pages[1])
```

15.2.8　[open|close|readfrom|whole|stream

]

流有些特殊。当你的索引或键命中一个流时，你会得到一个流对象和字典对象。字典你可以用通常的方式访问，而流有以下方法：

```
okay   = openstream(<pdfe stream>,[decode])
         closestream(<pdfe stream>
str, n = readfromstream(<pdfe stream>
str, n = readwholestream(<pdfe stream>,[decode])
```

你可以分块读取，也可以一次性读取全部。分块读取时，你需要自己打开和关闭流。n 值指示读取的长度。decode 参数控制是否对流数据进行解压。

与字典一样，你也可以用通常的 Lua 方式访问流字典中的字段。当你 “调用流时，你会得到内容。你可以传递一个布尔值，指示是否需要解压流。

15.2.9　getfrom[dictionary|array

]

除了前面描述的接口之外，还有一个更低级别的接口可用。

```
key, type, value, detail = getfromdictionary(<pdfe dictionary>,index)
type, value, detail = getfromarray(<pdfe array>,index)
```

类型	含义	值	详情
0	无	nil	
1	空值	nil	
2	布尔值	布尔值	
3	整数	整数	
4	数字	浮点数	
5	名称	字符串	
6	字符串	字符串	十六进制
7	数组	数组对象	大小
8	字典	字典对象	大小
9	流	流对象	字典大小
10	引用	整数	

hex 字符串（在 PDF 文件中）由 <> 包围，而普通字符串由括号界定。

15.2.10　[dictionary|arraytotable

]

字典或表中的所有条目都可以通过以下命令获取，返回值是一个哈希表或索引表。

```
hash = dictionarytotable(<pdfe dictionary>
list = arraytotable(<pdfe array>
```

你可以通过以下方式获取页面列表：

```
{ { <pdfe dictionary>, size, objnum }, ... } = pagestotable(<pdfe document>
```

15.2.11　getfromreference

因为可能存在未解析的引用，可以通过以下方式解析引用对象：

```
type, <pdfe dictionary|array|stream>, detail = getfromreference(<pdfe reference>
```

因此，作为第二个值，你会得到一个新的 pdfe userdata 对象，你可以查询它。

15.3　内存流

pdfe.new 接受三个参数：

值	说明

stream	这是一个（用低级 Lua 术语来说）轻量级 userdata 对象，即指向字节序列的指针
length	这是流的长度，以字节为单位（流可以包含嵌入的零字节）
name	可选，这是一个唯一标识符，用于对流进行哈希处理，以便多个流不会占用更多内存

第三个参数是可选的。如果未给出，函数将像处理普通文件一样返回一个 pdfe 文档对象；否则它将返回一个文件名，可在其他地方（例如在图像库中）用作伪文件来引用该流。

你也可以传递一个 Lua 字符串，而不是轻量级 userdata 流（这实际上不太稳定，但当你来自一个库时很方便），在这种情况下，给定的长度（至多）是字符串长度。

该函数返回一个 pdfe 对象和一个字符串。该字符串可以在 img 库中代替文件名使用。当你将该对象用作图像时，需要防止它被垃圾回收（例如通过将其存储在某处）。

内存流及其在图像库中的使用都是实验性的，可能会发生变化。如果你想知道这在哪里可以使用：当你使用 graphicmagick 的 swiglib 库时，它可以返回这样一个 userdata 对象。这允许在内存中进行转换并将结果直接传递给后端。这在一遍式工作流中可能会节省一些运行时间。此功能目前不用于生产环境，我们可能会提供更好的实现。

15.4　pdfscanner

pdfscanner 库允许解释 PDF 内容流和 /ToUnicode（cmap）流。你可以从 pdfe 库获取这些流，如前面的章节所述。此库中只有一个顶层函数：

```
pdfscanner.scan <pdfe stream>, <table> operatortable, <table> info)
pdfscanner.scan <pdfe array>, <table> operatortable, <table> info)
pdfscanner.scan <string>, <table> operatortable, <table> info)
```

第一个参数应该是一个 Lua 字符串,或者来自 pdfe 库的流或数组对象。第二个参数 operatortable 应该是一个 Lua 表，其中键是 PDF 运算符名称字符串，值是（由你定义的）用于处理这些运算符的 Lua 函数。每当扫描器在内容流中找到这些 PDF 运算符之一时，就会调用这些函数。这些函数以两个参数被调用：scanner 对象本身，以及作为 pdfscanner.scan 第三个参数传递的 info 表。

在内部，pdfscanner.scan 遍历流中的 PDF 运算符，在内部栈上收集操作数，直到找到一个 PDF 运算符。如果该 PDF 运算符的名称存在于 operatortable 中，则执行关联的函数。函数运行后（或没有函数可执行时），内部操作数栈会被清空，为下一个运算符做准备，然后继续处理。

处理函数的 scanner 参数是必需的，因为它提供了各种方法来从内部操作数栈中获取实际操作数。操作数栈中获取实际操作数。

处理 PDF 文档流的一个简单示例如下：

```
local operatortable = { }

operatortable.Do = function(scanner,info)
```

```
    local resources = info.resources
    if resources then
        local val     = scanner:pop()
        local name    = val[2]
        local xobject = resources.XObject
        print(info.space .. "Uses XObject " .. name)
        local resources = xobject.Resources
        if resources then
            local newinfo =  {
                space     = info.space .. "  ",
                resources = resources,
            }
            pdfscanner.scan(entry, operatortable, newinfo)
        end
    end
end

local function Analyze(filename)
    local doc = pdfe.open(filename)
    if doc then
        local pages = doc.Pages
        for i=1,#pages do
            local page = pages[i]
            local info = {
              space     = "  " ,
              resources = page.Resources,
            }
            print("Page " .. i)
         -- pdfscanner.scan(page.Contents,operatortable,info)
            pdfscanner.scan(page.Contents(),operatortable,info)
        end
    end
end

Analyze("foo.pdf")
```

此示例遍历 PDF 中的所有实际内容，并打印找到的 XObject 名称。虽然这段代码演示了不少 pdfe 函数，但我们还是关注 pdfscanner 类型特有的代码。

自底向上看，以下行运行扫描器，第一个参数给出了 PDF 页面的顶层内容。

第三个参数 info 包含两个条目：space 用于缩进打印输出，resources 是必需的，以便嵌入的 XForms 能够找到自己的内容。

第二个参数 operatortable 为单个 PDF 运算符 Do 定义了一个处理函数。函数 Do 打印当前 XObject 的名称，然后在该 XObject 实际上是一个 /Form 的条件下，为该对象的内容流启动一个新的扫描器。那个嵌套扫描器使用新的 info 参数调用，其中 space 值已更新，以便输出的缩进良好地嵌套；还带有一个新的 resources 字段，以帮助下一次迭代正确处理任何其他嵌入的 XObject。

当然，这在实践中不是一个非常有用的示例，但对于演示 pdfscanner 来说，它的长度刚好。它只使用了一个 scanner 方法：scanner:pop()。该函数弹出内部栈的顶部操作数，并返回一个 Lua 表，其中索引 1 处的对象是一个表示操作数类型的字符串，索引 2 处的对象是其值。

可能的操作数类型及关联的 Lua 值类型列表如下：

类型	Lua 类型
integer	<number>
real	<number>
boolean	<boolean>
name	<string>
operator	<string>
string	<string>
array	<table>
dict	<table>

对于 integer 或 real，值始终是一个 Lua（浮点）数字。对于 name，前导斜杠总是被去掉。

对于 string，请注意 PDF 实际上在 PDF 文档的不同部分支持不同类型的字符串（具有不同的编码），因此你可能需要对某些结果重新编码；pdfscanner 始终输出字节流而不进行任何重新编码。pdfscanner 不区分字面字符串和十六进制字符串（十六进制值会被解码），并且它将内嵌图像的流数据视为一个字符串，即 EI 的唯一操作数。

对于 array，表内容是 pop 返回值的列表；对于 dict，表键是 PDF 名称字符串，值是 pop 返回值。

还有一些其他方法可以供你向 scanner 调用：

方法	说明
pop	见上文
popnumber	仅返回 real 或 integer 的值
popname	仅返回 name 的值
popstring	仅返回 string 的值
poparray	仅返回 array 的值
popdictionary	仅返回 dict 的值
popboolean	仅返回 boolean 的值
done	中止此 scan() 调用的进一步处理

pop* 是便捷函数，当你事先知道操作数的类型时（在 PDF 中你通常确实知道）会很方便。例如，Do 函数本可以使用 local name = scanner:popname()，因为 Do 运算符的唯一操作数始终是一个 PDF 名称对象。

done 函数允许你在已经了解了所有想了解的内容后中止流的处理。这在解析 /ToUnicode 时很方便，因为通常有一些你不感兴趣的尾部垃圾数据。如果没有 done，处理只会在流的末尾结束，可能会浪费 CPU 周期。

我们保留了旧的名称 *popNumber*、*popName*、*popString*、*popArray*、*popDict* 和 *popBool*。

索引

Aleph, 21, 28
e-TEX, 18
Lua, 4
Lua+byte code, 34
Lua+extensions, 38
Lua+interpreter, 34
Lua+libraries, 38, 42
Lua+modules, 42
MetaPost, 179
MetaPost+mplib, 179
Omega, 28
OpenType, 188
PDF, 213
PDF+`pdfe`, 218
PDF+analyze, 218
PDF+annotations, 215, 218
PDF+backend, 22
PDF+catalog, 213
PDF+color stack, 218
PDF+compression, 214
PDF+date, 43, 214
PDF+fonts, 218
PDF+info, 213
PDF+margins, 215
PDF+matrix, 216
PDF+memory streams, 222
PDF+objects, 215–218
PDF+options, 214
PDF+page attributes, 213
PDF+page resources, 213
PDF+pages, 218
PDF+positioning, 215
PDF+positions, 215
PDF+precision, 214
PDF+print to, 216
PDF+resolution, 215
PDF+scanner, 223
PDF+trailer, 213, 214
PDF+unicode, 214
PDF+version, 213
PDF+xform attributes, 213
PDF+xform resources, 213
pdfTEX, 18
TEX, 17
TrueType, 188
Type 1, 189
Unicode, 7, 8
Unicode+math, 68
WEBC, 22

adjust, 87
`apply_afmfile`, 189
`apply_featurefile`, 189
`arraytotable`, 222
`attribute`, 149
attributes, 8, 9, 113, 149

backend, 22, 213
`badness`, 161
banner, 6
boundary, 91
`box`, 149, 152
boxes, 4, 9, 152
`boxes`, 178
boxes+reuse, 153

boxes+split, 153

callbacks, 125
callbacks+PDF file, 138
callbacks+building pages, 130
callbacks+closing files, 128
callbacks+contributions, 130, 132
callbacks+data files, 127
callbacks+dump, 136
callbacks+editing, 137
callbacks+errors, 136, 137
callbacks+files, 137
callbacks+font files, 126, 127
callbacks+fonts, 139, 140
callbacks+format file, 126
callbacks+hyphenation, 134
callbacks+image content, 139
callbacks+image files, 127
callbacks+input buffer, 129
callbacks+inserts, 131
callbacks+job run, 136
callbacks+jobname, 130
callbacks+kerning, 135
callbacks+ligature building, 135
callbacks+linebreaks, 131–133
callbacks+math, 135
callbacks+opening files, 128
callbacks+output, 134
callbacks+output buffer, 129
callbacks+output file, 126
callbacks+packing, 133, 134
callbacks+pages, 136
callbacks+reader, 128
callbacks+readers, 128
callbacks+rules, 134
callbacks+synctex, 138
callbacks+wrapping up, 138
`catcode`, 151
catcodes, 12
`char_depth`, 186
`char_height`, 185
`char_width`, 185
characters, 44
characters+codes, 151
`check_permission`, 173
`check_discretionaries`, 115
`check_discretionary`, 115
`close`, 188, 219
`closeinput`, 167
`closestream`, 221
command line, 34
configuration, 164
convert commands, 148
`copy`, 105, 177
`copy_list`, 105
`count`, 108, 149
`cprint`, 157
csnames, 32
`current_attr`, 105

`default_texmfcnf`, 170
`definefont`, 160
`delcode`, 151
`dictionarytotable`, 222
`dimen`, 149
`dimensions`, 106
direct nodes, 115
directions, 28, 91
discretionaries, 49, 87
`done`, 225

`enableprimitives`, 160
`end_of_math`, 110
engines, 17
`error`, 159
errors, 13, 159
exceptions, 48
`execute`, 180
`expand_braces`, 172
`expand_path`, 172
`expand_var`, 172

expansion, 16
extraprimitives, 161

family_font, 115
fields, 84, 104, 175, 181, 189
files+binary, 32
files+finding, 170
files+map, 213
files+recording, 170
find, 125
find_attribute, 114
find_file, 170
finish, 159, 181
fio.getposition, 41
fio.read2dot14, 41
fio.readbytes, 41
fio.readbytetable, 41
fio.readcardinal1, 41
fio.readcardinal2, 41
fio.readcardinal3, 41
fio.readcardinal4, 41
fio.readcardinaltable, 41
fio.readfixed2, 41
fio.readfixed4, 41
fio.readinteger1, 41
fio.readinteger2, 41
fio.readinteger3, 41
fio.readinteger4, 41
fio.readintegertable, 41
fio.setposition, 41
fio.skipposition, 41
first_glyph, 111
flatten_discretionaries, 115
flush_list, 104
flush_node, 104
flush_properties_table, 122
fontidentifier, 158
fontloader+tables, 189
fontname, 158
fonts, 14, 52
fonts+current, 66
fonts+define, 64
fonts+defining, 160
fonts+extend, 65
fonts+id, 66
fonts+information, 187
fonts+iterate, 66
fonts+library, 63
fonts+loading, 187
fonts+real, 58
fonts+tables, 52
fonts+tfm, 63
fonts+vf, 63
fonts+virtual, 58, 59, 62, 63
force_synctex_line, 164
force_synctex_tag, 164
forcehmode, 160
format, 7, 32
free, 104

get, 144
get_attribute, 114
get_boolean, 186
get_numeric, 186
get_path, 186
get_properties_table, 122
get_string, 186
get_synctex_line, 164
get_synctex_mode, 164
get_synctex_tag, 164
getarray, 221
getattribute, 149
getboolean, 220
getbox, 149, 152, 220
getboxresourcedimensions, 153
getcatalog, 213, 220
getcatcode, 151
getcompresslevel, 214
getcount, 149
getcreationdate, 214

getdecimaldigits, 214
getdelcode, 151
getdelcodes, 151
getdestmargin, 215
getdictionary, 221
getdimen, 149
getfontname, 215
getfontobjnum, 215
getfontsize, 215
getfromarray, 221
getfromdictionary, 221
getfromreference, 222
getfromstream, 221
getgentounicode, 214
getglue, 113, 149
gethjcode, 50
gethpos, 215
getignoreunknownimages, 214
getimageresolution, 215
getinclusionerrorlevel, 214
getinfo, 213, 220
getinteger, 220
getlastannot, 215
getlastlink, 215
getlastobj, 215
getlccode, 151
getlinkmargin, 215
getlist, 154
getlocallevel, 163
getmajorversion, 213
getmarginmargin, 215
getmark, 149
getmath, 153
getmathcode, 151
getmathcodes, 151
getmatrix, 216
getmaxobjnum, 215
getmemoryusage, 220
getminorversion, 213
getmuglue, 149
getmuskip, 149
getname, 220
getnames, 213
getnest, 155
getnofobjects, 220
getnofpages, 220
getnumber, 220
getobjcompresslevel, 214
getobjtype, 215
getomitcharset, 214
getomitcidset, 214
getomitinfo, 214
getomitmediabox, 214
getomitprocset, 214
getorigin, 215
getpage, 220
getpageattributes, 213
getpageref, 218
getpageresources, 213
getpagesattributes, 213
getpagestate, 163
getpkresolution, 215
getpos, 215
getptexprefix, 214
getrecompress, 214
getretval, 215
getsfcode, 151
getsize, 220
getskip, 149
getstatus, 219
getstream, 221
getstring, 220
getsuppressoptionalinfo, 214
getthreadmargin, 215
gettoks, 149
gettrailer, 213, 220
gettrailerid, 214
getuccode, 151
getversion, 220
getvpos, 215

getxformattributes, 213
getxformmargin, 215
getxformname, 215
getxformresources, 213
glue, 88
glue, 149
glyphs, 44, 89
graphics, 174

has_attribute, 114
has_field, 104
has_glyph, 110
hash, 160
hashtokens, 160
hasmatrix, 216
helpers, 158
history, 17
hpack, 106
hyperlinks, 32
hyphenate, 50
hyphenation, 44, 46, 48
hyphenation+discretionaries, 49
hyphenation+exceptions, 48
hyphenation+how it works, 49
hyphenation+patterns, 48

I/O, 167
id, 103
images, 174
images+MetaPost, 179
images+immediate, 177
images+injection, 177
images+library, 174
images+mplib, 179
images+object, 177
images+types, 178
immediateobj, 216
immediatewrite, 177
immediatewriteobject, 177
in_name_ok, 172
in_name_ok_silent_extended, 172
info, 187
init_prog, 172
init_rand, 164
initialization, 34, 160
initialization+bitmaps, 172
insert_after, 111
insert_before, 110
insertions, 86
is_char, 108
is_glyph, 108
is_node, 103
is_zero_glue, 113
isattribute, 149
isbox, 149
iscount, 149
isdimen, 149
isglue, 149
ismuglue, 149
ismuskip, 149
isskip, 149
istoks, 149

kerning, 111
kerns, 89
kerns+suppress, 14

languages, 44
last items, 148
last_node, 112
lccode, 151
length, 108
libraries+kpse, 169
libraries+lua, 141
libraries+status, 141
libraries+texconfig, 164
libraries+texio, 167
libraries+tex, 144
libraries+token, 168
ligatures+suppress, 14
ligaturing, 111
linebreak, 162

linebreaks, 162
list, 125, 141
lists, 85, 154
lists, 154
locales, 42
lookup, 171
lua_math_random, 164
lua_math_randomseed, 164

main loop, 46
map files, 213
marks, 16, 86
math, 13, 68
math+Unicode, 68
math+accents, 80
math+cramped, 71
math+delimiters, 81
math+extensibles, 81
math+flattening, 83
math+last line, 81
math+nodes, 88, 92
math+parameters, 72, 74, 153
math+penalties, 79
math+radicals, 80
math+scripts, 81, 82
math+spacing, 71, 78
math+stacks, 71
math+styles, 70, 71, 82
math+text, 82
math+tracing, 83
mathcode, 151
memory, 31
mlist_to_hlist, 107
muglue, 149
muskip, 149

nest, 155
nesting, 155, 163
new, 104, 170, 174, 179, 219, 222
newcolorstack, 218
newline, 32
next, 105
node, 178
nodes, 4, 8, 84
nodes+adjust, 87
nodes+attributes, 113
nodes+boundary, 91
nodes+direct, 115
nodes+direction, 91
nodes+discretionaries, 87
nodes+functions, 103
nodes+glue, 88
nodes+glyph, 89
nodes+insertions, 86
nodes+kerns, 89
nodes+lists, 85
nodes+marks, 86
nodes+math, 88, 92
nodes+paragraphs, 91, 92
nodes+penalty, 89
nodes+properties, 122
nodes+rules, 85
nodes+text, 84
normal_rand, 164
number, 158

obj, 217
open, 188, 219
openstream, 221
os.env, 40
os.exec, 40
os.gettimeofday, 40
os.kpsepopen, 40
os.name, 40
os.selfdir, 40
os.setenv, 40
os.sleep, 40
os.socketgettime, 40
os.socketsleep, 40
os.spawn, 40
os.times, 40

os.tmpdir, 40
os.type, 40
os.uname, 40
out_name_ok, 173
out_name_ok_silent_extended, 173

pages, 153, 163
paragraphs, 91, 92
paragraphs+reset, 162
parameters+internal, 144
parameters+math, 153
patterns, 48
pen_info, 185
penalty, 89
pop, 225
poparray, 225
popboolean, 225
popdictionary, 225
popname, 225
popnumber, 225
popstring, 225
postexhyphenchar, 50
posthyphenchar, 50
preexhyphenchar, 50
prehyphenchar, 50
prepend_prevdepth, 106
prev, 105
primitives, 13, 160
primitives, 161
print, 156, 216
printing, 156
properties, 122
protect_glyph, 112
protect_glyphs, 112
protrusion, 92
protrusion_skippable, 112
ptr, 155

rangedimensions, 106
readable_file, 172
readfromstream, 221
readfromwholestream, 221
record_input_file, 170
record_output_file, 170
refobj, 217
register, 125
registerannot, 218
registers, 149, 152
remove, 110
reserveobj, 218
resetmessages, 141
resetparagraph, 162
romannumeral, 158
round, 158
rules, 85
run, 159

saveboxresource, 153
scale, 158
scan, 176, 223
scan_argument, 168
scan_code, 168
scan_csname, 168
scan_dimen, 168
scan_float, 168
scan_glue, 168
scan_int, 168
scan_keyword, 168
scan_keyword_cs, 168
scan_list, 168
scan_real, 168
scan_string, 168
scan_toks, 168
scan_word, 168
scantoks, 149
set, 144
set_attribute, 114
set_program_name, 170
set_properties_mode, 122
set_synctex_line, 164
set_synctex_mode, 164

set_synctex_no_files, 164
set_synctex_tag, 164
setattribute, 149
setbox, 149, 152
setcatalog, 213
setcatcode, 151
setcompresslevel, 214
setcount, 149
setdecimaldigits, 214
setdelcode, 151
setdelcodes, 151
setdestmargin, 215
setdimen, 149
setescape, 167
setexitcode, 141
setfontattributes, 218
setgentounicode, 214
setglue, 112, 149
sethjcode, 50
setignoreunknownimages, 214
setimageresolution, 215
setinclusionerrorlevel, 214
setinfo, 213
setlastannot, 215
setlastlink, 215
setlastobj, 215
setlccode, 151
setlinkmargin, 215
setlist, 154
setmajorversion, 213
setmarginmargin, 215
setmath, 153
setmathcode, 151
setmathcodes, 151
setminorversion, 213
setmuglue, 149
setmuskip, 149
setnames, 213
setobjcompresslevel, 214
setomitcharset, 214
setomitcidset, 214
setomitinfo, 214
setomitmediabox, 214
setomitprocset, 214
setorigin, 215
setpageattributes, 213
setpageresources, 213
setpagesattributes, 213
setpkresolution, 215
setptexprefix, 214
setrecompress, 214
setsfcode, 151
setskip, 149
setsuppressoptionalinfo, 214
setthreadmargin, 215
settoks, 149
settrailer, 213
settrailerid, 214
setuccode, 151
setxformattributes, 213
setxformmargin, 215
setxformresources, 213
sfcode, 151
sha2.digest256, 42
sha2.digest384, 42
sha2.digest512, 42
shipout, 163
shipout, 163
show_context, 159
show_path, 173
sio.getposition, 42
sio.read2dot14, 42
sio.readbytes, 42
sio.readbytetable, 42
sio.readcardinal1, 42
sio.readcardinal2, 42
sio.readcardinal3, 42
sio.readcardinal4, 42
sio.readcardinaltable, 42
sio.readfixed2, 42

sio.readfixed4, 42
sio.readinteger1, 42
sio.readinteger2, 42
sio.readinteger3, 42
sio.readinteger4, 42
sio.readintegertable, 42
sio.setposition, 42
sio.skipposition, 42
skip, 149
slide, 114
sp, 159
space, 32
spaces+suppress, 14
splitbox, 153
sprint, 156
statistics, 180
string.bytepairs, 39
string.bytes, 39
string.characterpairs, 39
string.characters, 39
string.explode, 39
string.utfcharacter, 40
string.utfcharacters, 39
string.utflength, 40
string.utfvalue, 40
string.utfvalues, 39
subtype, 103
subtypes, 84
synctex, 164

tail, 108
testing, 43
text+math, 82
to_table, 188
todirect, 115
tokens, 168
tokens+scanning, 15
toks, 149
tonode, 115
tostring, 115
tprint, 157
traverse, 108
traverse_char, 109
traverse_glyph, 109
traverse_id, 109
traverse_list, 110
triggerbuildpage, 153
type, 103, 220
types, 103, 178

uccode, 151
unencrypt, 219
uniform_rand, 164
uniformdeviate, 164
unprotect_glyph, 111
unprotect_glyphs, 111
unset_attribute, 114
useboxresource, 153

values, 84
var_value, 173
version, 6, 141
version, 141, 173, 179
vpack, 106

whatsits, 103
write, 112, 158, 167, 177
write_nl, 167